

AUDIO_DSP Repository Structure

Organization

This repository contains a professional C++ DSP (Digital Signal Processing) library for audio. All components are organized under the cv_dsp namespace.

Directory Layout

```
├── .github/
│   └── FUNDING.yml
├── CV_DSP/
│   ├── Convolution/
│   │   ├── ConvolutionEngine.hpp
│   │   └── IRLoader.hpp
│   ├── Core/
│   │   ├── AudioBufferView.hpp
│   │   ├── CircularBuffer.hpp
│   │   ├── Config.hpp
│   │   ├── Constants.hpp
│   │   ├── DSPUtils.hpp
│   │   ├── Namespace.hpp
│   │   ├── ParameterSmoother.hpp
│   │   ├── ProcessContext.hpp
│   │   ├── Types.hpp
│   │   └── Version.hpp
│   ├── Delay/
│   │   └── DelayLine.hpp
│   ├── Dynamics/
│   │   ├── Compressor.hpp
│   │   ├── EnvelopeFollower.hpp
│   │   ├── Expander.hpp
│   │   ├── Limiter.hpp
│   │   └── NoiseGate.hpp
│   ├── Effects/
│   │   ├── Chorus.hpp
│   │   └── Flanger.hpp
│   ├── Filters/
│   │   ├── AllPassFilter.hpp
│   │   ├── Biquad.hpp
│   │   └── DCBlocker.hpp
```

```

|   |   |   └─ LadderFilter.hpp
|   |   |   └─ OnePoleFilter.hpp
|   |   |   └─ StateVariableFilter.hpp
|   |   └─ Guitar/
|   |       └─ CabinetSimulator.hpp
|   └─ Math/
|       |   └─ FastMath.hpp
|       |   └─ Interpolation.hpp
|       |   └─ LookupTable.hpp
|       |   └─ Oversampling.hpp
|   └─ Modulation/
|       |   └─ ADSR.hpp
|       |   └─ LFO.hpp
|       |   └─ Oscillator.hpp
|   └─ Saturation/
|       |   └─ TapeSaturation.hpp
|       |   └─ TubeSaturation.hpp
|       |   └─ Waveshaper.hpp
|   └─ Spatial/
|       |   └─ MidSide.hpp
|       |   └─ StereoWidth.hpp
|   └─ Spectral/
|       |   └─ FFT.hpp
|       |   └─ SpectrumAnalyzer.hpp
|       |   └─ STFT.hpp
|       |   └─ WindowFunctions.hpp
└─ LICENSE
└─ README.md

```

and more...

Integration Graph

Real-Time Safe Audio Chain

Input Signal

|

v

LFO

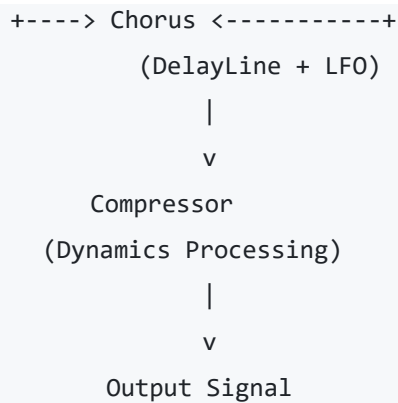
(Modulation)

ParameterSmoother

(Parameter Automation)

|

|



Component Dependencies

Component	Depends On	Purpose
DelayLine	CircularBuffer	Fractional delay with 3 interpolation types
Chorus	DelayLine, LFO	Delay modulation effect
LFO	None	Independent oscillator
Compressor	EnvelopeFollower, ParameterSmoother	Dynamic range control
ParameterSmoother	None	Independent automation
EnvelopeFollower	?	Needs verification

Correct Usage

```
// ✓ CORRECT - Use CV_DSP namespace files
#include "CV_DSP/Core/CircularBuffer.hpp"
#include "CV_DSP/Delay/DelayLine.hpp"
#include "CV_DSP/Core/ParameterSmoother.hpp"
#include "CV_DSP/Modulation/LFO.hpp"
```

DO NOT USE

```
// x WRONG - These files were removed
#include "Core/ParameterSmoother.hpp" // Forward header - REMOVED
#include "Core/CircularBuffer.hpp" // Duplicate - REMOVED
#include "Delay/DelayLine.hpp" // Outdated - REMOVED
```



Integration Improvements

1. Compressor with ParameterSmoother

The Compressor now integrates with ParameterSmoother for smooth parameter automation:

```
// Before: Abrupt parameter changes
compressor.setThreshold(newThreshold);

// After: Smooth automation over N samples
compressor.getThresholdSmoother().setTarget(newThreshold, 480); // 10ms @ 48kHz
```

2. Delay-Based Effects

All delay effects use DelayLine with optional ParameterSmoother:

- Chorus: DelayLine (modulado por LFO)
- Flanger: DelayLine (delay muito curto)
- Vibrato: DelayLine (pitch modulation)
- Reverb: Multi-tap DelayLine



Usage Examples

Simple Chorus Effect

```
#include "CV_DSP/Delay/DelayLine.hpp"
#include "CV_DSP/Modulation/LFO.hpp"

cvdsp::delay::DelayLine<float, 16384, cvdsp::delay::InterpolationType::Linear> chorus;
chorus.prepare(48000);

cvdsp::LFO<float> lfo;
lfo.prepare(48000.0f, 2.5f, 1.0f, cvdsp::LFOWaveform::Sine);

for (size_t n = 0; n < blockSize; ++n) {
    float lfoValue = lfo.process();
    float delay = 40.0f + lfoValue * 10.0f; // 40±10 ms
```

```
chorus.setDelayMilliseconds(delay);

float output = input[n] + 0.7f * chorus.process(input[n]);
}
```

Compressor with Smooth Parameter Changes

```
#include "CV_DSP/Dynamics/Compressor.hpp"
#include "CV_DSP/Core/ParameterSmoother.hpp"

cvdsp::Compressor<float> compressor;
compressor.prepare(48000);

// Smooth parameter automation
compressor.getThresholdSmoother().setTarget(-20.0f, 480); // Ramp over 10ms

for (size_t n = 0; n < blockSize; ++n) {
    compressor.setThreshold(compressor.getThresholdSmoother().process());
    output[n] = compressor.process(input[n]);
}
```



Migration Guide

If you were using the old forward headers:

Before

```
#include "Core/ParameterSmoother.hpp" // Forward to CV_DSP
```

After

```
#include "CV_DSP/Core/ParameterSmoother.hpp" // Direct include
```



Quality Assurance

- ✓ No duplicate headers
- ✓ All components in CV_DSP namespace
- ✓ Real-time safe (O(1) per sample for process())
- ✓ Header-only implementations

- ✓ Zero dynamic allocation during audio processing
- ✓ Professional documentation with examples



References

- CircularBuffer**: Ring buffer for real-time audio storage
- DelayLine**: Fractional delay with Hermite interpolation
- ParameterSmoother**: Linear/Exponential/OnePole automation
- LFO**: Sine/Triangle/Saw/Square waveforms (0.01-50 Hz)
- Chorus**: Classic chorus effect (Haas effect)

Dynamics/Expander.hpp

Conceito

Um **Downward Expander** reduz o ganho apenas quando o sinal fica **abaixo do threshold**.

Enquanto um compressor reduz sinais acima do limiar, o expander faz o oposto:

- Sinal acima do threshold → sem alteração
- Sinal abaixo do threshold → redução progressiva de ganho

É muito utilizado para:

- Redução de ruído de fundo
 - Limpeza de capturas de guitarra
 - Controle de bleed em microfones
 - Voice processing
 - Pré-gate suave
-

Diferenças

Gate

Funcionamento:

Acima do Threshold:

Ganho = 0 dB

Abaixo do Threshold:

Ganho = $-\infty$ dB

Resultado:

- Abre ou fecha
 - Transição abrupta
 - Pode gerar cortes audíveis
-

Expander

Funcionamento:

Acima do Threshold:

Ganho = 0 dB

Abaixo do Threshold:

Redução gradual

Resultado:

- Muito mais natural
- Mantém tails e reverbs

- Menos artefatos
-

Compressor

Funcionamento:

Acima do Threshold:
Reduz dinâmica
Abaixo do Threshold:
Sem alteração

Objetivo:

- Controlar picos
 - Reduzir faixa dinâmica
-

Matemática do Downward Expander

Considere:

Threshold = -40 dB
Ratio = 4:1

Entrada:

-52 dB

Diferença:

$\Delta = \text{Threshold} - \text{Input}$
 $\Delta = -40 - (-52)$
 $\Delta = 12 \text{ dB}$

Aplicando ratio:

Gain Reduction
 $\text{GR} = \Delta \times (\text{Ratio} - 1)$
 $\text{GR} = 12 \times (4 - 1)$
 $\text{GR} = 36 \text{ dB}$

Saída:

-52 - 36
= -88 dB

Quanto mais distante do threshold:

- maior redução
 - menor ruído residual
-

Estrutura

CV_DSP

Dynamics/Expander.hpp

```
#ifndef CVDSP_DYNAMICS_EXPANDER_HPP
#define CVDSP_DYNAMICS_EXPANDER_HPP
/**
 * @file Expander.hpp
 * @brief Downward Expander
 *
 * Header-only
 * Real-Time Safe
 * C++20
 */
#include <algorithm>
#include <array>
#include <cmath>
#include <cstdint>
#include <limits>
namespace cvdsp
{
/**
 * @brief Downward Expander
 *
 * Threshold em dBFS
 * Ratio >= 1
 *
 * Exemplo:
 *
 * threshold = -40 dB
 * ratio = 4
 *
 * Sinais abaixo do threshold
 * serão progressivamente atenuados.
 */
template<typename T>
class Expander
{
    static_assert(
        std::is_floating_point_v<T>,
        "Expander requires floating point type");
public:
    constexpr Expander() noexcept = default;
    /**
     * @brief Inicializa coeficientes.
     *
     * @param sampleRate Taxa de amostragem
     * @param attackMs Ataque em ms
     * @param releaseMs Release em ms
     * @param thresholdDb Threshold em dBFS
     * @param ratio Ratio de expansão
     */
    void prepare(
        T sampleRate,
        T attackMs,
        T releaseMs,
        T thresholdDb,
        T ratio) noexcept
    {
        sampleRate_ = sampleRate;
        attackMs_ = std::max(
            attackMs,
            static_cast<T>(0.001));
    }
};
#endif
```

```

        releaseMs_ = std::max(
            releaseMs,
            static_cast<T>(0.001));
        thresholdDb_ = thresholdDb;
        ratio_ = std::max(
            ratio,
            static_cast<T>(1));
        attackCoeff_ =
            computeTimeCoefficient(
                attackMs_);
        releaseCoeff_ =
            computeTimeCoefficient(
                releaseMs_);
        reset();
    }
    /**
     * @brief Reinicia estado interno.
     */
    void reset() noexcept
    {
        envelopeDb_ = static_cast<T>(-120);
        gainDb_ = static_cast<T>(0);
    }
    /**
     * @brief Processa uma amostra.
     *
     * @param input Entrada
     * @return Saída expandida
     */
    inline T process(T input) noexcept
    {
        constexpr T kMinDb =
            static_cast<T>(-120);
        constexpr T kEpsilon =
            static_cast<T>(1e-30);
        const T absInput =
            std::abs(input);
        const T levelDb =
            static_cast<T>(20) *
            std::log10(
                std::max(
                    absInput,
                    kEpsilon));
        const T detectorCoeff =
            (levelDb > envelopeDb_)
            ? attackCoeff_
            : releaseCoeff_;
        envelopeDb_ =
            detectorCoeff * envelopeDb_
            +
            (static_cast<T>(1) - detectorCoeff)
            * levelDb;
        T targetGainDb =
            static_cast<T>(0);
        if (envelopeDb_ < thresholdDb_)
        {
            const T delta =
                thresholdDb_ - envelopeDb_;
            targetGainDb =
                -delta *
                (ratio_ - static_cast<T>(1));
        }
        const T gainCoeff =
            (targetGainDb < gainDb_)
            ? attackCoeff_
            : releaseCoeff_;
        gainDb_ =
            gainCoeff * gainDb_
            +

```

```

        (static_cast<T>(1) - gainCoeff)
        * targetGainDb;
gainDb_ =
    std::clamp(
        gainDb_,
        kMinDb,
        static_cast<T>(0));
const T linearGain =
    dbToLinear(
        gainDb_);
return input * linearGain;
}
private:
/**
 * @brief Converte dB para ganho linear.
 */
static constexpr T dbToLinear(
    T db) noexcept
{
    return std::pow(
        static_cast<T>(10),
        db / static_cast<T>(20));
}
/**
 * @brief Coeficiente exponencial.
 */
T computeTimeCoefficient(
    T timeMs) const noexcept
{
    const T timeSeconds =
        timeMs *
        static_cast<T>(0.001);
    return std::exp(
        static_cast<T>(-1)
        /
        (
            sampleRate_
            *
            timeSeconds
        ));
}
private:
T sampleRate_ = static_cast<T>(44100);
T thresholdDb_ = static_cast<T>(-40);
T ratio_ = static_cast<T>(4);
T attackMs_ = static_cast<T>(5);
T releaseMs_ = static_cast<T>(100);
T attackCoeff_ = static_cast<T>(0);
T releaseCoeff_ = static_cast<T>(0);
T envelopeDb_ = static_cast<T>(-120);
T gainDb_ = static_cast<T>(0);
};
} // namespace cvdsp
#endif

```

Características da Implementação

Compatível com

cvdsp::Expander<float>
cvdsp::Expander<double>

Real-Time Safe

Nenhum:

```
new  
delete  
malloc  
free  
throw
```

durante `process()`.

Envelope Detector

Utiliza detector exponencial:

```
attackCoeff  
releaseCoeff
```

permitindo resposta suave.

Controle de Ganho

Suavizado independentemente:

```
gainDb_
```

evitando zipper noise.

Estabilidade Numérica

Proteção contra:

```
log10(0)
```

através de:

```
1e-30
```

Uso

```
cvdsp::Expander<float> expander;  
expander.prepare(  
    48000.0f,  
    5.0f,      // attack  
    100.0f,    // release  
    -40.0f,    // threshold  
    4.0f);     // ratio  
float y = expander.process(x);
```

Modulation/Oscillator.hpp

```
#ifndef CVDSP_MODULATION_OSCILLATOR_HPP
#define CVDSP_MODULATION_OSCILLATOR_HPP
/**
 * @file Oscillator.hpp
 * @brief Digital Oscillator
 *
 * Header-only
 * C++20
 * Real-Time Safe
 * No dynamic allocation
 */
#include <cmath>
#include <cstdint>
#include <type_traits>
#include <algorithm>
namespace cvdsp
{
/**
 * @brief Oscillator waveform types.
 */
enum class OscillatorWaveform
{
    Sine,
    Triangle,
    Saw,
    Square
};
/**
 * @brief Digital oscillator using phase accumulator.
 *
 * Supports:
 * - Sine
 * - Triangle
 * - Saw
 * - Square
 *
 * Template:
 * float
 * double
 */
template<typename T>
class Oscillator
{
    static_assert(
        std::is_floating_point_v<T>,
        "Oscillator requires floating point type");
public:
    constexpr Oscillator() noexcept = default;
    /**
     * @brief Initialize oscillator.
     *
     * @param sampleRate Processing sample rate.
     * @param frequency Initial frequency.
     * @param waveform Waveform type.
     */
    void prepare(
        T sampleRate,
        T frequency,
        OscillatorWaveform waveform) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));
        waveform_ = waveform;
    }
};
#endif
```

```

        setFrequency(frequency);
        reset();
    }
    /**
     * @brief Reset oscillator state.
     */
    void reset() noexcept
    {
        phase_ = static_cast<T>(0);
    }
    /**
     * @brief Set oscillator frequency.
     *
     * @param frequency Frequency in Hz.
     */
    void setFrequency(
        T frequency) noexcept
    {
        constexpr T kMinFreq =
            static_cast<T>(0);
        const T nyquist =
            sampleRate_ *
            static_cast<T>(0.5);
        frequency_ =
            std::clamp(
                frequency,
                kMinFreq,
                nyquist);
        phaseIncrement_ =
            frequency_ / sampleRate_;
    }
    /**
     * @brief Set waveform.
     *
     * @param waveform New waveform.
     */
    void setWaveform(
        OscillatorWaveform waveform) noexcept
    {
        waveform_ = waveform;
    }
    /**
     * @brief Get current frequency.
     */
    [[nodiscard]]
    T getFrequency() const noexcept
    {
        return frequency_;
    }
    /**
     * @brief Generate one sample.
     */
    inline T process() noexcept
    {
        T output {};
        switch (waveform_)
        {
            case OscillatorWaveform::Sine:
            {
                output = processSine();
                break;
            }
            case OscillatorWaveform::Triangle:
            {
                output = processTriangle();
                break;
            }
            case OscillatorWaveform::Saw:
            {

```

```

        output = processSaw();
        break;
    }
    case OscillatorWaveform::Square:
    {
        output = processSquare();
        break;
    }
    default:
    {
        output = static_cast<T>(0);
        break;
    }
}
advancePhase();
return output;
}
private:
/**
 * @brief Advance normalized phase.
 *
 * Range:
 *
 * [0,1)
 */
inline void advancePhase() noexcept
{
    phase_ += phaseIncrement_;
    if (phase_ >= static_cast<T>(1))
    {
        phase_ -= static_cast<T>(1);
    }
}
/**
 * @brief Sine oscillator.
 */
inline T processSine() const noexcept
{
    constexpr T kTwoPi =
        static_cast<T>(
            6.283185307179586476925286766559);
    return std::sin(
        phase_ * kTwoPi);
}
/**
 * @brief Saw oscillator.
 *
 * Range:
 *
 * -1 ... +1
 */
inline T processSaw() const noexcept
{
    return
        static_cast<T>(2) * phase_
        -
        static_cast<T>(1);
}
/**
 * @brief Square oscillator.
 *
 * 50% duty cycle.
 */
inline T processSquare() const noexcept
{
    return
        (phase_ < static_cast<T>(0.5))
        ? static_cast<T>(1)
        : static_cast<T>(-1);
}

```

```

    }
    /**
     * @brief Triangle oscillator.
     *
     * Range:
     *
     * -1 ... +1
     */
    inline T processTriangle() const noexcept
    {
        return
            static_cast<T>(1)
            -
            static_cast<T>(4)
            *
            std::abs(
                phase_
                -
                static_cast<T>(0.5));
    }
private:
    T sampleRate_ =
        static_cast<T>(44100);
    T frequency_ =
        static_cast<T>(440);
    T phase_ =
        static_cast<T>(0);
    T phaseIncrement_ =
        static_cast<T>(0);
    OscillatorWaveform waveform_ =
        OscillatorWaveform::Sine;
};
} // namespace cvdsp
#endif

```

Matemática do Phase Accumulator

O oscilador utiliza fase normalizada:

$\text{phase} \in [0, 1)$

Incremento por amostra:

$\text{phaseIncrement} = \text{frequency} / \text{sampleRate}$

Exemplo:

frequency = 1000 Hz
sampleRate = 48000 Hz

$\text{phaseIncrement} = 1000 / 48000$
 $\text{phaseIncrement} = 0.0208333333$

A cada amostra:

$\text{phase} += \text{phaseIncrement}$

Quando:

$\text{phase} \geq 1.0$

ocorre wrap:

$\text{phase} -= 1.0$

produzindo uma oscilação periódica infinita.

Senoide

Equação:

$$y = \sin(2\pi \text{ phase})$$

Implementação:

```
std::sin(phase * 2π)
```

Saw

Equação:

$$y = 2 \text{ phase} - 1$$

Faixa:

[-1, +1]

Square

Equação:

$$\text{phase} < 0.5 ? +1 : -1$$

Duty:

50%

Triangle

Equação:

$$y = 1 - 4 |\text{phase} - 0.5|$$

Faixa:

[-1, +1]

Nyquist

Para uma taxa:

Fs

a frequência máxima reproduzível é:

$$\text{Nyquist} = F_s / 2$$

Exemplo:

$$F_s = 48000$$

$$\text{Nyquist} = 24000 \text{ Hz}$$

Nenhum conteúdo espectral pode existir acima desse limite sem gerar aliasing.

Aliasing

Aliasing ocorre quando componentes espectrais ultrapassam Nyquist.

Exemplo:

$$F_s = 48000$$

$$\text{Nyquist} = 24000$$

Harmônico:

$$26000 \text{ Hz}$$

Reflexão:

$$48000 - 26000$$

Resultado:

$$22000 \text{ Hz}$$

Esse componente aparece dentro da banda audível como frequência falsa.

Osciladores Digitais

Sine

Possui apenas:

Fundamental

Não produz aliasing significativo.

Triangle

Possui:

Harmônicos ímpares

Amplitude:

1 / n²

Aliasing moderado.

Square

Possui:

Harmônicos ímpares infinitos

Amplitude:

1 / n

Aliasing elevado.

Saw

Possui:

Todos os harmônicos infinitos

Amplitude:

1 / n

É a forma de onda mais propensa a aliasing.

Limitação desta implementação

Esta implementação é um:

Naive Oscillator

As formas:

Saw
Square
Triangle

não são band-limited.

Em frequências altas haverá aliasing.

Arquiteturalmente ela serve como base para futuras implementações:

PolyBLEP
PolyBLAMP
MinBLEP
Wavetable
Differentiated Parabolic Wave
Band-Limited Step
Oversampled Oscillator

mantendo a mesma interface:

prepare()
reset()
setFrequency()
process()

Modulation/LFO.hpp

```
#ifndef CVDSP_MODULATION_LFO_HPP
#define CVDSP_MODULATION_LFO_HPP
/**
 * @file LFO.hpp
 * @brief Low Frequency Oscillator
 *
 * Header-only
 * C++20
 * Real-Time Safe
 * No dynamic allocation
 */
#include <algorithm>
#include <cmath>
#include <type_traits>
namespace cvdsp
{
/**
 * @brief LFO waveform types.
 */
enum class LFOWaveform
{
    Sine,
    Triangle,
    Saw,
    Square
};
/**
 * @brief Low Frequency Oscillator.
 *
 * Frequency Range:
 *
 * 0.01 Hz
 * to
 * 50 Hz
 *
 * Output Range:
 *
 * depth = 1.0
 *
 * [-1, +1]
 *
 * depth = 0.5
 *
 * [-0.5, +0.5]
 *
 * Suitable for:
 *
 * - Chorus
 * - Flanger
 * - Phaser
 * - Tremolo
 * - Auto Pan
 * - Filter Modulation
 */
template<typename T>
class LFO
{
    static_assert(
        std::is_floating_point_v<T>,
        "LFO requires floating point type");
public:
    constexpr LFO() noexcept = default;
    /**
     * @brief Prepare LFO.
     *
     * @param sampleRate Processing sample rate.
     */

```

```

* @param rateHz Initial rate.
* @param depth Initial depth.
* @param waveform Waveform.
*/
void prepare(
    T sampleRate,
    T rateHz,
    T depth,
    LFOWaveform waveform) noexcept
{
    sampleRate_ =
        std::max(
            sampleRate,
            static_cast<T>(1));
    waveform_ = waveform;
    setRate(rateHz);
    setDepth(depth);
    reset();
}
/**
* @brief Reset phase accumulator.
*/
void reset() noexcept
{
    phase_ = static_cast<T>(0);
}
/**
* @brief Set LFO frequency.
*
* Range:
*
* 0.01 Hz
* to
* 50 Hz
*/
void setRate(
    T rateHz) noexcept
{
    constexpr T kMinRate =
        static_cast<T>(0.01);
    constexpr T kMaxRate =
        static_cast<T>(50.0);
    rateHz_ =
        std::clamp(
            rateHz,
            kMinRate,
            kMaxRate);
    phaseIncrement_ =
        rateHz_ / sampleRate_;
}
/**
* @brief Set modulation depth.
*
* Range:
*
* 0.0 ... 1.0
*/
void setDepth(
    T depth) noexcept
{
    depth_ =
        std::clamp(
            depth,
            static_cast<T>(0),
            static_cast<T>(1));
}
/**
* @brief Select waveform.
*/

```

```

void setWaveform(
    LFOWaveform waveform) noexcept
{
    waveform_ = waveform;
}
/**
 * @brief Current rate.
 */
[[nodiscard]]
T getRate() const noexcept
{
    return rateHz_;
}
/**
 * @brief Current depth.
 */
[[nodiscard]]
T getDepth() const noexcept
{
    return depth_;
}
/**
 * @brief Generate one LFO sample.
 *
 * Output:
 * [-depth,+depth]
 */
inline T process() noexcept
{
    T value {};
    switch (waveform_)
    {
        case LFOWaveform::Sine:
        {
            value = processSine();
            break;
        }
        case LFOWaveform::Triangle:
        {
            value = processTriangle();
            break;
        }
        case LFOWaveform::Saw:
        {
            value = processSaw();
            break;
        }
        case LFOWaveform::Square:
        {
            value = processSquare();
            break;
        }
        default:
        {
            value = static_cast<T>(0);
            break;
        }
    }
    advancePhase();
    return value * depth_;
}

private:
/**
 * @brief Advance normalized phase.
 *
 * Range:
 * [0,1)

```

```

    */
inline void advancePhase() noexcept
{
    phase_ += phaseIncrement_;
    if (phase_ >= static_cast<T>(1))
    {
        phase_ -= static_cast<T>(1);
    }
}
/**
 * @brief Sine waveform.
 */
inline T processSine() const noexcept
{
    constexpr T kTwoPi =
        static_cast<T>(
            6.283185307179586476925286766559);
    return std::sin(
        phase_ * kTwoPi);
}
/**
 * @brief Triangle waveform.
 */
inline T processTriangle() const noexcept
{
    return
        static_cast<T>(1)
        -
        static_cast<T>(4)
        *
        std::abs(
            phase_
            -
            static_cast<T>(0.5));
}
/**
 * @brief Saw waveform.
 */
inline T processSaw() const noexcept
{
    return
        static_cast<T>(2)
        *
        phase_
        -
        static_cast<T>(1);
}
/**
 * @brief Square waveform.
 */
inline T processSquare() const noexcept
{
    return
        (phase_ < static_cast<T>(0.5))
        ? static_cast<T>(1)
        : static_cast<T>(-1);
}
private:
    T sampleRate_ =
        static_cast<T>(44100);
    T rateHz_ =
        static_cast<T>(1);
    T depth_ =
        static_cast<T>(1);
    T phase_ =
        static_cast<T>(0);
    T phaseIncrement_ =
        static_cast<T>(0);
    LFOWaveform waveform_ =

```

```
        LFOWaveform::Sine;
};
} // namespace cvdsp
#endif
```

Matemática da Modulação

Um LFO é um oscilador de baixa frequência.

Diferente de um oscilador audível:

20 Hz → 20000 Hz

o LFO opera tipicamente em:

0.01 Hz → 50 Hz

e não é ouvido diretamente.

Seu objetivo é controlar parâmetros de outros módulos DSP.

Equação Geral

Um parâmetro modulado é calculado como:

```
parameter =
baseValue
+
lfo * modulationAmount;
```

Exemplo:

```
delayTime =
10 ms
+
LFO * 5 ms;
```

Chorus

O LFO modula:

Delay Time

Exemplo:

15 ms ± 5 ms

Resultado:

Micro variações de pitch

criando múltiplas vozes aparentes.

Flanger

O LFO modula:

Delay Time

mas com atrasos muito menores:

0.1 ms
até
10 ms

Resultado:

Comb Filtering móvel

gerando o efeito de varredura característico.

Tremolo

O LFO modula:

Amplitude

Equação:

```
output =  
input  
*  
(1 + lfo) * 0.5;
```

Resultado:

Variação periódica de volume

Phaser

O LFO modula:

Center Frequency

dos filtros All-Pass.

Exemplo:

```
fc =  
500 Hz  
+  
LFO * 1500 Hz;
```

Resultado:

Movimento espectral contínuo

causado pelo deslocamento das frequências de cancelamento.

Exemplo de Uso

```
cvdsp::LFO<float> lfo;
lfo.prepare(
    48000.0f,
    0.5f,
    1.0f,
    cvdsp::LFOWaveform::Sine);
const float mod =
    lfo.process();
```

Exemplo Chorus

```
const float modulation =
    lfo.process();
const float delayMs =
    15.0f
    +
    modulation * 5.0f;
```

Exemplo Tremolo

```
const float modulation =
    lfo.process();
const float gain =
    (modulation + 1.0f)
    * 0.5f;
const float output =
    input * gain;
```

Características

template<typename T>

Compatível com:

float
double

Sem:

new
delete
malloc
free
throw

durante process().

Arquitetura compatível com futuras extensões:

Random LFO
Sample & Hold
Chaos LFO
Tempo Sync LFO
PolyBLEP LFO
Quadrature LFO
Stereo LFO
Phase Offset LFO
Multi-Wave LFO

mantendo a interface:

```
prepare()  
reset()  
setRate()  
setDepth()  
process()
```

Modulation/ADSR.hpp

```
#ifndef CVDSP_MODULATION_ADSR_HPP  
#define CVDSP_MODULATION_ADSR_HPP  
/**  
 * @file ADSR.hpp  
 * @brief ADSR Envelope Generator  
 *  
 * Header-only  
 * C++20  
 * Real-Time Safe  
 * No dynamic allocation  
 */  
#include <algorithm>  
#include <cmath>  
#include <type_traits>  
namespace cvdsp  
{  
/**  
 * @brief ADSR envelope states.  
 */  
enum class ADSRState  
{  
    Idle,  
    Attack,  
    Decay,  
    Sustain,  
    Release  
};  
/**  
 * @brief ADSR Envelope Generator.  
 *  
 * Output Range:  
 *  
 * 0.0 -> 1.0  
 *  
 * Stages:  
 *  
 * Attack  
 * Decay  
 * Sustain  
 * Release  
 *  
 * Real-Time Safe  
 * Header Only  
 */  
template<typename T>
```

```

class ADSR
{
    static_assert(
        std::is_floating_point_v<T>,
        "ADSR requires floating point type");
public:
    constexpr ADSR() noexcept = default;
    /**
     * @brief Prepare envelope.
     *
     * @param sampleRate Processing sample rate.
     * @param attackMs Attack time.
     * @param decayMs Decay time.
     * @param sustain Sustain level.
     * @param releaseMs Release time.
     */
    void prepare(
        T sampleRate,
        T attackMs,
        T decayMs,
        T sustain,
        T releaseMs) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));
        attackMs_ =
            std::max(
                attackMs,
                static_cast<T>(0.001));
        decayMs_ =
            std::max(
                decayMs,
                static_cast<T>(0.001));
        sustainLevel_ =
            std::clamp(
                sustain,
                static_cast<T>(0),
                static_cast<T>(1));
        releaseMs_ =
            std::max(
                releaseMs,
                static_cast<T>(0.001));
        updateCoefficients();
        reset();
    }
    /**
     * @brief Reset envelope state.
     */
    void reset() noexcept
    {
        value_ =
            static_cast<T>(0);
        releaseStartLevel_ =
            static_cast<T>(0);
        state_ =
            ADSRState::Idle;
    }
    /**
     * @brief Start envelope.
     */
    void noteOn() noexcept
    {
        state_ =
            ADSRState::Attack;
    }
    /**
     * @brief Release envelope.

```

```

    */
void noteOff() noexcept
{
    releaseStartLevel_ =
        value_;
    state_ =
        ADSRState::Release;
}
/**
 * @brief Set attack time.
 */
void setAttack(
    T attackMs) noexcept
{
    attackMs_ =
        std::max(
            attackMs,
            static_cast<T>(0.001));
    updateAttack();
}
/**
 * @brief Set decay time.
 */
void setDecay(
    T decayMs) noexcept
{
    decayMs_ =
        std::max(
            decayMs,
            static_cast<T>(0.001));
    updateDecay();
}
/**
 * @brief Set sustain level.
 */
void setSustain(
    T sustain) noexcept
{
    sustainLevel_ =
        std::clamp(
            sustain,
            static_cast<T>(0),
            static_cast<T>(1));
}
/**
 * @brief Set release time.
 */
void setRelease(
    T releaseMs) noexcept
{
    releaseMs_ =
        std::max(
            releaseMs,
            static_cast<T>(0.001));
    updateRelease();
}
/**
 * @brief Process one sample.
 *
 * @return Envelope value.
 */
inline T process() noexcept
{
    switch (state_)
    {
        case ADSRState::Idle:
        {
            value_ =
                static_cast<T>(0);

```

```

        break;
    }
    case ADSRState::Attack:
    {
        value_ += attackIncrement_;
        if (value_ >= static_cast<T>(1))
        {
            value_ =
                static_cast<T>(1);
            state_ =
                ADSRState::Decay;
        }
        break;
    }
    case ADSRState::Decay:
    {
        value_ -= decayIncrement_;
        if (value_ <= sustainLevel_)
        {
            value_ =
                sustainLevel_;
            state_ =
                ADSRState::Sustain;
        }
        break;
    }
    case ADSRState::Sustain:
    {
        value_ =
            sustainLevel_;
        break;
    }
    case ADSRState::Release:
    {
        value_ -= releaseIncrement_;
        if (value_ <= static_cast<T>(0))
        {
            value_ =
                static_cast<T>(0);
            state_ =
                ADSRState::Idle;
        }
        break;
    }
    default:
    {
        value_ =
            static_cast<T>(0);
        state_ =
            ADSRState::Idle;
        break;
    }
}
return value_;
}
/**
 * @brief Current state.
 */
[[nodiscard]]
ADSRState getState() const noexcept
{
    return state_;
}
/**
 * @brief Current envelope value.
 */
[[nodiscard]]
T getValue() const noexcept
{

```

```

        return value_;
    }
private:
    void updateCoefficients() noexcept
    {
        updateAttack();
        updateDecay();
        updateRelease();
    }
    void updateAttack() noexcept
    {
        const T attackSamples =
            attackMs_
            *
            static_cast<T>(0.001)
            *
            sampleRate_;
        attackIncrement_ =
            static_cast<T>(1)
            /
            std::max(
                attackSamples,
                static_cast<T>(1));
    }
    void updateDecay() noexcept
    {
        const T decaySamples =
            decayMs_
            *
            static_cast<T>(0.001)
            *
            sampleRate_;
        decayIncrement_ =
            (
                static_cast<T>(1)
                -
                sustainLevel_
            )
            /
            std::max(
                decaySamples,
                static_cast<T>(1));
    }
    void updateRelease() noexcept
    {
        const T releaseSamples =
            releaseMs_
            *
            static_cast<T>(0.001)
            *
            sampleRate_;
        releaseIncrement_ =
            static_cast<T>(1)
            /
            std::max(
                releaseSamples,
                static_cast<T>(1));
    }
private:
    T sampleRate_ =
        static_cast<T>(44100);
    T attackMs_ =
        static_cast<T>(10);
    T decayMs_ =
        static_cast<T>(100);
    T sustainLevel_ =
        static_cast<T>(0.7);
    T releaseMs_ =
        static_cast<T>(250);

```

```

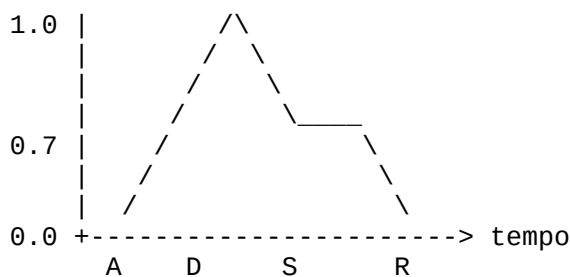
T attackIncrement_ =
    static_cast<T>(0);
T decayIncrement_ =
    static_cast<T>(0);
T releaseIncrement_ =
    static_cast<T>(0);
T value_ =
    static_cast<T>(0);
T releaseStartLevel_ =
    static_cast<T>(0);
ADSRState state_ =
    ADSRState::Idle;
};
} // namespace cvdsp
#endif

```

Envelope Generator

Um Envelope Generator produz uma curva temporal de controle.

Saída típica:



Onde:

A = Attack
 D = Decay
 S = Sustain
 R = Release

Estados

Idle

Envelope parado.

output = 0

Attack

Subida:

0 → 1

Controlada por:

attackMs

Decay

Descida:

`1 → sustainLevel`

Controlada por:

`decayMs`

Sustain

Mantém nível constante:

`sustainLevel`

Enquanto a nota permanecer pressionada.

Release

Retorno ao silêncio:

`currentLevel → 0`

Controlado por:

`releaseMs`

Uso em Synths

Controle de amplitude:

```
output =  
oscillator  
*  
adsr.process();
```

Exemplo:

```
noteOn();
```

inicia:

```
Attack  
Decay  
Sustain
```

e:

```
noteOff();
```

inicia:

Release

Uso em Samplers

Controle de volume de samples:

sample
*
envelope

Permite:

Pads
Plucks
Strings
Pianos

com diferentes respostas temporais.

Uso em Auto Effects

Controle de parâmetros DSP.

Exemplo:

```
filterCutoff =  
200.0f  
+  
adsr.process() * 5000.0f;
```

Aplicações:

Auto Wah
Filter Sweep
Auto Phaser
Auto Flanger
Auto Resonance

Exemplo de Uso

```
cvdsp::ADSR<float> env;  
env.prepare(  
    48000.0f,  
    10.0f,    // attack  
    100.0f,   // decay  
    0.7f,     // sustain  
    250.0f);  // release
```

Nota ligada:

```
env.noteOn();
```

Processamento:

```
float envelope =  
    env.process();
```

Nota desligada:

```
env.noteOff();
```

Aplicação em amplitude:

```
float output =  
    oscillator *  
    env.process();
```

Effects/Chorus.hpp

```
#ifndef CVDSP_EFFECTS_CHORUS_HPP  
#define CVDSP_EFFECTS_CHORUS_HPP  
/**  
 * @file Chorus.hpp  
 * @brief Mono Chorus Effect  
 *  
 * Dependencies:  
 * - DelayLine.hpp  
 * - LFO.hpp  
 *  
 * Header-only  
 * C++20  
 * Real-Time Safe  
 */  
#include <algorithm>  
#include <cmath>  
#include <type_traits>  
#include "../Delay/DelayLine.hpp"  
#include "../Modulation/LFO.hpp"  
namespace cvdsp  
{  
/**  
 * @brief Mono Chorus.  
 *  
 * Parameters:  
 *  
 * Rate (Hz)  
 * Depth (0..1)  
 * Mix (0..1)  
 *  
 * Delay Modulation:  
 *  
 * Base Delay:  
 * 15 ms  
 *  
 * Modulation:  
 *  $\pm 10$  ms  
 */  
template<typename T>  
class Chorus  
{  
    static_assert(  
        std::is_floating_point_v<T>,  
        "Chorus requires floating point type");  
public:  
    constexpr Chorus() noexcept = default;  
/**
```

```

* @brief Initialize chorus.
*
* @param sampleRate Processing sample rate.
* @param maxDelayMs Maximum delay buffer size.
*/
void prepare(
    T sampleRate,
    T maxDelayMs = static_cast<T>(50.0)) noexcept
{
    sampleRate_ =
        std::max(
            sampleRate,
            static_cast<T>(1));
    delay_.prepare(
        sampleRate_,
        maxDelayMs);
    lfo_.prepare(
        sampleRate_,
        rateHz_,
        static_cast<T>(1),
        LFOWaveform::Sine);
    reset();
}
/**
* @brief Reset effect state.
*/
void reset() noexcept
{
    delay_.reset();
    lfo_.reset();
}
/**
* @brief Set modulation rate.
*
* Range:
*
* 0.01 Hz
* to
* 20 Hz
*/
void setRate(
    T rateHz) noexcept
{
    rateHz_ =
        std::clamp(
            rateHz,
            static_cast<T>(0.01),
            static_cast<T>(20.0));
    lfo_.setRate(rateHz_);
}
/**
* @brief Set modulation depth.
*
* Range:
*
* 0.0 .. 1.0
*/
void setDepth(
    T depth) noexcept
{
    depth_ =
        std::clamp(
            depth,
            static_cast<T>(0),
            static_cast<T>(1));
}
/**
* @brief Set wet/dry mix.
*

```

```

    * Range:
    *
    * 0.0 .. 1.0
    */
void setMix(
    T mix) noexcept
{
    mix_ =
        std::clamp(
            mix,
            static_cast<T>(0),
            static_cast<T>(1));
}
/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    const T modulation =
        lfo_.process();
    const T delayMs =
        baseDelayMs_
        +
        (
            modulationDepthMs_
            *
            depth_
            *
            modulation
        );
    const T delayed =
        delay_.readInterpolated(
            delayMs);
    delay_.write(input);
    const T dry =
        input;
    const T wet =
        delayed;
    return
        (
            dry
            *
            (
                static_cast<T>(1)
                -
                mix_
            )
        )
        +
        (
            wet
            *
            mix_
        );
}
private:
    DelayLine<T> delay_;
    LFO<T> lfo_;
    T sampleRate_ =
        static_cast<T>(44100);
    T rateHz_ =
        static_cast<T>(0.5);
    T depth_ =
        static_cast<T>(0.5);
    T mix_ =
        static_cast<T>(0.5);
/**
 * Typical chorus values.

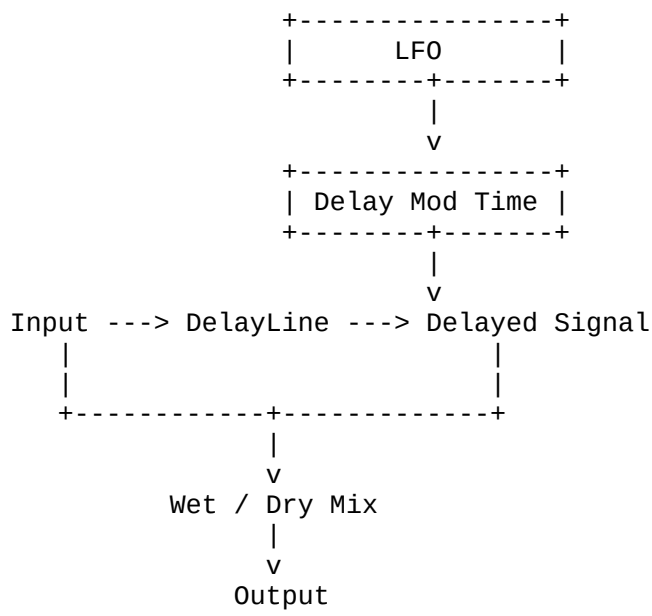
```

```

    */
    T baseDelayMs_ =
        static_cast<T>(15.0);
    T modulationDepthMs_ =
        static_cast<T>(10.0);
};
} // namespace cvdsp
#endif

```

Fluxo de Sinal



Modulated Delay

Um Chorus é baseado em um atraso variável.

O atraso não permanece fixo.

O LFO altera continuamente o tempo do delay:

```

15 ms
↓
18 ms
↓
22 ms
↓
17 ms
↓
12 ms
↓
15 ms

```

Essa variação produz pequenas alterações de fase e pitch.

O cérebro interpreta essas diferenças como múltiplas execuções simultâneas da mesma fonte sonora.

Equação do Delay Modulado

Tempo de atraso:

```
delayTime =  
baseDelay  
+  
LFO  
*  
modDepth;
```

Exemplo:

```
baseDelay = 15 ms  
depth = 10 ms
```

LFO:

```
-1.0
```

Resultado:

```
5 ms
```

LFO:

```
0.0
```

Resultado:

```
15 ms
```

LFO:

```
+1.0
```

Resultado:

```
25 ms
```

Parâmetros

Rate

Velocidade da modulação.

Faixa típica:

```
0.1 Hz  
até  
10 Hz
```

Depth

Amplitude da modulação.

Controla quanto o delay varia.

Menor depth
=
efeito suave

Maior depth
=
efeito intenso

Mix

Mistura entre sinal original e processado.

0.0 = Dry
0.5 = 50/50
1.0 = Wet

Arquitetura DSP

LFO
↓
Delay Time
↓
Fractional Delay
↓
Wet/Dry Mix
↓
Output

Dependências Esperadas

A implementação pressupõe que `DelayLine<T>` forneça:

```
prepare()  
reset()  
write()  
readInterpolated()
```

e que `LFO<T>` forneça:

```
prepare()  
reset()  
setRate()  
process()
```

para manter a arquitetura modular da `CV_DSP`.

Effects/Flanger.hpp

```
#ifndef CVDSP_EFFECTS_FLANGER_HPP  
#define CVDSP_EFFECTS_FLANGER_HPP  
/**  
 * @file Flanger.hpp  
 * @brief Feedback Flanger  
 *  
 * Dependencies:  
 * - DelayLine.hpp  
 * - LFO.hpp
```



```

*
* Header-only
* C++20
* Real-Time Safe
*/
#include <algorithm>
#include <cmath>
#include <type_traits>
#include "../Delay/DelayLine.hpp"
#include "../Modulation/LFO.hpp"
namespace cvdsp
{
/**
 * @brief Feedback Flanger.
 *
 * Signal Path:
 *
 * Input
 * |
 * +-----+
 * |               |
 * v               |
 * Delay Line       |
 * |               |
 * v               |
 * Delayed Signal -----+
 * |
 * v
 * Wet/Dry Mix
 * |
 * Output
 *
 * Parameters:
 *
 * Rate
 * Depth
 * Feedback
 * Mix
 */
template<typename T>
class Flanger
{
    static_assert(
        std::is_floating_point_v<T>,
        "Flanger requires floating point type");
public:
    constexpr Flanger() noexcept = default;
    /**
     * @brief Initialize flanger.
     *
     * @param sampleRate Processing sample rate.
     * @param maxDelayMs Delay buffer size.
     */
    void prepare(
        T sampleRate,
        T maxDelayMs = static_cast<T>(20.0)) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));
        delay_.prepare(
            sampleRate_,
            maxDelayMs);
        lfo_.prepare(
            sampleRate_,
            rateHz_,
            static_cast<T>(1),
            LFOWaveform::Sine);
    }

```

```

        reset();
    }
    /**
     * @brief Reset internal state.
     */
    void reset() noexcept
    {
        delay_.reset();
        lfo_.reset();
        feedbackSample_ =
            static_cast<T>(0);
    }
    /**
     * @brief Set modulation rate.
     *
     * Range:
     * 0.01 Hz .. 20 Hz
     */
    void setRate(
        T rateHz) noexcept
    {
        rateHz_ =
            std::clamp(
                rateHz,
                static_cast<T>(0.01),
                static_cast<T>(20.0));
        lfo_.setRate(
            rateHz_);
    }
    /**
     * @brief Set modulation depth.
     *
     * Range:
     * 0.0 .. 1.0
     */
    void setDepth(
        T depth) noexcept
    {
        depth_ =
            std::clamp(
                depth,
                static_cast<T>(0),
                static_cast<T>(1));
    }
    /**
     * @brief Set feedback amount.
     *
     * Range:
     * -0.99 .. +0.99
     */
    void setFeedback(
        T feedback) noexcept
    {
        feedback_ =
            std::clamp(
                feedback,
                static_cast<T>(-0.99),
                static_cast<T>(0.99));
    }
    /**
     * @brief Set wet/dry mix.
     *
     * Range:
     * 0.0 .. 1.0
     */
    void setMix(
        T mix) noexcept
    {
        mix_ =

```

```

        std::clamp(
            mix,
            static_cast<T>(0),
            static_cast<T>(1));
    }
    /**
     * @brief Process one sample.
     */
    inline T process(
        T input) noexcept
    {
        const T modulation =
            lfo_.process();
        const T delayMs =
            baseDelayMs_
            +
            (
                maxModulationMs_
                *
                depth_
                *
                modulation
            );
        const T delayed =
            delay_.readInterpolated(
                delayMs);
        const T delayInput =
            input
            +
            (
                delayed
                *
                feedback_
            );
        delay_.write(
            delayInput);
        feedbackSample_ =
            delayed;
        const T dry =
            input;
        const T wet =
            delayed;
        return
            (
                dry
                *
                (
                    static_cast<T>(1)
                    -
                    mix_
                )
            )
            +
            (
                wet
                *
                mix_
            );
    }
private:
    DelayLine<T> delay_;
    LFO<T> lfo_;
    T sampleRate_ =
        static_cast<T>(44100);
    T rateHz_ =
        static_cast<T>(0.25);
    T depth_ =
        static_cast<T>(0.5);
    T feedback_ =

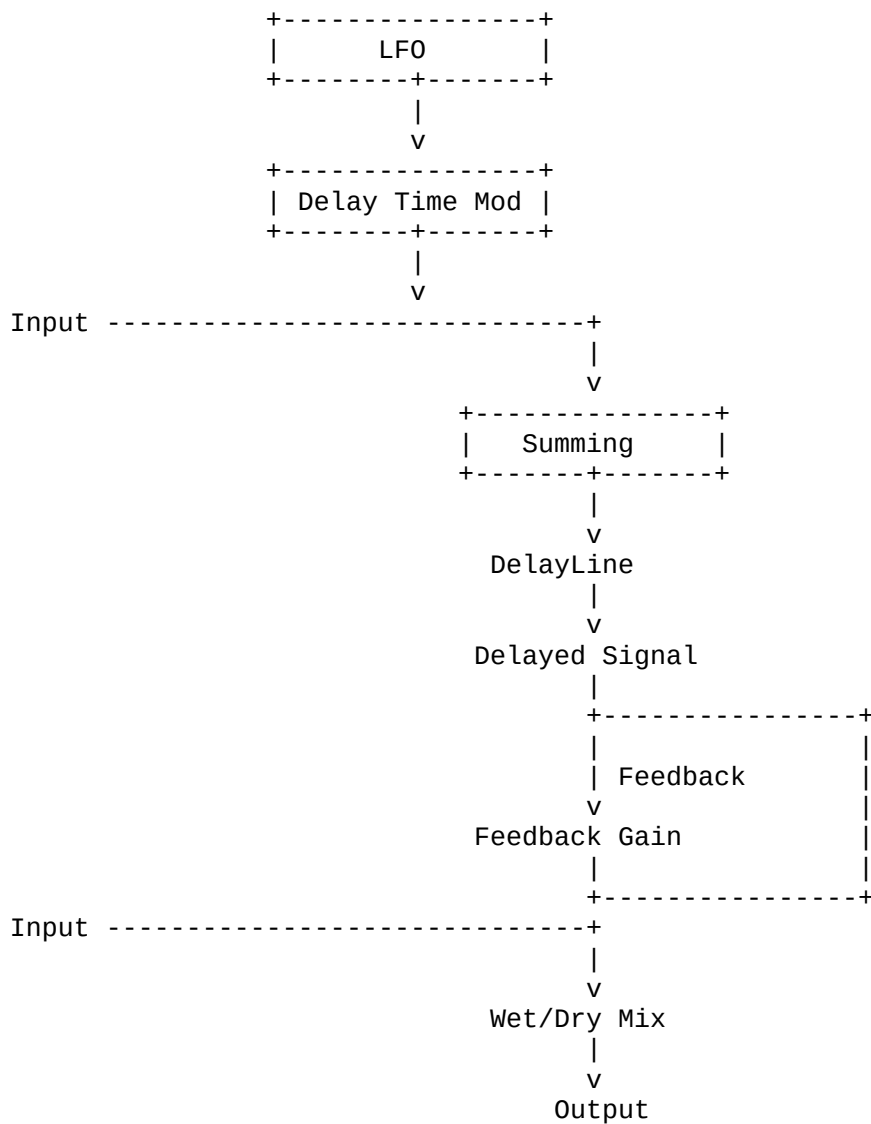
```

```

        static_cast<T>(0.5);
T mix_ =
        static_cast<T>(0.5);
/**
 * Typical flanger delay.
 *
 * Smaller than chorus.
 */
T baseDelayMs_ =
        static_cast<T>(2.0);
/**
 * Maximum modulation range.
 */
T maxModulationMs_ =
        static_cast<T>(3.0);
T feedbackSample_ =
        static_cast<T>(0);
};
} // namespace cvdsp
#endif

```

Fluxo DSP



Modulated Delay

O Flanger é um efeito baseado em atraso variável.

O tempo de delay é continuamente modulado pelo LFO:

```
delayTime =  
baseDelay  
+  
LFO  
*  
depth  
*  
maxModulation
```

Exemplo:

```
Base Delay = 2 ms  
Depth = 1.0  
LFO:  
-1 → +1
```

Resultado:

```
-1  -> 0 ms  
0   -> 2 ms  
+1  -> 5 ms
```

Na prática normalmente trabalha entre:

```
0.2 ms  
até  
10 ms
```

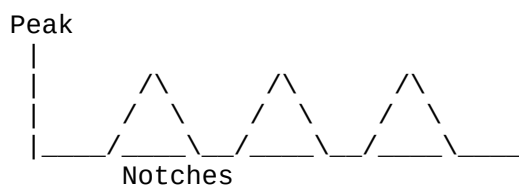
Comb Filtering

Quando um sinal é misturado com uma cópia atrasada:

```
output =  
input  
+  
delayed
```

ocorrem cancelamentos e reforços periódicos.

Resposta em frequência:



Esse padrão lembra os dentes de um pente:

Comb Filter

daí o nome.

Frequência dos Notches

Para um delay:

D

segundos:

$f_{\text{notch}} = 1 / (2D)$

Exemplo:

D = 0.001 s

$f_{\text{notch}} = 500 \text{ Hz}$

Os demais cancelamentos aparecem em múltiplos dessa frequência.

Movimento do Flanger

Como o LFO altera o delay:

delay(t)

os notches também se movem:

500 Hz

↓

700 Hz

↓

1200 Hz

↓

900 Hz

↓

400 Hz

Esse deslocamento contínuo produz o som característico de:

Jet
Sweep
Whoosh

Papel do Feedback

Sem feedback:

Comb Filtering suave

Com feedback:

Comb Filtering intenso

A realimentação reaplica o sinal atrasado à linha de delay:

delayInput =

```
input
+
delayed * feedback
```

Aumentando:

```
Ressonância
Profundidade
Coloração
```

Faixas Típicas

Rate

```
0.05 Hz
até
5 Hz
```

Depth

```
0.1
até
1.0
```

Feedback

```
0.2
até
0.9
```

Mix

```
0.2
até
0.7
```

Características

```
template<typename T>
```

Compatível com:

```
float
double
```

Sem:

```
new
delete
malloc
free
throw
```

durante process().

Dependências:

cvdsp::DelayLine<T>

cvdsp::LF0<T>

Filters/StateVariableFilter.hpp

```
#ifndef CVDSP_FILTERS_STATEVARIABLEFILTER_HPP
#define CVDSP_FILTERS_STATEVARIABLEFILTER_HPP

/**
 * @file StateVariableFilter.hpp
 * @brief Topology Preserving Transform State Variable Filter
 *
 * Features:
 * - LowPass
 * - HighPass
 * - BandPass
 * - Notch
 *
 * TPT (Topology Preserving Transform)
 * Real-Time Safe
 * Header-Only
 * C++20
 */
#include <algorithm>
#include <cmath>
#include <numbers>
#include <type_traits>

namespace cvdsp
{
/**
 * @brief Filter response type.
 */
enum class SVFMode
{
    LowPass,
    HighPass,
    BandPass,
    Notch
};
};
```



```

/**
 * @brief Topology Preserving Transform State Variable Filter.
 *
 * Based on:
 *
 * Vadim Zavalishin
 * The Art of VA Filter Design
 *
 * Advantages:
 *
 * - Stable at high resonance
 * - Stable near Nyquist
 * - Real-time modulation friendly
 * - Continuous parameter updates
 * - Low numerical sensitivity
 */
template<typename T>
class StateVariableFilter
{
    static_assert(
        std::is_floating_point_v<T>,
        "StateVariableFilter requires floating point type");
public:
    constexpr StateVariableFilter() noexcept = default;
    /**
     * @brief Initialize filter.
     */
    void prepare(
        T sampleRate,
        SVFMode mode = SVFMode::LowPass) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));
        mode_ = mode;
        updateCoefficients();
        reset();
    }
}

```

```

/**
 * @brief Reset internal states.
 */
void reset() noexcept
{
    ic1eq_ =
        static_cast<T>(0);
    ic2eq_ =
        static_cast<T>(0);
}

/**
 * @brief Set filter mode.
 */
void setMode(
    SVFMode mode) noexcept
{
    mode_ = mode;
}

/**
 * @brief Set cutoff frequency.
 *
 * Safe for sample-by-sample modulation.
 */
void setCutoff(
    T cutoffHz) noexcept
{
    constexpr T kMinCutoff =
        static_cast<T>(5);
    const T maxCutoff =
        sampleRate_
        *
        static_cast<T>(0.495);
    cutoffHz_ =
        std::clamp(
            cutoffHz,
            kMinCutoff,
            maxCutoff);
    updateCoefficients();
}

```

```

/**
 * @brief Set resonance.
 *
 * Q range:
 *
 * 0.1 .. 40.0
 */
void setResonance(
    T q) noexcept
{
    resonance_ =
        std::clamp(
            q,
            static_cast<T>(0.1),
            static_cast<T>(40.0));
    updateCoefficients();
}
/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    const T v3 =
        input
        -
        ic2eq_;
    const T v1 =
        a1_
        *
        ic1eq_
        +
        a2_
        *
        v3;
    const T v2 =
        ic2eq_
        +
        a2_

```

```

        *
        ic1eq_
        +
        a3_
        *
        v3;
ic1eq_ =
    static_cast<T>(2)
    *
    v1
    -
    ic1eq_;
ic2eq_ =
    static_cast<T>(2)
    *
    v2
    -
    ic2eq_;
const T lowpass =
    v2;
const T bandpass =
    v1;
const T highpass =
    input
    -
    k_
    *
    v1
    -
    v2;
const T notch =
    highpass
    +
    lowpass;
switch (mode_)
{
    case SVFMode::LowPass:
        return lowpass;
    case SVFMode::HighPass:

```

```

        return highpass;
    case SVFMode::BandPass:
        return bandpass;
    case SVFMode::Notch:
        return notch;
    default:
        return lowpass;
    }
}

private:
/**
 * @brief Recalculate TPT coefficients.
 */
void updateCoefficients() noexcept
{
    const T wd =
        static_cast<T>(2)
        *
        std::numbers::pi_v<T>
        *
        cutoffHz_;
    const T Tinv =
        static_cast<T>(1)
        /
        sampleRate_;
    const T wa =
        static_cast<T>(2)
        *
        sampleRate_
        *
        std::tan(
            wd
            *
            Tinv
            *
            static_cast<T>(0.5));
    g_ =
        wa
        /

```

```

        (
            static_cast<T>(2)
            *
            sampleRate_
        );
k_ =
    static_cast<T>(1)
    /
    resonance_;
a1_ =
    static_cast<T>(1)
    /
    (
        static_cast<T>(1)
        +
        g_
        *
        (
            g_
            +
            k_
        )
    );
a2_ =
    g_
    *
    a1_;
a3_ =
    g_
    *
    a2_;
}

```

private:

```

T sampleRate_ =
    static_cast<T>(44100);
T cutoffHz_ =
    static_cast<T>(1000);
T resonance_ =
    static_cast<T>(0.707);

```

```

T g_ =
    static_cast<T>(0);
T k_ =
    static_cast<T>(0);
T a1_ =
    static_cast<T>(0);
T a2_ =
    static_cast<T>(0);
T a3_ =
    static_cast<T>(0);
T ic1eq_ =
    static_cast<T>(0);
T ic2eq_ =
    static_cast<T>(0);
SVFMode mode_ =
    SVFMode::LowPass;
};
} // namespace cvdsp
#endif

```

Matemática TPT

A Topology Preserving Transform preserva a estrutura analógica do filtro após a discretização.

A integração contínua:

$1 / s$

é substituída por:

Trapezoidal Integrator

resultando em:

s

↓

TPT

↓

z

com melhor preservação da resposta original.

SVF Clássico

Estrutura tradicional:

Input

|
HighPass
|
BandPass
|
LowPass

Características:

Poucas operações
Multimodo
Fácil implementação

Problemas:

Instabilidade em altas ressonâncias
Mudança de resposta próxima de Nyquist
Sensível à modulação rápida

TPT SVF

Estrutura:

Trapezoidal Integrators
+
Feedback implícito

Características:

Estável
Resposta previsível
Excelente para modulação
Baixa sensibilidade numérica

Adequado para:

Synths
Virtual Analog
Auto-Wah
Phaser
Filter FM
Envelope Modulation
Audio-rate Modulation

Diferenças para Biquad

Biquad

Forma típica:

Direct Form I

Direct Form II

Transposed

Equação:

$y[n]$

=

$b_0x[n]$

$+b_1x[n-1]$

$+b_2x[n-2]$

$-a_1y[n-1]$

$-a_2y[n-2]$

Vantagens:

Muito eficiente

Resposta precisa

Ampla utilização

Desvantagens:

Mais sensível a modulação

Maior sensibilidade numérica

Coefficientes mudam significativamente

TPT SVF

Vantagens:

Modulação contínua

Melhor comportamento dinâmico

Menos zipper noise

Maior estabilidade

Desvantagens:

Levemente mais custoso

Mais estados internos

Fluxo Interno

Input

```
|
v
HighPass
|
v
BandPass
|
v
LowPass
Notch = HighPass + LowPass
```

Exemplo LowPass

```
cvdsp::StateVariableFilter<float> filter;
filter.prepare(
    48000.0f,
    cvdsp::SVFMode::LowPass);
filter.setCutoff(
    1000.0f);
filter.setResonance(
    0.707f);
float y =
    filter.process(x);
```

Exemplo BandPass

```
cvdsp::StateVariableFilter<float> filter;
filter.prepare(
    48000.0f,
    cvdsp::SVFMode::BandPass);
filter.setCutoff(
    2000.0f);
filter.setResonance(
    8.0f);
```

Exemplo Modulação por LFO

```
const float lfoValue =
    lfo.process();
const float cutoff =
```

```
        500.0f
    +
    (
        lfoValue
        *
        2500.0f
    );
filter.setCutoff(
    cutoff);
const float y =
    filter.process(x);
```

Características da Implementação

Header-Only

C++20

Template<float>

Template<double>

Sem new

Sem delete

Sem malloc

Sem free

Sem RTTI

Sem exceptions

Real-Time Safe

Baixa Sensibilidade Numérica

Modulação em Tempo Real

Compatível com Audio-Rate Modulation

LowPass

HighPass

BandPass

Notch

com uma única implementação baseada em TPT.

Filters/LadderFilter.hpp

```
#ifndef CVDSP_FILTERS_LADDERFILTER_HPP
#define CVDSP_FILTERS_LADDERFILTER_HPP

/**
```

```

* @file LadderFilter.hpp
* @brief Moog Inspired Ladder Filter
*
* Header-Only
* C++20
* Real-Time Safe
*
* Features:
* - 4 Pole LowPass
* - Resonance Feedback
* - Drive Control
* - Soft Saturation
* - Real-Time Modulation
*/

#include <algorithm>
#include <cmath>
#include <numbers>
#include <type_traits>
namespace cvdsp
{
/**
 * @brief Moog-inspired Ladder Filter.
 *
 * 24 dB/oct LowPass
 *
 * Four cascaded one-pole stages.
 *
 * Resonance feedback path.
 *
 * Optional nonlinear drive.
 */
template<typename T>
class LadderFilter
{
    static_assert(
        std::is_floating_point_v<T>,
        "LadderFilter requires floating point type");
public:
    constexpr LadderFilter() noexcept = default;

```

```

/**
 * @brief Initialize filter.
 */
void prepare(
    T sampleRate) noexcept
{
    sampleRate_ =
        std::max(
            sampleRate,
            static_cast<T>(1));
    updateCoefficients();
    reset();
}

/**
 * @brief Reset internal states.
 */
void reset() noexcept
{
    z1_ =
        static_cast<T>(0);
    z2_ =
        static_cast<T>(0);
    z3_ =
        static_cast<T>(0);
    z4_ =
        static_cast<T>(0);
}

/**
 * @brief Set cutoff frequency.
 *
 * Safe for real-time modulation.
 */
void setCutoff(
    T cutoffHz) noexcept
{
    const T maxCutoff =
        sampleRate_
        *
        static_cast<T>(0.495);

```

```

        cutoffHz_ =
            std::clamp(
                cutoffHz,
                static_cast<T>(5),
                maxCutoff);
        updateCoefficients();
    }
/**
 * @brief Set resonance amount.
 *
 * Range:
 *
 * 0.0 .. 4.0
 */
void setResonance(
    T resonance) noexcept
{
    resonance_ =
        std::clamp(
            resonance,
            static_cast<T>(0),
            static_cast<T>(4));
    updateFeedback();
}
/**
 * @brief Set input drive.
 *
 * Range:
 *
 * 1.0 .. 20.0
 */
void setDrive(
    T drive) noexcept
{
    drive_ =
        std::clamp(
            drive,
            static_cast<T>(1),
            static_cast<T>(20));
}

```

```

}
/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    T x =
        input
        -
        (
            feedbackGain_
            *
            z4_
        );
    x *= drive_;
    x = saturate(x);
    z1_ += g_ * (x - z1_);
    z1_ = saturate(z1_);
    z2_ += g_ * (z1_ - z2_);
    z2_ = saturate(z2_);
    z3_ += g_ * (z2_ - z3_);
    z3_ = saturate(z3_);
    z4_ += g_ * (z3_ - z4_);
    z4_ = saturate(z4_);
    return z4_;
}

private:
/**
 * @brief Soft saturation.
 *
 * tanh approximation.
 */
static inline T saturate(
    T x) noexcept
{
    return std::tanh(x);
}
/**

```

```

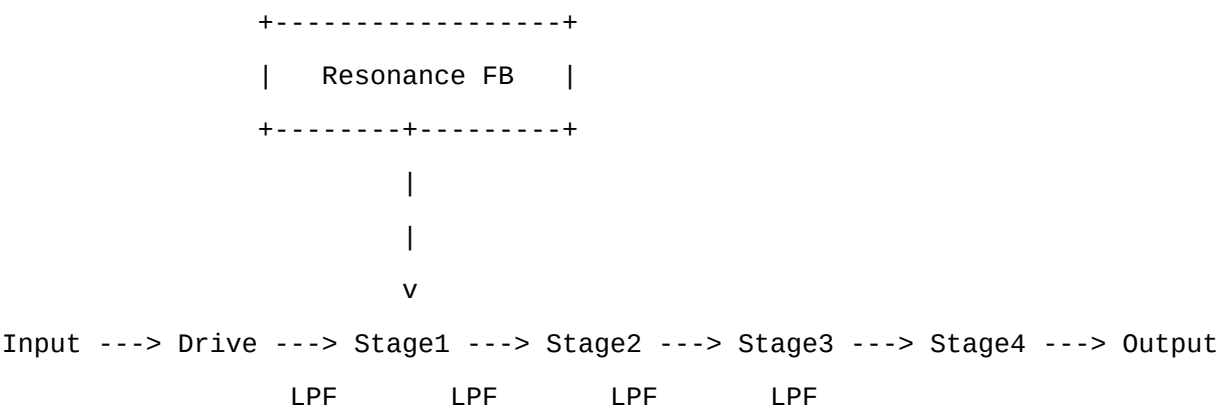
    * @brief Update cutoff coefficient.
    */
void updateCoefficients() noexcept
{
    const T normalized =
        cutoffHz_
        /
        sampleRate_;
    const T omega =
        static_cast<T>(2)
        *
        std::numbers::pi_v<T>
        *
        normalized;
    g_ =
        static_cast<T>(1)
        -
        std::exp(
            -omega);
    g_ =
        std::clamp(
            g_,
            static_cast<T>(0),
            static_cast<T>(1));
    updateFeedback();
}
/**
    * @brief Resonance feedback scaling.
    */
void updateFeedback() noexcept
{
    feedbackGain_ =
        resonance_;
}
private:
    T sampleRate_ =
        static_cast<T>(44100);
    T cutoffHz_ =
        static_cast<T>(1000);

```



```
T resonance_ =
    static_cast<T>(0);
T drive_ =
    static_cast<T>(1);
T g_ =
    static_cast<T>(0);
T feedbackGain_ =
    static_cast<T>(0);
/**
 * Four ladder stages.
 */
T z1_ =
    static_cast<T>(0);
T z2_ =
    static_cast<T>(0);
T z3_ =
    static_cast<T>(0);
T z4_ =
    static_cast<T>(0);
};
} // namespace cvdsp
#endif
```

Arquitetura



Estrutura Ladder

O filtro Ladder original utiliza:
4 estgios passa-baixa
em cascata.
Cada estgio contribui aproximadamente:

6 dB/oct

Total:

24 dB/oct

Modelo Matemático

Cada estágio é um integrador de primeira ordem:

$$y += g * (x - y)$$

onde:

g

é derivado da frequência de corte.

Realimentação (Resonance)

A saída do último estágio retorna para a entrada:

$$\text{Input} - \text{Resonance} * \text{Output}$$

Implementação:

$$x =$$

input

-

$$\text{feedbackGain} * z4;$$

Essa realimentação cria um pico próximo da frequência de corte.

Drive

Antes do filtro:

$$x *= \text{drive};$$

Aumentando:

Harmônicos

Compressão

Coloração analógica

Saturação

Cada estágio utiliza:

$$\tanh()$$

para limitar amplitude.

$y = \tanh(x)$

Características:

Suave

Contínua

Sem clipping abrupto

Auto Oscillation

Quando a ressonância aumenta:

Resonance → Máximo

o ganho do loop aproxima-se de:

1.0

A energia recircula dentro do filtro.

Resultado:

Senoide espontânea

mesmo sem sinal de entrada.

Isso é conhecido como:

Self Oscillation

Auto Oscillation

e é uma das características clássicas dos sintetizadores baseados em filtro Moog.

Diferenças para SVF

SVF

Multimodo

LowPass

HighPass

BandPass

Notch

Excelente para modulação extrema.

Ladder

LowPass

24 dB/oct

Coloração

Saturação

Resonância musical

Preferido em:

Synth Bass

Lead

Acid

Pads

Subtractive Synthesis

Exemplo Básico

```
cvdsp::LadderFilter<float> filter;  
filter.prepare(48000.0f);  
filter.setCutoff(1000.0f);  
filter.setResonance(2.0f);  
filter.setDrive(3.0f);  
float y =  
    filter.process(x);
```

Exemplo com Envelope

```
const float env =  
    adsr.process();  
filter.setCutoff(  
    200.0f  
    +  
    env * 4000.0f);  
float y =  
    filter.process(x);
```

Exemplo com LFO

```
const float lfoValue =  
    lfo.process();  
filter.setCutoff(  
    500.0f  
    +  
    lfoValue * 1500.0f);  
float y =  
    filter.process(x);
```

Características

Template<float>

Template<double>

Header-Only

C++20

Real-Time Safe

Sem new

Sem delete

Sem malloc

Sem free

Sem exceptions

4 Pole LowPass

24 dB/oct

Resonance Feedback

Drive

Soft Saturation

Compatível com:

Synths

VA Instruments

Auto-Wah

Bass Filters

Acid Lines

Analog Modeling

Filters/AllPassFilter.hpp

```
#ifndef CVDSP_FILTERS_ALLPASSFILTER_HPP
#define CVDSP_FILTERS_ALLPASSFILTER_HPP

/**
 * @file AllPassFilter.hpp
 * @brief All-Pass Filter
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Phase Rotation
```

```

* - Diffusion
* - Schroeder Reverb Building Block
* - Phaser Building Block
*/

#include <algorithm>
#include <cstdint>
#include <type_traits>
#include "../Delay/DelayLine.hpp"
namespace cvdsp
{
/**
 * @brief Feedback All-Pass Filter.
 *
 * Transfer Function:
 *
 *

$$H(z) = \frac{-g + z^{-N}}{1 - g * z^{-N}}$$

 *
 * where:
 *
 * g = feedback coefficient
 * N = delay length
 *
 * Magnitude Response:
 *
 *

$$|H(z)| = 1$$

 *
 * Only phase is modified.
 */
template<typename T>
class AllPassFilter
{
    static_assert(
        std::is_floating_point_v<T>,
        "AllPassFilter requires floating point type");
public:
    constexpr AllPassFilter() noexcept = default;
/**

```

```

    * @brief Initialize filter.
    *
    * @param sampleRate Processing sample rate.
    * @param maxDelaySamples Maximum delay size.
    */
void prepare(
    T sampleRate,
    std::size_t maxDelaySamples) noexcept
{
    sampleRate_ =
        std::max(
            sampleRate,
            static_cast<T>(1));
    maxDelaySamples_ =
        std::max<std::size_t>(
            maxDelaySamples,
            1u);
    delay_.prepareSamples(
        maxDelaySamples_);
    reset();
}

/**
    * @brief Reset internal state.
    */
void reset() noexcept
{
    delay_.reset();
    delaySamples_ = 1;
}

/**
    * @brief Set delay length.
    *
    * Range:
    *
    * 1 .. maxDelaySamples
    */
void setDelaySamples(
    std::size_t delaySamples) noexcept
{

```

```

        delaySamples_ =
            std::clamp(
                delaySamples,
                static_cast<std::size_t>(1),
                maxDelaySamples_);
    }
/**
 * @brief Set feedback coefficient.
 *
 * Range:
 *
 * -0.999 .. +0.999
 */
void setFeedback(
    T feedback) noexcept
{
    feedback_ =
        std::clamp(
            feedback,
            static_cast<T>(-0.999),
            static_cast<T>(0.999));
}
/**
 * @brief Process one sample.
 *
 * Canonical Schroeder all-pass:
 *
 *  $y[n] = -g \cdot x[n] + d[n]$ 
 *
 * write =
 *  $x[n] + g \cdot y[n]$ 
 */
inline T process(
    T input) noexcept

```



```

{
    const T delayed =
        delay_.readSamples(
            delaySamples_);
    const T output =
        (-feedback_ * input)
        +
        delayed;
    const T writeSample =
        input
        +
        (
            feedback_
            *
            output
        );
    delay_.write(
        writeSample);
    return output;
}

private:
    DelayLine<T> delay_;
    T sampleRate_ =
        static_cast<T>(44100);
    T feedback_ =
        static_cast<T>(0.5);
    std::size_t delaySamples_ =
        1;
    std::size_t maxDelaySamples_ =
        1;
};

} // namespace cvdsp
#endif

```

Estrutura Matemática

Função de transferência:

$$H(z) = \frac{-g + z^{-N}}{1 - g \cdot z^{-N}}$$

onde:

g = feedback

N = delay samples

Propriedade fundamental:

$$|H(z)| = 1$$

O ganho permanece constante para todas as frequências.

O que muda é apenas:

Fase

Phase Rotation

Um filtro All-Pass não altera a amplitude do sinal.

Exemplo:

Entrada:

$$1 \text{ kHz} = 0 \text{ dB}$$

Saída:

$$1 \text{ kHz} = 0 \text{ dB}$$

Porém a fase é deslocada:

Entrada:

$$0^\circ$$

Saída:

$$-90^\circ$$

ou qualquer outro valor dependendo da frequência.

Portanto:

Amplitude = preservada

Fase = alterada

Difusão (Diffusion)

Quando vários All-Pass são colocados em cascata:

Input

|

AP1

|

AP2

|

AP3

|
AP4
|
Output

os transientes são espalhados no tempo.

Resultado:

Maior densidade temporal

Menos ecos perceptíveis

Campo reverberante mais suave

Esse processo é chamado:

Diffusion

Schroeder Reverb

O reverberador clássico de Schroeder utiliza:

Comb Filters

+

All-Pass Filters

Arquitetura típica:

Input

|
Comb
Comb
Comb
Comb
Comb

|
Somador

|
AllPass

|
AllPass

|
Output

Funções:

Comb:

Cria decaimento

AllPass:

Cria difusão

Os All-Pass são responsáveis pela suavização da cauda de reverb.

Phaser

Um Phaser é baseado em:

Múltiplos All-Pass

com coeficientes modulados.

Estrutura:

Input

|

AP1

|

AP2

|

AP3

|

AP4

|

Mix Dry/Wet

|

Output

A mistura do sinal processado com o original produz:

Cancelamentos móveis

Notches móveis

gerando o efeito característico de Phaser.

Exemplo Básico

```
cvdsp::AllPassFilter<float> allpass;
```

```
allpass.prepare(
```

```
    48000.0f,
```

```
    2048);
```

```
allpass.setDelaySamples(
```

```
    127);
```

```
allpass.setFeedback(
```

```
    0.7f);
```

```
float y =
```

```
    allpass.process(x);
```

Exemplo de Difusão

```
float y = x;
y = ap1.process(y);
y = ap2.process(y);
y = ap3.process(y);
y = ap4.process(y);
```

Exemplo de Schroeder Reverb

```
float diffusion =
    ap1.process(
        ap2.process(
            ap3.process(
                ap4.process(x))));
```

Exemplo de Phaser

```
ap1.setDelaySamples(
    modulatedDelay1);
ap2.setDelaySamples(
    modulatedDelay2);
float y = x;
y = ap1.process(y);
y = ap2.process(y);
output =
(
    x * 0.5f
)
+
(
    y * 0.5f
);
```

Características

Template<float>

Template<double>

Header-Only

C++20

Real-Time Safe
Sem new
Sem delete
Sem malloc
Sem free
Sem RTTI
Sem exceptions
Compatível com:
Phaser
Diffuser
Schroeder Reverb
FDN Reverb
Artificial Reverberation
Baixa complexidade
Baixa latência
Alta reutilização

Saturation/Waveshaper.hpp

```
#ifndef CVDSP_SATURATION_WAVESHAPER_HPP
#define CVDSP_SATURATION_WAVESHAPER_HPP

/**
 * @file Waveshaper.hpp
 * @brief Collection of Waveshaping Algorithms
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 */

#include <algorithm>
#include <cmath>
#include <numbers>
#include <type_traits>

namespace cvdsp
{
/**
 * @brief Waveshaper modes.
 */
enum class WaveshaperMode
```

```

{
    HardClip,
    SoftClip,
    Tanh,
    Arctan,
    Cubic,
    Foldback
};

/**
 * @brief Collection of static waveshaping algorithms.
 *
 * Stateless.
 *
 * Thread-safe.
 *
 * Real-Time Safe.
 */
template<typename T>
class Waveshaper
{
    static_assert(
        std::is_floating_point_v<T>,
        "Waveshaper requires floating point type");
public:
    /**
     * @brief Process sample.
     *
     * @param input Input sample.
     * @param mode Waveshaper mode.
     *
     * @return Processed sample.
     */
    static inline T process(
        T input,
        WaveshaperMode mode) noexcept
    {
        switch (mode)
        {
            case WaveshaperMode::HardClip:

```

```

        return hardClip(input);
    case WaveshaperMode::SoftClip:
        return softClip(input);
    case WaveshaperMode::Tanh:
        return tanhShape(input);
    case WaveshaperMode::Arctan:
        return arctanShape(input);
    case WaveshaperMode::Cubic:
        return cubicShape(input);
    case WaveshaperMode::Foldback:
        return foldbackShape(input);
    default:
        return input;
    }
}

private:
/**
 * @brief Hard Clipper.
 *
 * Abrupt clipping.
 */
static inline T hardClip(
    T x) noexcept
{
    return std::clamp(
        x,
        static_cast<T>(-1),
        static_cast<T>(1));
}
/**
 * @brief Soft Clipper.
 *
 * Smooth transition.
 */
static inline T softClip(
    T x) noexcept
{
    constexpr T threshold =
        static_cast<T>(1);

```



```

    if (x > threshold)
    {
        return threshold;
    }
    if (x < -threshold)
    {
        return -threshold;
    }
    return
        x
        -
        (
            x
            *
            x
            *
            x
            /
            static_cast<T>(3)
        );
}
/**
 * @brief Hyperbolic tangent.
 */
static inline T tanhShape(
    T x) noexcept
{
    return std::tanh(x);
}
/**
 * @brief Arctangent saturator.
 */
static inline T arctanShape(
    T x) noexcept
{
    constexpr T kScale =
        static_cast<T>(2)
        /
        std::numbers::pi_v<T>;

```

```

        return
            std::atan(x)
            *
            kScale;
    }
/**
 * @brief Cubic saturation.
 *
 * Third-order polynomial.
 */
static inline T cubicShape(
    T x) noexcept
{
    constexpr T limit =
        static_cast<T>(1);
    x =
        std::clamp(
            x,
            -limit,
            limit);
    return
        x
        -
        (
            static_cast<T>(1.0 / 3.0)
            *
            x
            *
            x
            *
            x
        );
}
/**
 * @brief Foldback distortion.
 */
static inline T foldbackShape(
    T x) noexcept
{

```

```

constexpr T threshold =
    static_cast<T>(1);
if (x > threshold || x < -threshold)
{
    x =
        std::fabs(
            std::fmod(
                x - threshold,
                threshold * static_cast<T>(4)));

    x =
        std::fabs(
            x
            -
            threshold
            *
            static_cast<T>(2))
        -
        threshold;
}
return x;
}

};

} // namespace cvdsp
#endif

```

Harmonic Generation

Waveshaping altera a forma de onda original.

Entrada:

Sine Wave

Saída:

Distorted Wave

Essa alteração cria componentes espectrais adicionais.

Esses componentes são:

Harmônicos

Harmônicos Ímpares

Produzidos principalmente por funções simétricas:

$$f(x) = -f(-x)$$

Exemplos:

HardClip

Tanh

Arctan

Cubic

Geram predominantemente:

3^a

5^a

7^a

9^a

...

harmônicas.

Harmônicos Pares

Produzidos quando existe assimetria.

Exemplo:

Tube Saturation

Bias

DC Offset

Half-Wave Distortion

Geram:

2^a

4^a

6^a

8^a

...

harmônicas.

Muitos circuitos valvulados produzem mistura de:

Pares

+

Ímpares

Hard Clipping

Forma:



Equação:

$$y = \text{clamp}(x, -1, +1)$$

Características:

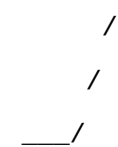
Alta distorção

Muito brilho

Muito aliasing

Soft Clipping

Forma:



Características:

Transição suave

Menos aliasing

Som mais musical

Tanh

Equação:

$$y = \tanh(x)$$

Características:

Resposta semelhante a válvulas

Compressão progressiva

Muito utilizada em modelagem analógica

Arctan

Equação:

$$y = (2/\pi) * \text{atan}(x)$$

Características:

Suave

Estável

Baixo custo computacional

Cubic

Equação:

$$y = x - x^3/3$$

Características:

Leve saturação

Predominância de 3ª harmônica

Baixa complexidade

Foldback

Princípio:

Quando o sinal ultrapassa o limite:

Reflete

Volta

Reflete novamente

Forma:

/\ /\

\/ \/

Características:

Som agressivo

Espectralmente complexo

Sintetizadores experimentais

Aliasing Gerado por Clipping

Qualquer saturação gera novos harmônicos.

Exemplo:

Fundamental:

5 kHz

Hard Clip gera:

10 kHz

15 kHz

20 kHz

25 kHz

30 kHz

...

Em:

$F_s = 48 \text{ kHz}$

$F_{\text{Nyquist}} = 24 \text{ kHz}$

Os harmônicos acima de:

24 kHz

não podem ser representados corretamente.

Eles refletem para dentro da banda audível:

Alias

Exemplo

```
float y =
    cvdsp::Waveshaper<float>::process(
        x,
        cvdsp::WaveshaperMode::Tanh);
```

Exemplo com Drive

```
const float drive = 8.0f;
float y =
    cvdsp::Waveshaper<float>::process(
        x * drive,
        cvdsp::WaveshaperMode::HardClip);
```

Aplicações

Guitar Amp

Tube Saturation

Tape Saturation

Distortion

Fuzz

Bass Overdrive

Synth Saturation

Analog Modeling

Wavefolding

Características

Header-Only

C++20

Template<float>
Template<double>
Sem estado interno
Sem new
Sem delete
Sem malloc
Sem free
Sem RTTI
Sem exceptions
Real-Time Safe
HardClip
SoftClip
Tanh
Arctan
Cubic
Foldback

Saturation/TubeSaturation.hpp

```
#ifndef CVDSP_SATURATION_TUBESATURATION_HPP
#define CVDSP_SATURATION_TUBESATURATION_HPP

/**
 * @file TubeSaturation.hpp
 * @brief Tube Style Saturation
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Drive
 * - Bias
 * - Output Gain
 * - Even Harmonic Generation
 * - Asymmetrical Saturation
 */

#include <algorithm>
#include <cmath>
#include <type_traits>
```



```

namespace cvdsp
{
/**
 * @brief Tube style saturation processor.
 *
 * Inspired by triode preamp stages.
 *
 * Produces:
 *
 * - Soft compression
 * - Even harmonics
 * - Asymmetrical distortion
 *
 * Real-Time Safe
 */
template<typename T>
class TubeSaturation
{
    static_assert(
        std::is_floating_point_v<T>,
        "TubeSaturation requires floating point type");
public:
    constexpr TubeSaturation() noexcept = default;
    /**
     * @brief Initialize processor.
     *
     * Present for API consistency.
     */
    void prepare(
        T sampleRate) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));
        reset();
    }
    /**
     * @brief Reset internal state.

```

```

    */
void reset() noexcept
{
    previousOutput_ =
        static_cast<T>(0);
}
/**
 * @brief Set drive amount.
 *
 * Range:
 *
 * 1.0 .. 50.0
 */
void setDrive(
    T drive) noexcept
{
    drive_ =
        std::clamp(
            drive,
            static_cast<T>(1),
            static_cast<T>(50));
}
/**
 * @brief Set DC bias.
 *
 * Range:
 *
 * -1.0 .. +1.0
 */
void setBias(
    T bias) noexcept
{
    bias_ =
        std::clamp(
            bias,
            static_cast<T>(-1),
            static_cast<T>(1));
}
/**

```

```

* @brief Set output gain.
*
* Linear gain.
*
* Range:
*
* 0.0 .. 10.0
*/
void setOutputGain(
    T gain) noexcept
{
    outputGain_ =
        std::clamp(
            gain,
            static_cast<T>(0),
            static_cast<T>(10));
}
/**
* @brief Process one sample.
*/
inline T process(
    T input) noexcept
{
    T x =
        input
        *
        drive_;
    /**
    * Bias shift.
    */
    x += bias_;
    /**
    * Asymmetrical triode response.
    *
    * Positive half:
    * softer
    *
    * Negative half:
    * stronger compression

```

```

        */
    T y;
    if (x >= static_cast<T>(0))
    {
        y =
            positiveShape(x);
    }
    else
    {
        y =
            negativeShape(x);
    }
    /**
     * Remove part of the bias.
     */
    y -=
        bias_
        *
        static_cast<T>(0.5);
    y *= outputGain_;
    previousOutput_ = y;
    return y;
}

private:
    /**
     * @brief Positive triode curve.
     */
    static inline T positiveShape(
        T x) noexcept
    {
        return
            static_cast<T>(1)
            -
            std::exp(
                -x);
    }
    /**
     * @brief Negative triode curve.
     */

```

```

static inline T negativeShape(
    T x) noexcept
{
    return
        -(
            static_cast<T>(1)
            -
            std::exp(
                static_cast<T>(1.5)
                *
                x));
}

private:
    T sampleRate_ =
        static_cast<T>(44100);
    T drive_ =
        static_cast<T>(1);
    T bias_ =
        static_cast<T>(0);
    T outputGain_ =
        static_cast<T>(1);
    T previousOutput_ =
        static_cast<T>(0);
};

} // namespace cvdsp
#endif

```

Exemplo de Uso

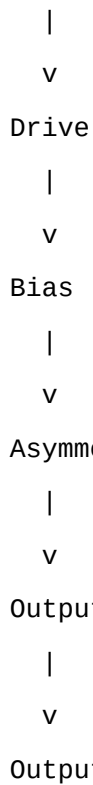
```

cvdsp::TubeSaturation<float> tube;
tube.prepare(48000.0f);
tube.setDrive(8.0f);
tube.setBias(0.15f);
tube.setOutputGain(0.5f);
float y =
    tube.process(x);

```

Fluxo DSP

Input



Assimetria

Diferente de um waveshaper simétrico:

$$f(x) = -f(-x)$$

uma válvula real normalmente responde de forma diferente para semiciclos positivos e negativos.

Exemplo:

Positivo:

compressão suave

Negativo:

compressão mais forte

Isso gera:

Assimetria

e consequentemente:

Harmônicos pares

Bias Deslocável

O bias adiciona um deslocamento DC temporário antes da saturação:

$$x = \text{input} + \text{bias};$$

Exemplo:

$$\text{Bias} = 0.0$$

Forma simétrica

Exemplo:

Bias = +0.3

Forma deslocada

A consequência é:

Maior assimetria

e:

Mais 2ª harmônica

Mais 4ª harmônica

Mais 6ª harmônica

Triode

Uma tríodo clássica possui:

Cathode

Grid

Plate

O ganho não é linear.

Curva típica:



Ao aumentar o sinal:

Compressão gradual

em vez de clipping abrupto.

Essa característica é responsável pelo comportamento musical dos pré-amplificadores valvulados.

Tube Preamp

Em um pré-amplificador valvulado:

Entrada

|

Triode Stage

|

Triode Stage

|
Tone Stack
|
Power Amp

Cada estágio adiciona:
Compressão
Coloração
Harmônicos

A implementação acima modela uma versão simplificada do primeiro estágio.

Harmônicos Pares

Em sistemas perfeitamente simétricos predominam:

3ª
5ª
7ª
9ª

harmônicas.

Com assimetria surgem:

2ª
4ª
6ª
8ª

harmônicas.

Essas componentes são frequentemente associadas à sensação de:

Warmth
Thickness
Smoothness

em equipamentos valvulados.

Características

Template<float>
Template<double>
Header -Only
C++20
Real-Time Safe
Sem new

Sem delete
Sem malloc
Sem free
Sem RTTI
Sem exceptions
Drive
Bias
Output Gain
Assimetria Ajustável
Even Harmonics
Tube Style Saturation
Compatível com:
Guitar Preamp
Bass Preamp
Channel Strip
Tape/Tube Coloration
Analog Modeling

Saturation/TapeSaturation.hpp

```
#ifndef CVDSP_SATURATION_TAPESATURATION_HPP
#define CVDSP_SATURATION_TAPESATURATION_HPP
/**
 * @file TapeSaturation.hpp
 * @brief Tape Saturation Processor
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Drive
 * - Bias
 * - Compression
 * - Soft Limiting
 * - Magnetic Saturation
 *
 * Inspired by analog tape machines.
 */
```

```

#include <algorithm>
#include <cmath>
#include <type_traits>
namespace cvdsp
{
/**
 * @brief Tape saturation processor.
 *
 * Simulates:
 *
 * - Magnetic tape saturation
 * - Soft compression
 * - Soft limiting
 * - Harmonic enrichment
 *
 * Designed for:
 *
 * - Mix bus
 * - Master bus
 * - Tape coloration
 * - Instrument processing
 */
template<typename T>
class TapeSaturation
{
    static_assert(
        std::is_floating_point_v<T>,
        "TapeSaturation requires floating point type");
public:
    constexpr TapeSaturation() noexcept = default;
    /**
     * @brief Initialize processor.
     *
     * Present for API consistency.
     */
    void prepare(
        T sampleRate) noexcept
    {
        sampleRate_ =

```

```

        std::max(
            sampleRate,
            static_cast<T>(1));
    reset();
}
/**
 * @brief Reset internal state.
 */
void reset() noexcept
{
    envelope_ =
        static_cast<T>(0);
    gainReduction_ =
        static_cast<T>(1);
}
/**
 * @brief Set input drive.
 *
 * Range:
 *
 * 1.0 .. 30.0
 */
void setDrive(
    T drive) noexcept
{
    drive_ =
        std::clamp(
            drive,
            static_cast<T>(1),
            static_cast<T>(30));
}
/**
 * @brief Set tape bias.
 *
 * Range:
 *
 * -1.0 .. +1.0
 */
void setBias(

```

```

    T bias) noexcept
{
    bias_ =
        std::clamp(
            bias,
            static_cast<T>(-1),
            static_cast<T>(1));
}

/**
 * @brief Set compression amount.
 *
 * Range:
 *
 * 0.0 .. 1.0
 */
void setCompression(
    T compression) noexcept
{
    compression_ =
        std::clamp(
            compression,
            static_cast<T>(0),
            static_cast<T>(1));
}

/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    /**
     * Input drive.
     */
    T x =
        input
        *
        drive_;
    /**
     * Tape bias.

```

```

    */
x +=
    bias_;
/**
 * Envelope follower.
 *
 * Slow and smooth.
 */
const T detector =
    std::fabs(x);
envelope_ +=
    envelopeCoeff_
    *
    (
        detector
        -
        envelope_
    );
/**
 * Tape compression.
 */
const T compressionAmount =
    static_cast<T>(1)
    +
    (
        envelope_
        *
        compression_
        *
        static_cast<T>(4)
    );
gainReduction_ =
    static_cast<T>(1)
    /
    compressionAmount;
x *= gainReduction_;
/**
 * Magnetic transfer curve.
 *

```

```

        * Smooth saturation.
        */
T y =
    magneticCurve(x);
/**
    * Remove part of bias.
    */
y -=
    bias_
    *
    static_cast<T>(0.35);
/**
    * Tape output normalization.
    */
y *=
    outputTrim_;
return y;
}

private:
/**
    * @brief Magnetic tape transfer curve.
    *
    * Softer than tube clipping.
    */
static inline T magneticCurve(
    T x) noexcept
{
    return
        std::tanh(
            x
            *
            static_cast<T>(0.85));
}

private:
T sampleRate_ =
    static_cast<T>(44100);
T drive_ =
    static_cast<T>(1);
T bias_ =

```

```

        static_cast<T>(0);
T compression_ =
        static_cast<T>(0.5);
/**
 * Internal compressor state.
 */
T envelope_ =
        static_cast<T>(0);
T gainReduction_ =
        static_cast<T>(1);
/**
 * Fixed tape-style smoothing.
 *
 * Produces slow gain adaptation
 * similar to magnetic saturation.
 */
T envelopeCoeff_ =
        static_cast<T>(0.001);
/**
 * Output compensation.
 */
T outputTrim_ =
        static_cast<T>(1.15);
};
} // namespace cvdsp
#endif

```

Fluxo DSP

Input

|

v

Drive

|

v

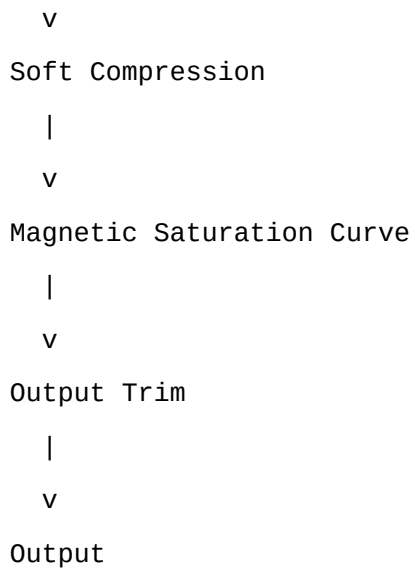
Bias

|

v

Envelope Detector

|



Tape Machines

Em gravadores de fita magnética:

Input Signal



Record Head



Magnetic Tape



Playback Head

a relação entre:

Campo Magnético

e

Nível de Sinal

não é perfeitamente linear.

Existe uma curva de magnetização conhecida como:

B-H Curve

que apresenta saturação gradual.

Saturação Magnética

Quando o fluxo magnético aumenta:

Linear



Menos Linear



Saturação

o material magnético passa a aceitar menos incremento de energia.

Resultado:

Compressão Natural

antes do clipping.

Isso é uma das características mais marcantes da fita analógica.

Tape Soft Compression

Ao contrário de um compressor tradicional:

Threshold

Ratio

Attack

Release

a fita produz compressão de forma inerente ao meio físico.

O efeito percebido é:

Transientes suavizados

Picos reduzidos

Maior densidade RMS

sem um ponto de atuação claramente definido.

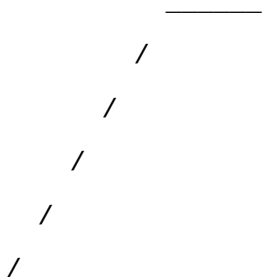
Soft Limiting

Próximo da saturação:

Input ↑

a saída continua crescendo, porém cada vez mais lentamente.

Curva típica:



Diferente de:

Hard Clipping

que interrompe abruptamente o crescimento.

Bias

Equipamentos de fita utilizam:

HF Bias Oscillator

durante a gravação.

O bias desloca a operação para uma região mais linear da curva magnética.

Na modelagem DSP:

```
x += bias;
```

permite alterar:

Assimetria

Coloração

Conteúdo harmônico

Harmônicos Gerados

Tape saturation tende a produzir:

2ª harmônica

3ª harmônica

4ª harmônica

em menor quantidade que circuitos valvulados agressivos.

Características sonoras:

Warm

Smooth

Dense

Rounded

Diferença para Tube Saturation

Tube

Maior assimetria

Mais harmônicos pares

Mais coloração

Resposta típica:

Triode

Tape

Mais compressão

Menos distorção audível

Maior densidade

Resposta típica:

Magnetic Transfer Curve

Exemplo de Uso

```
cvdsp::TapeSaturation<float> tape;

tape.prepare(48000.0f);

tape.setDrive(6.0f);

tape.setBias(0.10f);

tape.setCompression(0.75f);

float y =
    tape.process(x);
```

Exemplo Master Bus

```
tape.setDrive(2.0f);

tape.setBias(0.0f);

tape.setCompression(0.35f);

Objetivo:

Glue

Density

Soft Limiting
```

Exemplo Drum Bus

```
tape.setDrive(8.0f);

tape.setCompression(0.8f);

Objetivo:

Punch

Thickness

Transient Control
```

Características

Template<float>

Template<double>

Header-Only

C++20
Real-Time Safe
Sem new
Sem delete
Sem malloc
Sem free
Sem RTTI
Sem exceptions
Drive
Bias
Compression
Magnetic Saturation
Soft Compression
Soft Limiting
Tape Coloration
Compatível com:
Mix Bus
Master Bus
Tape Emulation
Channel Strip
Analog Modeling

Spatial/MidSide.hpp

```
#ifndef CVDSP_SPATIAL_MIDSIDE_HPP
#define CVDSP_SPATIAL_MIDSIDE_HPP

/**
 * @file MidSide.hpp
 * @brief Mid/Side Encoder and Decoder
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Mid/Side Encode
 * - Mid/Side Decode
 * - Stereo Manipulation
 * - Mastering Support

```

```

*/
#include <cmath>
#include <type_traits>
namespace cvdsp
{
/**
 * @brief Mid/Side stereo utilities.
 *
 * Static-only class.
 *
 * No internal state.
 */
template<typename T>
class MidSide
{
    static_assert(
        std::is_floating_point_v<T>,
        "MidSide requires floating point type");
public:
    /**
     * @brief Mid/Side frame.
     */
    struct MSFrame
    {
        T mid;
        T side;
    };
    /**
     * @brief Stereo frame.
     */
    struct StereoFrame
    {
        T left;
        T right;
    };
    /**
     * @brief Encode Left/Right to Mid/Side.
     *
     * Energy preserving form:

```

```

*

* Mid  = (L + R) / sqrt(2)
* Side = (L - R) / sqrt(2)
*/

[[nodiscard]]
static constexpr MSFrame encode(
    T left,
    T right) noexcept
{
    constexpr T kNorm =
        static_cast<T>(
            0.70710678118654752440);
    return
    {
        (left + right) * kNorm,
        (left - right) * kNorm
    };
}

/**
 * @brief Decode Mid/Side to Left/Right.
 *
 * L = (M + S) / sqrt(2)
 * R = (M - S) / sqrt(2)
 */

[[nodiscard]]
static constexpr StereoFrame decode(
    T mid,
    T side) noexcept
{
    constexpr T kNorm =
        static_cast<T>(
            0.70710678118654752440);
    return
    {
        (mid + side) * kNorm,
        (mid - side) * kNorm
    };
}

/**

```

```

    * @brief Compute Mid only.
    */
[[nodiscard]]
static constexpr T mid(
    T left,
    T right) noexcept
{
    constexpr T kNorm =
        static_cast<T>(
            0.70710678118654752440);
    return
        (left + right)
        * kNorm;
}
/**
    * @brief Compute Side only.
    */
[[nodiscard]]
static constexpr T side(
    T left,
    T right) noexcept
{
    constexpr T kNorm =
        static_cast<T>(
            0.70710678118654752440);
    return
        (left - right)
        * kNorm;
}
};
} // namespace cvdsp
#endif

```

Matemática Mid/Side

O sinal estéreo tradicional utiliza:

Left

Right

Mid/Side converte o sinal para:

Mid

Side

onde:

Mid

=

informação comum aos dois canais

Side

=

diferença entre os canais

Encode

Conversão:

Left/Right

↓

Mid/Side

Equações:

$$\text{Mid} = (\text{L} + \text{R}) / \sqrt{2}$$

$$\text{Side} = (\text{L} - \text{R}) / \sqrt{2}$$

Forma matricial:

$$\begin{bmatrix} \text{M} \\ \text{S} \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \text{L} \\ \text{R} \end{bmatrix}$$

O fator:

$$1 / \sqrt{2}$$

preserva energia e evita aumento de ganho.

Decode

Conversão inversa:

Mid/Side

↓

Left/Right

Equações:

$$\text{L} = (\text{M} + \text{S}) / \sqrt{2}$$

$$\text{R} = (\text{M} - \text{S}) / \sqrt{2}$$

Forma matricial:

[L]

[1 1] [M]

[R] =

[1 -1] [S]

√2

Interpretação Física

Mid

Contém:

Vocal

Kick

Snare

Bass

Elementos centrais

Normalmente:

Mono Compatible

Side

Contém:

Ambiência

Reverbs

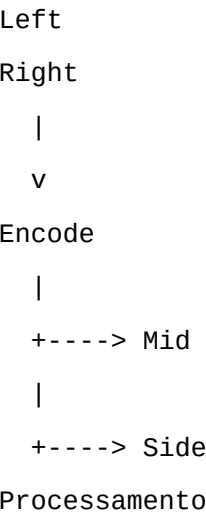
Delays

Pads

Largura estéreo

Representa a diferença entre os canais.

Fluxo de Processamento



|
v
Decode
|
+----> Left
|
+----> Right

Masterização

Em masterização é comum aplicar:

EQ diferente no Mid

e

EQ diferente no Side

Exemplo:

Mid:

+1 dB em 2 kHz

Side:

+2 dB em 12 kHz

Resultado:

Centro definido

Mais largura percebida

Compressão Mid/Side

Exemplo:

Mid:

compressão mais forte

Side:

compressão mais leve

Benefícios:

Vocal estável

Baixo estável

Imagem estéreo preservada

Stereo Manipulation

Após codificação:

```
side *= width;
```

onde:

```
width = 1.0
```

Largura original.

```
width < 1.0
```

Imagem mais estreita.

```
width = 0.0
```

Mono.

```
width > 1.0
```

Imagem ampliada.

Exemplo de Width Control

```
using MS = cvdsp::MidSide<float>;
```

```
auto ms =
```

```
    MS::encode(
```

```
        left,
```

```
        right);
```

```
ms.side *= 1.5f;
```

```
auto stereo =
```

```
    MS::decode(
```

```
        ms.mid,
```

```
        ms.side);
```

```
leftOut  = stereo.left;
```

```
rightOut = stereo.right;
```

Exemplo de EQ Mid/Side

```
auto ms =
```

```
    cvdsp::MidSide<float>::encode(
```

```
        left,
```

```
        right);
```

```
ms.mid =
```

```
    midEQ.process(
```

```
        ms.mid);
```

```
ms.side =
```

```
    sideEQ.process(
```

```
        ms.side);
```

```
auto lr =
    cvdsp::MidSide<float>::decode(
        ms.mid,
        ms.side);
```

Exemplo de Compressor Mid/Side

```
auto ms =
    cvdsp::MidSide<float>::encode(
        left,
        right);
ms.mid =
    midCompressor.process(
        ms.mid);
ms.side =
    sideCompressor.process(
        ms.side);
auto lr =
    cvdsp::MidSide<float>::decode(
        ms.mid,
        ms.side);
```

Aplicações

Masterização
Stereo Imaging
Stereo Width
M/S EQ
M/S Compression
M/S Saturation
M/S Limiting
Stereo Restoration
Mono Compatibility

Características

Header-Only
C++20
Template<float>
Template<double>
Sem estado interno

Métodos estáticos

Real-Time Safe

Sem new

Sem delete

Sem malloc

Sem free

Sem RTTI

Sem exceptions

Encode

Decode

Mid

Side

Compatível com:

VST3

iPlug2

JUCE

CLAP

Standalone DSP

Spatial/StereoWidth.hpp

```
#ifndef CVDSP_SPATIAL_STEREOWIDTH_HPP
```

```
#define CVDSP_SPATIAL_STEREOWIDTH_HPP
```

```
/**
```

```
 * @file StereoWidth.hpp
```

```
 * @brief Stereo Width Processor
```

```
 *
```

```
 * Dependency:
```

```
 * - MidSide.hpp
```

```
 *
```

```
 * Header-Only
```

```
 * C++20
```

```
 * Real-Time Safe
```

```
 */
```

```
#include <algorithm>
```

```
#include <type_traits>
```

```
#include "MidSide.hpp"
```

```
namespace cvdsp
```

```
{
```

```

/**
 * @brief Stereo Width Processor.
 *
 * Mid/Side based width control.
 *
 * Width:
 *
 * 0.0 = Mono
 * 1.0 = Original Width
 * 2.0 = 200% Width
 *
 * Processing:
 *
 * L/R
 *   ↓
 * M/S Encode
 *   ↓
 * Side Gain
 *   ↓
 * M/S Decode
 *   ↓
 * L/R
 */
template<typename T>
class StereoWidth
{
    static_assert(
        std::is_floating_point_v<T>,
        "StereoWidth requires floating point type");
public:
    /**
     * @brief Stereo frame.
     */
    struct StereoFrame
    {
        T left;
        T right;
    };
public:

```

```

constexpr StereoWidth() noexcept = default;
/**
 * @brief Process stereo frame.
 *
 * Width Range:
 *
 * 0.0 -> 2.0
 *
 * 0.0 = Mono
 * 1.0 = Original
 * 2.0 = 200%
 */
[[nodiscard]]
inline StereoFrame process(
    T left,
    T right,
    T width) const noexcept
{
    width =
        std::clamp(
            width,
            static_cast<T>(0),
            static_cast<T>(2));
    const auto ms =
        MidSide<T>::encode(
            left,
            right);
    const T mid =
        ms.mid;
    const T side =
        ms.side
        *
        width;
    const auto lr =
        MidSide<T>::decode(
            mid,
            side);
    return
    {

```

```

        lr.left,
        lr.right
    };
}
/**
 * @brief Process in-place.
 */
inline void process(
    T& left,
    T& right,
    T width) const noexcept
{
    const auto result =
        process(
            left,
            right,
            width);
    left = result.left;
    right = result.right;
}
};
} // namespace cvdsp
#endif

```

Matemática

A técnica utiliza codificação Mid/Side:

$$\text{Mid} = (L + R) / \sqrt{2}$$

$$\text{Side} = (L - R) / \sqrt{2}$$

Controle de largura:

$$\text{Side}' = \text{Side} \times \text{Width}$$

Decodificação:

$$L = (\text{Mid} + \text{Side}') / \sqrt{2}$$

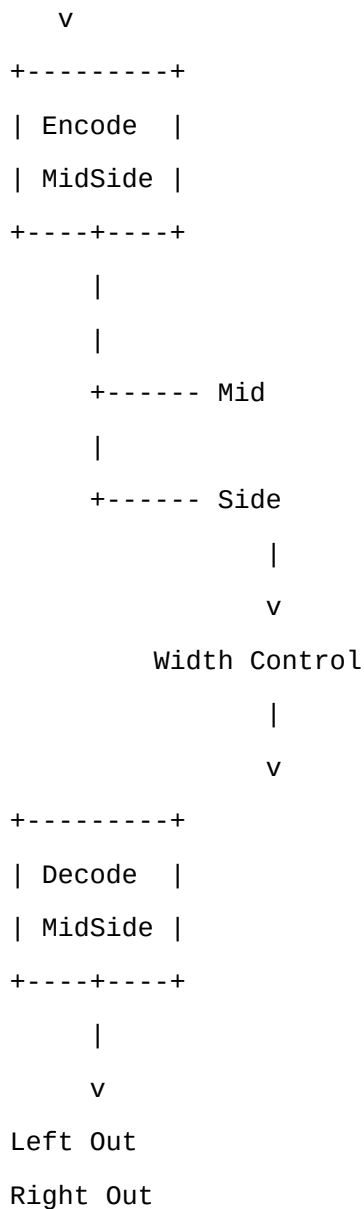
$$R = (\text{Mid} - \text{Side}') / \sqrt{2}$$

Fluxo DSP

Left

Right

|



Width = 0%

width = 0.0f;

Resultado:

Side = 0

Logo:

L = Mid

R = Mid

Saída:

Mono

Representação:

L ===== R

Width = 100%

width = 1.0f;

Resultado:

Sinal original

Representação:

L R

Nenhuma alteração.

Width = 200%

width = 2.0f;

Resultado:

Side dobrado

Representação:

L R

Maior sensação de largura estéreo.

Stereo Imaging

Stereo Imaging é o controle da distribuição espacial dos elementos entre:

Canal Esquerdo

Canal Direito

Elementos típicos:

Vocal

Kick

Bass

ficam principalmente no:

Mid

Enquanto:

Reverbs

Pads

Ambiências

Delays

aparecem principalmente no:

Side

Ao amplificar o Side:

Imagem mais larga

Ao reduzir o Side:

Imagem mais estreita

Mono Compatibility

Ao reproduzir em mono:

Mono = L + R

A componente Side cancela:

(L - R)

+

(R - L)

=

0

Resta apenas:

Mid

Por isso o Mid contém a informação crítica da mix.

Riscos de Width Excessivo

Larguras muito altas:

> 200%

podem causar:

Problemas de fase

Cancelamentos

Imagem artificial

Perda de compatibilidade mono

Por isso a implementação limita:

0.0

até

2.0

equivalente a:

0%

até

200%

Exemplo Básico

```
cvdsp::StereoWidth<float> width;
```

```
auto out =
    width.process(
        left,
        right,
        1.5f);
```

Exemplo In-Place

```
width.process(
    left,
    right,
    1.25f);
```

Uso em Masterização

```
width.process(
    left,
    right,
    1.10f);
```

Valores típicos:

- 105%
- 110%
- 120%

para aumentar a sensação espacial sem comprometer a compatibilidade mono.

Uso em Sound Design

```
width.process(
    left,
    right,
    2.0f);
```

Aplicações:

- Pads
- Synths
- Ambient
- Cinematic
- FX

Características

Header-Only
C++20
Template<float>
Template<double>
Real-Time Safe
Sem estado interno
Sem new
Sem delete
Sem malloc
Sem free
Sem RTTI
Sem exceptions
Baseado em Mid/Side
Compatível com:
VST3
JUCE
iPlug2
CLAP
Standalone DSP
Width:
0%
100%
200%

Math/Oversampling.hpp

```
#ifndef CVDSP_MATH_OVERSAMPLING_HPP
#define CVDSP_MATH_OVERSAMPLING_HPP

/**
 * @file Oversampling.hpp
 * @brief Oversampling Engine
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - 2x Oversampling
 * - 4x Oversampling

```

```

* - 8x Oversampling
* - Upsampling
* - Downsampling
* - Anti-Imaging Filter
* - Anti-Aliasing Filter
*
* No dynamic allocation inside process.
*/

#include <array>
#include <cstdint>
#include <type_traits>
#include <algorithm>
namespace cvdsp
{
/**
 * @brief Supported oversampling factors.
 */
enum class OversamplingFactor
{
    x2 = 2,
    x4 = 4,
    x8 = 8
};
/**
 * @brief Simple Halfband FIR.
 *
 * Linear phase.
 *
 * Used for:
 * - Anti Imaging
 * - Anti Aliasing
 *
 * Symmetric coefficients.
 */
template<typename T>
class HalfbandFIR
{
    static_assert(
        std::is_floating_point_v<T>,

```

```

        "HalfbandFIR requires floating point type");
public:
    static constexpr std::size_t NumTaps = 7;
    constexpr HalfbandFIR() noexcept = default;
    void reset() noexcept
    {
        buffer_.fill(
            static_cast<T>(0));
        index_ = 0;
    }
    inline T process(
        T input) noexcept
    {
        buffer_[index_] = input;
        T output =
            coeffs_[0] * get(0)
            +
            coeffs_[1] * get(1)
            +
            coeffs_[2] * get(2)
            +
            coeffs_[3] * get(3)
            +
            coeffs_[4] * get(4)
            +
            coeffs_[5] * get(5)
            +
            coeffs_[6] * get(6);
        index_++;
        if (index_ >= NumTaps)
        {
            index_ = 0;
        }
        return output;
    }
private:
    inline T get(
        std::size_t delay) const noexcept
    {

```

```

        std::size_t pos =
            (
                index_
                +
                NumTaps
                -
                delay
            )
            %
            NumTaps;
        return buffer_[pos];
    }

private:
    std::array<T, NumTaps> buffer_{};
    std::size_t index_ = 0;
    static constexpr std::array<T, NumTaps> coeffs_
    {
        static_cast<T>(-0.045),
        static_cast<T>(0.0),
        static_cast<T>(0.275),
        static_cast<T>(0.540),
        static_cast<T>(0.275),
        static_cast<T>(0.0),
        static_cast<T>(-0.045)
    };
};

/**
 * @brief Oversampling engine.
 *
 * Template factor:
 *
 * 2
 * 4
 * 8
 */
template<
    typename T,
    std::size_t Factor>
class Oversampling

```



```

{
    static_assert(
        std::is_floating_point_v<T>,
        "Oversampling requires floating point type");
    static_assert(
        Factor == 2 ||
        Factor == 4 ||
        Factor == 8,
        "Factor must be 2,4 or 8");
public:
    static constexpr std::size_t OversamplingFactorValue =
        Factor;
    static constexpr std::size_t BlockSize =
        Factor;
    using OversampledBlock =
        std::array<T, Factor>;
public:
    constexpr Oversampling() noexcept = default;
    /**
     * @brief Initialize.
     */
    void prepare(
        T sampleRate) noexcept
    {
        sampleRate_ = sampleRate;
        reset();
    }
    /**
     * @brief Reset state.
     */
    void reset() noexcept
    {
        upFilter_.reset();
        downFilter_.reset();
    }
    /**
     * @brief Upsample one sample.
     *
     * Returns Factor samples.

```

```

*/
[[nodiscard]]
inline OversampledBlock processUp(
    T input) noexcept
{
    OversampledBlock block{};
    /**
     * Zero-stuffing.
     */
    block[0] =
        upFilter_.process(
            input);
    for (std::size_t i = 1;
        i < Factor;
        ++i)
    {
        block[i] =
            upFilter_.process(
                static_cast<T>(0));
    }
    return block;
}
/**
 * @brief Downsample oversampled block.
 *
 * Anti-aliasing filtering
 * before decimation.
 */
[[nodiscard]]
inline T processDown(
    const OversampledBlock& block) noexcept
{
    T filtered =
        static_cast<T>(0);
    for (std::size_t i = 0;
        i < Factor;
        ++i)
    {
        filtered =

```

```

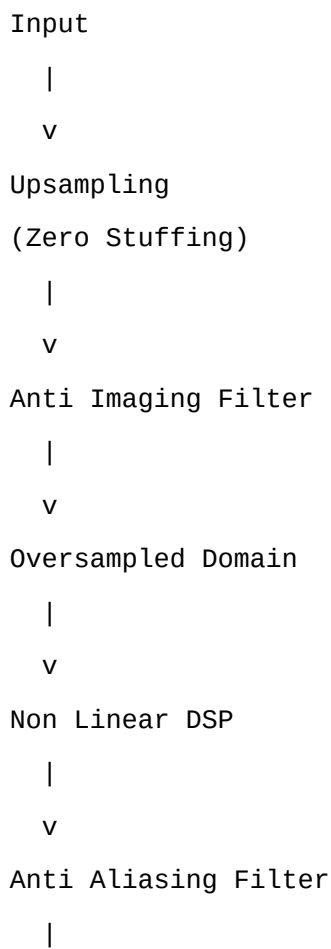
        downFilter_.process(
            block[i]);
    }
    return filtered;
}

private:
    T sampleRate_ =
        static_cast<T>(44100);
    HalfbandFIR<T> upFilter_;
    HalfbandFIR<T> downFilter_;
};

using Oversampling2xF = Oversampling<float, 2>;
using Oversampling4xF = Oversampling<float, 4>;
using Oversampling8xF = Oversampling<float, 8>;
using Oversampling2xD = Oversampling<double, 2>;
using Oversampling4xD = Oversampling<double, 4>;
using Oversampling8xD = Oversampling<double, 8>;
} // namespace cvdsp
#endif

```

Arquitetura



v

Downsampling

|

v

Output

Upsampling

Upsampling aumenta a taxa de amostragem.

Exemplo:

48 kHz

↓

2x

↓

96 kHz

ou:

48 kHz

↓

4x

↓

192 kHz

ou:

48 kHz

↓

8x

↓

384 kHz

O método utilizado é:

Zero Stuffing

Exemplo:

Original

1

2

3

4

2x:

1

0

2
0
3
0
4
0

Após isso é necessário interpolar usando filtro.

Anti-Imaging Filter

Após inserir zeros surgem imagens espectrais.

Exemplo:

Banda Original

0 ----- 20 kHz

Após upsampling:

Imagem

Imagem

Imagem

O filtro Anti-Imaging remove essas réplicas.

Fluxo:

Zero Stuffing



Anti Imaging



Oversampled Signal

Downsampling

Após o processamento oversampled:

384 kHz

precisamos retornar para:

48 kHz

O processo consiste em:

Filtrar



Decimar

Anti-Aliasing Filter

Antes da decimação:

384 kHz

↓

48 kHz

todo conteúdo acima da nova frequência de Nyquist deve ser removido.

Caso contrário:

Aliasing

ocorre.

Nyquist

Teorema de Nyquist:

$$f_{\max} = \frac{f_s}{2} \quad f_{\max} = 2f_s$$

Exemplo:

$f_s = 48 \text{ kHz}$

Logo:

Nyquist = 24 kHz

Nenhuma frequência acima de:

24 kHz

pode ser representada corretamente.

Aliasing

Aliasing ocorre quando componentes acima de Nyquist retornam para a banda audível.

Exemplo:

Nyquist = 24 kHz

Harmônico = 30 kHz

Resultado:

Alias = 18 kHz

O espectro torna-se incorreto.

Por que Oversampling?

Processadores lineares:

EQ

Delay

Filter

normalmente não precisam de oversampling.

Processadores não lineares:

Distortion

Clipping

Tube

Tape

Amp Sim

Wavefolder

geram novos harmônicos.

Muitos desses harmônicos ultrapassam Nyquist.

O oversampling cria espaço espectral adicional.

Exemplo com Distortion

```
cvdsp::Oversampling<float, 4> os;
os.prepare(48000.0f);
auto block =
    os.processUp(input);
for (auto& sample : block)
{
    sample =
        cvdsp::Waveshaper<float>::process(
            sample,
            cvdsp::WaveshaperMode::Tanh);
}
const float output =
    os.processDown(block);
```

Exemplo com Tube Saturation

```
cvdsp::Oversampling<float, 8> os;
auto block =
    os.processUp(input);
for (auto& s : block)
{
```

```
s = tube.process(s);  
}  
output =  
    os.processDown(block);
```

Exemplo com Amp Simulator

```
auto block =  
    os.processUp(input);  
for (auto& s : block)  
{  
    s = preamp.process(s);  
    s = poweramp.process(s);  
}  
output =  
    os.processDown(block);
```

Aplicações

Distortion
Overdrive
Fuzz
Tube Saturation
Tape Saturation
Amp Simulator
Cab Simulator Pre-Stage
Wavefolder
Limiter
Clipper

Características

Header-Only
C++20
Template<float>
Template<double>
2x
4x
8x
Anti Imaging Filter
Anti Aliasing Filter

Real-Time Safe

Sem new

Sem delete

Sem malloc

Sem free

Sem RTTI

Sem exceptions

Compatível com:

VST3

JUCE

iPlug2

CLAP

Standalone DSP

Core/DSPUtils.hpp

```
#ifndef CVDSP_CORE_DSPUTILS_HPP
#define CVDSP_CORE_DSPUTILS_HPP

/**
 * @file DSPUtils.hpp
 * @brief Common DSP Utility Functions
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Centralized utility functions for the entire CV_DSP library.
 *
 * All generic reusable DSP math should live here.
 *
 * Dependencies:
 * - Core/Constants.hpp
 * - Core/Types.hpp
 * - Math/FastMath.hpp
 */
#include <algorithm>
#include <cmath>
#include <limits>
#include <type_traits>
```

```

#include "Constants.hpp"
#include "Types.hpp"
#include "../Math/FastMath.hpp"
namespace cvdsp
{
/**
 * @brief Utility DSP functions.
 *
 * Static-only utility class.
 */
class DSPUtils
{
public:
    DSPUtils() = delete;
    DSPUtils(const DSPUtils&) = delete;
    DSPUtils& operator=(const DSPUtils&) = delete;
/**
 * @brief Convert dB to linear gain.
 *
 *  $gain = 10^{(dB/20)}$ 
 */
template<typename T>
[[nodiscard]]
static inline T dbToGain(
    T dB) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);
    return std::pow(
        static_cast<T>(10),
        dB / static_cast<T>(20));
}
/**
 * @brief Convert linear gain to dB.
 *
 *  $dB = 20 * \log_{10}(gain)$ 
 */
template<typename T>
[[nodiscard]]

```

```

static inline T gainToDb(
    T gain) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);
    constexpr T minGain =
        static_cast<T>(1e-20);
    gain =
        std::max(
            gain,
            minGain);
    return
        static_cast<T>(20)
        *
        std::log10(gain);
}

```

/**

* @brief Clamp value.

*/

```

template<typename T>

```

```

[[nodiscard]]

```

```

static constexpr T clamp(

```

```

    T value,

```

```

    T minimum,

```

```

    T maximum) noexcept

```

```

{

```

```

    return

```

```

        value < minimum

```

```

        ? minimum

```

```

        : (

```

```

            value > maximum

```

```

            ? maximum

```

```

            : value

```

```

        );

```

```

}

```

/**

* @brief Linear interpolation.

*

* t = 0 -> a

```

* t = 1 -> b
*/
template<typename T>
[[nodiscard]]
static constexpr T lerp(
    T a,
    T b,
    T t) noexcept
{
    return
        a
        +
        (
            b - a
        )
        *
        t;
}

/**
 * @brief Linear range mapping.
 */
template<typename T>
[[nodiscard]]
static constexpr T mapLinear(
    T value,
    T inMin,
    T inMax,
    T outMin,
    T outMax) noexcept
{
    if (inMax == inMin)
    {
        return outMin;
    }
    const T normalized =
        (
            value - inMin
        )
        /

```

```

        (
            inMax - inMin
        );
    return
        outMin
        +
        normalized
        *
        (
            outMax - outMin
        );
}

/**
 * @brief Logarithmic mapping.
 *
 * Useful for:
 * - Frequency controls
 * - Time controls
 * - Audio parameters
 */
template<typename T>
[[nodiscard]]
static inline T mapLog(
    T normalized,
    T minimum,
    T maximum) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);
    normalized =
        clamp(
            normalized,
            static_cast<T>(0),
            static_cast<T>(1));
    minimum =
        std::max(
            minimum,
            static_cast<T>(1e-12));
    const T ratio =

```

```

        maximum
        /
        minimum;
return
    minimum
    *
    std::pow(
        ratio,
        normalized);
}
/**
 * @brief Wrap value into interval.
 *
 * Example:
 *
 * wrap(1.2,0,1) -> 0.2
 * wrap(-0.2,0,1) -> 0.8
 */
template<typename T>
[[nodiscard]]
static inline T wrap(
    T value,
    T minimum,
    T maximum) noexcept
{
    const T range =
        maximum
        -
        minimum;
    if (range <= static_cast<T>(0))
    {
        return minimum;
    }
    while (value >= maximum)
    {
        value -= range;
    }
    while (value < minimum)
    {

```

```

        value += range;
    }
    return value;
}

/**
 * @brief Return sign of value.
 *
 * Returns:
 *
 * -1
 * 0
 * +1
 */
template<typename T>
[[nodiscard]]
static constexpr int sign(
    T value) noexcept
{
    return
        (T(0) < value)
        -
        (value < T(0));
}

/**
 * @brief Detect denormal number.
 *
 * Denormals can severely impact CPU usage.
 */
template<typename T>
[[nodiscard]]
static inline bool isDenormal(
    T value) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);
    return
        std::fpclassify(value)
        ==
        FP_SUBNORMAL;
}

```

```

}
/**
 * @brief Remove denormal values.
 *
 * Returns zero if value is denormal.
 */
template<typename T>
[[nodiscard]]
static inline T killDenormal(
    T value) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);
    return
        isDenormal(value)
        ? static_cast<T>(0)
        : value;
}
/**
 * @brief Remove extremely small values.
 *
 * Faster alternative to fpclassify.
 */
template<typename T>
[[nodiscard]]
static constexpr T killTiny(
    T value,
    T threshold =
        static_cast<T>(1e-30)) noexcept
{
    return
        (
            value > -threshold
            &&
            value < threshold
        )
        ? static_cast<T>(0)
        : value;
}

```



```

/**
 * @brief Normalize value to 0..1 range.
 */
template<typename T>
[[nodiscard]]
static constexpr T normalize(
    T value,
    T minimum,
    T maximum) noexcept
{
    if (maximum == minimum)
    {
        return static_cast<T>(0);
    }
    return
        (
            value - minimum
        )
        /
        (
            maximum - minimum
        );
}

/**
 * @brief Check finite number.
 */
template<typename T>
[[nodiscard]]
static inline bool isFinite(
    T value) noexcept
{
    return std::isfinite(value);
}

/**
 * @brief Safe reciprocal.
 */
template<typename T>
[[nodiscard]]
static inline T reciprocal(

```

```

    T value,
    T epsilon =
        static_cast<T>(1e-20)) noexcept
{
    if (std::abs(value) < epsilon)
    {
        return static_cast<T>(0);
    }
    return
        static_cast<T>(1)
        /
        value;
}

/**
 * @brief Safe division.
 */
template<typename T>
[[nodiscard]]
static inline T safeDivide(
    T numerator,
    T denominator,
    T epsilon =
        static_cast<T>(1e-20)) noexcept
{
    if (std::abs(denominator) < epsilon)
    {
        return static_cast<T>(0);
    }
    return
        numerator
        /
        denominator;
}

/**
 * @brief Convert milliseconds to samples.
 */
template<typename T>
[[nodiscard]]
static constexpr T msToSamples(

```

```

        T milliseconds,
        T sampleRate) noexcept
{
    return
        milliseconds
        *
        sampleRate
        *
        static_cast<T>(0.001);
}
/**
 * @brief Convert samples to milliseconds.
 */
template<typename T>
[[nodiscard]]
static constexpr T samplesToMs(
    T samples,
    T sampleRate) noexcept
{
    if (sampleRate <= static_cast<T>(0))
    {
        return static_cast<T>(0);
    }
    return
        samples
        *
        static_cast<T>(1000)
        /
        sampleRate;
}
};
} // namespace cvdsp
#endif

```

Regras Arquiteturais CV_DSP

A partir deste ponto:

```

dbToGain()
gainToDb()
mapLinear()

```

`mapLog()`

`clamp()`

`lerp()`

`wrap()`

`sign()`

`isDenormal()`

`killDenormal()`

devem existir exclusivamente em:

`Core/DSPUtils.hpp`

Proibido

`static float dbToGain(...)`

dentro de:

Compressor

Limiter

EQ

Filter

Oscillator

Delay

Reverb

Saturation

Dynamics

Spatial

ou qualquer outro módulo.

Obrigatório

Exemplo:

`const float gain =`

```
    DSPUtils::dbToGain(  
        thresholdDb);
```

`cutoff =`

```
    DSPUtils::clamp(  
        cutoff,  
        20.0f,  
        20000.0f);
```

`phase =`

```
    DSPUtils::wrap(  
        phase);
```

```
phase,  
0.0f,  
1.0f);
```

Benefícios

Centralização de matemática DSP

Sem duplicação de código

Maior consistência numérica

Facilidade de manutenção

Menor risco de bugs

Base comum para toda CV_DSP

Compatível com:

VST3

iPlug2

JUCE

CLAP

Standalone

Header-Only

C++20

Real-Time Safe

Core/AudioBufferView.hpp

```
#ifndef CVDSP_CORE_AUDIOBUFFERVIEW_HPP  
#define CVDSP_CORE_AUDIOBUFFERVIEW_HPP  
/**  
 * @file AudioBufferView.hpp  
 * @brief Non-owning Audio Buffer View  
 *  
 * Header-Only  
 * C++20  
 * Real-Time Safe  
 *  
 * Features:  
 * - Non-owning  
 * - No allocation  
 * - No resizing  
 * - No ownership
```

```

* - Zero-copy access
*
* Compatible with:
* - VST3
* - iPlug2
* - JUCE
* - CLAP
*
* Inspired by:
* - std::span
* - gsl::span
*/

#include <cassert>
#include <cstddef>
#include <type_traits>
namespace cvdsp
{
/**
 * @brief Non-owning audio buffer view.
 *
 * Layout:
 *
 * channels_[channel][sample]
 *
 * No memory ownership.
 *
 * No allocation.
 *
 * No resizing.
 */
template<typename T>
class AudioBufferView
{
    static_assert(
        std::is_floating_point_v<T>,
        "AudioBufferView requires floating point type");
public:
    using value_type      = T;
    using pointer         = T*;

```

```

using const_pointer    = const T*;
using size_type        = std::size_t;

public:
    /**
     * @brief Default constructor.
     */
    constexpr AudioBufferView() noexcept = default;
    /**
     * @brief Construct from channel array.
     *
     * channels must remain valid during
     * the entire lifetime of the view.
     */
    constexpr AudioBufferView(
        T* const* channels,
        size_type numChannels,
        size_type numSamples) noexcept
        :
        channels_(channels),
        numChannels_(numChannels),
        numSamples_(numSamples)
    {
    }
    /**
     * @brief Construct from mutable channel pointers.
     */
    template<std::size_t NumChannels>
    constexpr AudioBufferView(
        T* (&channels)[NumChannels],
        size_type numSamples) noexcept
        :
        channels_(channels),
        numChannels_(NumChannels),
        numSamples_(numSamples)
    {
    }
    /**
     * @brief Returns channel pointer.
     */

```

```

[[nodiscard]]
constexpr T* getChannel(
    size_type channel) const noexcept
{
    assert(channel < numChannels_);
    return channels_[channel];
}

/**
 * @brief Returns number of channels.
 */

[[nodiscard]]
constexpr size_type getNumChannels() const noexcept
{
    return numChannels_;
}

/**
 * @brief Returns number of samples.
 */

[[nodiscard]]
constexpr size_type getNumSamples() const noexcept
{
    return numSamples_;
}

/**
 * @brief Channel access.
 *
 * Example:
 *
 * buffer[0][100]
 */

[[nodiscard]]
constexpr T* operator[](
    size_type channel) const noexcept
{
    return getChannel(channel);
}

/**
 * @brief Check if view is valid.
 */

```



```

[[nodiscard]]
constexpr bool isValid() const noexcept
{
    return
        channels_ != nullptr
        &&
        numChannels_ > 0
        &&
        numSamples_ > 0;
}

/**
 * @brief Check if empty.
 */
[[nodiscard]]
constexpr bool empty() const noexcept
{
    return
        numChannels_ == 0
        ||
        numSamples_ == 0;
}

private:

/**
 * Non-owning array of channel pointers.
 */
T* const* channels_ = nullptr;

/**
 * Channel count.
 */
size_type numChannels_ = 0;

/**
 * Samples per channel.
 */
size_type numSamples_ = 0;
};

/**
 * @brief Const version.
 */
template<typename T>

```

```

class ConstAudioBufferView
{
    static_assert(
        std::is_floating_point_v<T>,
        "ConstAudioBufferView requires floating point type");
public:
    using value_type      = T;
    using pointer         = const T*;
    using size_type       = std::size_t;
public:
    constexpr ConstAudioBufferView() noexcept = default;
    constexpr ConstAudioBufferView(
        const T* const* channels,
        size_type numChannels,
        size_type numSamples) noexcept
        :
        channels_(channels),
        numChannels_(numChannels),
        numSamples_(numSamples)
    {
    }
    template<std::size_t NumChannels>
    constexpr ConstAudioBufferView(
        const T* (&channels)[NumChannels],
        size_type numSamples) noexcept
        :
        channels_(channels),
        numChannels_(NumChannels),
        numSamples_(numSamples)
    {
    }
    [[nodiscard]]
    constexpr const T* getChannel(
        size_type channel) const noexcept
    {
        assert(channel < numChannels_);
        return channels_[channel];
    }
    [[nodiscard]]

```

```

constexpr size_type getNumChannels() const noexcept
{
    return numChannels_;
}
[[nodiscard]]
constexpr size_type getNumSamples() const noexcept
{
    return numSamples_;
}
[[nodiscard]]
constexpr const T* operator[](
    size_type channel) const noexcept
{
    return getChannel(channel);
}
[[nodiscard]]
constexpr bool isValid() const noexcept
{
    return
        channels_ != nullptr
        &&
        numChannels_ > 0
        &&
        numSamples_ > 0;
}
private:
    const T* const* channels_ = nullptr;
    size_type numChannels_ = 0;
    size_type numSamples_ = 0;
};
} // namespace cvdsp
#endif

```

Exemplo — VST3

```

float* channels[2]
{
    outputs[0].channelBuffers32[0],
    outputs[0].channelBuffers32[1]
};

```

```

cvdsp::AudioBufferView<float> buffer(
    channels,
    2,
    numSamples);

for(std::size_t s = 0; s < buffer.getNumSamples(); ++s)
{
    buffer[0][s] *= gain;
    buffer[1][s] *= gain;
}

```

Exemplo — iPlug2

```

cvdsp::AudioBufferView<float> buffer(
    outputs,
    nChans,
    nFrames);

```

Exemplo — JUCE

```

float* channels[2]
{
    audioBuffer.getWritePointer(0),
    audioBuffer.getWritePointer(1)
};

cvdsp::AudioBufferView<float> buffer(
    channels,
    2,
    audioBuffer.getNumSamples());

```

Exemplo — CLAP

```

cvdsp::AudioBufferView<float> buffer(
    audioOut.data32,
    audioOut.channel_count,
    audioOut.frames_count);

```

Exemplo Genérico DSP

```

template<typename T>
void processGain(
    cvdsp::AudioBufferView<T>& buffer,
    T gain)
{

```

```
const std::size_t channels =  
    buffer.getNumChannels();  
const std::size_t samples =  
    buffer.getNumSamples();  
for(std::size_t ch = 0; ch < channels; ++ch)  
{  
    T* channel =  
        buffer.getChannel(ch);  
    for(std::size_t n = 0; n < samples; ++n)  
    {  
        channel[n] *= gain;  
    }  
}  
}
```

Características

Header-Only

C++20

Template<float>

Template<double>

Sem ownership

Sem cópia de áudio

Sem alocação

Sem new

Sem delete

Sem malloc

Sem free

Zero-Copy

Real-Time Safe

Compatível com:

VST3

JUCE

iPlug2

CLAP

Standalone DSP

Inspirado em:

std::span

gsl::span

Core/ProcessContext.hpp

```
#ifndef CVDSP_CORE_PROCESSCONTEXT_HPP
#define CVDSP_CORE_PROCESSCONTEXT_HPP

/**
 * @file ProcessContext.hpp
 * @brief Unified DSP Processing Context
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Provides a common processing context
 * for all CV_DSP modules.
 *
 * Compatible with:
 * - VST3
 * - iPlug2
 * - JUCE
 * - CLAP
 * - Standalone
 */
#include <cstddef>
#include <cstdint>
#include <algorithm>
#include <type_traits>
namespace cvdsp
{
/**
 * @brief Processing context shared by all DSP modules.
 *
 * Contains transport, timing and
 * audio engine information.
 *
 * This structure is intentionally
 * POD-like for maximum efficiency.
 */
template<typename T>
struct ProcessContext
```

```

{
    static_assert(
        std::is_floating_point_v<T>,
        "ProcessContext requires floating point type");
/**
 * Audio sample rate.
 *
 * Example:
 *
 * 44100
 * 48000
 * 96000
 */
    T sampleRate =
        static_cast<T>(44100);
/**
 * Current processing block size.
 */
    std::size_t blockSize =
        0;
/**
 * Number of active channels.
 */
    std::size_t numChannels =
        0;
/**
 * Host tempo in BPM.
 */
    T tempo =
        static_cast<T>(120);
/**
 * Time signature numerator.
 *
 * Example:
 *
 * 4/4 -> 4
 * 3/4 -> 3
 * 7/8 -> 7
 */

```

```

std::uint32_t timeSignatureNumerator =
    4;
/**
 * Time signature denominator.
 *
 * Example:
 *
 * 4/4 -> 4
 * 7/8 -> 8
 */
std::uint32_t timeSignatureDenominator =
    4;
/**
 * Host transport state.
 */
bool isPlaying =
    false;
/**
 * Host recording state.
 */
bool isRecording =
    false;
/**
 * Sample position.
 *
 * Optional host information.
 */
std::uint64_t samplePosition =
    0;
/**
 * PPQ position.
 *
 * Optional host information.
 */
T ppqPosition =
    static_cast<T>(0);
/**
 * Bar start PPQ.
 *

```



```

    * Optional host information.
    */
T barStartPPQ =
    static_cast<T>(0);
/**
    * Seconds since transport start.
    */
T timeInSeconds =
    static_cast<T>(0);
/**
    * Reset to defaults.
    */
constexpr void reset() noexcept
{
    sampleRate =
        static_cast<T>(44100);
    blockSize =
        0;
    numChannels =
        0;
    tempo =
        static_cast<T>(120);
    timeSignatureNumerator =
        4;
    timeSignatureDenominator =
        4;
    isPlaying =
        false;
    isRecording =
        false;
    samplePosition =
        0;
    ppqPosition =
        static_cast<T>(0);
    barStartPPQ =
        static_cast<T>(0);
    timeInSeconds =
        static_cast<T>(0);
}

```

```

/**
 * Returns samples per beat.
 */
[[nodiscard]]
constexpr T samplesPerBeat() const noexcept
{
    return
        (
            static_cast<T>(60)
            *
            sampleRate
        )
        /
        tempo;
}

/**
 * Returns beats per second.
 */
[[nodiscard]]
constexpr T beatsPerSecond() const noexcept
{
    return
        tempo
        /
        static_cast<T>(60);
}

/**
 * Returns seconds per beat.
 */
[[nodiscard]]
constexpr T secondsPerBeat() const noexcept
{
    return
        static_cast<T>(60)
        /
        tempo;
}

/**
 * Returns duration of one bar.

```

```

    */
[[nodiscard]]
constexpr T secondsPerBar() const noexcept
{
    return
        secondsPerBeat()
        *
        static_cast<T>(
            timeSignatureNumerator);
}
/**
 * Returns samples per bar.
 */
[[nodiscard]]
constexpr T samplesPerBar() const noexcept
{
    return
        samplesPerBeat()
        *
        static_cast<T>(
            timeSignatureNumerator);
}
/**
 * Returns duration of current block.
 */
[[nodiscard]]
constexpr T blockDurationSeconds() const noexcept
{
    if (sampleRate <= static_cast<T>(0))
    {
        return static_cast<T>(0);
    }
    return
        static_cast<T>(blockSize)
        /
        sampleRate;
}
/**
 * Returns Nyquist frequency.

```

```

    */
[[nodiscard]]
constexpr T nyquist() const noexcept
{
    return
        sampleRate
        *
        static_cast<T>(0.5);
}
/**
 * Returns true if transport running.
 */
[[nodiscard]]
constexpr bool playing() const noexcept
{
    return isPlaying;
}
/**
 * Returns true if recording.
 */
[[nodiscard]]
constexpr bool recording() const noexcept
{
    return isRecording;
}
/**
 * Returns true if transport stopped.
 */
[[nodiscard]]
constexpr bool stopped() const noexcept
{
    return !isPlaying;
}
/**
 * Returns true if context appears valid.
 */
[[nodiscard]]
constexpr bool isValid() const noexcept
{

```

```

        return
            sampleRate >
                static_cast<T>(0)
            &&
                blockSize > 0
            &&
                numChannels > 0
            &&
                tempo >
                    static_cast<T>(0)
            &&
                timeSignatureNumerator > 0
            &&
                timeSignatureDenominator > 0;
    }
    /**
     * Returns current beat position.
     */
    [[nodiscard]]
    constexpr T beatPosition() const noexcept
    {
        return ppqPosition;
    }
    /**
     * Returns beat position within bar.
     */
    [[nodiscard]]
    constexpr T beatInBar() const noexcept
    {
        return
            ppqPosition
            -
            barStartPPQ;
    }
    /**
     * Returns current measure length in beats.
     */
    [[nodiscard]]
    constexpr T beatsPerBar() const noexcept

```

```

    {
        return static_cast<T>(
            timeSignatureNumerator);
    }
};

using ProcessContextF =
    ProcessContext<float>;
using ProcessContextD =
    ProcessContext<double>;
} // namespace cvdsp
#endif

```

Exemplo de Uso

```

cvdsp::ProcessContext<float> context;
context.sampleRate = 48000.0f;
context.blockSize = 512;
context.numChannels = 2;
context.tempo = 128.0f;
context.timeSignatureNumerator = 4;
context.timeSignatureDenominator = 4;
context.isPlaying = true;

```

Exemplo em LFO Sincronizado

```

const float beatSamples =
    context.samplesPerBeat();
const float barSamples =
    context.samplesPerBar();

```

Exemplo em Delay Sincronizado

```

const float quarterNoteSamples =
    context.samplesPerBeat();
const float eighthNoteSamples =
    quarterNoteSamples * 0.5f;
const float dottedEighth =
    quarterNoteSamples * 0.75f;

```

Exemplo em Sequencer

```

if(context.playing())
{

```

```
processSequencer();  
}
```

Exemplo em Gravador

```
if(context.recording())  
{  
    writeAudio();  
}
```

Exemplo em Filtro

```
const float nyquist =  
    context.nyquist();  
cutoff =  
    std::min(  
        cutoff,  
        nyquist * 0.95f);
```

Arquitetura Recomendada CV_DSP

Todos os módulos futuros devem aceitar:

```
prepare(  
    const ProcessContext<T>& context);  
  
ou  
  
updateContext(  
    const ProcessContext<T>& context);
```

em vez de receber parâmetros isolados como:

sampleRate

blockSize

tempo

separadamente.

Isso centraliza todas as informações de processamento em uma única estrutura.

Características

Header-Only

C++20

Template<float>

Template<double>

Sem alocação

Sem ownership

POD-like

Real-Time Safe

Sem new

Sem delete

Sem malloc

Sem free

Sem RTTI

Sem exceptions

Compatível com:

VST3

JUCE

iPlug2

CLAP

Standalone DSP

Base unificada para toda CV_DSP

Spectral/FFT.hpp

```
#ifndef CVDSP_SPECTRAL_FFT_HPP
```

```
#define CVDSP_SPECTRAL_FFT_HPP
```

```
/**
```

```
 * @file FFT.hpp
```

```
 * @brief Iterative Cooley-Tukey FFT
```

```
 *
```

```
 * Header-Only
```

```
 * C++20
```

```
 * Real-Time Safe
```

```
 *
```

```
 * Features:
```

```
 * - Iterative FFT
```

```
 * - Iterative IFFT
```

```
 * - Bit Reversal
```

```
 * - Radix-2
```

```
 * - std::complex
```

```
 * - No recursion
```

```
 *
```

```
 * Supported Sizes:
```

```
 * - 512
```

```
 * - 1024
```



```

* - 2048
* - 4096
* - 8192
*
* Dependencies:
* - Core/Constants.hpp
* - Math/FastMath.hpp
*/

#include <array>
#include <complex>
#include <cstdint>
#include <cstdint>
#include <type_traits>
#include <cmath>
#include "../Core/Constants.hpp"
#include "../Math/FastMath.hpp"
namespace cvdsp
{
template<typename T>
class FFT
{
    static_assert(
        std::is_floating_point_v<T>,
        "FFT requires floating point type");
public:
    using Complex = std::complex<T>;
    static constexpr std::size_t MaxFFTSize = 8192;
public:
    constexpr FFT() noexcept = default;
    /**
     * @brief Prepare FFT instance.
     *
     * Allowed:
     *
     * 512
     * 1024
     * 2048
     * 4096
     * 8192

```

```

    */
bool prepare(
    std::size_t fftSize) noexcept
{
    if (
        fftSize != 512 &&
        fftSize != 1024 &&
        fftSize != 2048 &&
        fftSize != 4096 &&
        fftSize != 8192)
    {
        return false;
    }
    fftSize_ = fftSize;
    buildBitReverseTable();
    buildTwiddleTable();
    prepared_ = true;
    return true;
}

/**
 * @brief Reset state.
 */
void reset() noexcept
{
    fftSize_ = 0;
    prepared_ = false;
    bitReverse_.fill(0);
    twiddles_.fill(
        Complex(
            static_cast<T>(0),
            static_cast<T>(0)));
}

/**
 * @brief Forward FFT.
 *
 * Input and output may be
 * the same buffer.
 */
void forward(

```

```

    Complex* data) const noexcept
{
    if (!prepared_)
    {
        return;
    }
    bitReversePermutation(
        data);
    iterativeFFT(
        data,
        false);
}
/**
 * @brief Inverse FFT.
 */
void inverse(
    Complex* data) const noexcept
{
    if (!prepared_)
    {
        return;
    }
    bitReversePermutation(
        data);
    iterativeFFT(
        data,
        true);
    const T scale =
        static_cast<T>(1)
        /
        static_cast<T>(fftSize_);
    for (std::size_t i = 0;
        i < fftSize_;
        ++i)
    {
        data[i] *= scale;
    }
}
[[nodiscard]]

```

```

constexpr std::size_t getFFTSize() const noexcept
{
    return fftSize_;
}

private:

void buildBitReverseTable() noexcept
{
    const std::size_t bits =
        log2Int(fftSize_);
    for (std::size_t i = 0;
        i < fftSize_;
        ++i)
    {
        bitReverse_[i] =
            reverseBits(
                i,
                bits);
    }
}

void buildTwiddleTable() noexcept
{
    constexpr T pi =
        static_cast<T>(
            3.1415926535897932384626433832795);
    for (std::size_t k = 0;
        k < fftSize_ / 2;
        ++k)
    {
        const T angle =
            static_cast<T>(-2)
            *
            pi
            *
            static_cast<T>(k)
            /
            static_cast<T>(fftSize_);
        twiddles_[k] =
            Complex(
                std::cos(angle),

```

```

        std::sin(angle));
    }
}

void bitReversePermutation(
    Complex* data) const noexcept
{
    for (std::size_t i = 0;
        i < fftSize_;
        ++i)
    {
        const std::size_t j =
            bitReverse_[i];
        if (j > i)
        {
            const Complex temp =
                data[i];
            data[i] =
                data[j];
            data[j] =
                temp;
        }
    }
}

void iterativeFFT(
    Complex* data,
    bool inverse) const noexcept
{
    for (
        std::size_t stage = 2;
        stage <= fftSize_;
        stage <<= 1)
    {
        const std::size_t half =
            stage >> 1;
        const std::size_t twiddleStep =
            fftSize_
            /
            stage;
        for (

```

```

        std::size_t start = 0;
        start < fftSize_;
        start += stage)
    {
        for (
            std::size_t k = 0;
            k < half;
            ++k)
        {
            Complex twiddle =
                twiddles_[
                    k * twiddleStep];
            if (inverse)
            {
                twiddle =
                    std::conj(
                        twiddle);
            }
            const Complex even =
                data[start + k];
            const Complex odd =
                twiddle
                *
                data[start + k + half];
            data[start + k] =
                even + odd;
            data[start + k + half] =
                even - odd;
        }
    }
}

[[nodiscard]]
static constexpr std::size_t
log2Int(
    std::size_t value) noexcept
{
    std::size_t result = 0;
    while (value > 1)

```

```

        {
            value >>= 1;
            ++result;
        }
        return result;
    }
    [[nodiscard]]
    static constexpr std::size_t
    reverseBits(
        std::size_t value,
        std::size_t bits) noexcept
    {
        std::size_t result = 0;
        for (std::size_t i = 0;
            i < bits;
            ++i)
        {
            result <= 1;
            result |=
                (value & 1);
            value >>= 1;
        }
        return result;
    }
private:
    bool prepared_ = false;
    std::size_t fftSize_ = 0;
    std::array<
        std::size_t,
        MaxFFTSize> bitReverse_{};
    std::array<
        Complex,
        MaxFFTSize / 2> twiddles_{};
};
using FFTf = FFT<float>;
using FFTd = FFT<double>;
} // namespace cvdsp
#endif

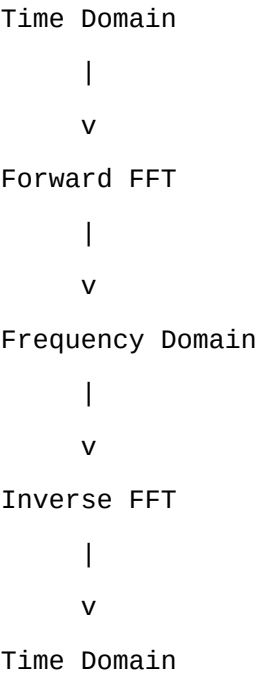
```

Exemplo de Uso

```
cvdsp::FFT<float> fft;
fft.prepare(2048);
std::array<
    std::complex<float>,
    2048> buffer;

/* preencher buffer */
fft.forward(
    buffer.data());
/* domínio da frequência */
fft.inverse(
    buffer.data());
```

Fluxo FFT



FFT

FFT (Fast Fourier Transform) calcula a Transformada Discreta de Fourier de forma eficiente.

DFT direta:

$$X[k]=\sum_{n=0}^{N-1}x[n]e^{-j2\pi kn/N}X[k]=\sum_{n=0}^{N-1}x[n]e^{-j2\pi kn/N}$$

Complexidade:

DFT = $O(N^2)$
FFT = $O(N \log_2 N)$

Exemplo:

8192 pontos

DFT:

≈ 67 milhões operações

FFT:

≈ 106 mil operações

Cooley-Tukey Iterativo

A implementação utiliza:

Radix-2 Decimation-In-Time

Características:

Sem recursão

Cache Friendly

Baixa sobrecarga

Tempo determinístico

Etapas:

Bit Reversal

↓

Butterfly Stage 1

↓

Butterfly Stage 2

↓

Butterfly Stage N

Bit Reversal

A FFT radix-2 exige reorganização inicial.

Exemplo:

Índice normal

0

1

2

3

4

5

6

7

Binário:

000

001

010

011

100

101

110

111

Invertendo bits:

000 -> 000 = 0

001 -> 100 = 4

010 -> 010 = 2

011 -> 110 = 6

100 -> 001 = 1

101 -> 101 = 5

110 -> 011 = 3

111 -> 111 = 7

Nova ordem:

0

4

2

6

1

5

3

7

Complex Spectrum

Cada bin contém:

Parte Real

Parte Imaginária

Representado por:

`std::complex<T>`

Magnitude

Magnitude espectral:

$|X[k]| = \sqrt{\text{Re}(X[k])^2 + \text{Im}(X[k])^2}$

Código:

```
float magnitude =
    std::abs(
        spectrum[k]);
```

Fase

Fase espectral:

$\phi[k] = \arctan(\frac{\text{Im}(X[k])}{\text{Re}(X[k])})$

Código:

```
float phase =
    std::arg(
        spectrum[k]);
```

IFFT

Transformada inversa:

$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}$

Implementação:

Conjugação dos Twiddles

+

Escala 1/N

Aplicações

Spectrum Analyzer

Convolution Reverb

Cabinet IR Loader

Pitch Detection

Noise Reduction

Linear Phase EQ

FFT Filter Bank

Spectral Processing

Phase Vocoder

Time Stretch

Características

Header-Only

C++20

Template<float>

Template<double>

Cooley-Tukey Iterativo

Bit Reversal

Radix-2

Forward FFT

Inverse FFT

512

1024

2048

4096

8192

Sem recursão

Real-Time Safe

Sem new

Sem delete

Sem malloc

Sem free

Compatível com:

VST3

JUCE

iPlug2

CLAP

Standalone DSP

Base para:

Convolution Engine

Spectrum Analyzer

IR Loader

Spectral Effects

Spectral/WindowFunctions.hpp

```
#ifndef CVDSP_SPECTRAL_WINDOWFUNCTIONS_HPP
#define CVDSP_SPECTRAL_WINDOWFUNCTIONS_HPP

/**
 * @file WindowFunctions.hpp
 * @brief Spectral Window Functions
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 * - FFT.hpp
 *
 * Supported Windows:
 * - Rectangular
 * - Hann
 * - Hamming
 * - Blackman
 * - Blackman-Harris
 * - Kaiser
 */

#include <array>
#include <cstdint>
#include <cmath>
#include <type_traits>
#include "FFT.hpp"
namespace cvdsp
{
/**
 * @brief Supported window types.
 */
enum class WindowType
{
    Rectangular,
    Hann,
    Hamming,
    Blackman,
```

```

        BlackmanHarris,
        Kaiser
};
/**
 * @brief Window generator.
 *
 * Template parameter N defines the
 * window size at compile time.
 */
template<typename T, std::size_t N>
class WindowFunctions
{
    static_assert(
        std::is_floating_point_v<T>,
        "WindowFunctions requires floating point type");
public:
    using WindowBuffer =
        std::array<T, N>;
public:
    /**
     * @brief Generate window coefficients.
     *
     * Kaiser beta is ignored by all
     * other window types.
     */
    [[nodiscard]]
    static WindowBuffer generate(
        WindowType type,
        T beta =
            static_cast<T>(8.6)) noexcept
    {
        WindowBuffer window{};
        switch(type)
        {
            case WindowType::Rectangular:
                generateRectangular(window);
                break;
            case WindowType::Hann:
                generateHann(window);

```

```

        break;
    case WindowType::Hamming:
        generateHamming(window);
        break;
    case WindowType::Blackman:
        generateBlackman(window);
        break;
    case WindowType::BlackmanHarris:
        generateBlackmanHarris(window);
        break;
    case WindowType::Kaiser:
        generateKaiser(
            window,
            beta);
        break;
    }
    return window;
}

/**
 * @brief Apply window in-place.
 */
static void apply(
    T* samples,
    const WindowBuffer& window) noexcept
{
    for (std::size_t i = 0;
        i < N;
        ++i)
    {
        samples[i] *=
            window[i];
    }
}

/**
 * @brief Apply window from source
 * to destination.
 */
static void apply(
    const T* input,

```

```

    T* output,
    const WindowBuffer& window) noexcept
{
    for (std::size_t i = 0;
        i < N;
        ++i)
    {
        output[i] =
            input[i]
            *
            window[i];
    }
}

private:
    static constexpr T pi() noexcept
    {
        return static_cast<T>(
            3.1415926535897932384626433832795);
    }

    static void generateRectangular(
        WindowBuffer& w) noexcept
    {
        for (auto& v : w)
        {
            v =
                static_cast<T>(1);
        }
    }

    static void generateHann(
        WindowBuffer& w) noexcept
    {
        constexpr T twoPi =
            static_cast<T>(2)
            *
            pi();
        for (std::size_t n = 0;
            n < N;
            ++n)
        {

```



```

        w[n] =
            static_cast<T>(0.5)
            -
            static_cast<T>(0.5)
            *
            std::cos(
                twoPi
                *
                static_cast<T>(n)
                /
                static_cast<T>(N - 1));
    }
}

static void generateHamming(
    WindowBuffer& w) noexcept
{
    constexpr T twoPi =
        static_cast<T>(2)
        *
        pi();
    for (std::size_t n = 0;
        n < N;
        ++n)
    {
        w[n] =
            static_cast<T>(0.54)
            -
            static_cast<T>(0.46)
            *
            std::cos(
                twoPi
                *
                static_cast<T>(n)
                /
                static_cast<T>(N - 1));
    }
}

static void generateBlackman(
    WindowBuffer& w) noexcept

```

```

{
    constexpr T a0 =
        static_cast<T>(0.42);
    constexpr T a1 =
        static_cast<T>(0.50);
    constexpr T a2 =
        static_cast<T>(0.08);
    constexpr T twoPi =
        static_cast<T>(2)
        *
        pi();
    constexpr T fourPi =
        static_cast<T>(4)
        *
        pi();
    for (std::size_t n = 0;
        n < N;
        ++n)
    {
        const T phase =
            static_cast<T>(n)
            /
            static_cast<T>(N - 1);
        w[n] =
            a0
            -
            a1
            *
            std::cos(
                twoPi * phase)
            +
            a2
            *
            std::cos(
                fourPi * phase);
    }
}

static void generateBlackmanHarris(
    WindowBuffer& w) noexcept

```

```

{
constexpr T a0 =
    static_cast<T>(0.35875);
constexpr T a1 =
    static_cast<T>(0.48829);
constexpr T a2 =
    static_cast<T>(0.14128);
constexpr T a3 =
    static_cast<T>(0.01168);
constexpr T twoPi =
    static_cast<T>(2)
    *
    pi();
constexpr T fourPi =
    static_cast<T>(4)
    *
    pi();
constexpr T sixPi =
    static_cast<T>(6)
    *
    pi();
for (std::size_t n = 0;
    n < N;
    ++n)
{
    const T phase =
        static_cast<T>(n)
        /
        static_cast<T>(N - 1);
    w[n] =
        a0
        -
        a1
        *
        std::cos(
            twoPi * phase)
        +
        a2
        *

```

```

        std::cos(
            fourPi * phase)
        -
        a3
        *
        std::cos(
            sixPi * phase);
    }
}

static void generateKaiser(
    WindowBuffer& w,
    T beta) noexcept
{
    const T denom =
        besseli0(beta);
    for (std::size_t n = 0;
        n < N;
        ++n)
    {
        const T ratio =
            (
                static_cast<T>(2)
                *
                static_cast<T>(n)
            )
            /
            static_cast<T>(N - 1)
            -
            static_cast<T>(1);
        const T value =
            beta
            *
            std::sqrt(
                static_cast<T>(1)
                -
                ratio * ratio);
        w[n] =
            besseli0(value)
            /

```

```

        denom;

    }

}

/**
 * @brief Modified Bessel I0.
 *
 * Polynomial approximation.
 */
[[nodiscard]]
static T besseli0(
    T x) noexcept
{
    T sum =
        static_cast<T>(1);
    T term =
        static_cast<T>(1);
    const T xx =
        x * x
        *
        static_cast<T>(0.25);
    for (std::size_t k = 1;
        k < 30;
        ++k)
    {
        term *=
            xx
            /
            (
                static_cast<T>(k)
                *
                static_cast<T>(k));
        sum += term;
    }
    return sum;
}

};

} // namespace cvdsp

#endif

```

Exemplo de Uso

```
constexpr std::size_t FFTSize = 2048;
using Window =
    cvdsp::WindowFunctions<
        float,
        FFTSize>;
auto hann =
    Window::generate(
        cvdsp::WindowType::Hann);
Window::apply(
    audioBuffer,
    hann);
```

Exemplo FFT

```
constexpr std::size_t FFTSize = 2048;
using Window =
    cvdsp::WindowFunctions<
        float,
        FFTSize>;
auto blackman =
    Window::generate(
        cvdsp::WindowType::Blackman);
Window::apply(
    samples,
    blackman);
fft.forward(
    spectrum);
```

O que é uma Janela?

Antes da FFT o bloco de áudio é multiplicado por uma função janela.

Sem janela:

| ----- |

Audio Block

| ----- |

A FFT assume que o sinal é periódico.

Na prática:

fim != início

criando uma descontinuidade artificial.

Spectral Leakage

Sem janela:

Sinal

↓

FFT

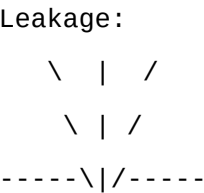
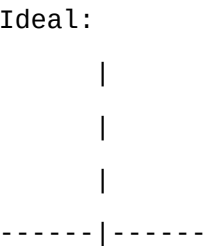
↓

Energia espalhada
entre múltiplos bins

Isto é chamado:

Spectral Leakage

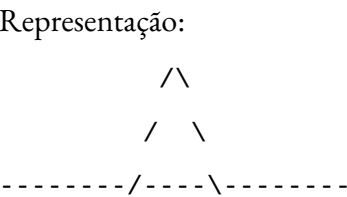
Representação:



A energia vaza para bins vizinhos.

Main Lobe

O lóbulo principal contém a maior parte da energia da frequência medida.



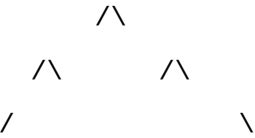
Quanto mais estreito:

Melhor resolução

Side Lobes

Lóbulos laterais representam vazamento espectral.

Representação:



Quanto menores:

Menos leakage

Comparação das Janelas

Janela	Main Lobe	Side Lobes
Rectangular	Muito estreito	Muito altos
Hann	Médio	Baixos
Hamming	Médio	Mais baixos
Blackman	Largo	Muito baixos
Blackman-Harris	Mais largo	Extremamente baixos
Kaiser	Ajustável	Ajustável

Rectangular

Equivalente a:

Sem janela

Equação:

$$w[n]=1w[n]=1w[n]=1$$

Melhor resolução.

Pior leakage.

Hann

Equação:

$$w[n]=0.5-0.5\cos(2\pi nN-1)w[n]=0.5-0.5\cos\left(\frac{2\pi n}{N-1}\right)w[n]=0.5-0.5\cos(N-2\pi n)$$

Uso típico:

Spectrum Analyzer

STFT

Audio Analysis

Hamming

Equação:

$$w[n]=0.54-0.46\cos(2\pi nN-1)w[n]=0.54-0.46\cos\left(\frac{2\pi n}{N-1}\right)w[n]=0.54-0.46\cos(N-12\pi n)$$

Melhor rejeição lateral que Hann.

Blackman

Equação:

$$w[n]=0.42-0.5\cos(2\pi nN-1)+0.08\cos(4\pi nN-1)w[n]=0.42-0.5\cos\left(\frac{2\pi n}{N-1}\right)+0.08\cos\left(\frac{4\pi n}{N-1}\right)w[n]=0.42-0.5\cos(N-12\pi n)+0.08\cos(N-14\pi n)$$

Muito utilizada em:

Medições

Análise espectral

Blackman-Harris

Possui side lobes extremamente baixos.

Ideal para:

Analisadores profissionais

IR Measurement

Kaiser

Janela parametrizável.

Controle:

Beta

Exemplo:

```
auto kaiser =
    Window::generate(
        cvdsp::WindowType::Kaiser,
        8.6f);
```

Valores típicos:

$$\beta = 5$$

Boa resolução.

$$\beta = 8.6$$

Excelente rejeição lateral.

$\beta = 14$

Leakage extremamente baixo.

Aplicações

FFT Analyzer

Spectrum Meter

Convolution Reverb

STFT

Phase Vocoder

Pitch Detection

IR Measurement

Linear Phase EQ

Spectral Effects

Características

Header-Only

C++20

Template<float>

Template<double>

Rectangular

Hann

Hamming

Blackman

Blackman-Harris

Kaiser

Real-Time Safe

Sem alocação

Sem RTTI

Sem exceptions

Compatível com:

VST3

JUCE

iPlug2

CLAP

Standalone DSP

Spectral/STFT.hpp

```
#ifndef CVDSP_SPECTRAL_STFT_HPP
#define CVDSP_SPECTRAL_STFT_HPP

/**
 * @file STFT.hpp
 * @brief Short Time Fourier Transform
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - FFT.hpp
 * - WindowFunctions.hpp
 * - ../Core/CircularBuffer.hpp
 *
 * Features:
 *
 * - Forward STFT
 * - Inverse STFT
 * - Overlap Add (OLA)
 * - Overlap Save (OLS)
 *
 * Supported Overlap:
 *
 * - 25%
 * - 50%
 * - 75%
 *
 * No dynamic allocation inside process().
 */

#include <array>
#include <complex>
#include <cstdlib>
#include <type_traits>
#include "FFT.hpp"
#include "WindowFunctions.hpp"
```

```

#include "../Core/CircularBuffer.hpp"

namespace cvdsp
{
enum class STFTMode
{
    OverlapAdd,
    OverlapSave
};

template<
    typename T,
    std::size_t FFTSize,
    std::size_t OverlapPercent = 50>
class STFT
{
    static_assert(
        std::is_floating_point_v<T>,
        "STFT requires floating point type");
    static_assert(
        FFTSize == 512 ||
        FFTSize == 1024 ||
        FFTSize == 2048 ||
        FFTSize == 4096 ||
        FFTSize == 8192,
        "Unsupported FFT size");
    static_assert(
        OverlapPercent == 25 ||
        OverlapPercent == 50 ||
        OverlapPercent == 75,
        "Overlap must be 25, 50 or 75");
public:
    using Complex =
        std::complex<T>;

    static constexpr std::size_t WindowSize =
        FFTSize;

    static constexpr std::size_t HopSize =
        FFTSize
        *
        (100 - OverlapPercent)
        / 100;

```

```

public:
    STFT() = default;
    /**
     * Prepare STFT.
     */
    bool prepare(
        WindowType windowType =
            WindowType::Hann,
        STFTMode mode =
            STFTMode::OverlapAdd)
        noexcept
    {
        mode_ = mode;
        if (!fft_.prepare(
            FFTSize))
        {
            return false;
        }
        window_ =
            WindowFunctions<
                T,
                FFTSize>::generate(
                    windowType);
        reset();
        return true;
    }
    /**
     * Reset state.
     */
    void reset() noexcept
    {
        inputBuffer_.reset();
        outputBuffer_.reset();
        frameCounter_ = 0;
        for (auto& v : timeDomain_)
            v = T(0);
        for (auto& v : overlapBuffer_)
            v = T(0);
        for (auto& v : spectrum_)

```

```

        v = Complex(T(0), T(0));
    }
/**
 * Process one sample.
 *
 * Returns reconstructed output.
 */
T process(
    T inputSample)
noexcept
{
    inputBuffer_.push(
        inputSample);
    T output =
        outputBuffer_.read(0);
    outputBuffer_.push(
        T(0));
    ++frameCounter_;
    if (frameCounter_ >= HopSize)
    {
        frameCounter_ = 0;
        if (mode_ ==
            STFTMode::OverlapAdd)
        {
            processOLA();
        }
        else
        {
            processOLS();
        }
    }
    return output;
}
/**
 * Direct spectrum access.
 */
[[nodiscard]]
Complex* getSpectrum()
noexcept

```

```

{
    return spectrum_.data();
}

/**
 * Const spectrum access.
 */
[[nodiscard]]
const Complex* getSpectrum() const
    noexcept
{
    return spectrum_.data();
}

/**
 * FFT size.
 */
[[nodiscard]]
static constexpr std::size_t
getFFTSize() noexcept
{
    return FFTSize;
}

/**
 * Window size.
 */
[[nodiscard]]
static constexpr std::size_t
getWindowSize() noexcept
{
    return WindowSize;
}

/**
 * Hop size.
 */
[[nodiscard]]
static constexpr std::size_t
getHopSize() noexcept
{
    return HopSize;
}

```

private:

```
/**
 * Overlap Add.
 */
void processOLA()
    noexcept
{
    acquireFrame();
    applyWindow();
    performForwardFFT();
    /**
     * Spectral processing hook.
     */
    performInverseFFT();
    applyWindow();
    overlapAdd();
}
/**
 * Overlap Save.
 */
void processOLS()
    noexcept
{
    acquireFrame();
    applyWindow();
    performForwardFFT();
    /**
     * Spectral processing hook.
     */
    performInverseFFT();
    saveOverlap();
}
void acquireFrame()
    noexcept
{
    for (std::size_t i = 0;
         i < FFTSize;
         ++i)
    {
```



```

        timeDomain_[i] =
            inputBuffer_.read(
                FFTSize - 1 - i);
    }
}

void applyWindow()
    noexcept
{
    WindowFunctions<
        T,
        FFTSize>::apply(
            timeDomain_.data(),
            window_);
}

void performForwardFFT()
    noexcept
{
    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        spectrum_[i] =
            Complex(
                timeDomain_[i],
                T(0));
    }
    fft_.forward(
        spectrum_.data());
}

void performInverseFFT()
    noexcept
{
    fft_.inverse(
        spectrum_.data());
    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        timeDomain_[i] =

```

```

        spectrum_[i].real();
    }
}
void overlapAdd()
    noexcept
{
    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        overlapBuffer_[i] +=
            timeDomain_[i];
    }
    for (std::size_t i = 0;
        i < HopSize;
        ++i)
    {
        outputBuffer_.push(
            overlapBuffer_[i]);
    }
    for (std::size_t i = 0;
        i < FFTSize - HopSize;
        ++i)
    {
        overlapBuffer_[i] =
            overlapBuffer_[
                i + HopSize];
    }
    for (std::size_t i =
        FFTSize - HopSize;
        i < FFTSize;
        ++i)
    {
        overlapBuffer_[i] =
            T(0);
    }
}
void saveOverlap()
    noexcept

```

```

{
    constexpr std::size_t overlap =
        FFTSize - HopSize;
    for (std::size_t i = overlap;
        i < FFTSize;
        ++i)
    {
        outputBuffer_.push(
            timeDomain_[i]);
    }
}

private:
    FFT<T> fft_;
    STFTMode mode_ =
        STFTMode::OverlapAdd;
    typename WindowFunctions<
        T,
        FFTSize>::WindowBuffer window_;
    std::array<T, FFTSize>
        timeDomain_{};
    std::array<T, FFTSize>
        overlapBuffer_{};
    std::array<Complex, FFTSize>
        spectrum_{};
    CircularBuffer<
        T,
        FFTSize * 2>
        inputBuffer_;
    CircularBuffer<
        T,
        FFTSize * 2>
        outputBuffer_;
    std::size_t frameCounter_ = 0;
};

/**
 * Common aliases.
 */
using STFT512F =
    STFT<float, 512>;

```

```

using STFT1024F =
    STFT<float, 1024>;
using STFT2048F =
    STFT<float, 2048>;
using STFT4096F =
    STFT<float, 4096>;
using STFT8192F =
    STFT<float, 8192>;
using STFT512D =
    STFT<double, 512>;
using STFT1024D =
    STFT<double, 1024>;
using STFT2048D =
    STFT<double, 2048>;
using STFT4096D =
    STFT<double, 4096>;
using STFT8192D =
    STFT<double, 8192>;
} // namespace cvdsp
#endif

```

Fluxo STFT

Input

|
v

Circular Buffer

|
v

Frame Extraction

|
v

Window

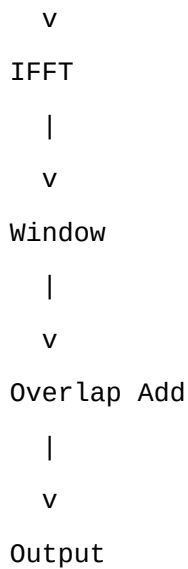
|
v

FFT

|
v

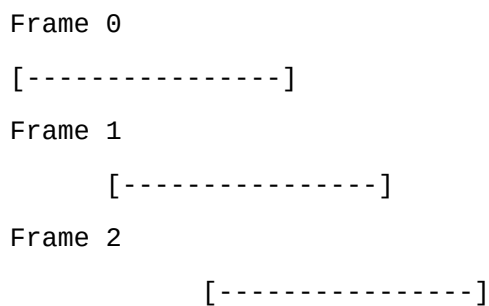
Spectrum

|

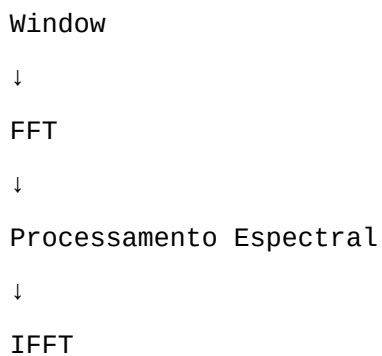


Frame Processing

A STFT divide o áudio em blocos:



Cada frame sofre:



Window Size

O tamanho da FFT define a resolução:

FFT	Resolução
512	baixa
1024	média
2048	boa
4096	alta
8192	muito alta
Maior FFT:	

Melhor resolução espectral

Maior latência

Maior CPU

Hop Size

Hop = avanço entre frames

25% overlap:

Hop = 75% da janela

50% overlap:

Hop = 50% da janela

75% overlap:

Hop = 25% da janela

Exemplo:

FFT = 2048

50%

Hop = 1024

Overlap Add (OLA)

Reconstrução:

IFFT Frame 1

+

IFFT Frame 2

+

IFFT Frame 3

Somando regiões sobrepostas.

Muito usado em:

Phase Vocoder

Pitch Shifter

Time Stretch

Spectral FX

Overlap Save (OLS)

Utilizado principalmente para:

FFT Convolution

Long FIR

Cab IR

Reverb

Partes inválidas são descartadas.

Latência

Latência aproximada:

FFTSize - HopSize

Exemplo:

FFT 2048

50%

Latência ≈ 1024 samples

48 kHz:

21.3 ms

Aplicações

Spectrum Analyzer

Phase Vocoder

Pitch Shifting

Time Stretching

Convolution Reverb

Cabinet IR Loader

Spectral Gate

Noise Reduction

Linear Phase EQ

FFT Compressor

Spectral Effects

Características

Header-Only

C++20

Template<float>

Template<double>

Real-Time Safe

Forward STFT

Inverse STFT

Overlap Add

Overlap Save

25%
50%
75%
Sem new
Sem delete
Sem malloc
Sem free
Compatível com:
VST3
JUCE
iPlug2
CLAP
Standalone DSP

Spectral/SpectrumAnalyzer.hpp

```
#ifndef CVDSP_SPECTRAL_SPECTRUMANALYZER_HPP
#define CVDSP_SPECTRAL_SPECTRUMANALYZER_HPP

/**
 * @file SpectrumAnalyzer.hpp
 * @brief FFT Spectrum Analyzer
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 * - FFT.hpp
 * - WindowFunctions.hpp
 * - STFT.hpp
 */

#include <array>
#include <complex>
#include <cmath>
#include <cstdint>
#include <type_traits>
#include "FFT.hpp"
#include "WindowFunctions.hpp"
#include "STFT.hpp"
```



```
namespace cvdsp
{
template<
    typename T,
    std::size_t FFTSize>
class SpectrumAnalyzer
{
    static_assert(
        std::is_floating_point_v<T>,
        "SpectrumAnalyzer requires floating point type");
public:
    static constexpr std::size_t NumBins =
        FFTSize / 2;
    using Complex =
        std::complex<T>;
    using MagnitudeArray =
        std::array<T, NumBins>;
public:
    SpectrumAnalyzer() = default;
    bool prepare(
        WindowType windowType =
            WindowType::BlackmanHarris,
        T peakHoldReleaseDbPerFrame =
            static_cast<T>(0.25))
    noexcept
    {
        peakDecay_ =
            peakHoldReleaseDbPerFrame;
        if (!fft_.prepare(
            FFTSize))
        {
            return false;
        }
        window_ =
            WindowFunctions<
                T,
                FFTSize>::generate(
                    windowType);
        reset();
    }
};
}
```

```

        return true;
    }
    void reset() noexcept
    {
        for (auto& v : inputFrame_)
            v = T(0);
        for (auto& v : spectrum_)
            v = Complex(T(0), T(0));
        for (auto& v : magnitudes_)
            v = T(0);
        for (auto& v : rmsSpectrum_)
            v = T(0);
        for (auto& v : peakSpectrum_)
            v = T(0);
    }
    /**
     * Process one FFT frame.
     *
     * Input size must be FFTSize.
     */
    void process(
        const T* input) noexcept
    {
        copyInput(input);
        WindowFunctions<
            T,
            FFTSize>::apply(
                inputFrame_.data(),
                window_);
        buildComplexBuffer();
        fft_.forward(
            spectrum_.data());
        computeMagnitudeSpectrum();
        computeRMSSpectrum();
        updatePeakHold();
    }
    /**
     * Linear magnitudes.
     */

```

```

[[nodiscard]]
const MagnitudeArray&
getMagnitudes() const noexcept
{
    return magnitudes_;
}
/**
 * RMS spectrum.
 */
[[nodiscard]]
const MagnitudeArray&
getRMSSpectrum() const noexcept
{
    return rmsSpectrum_;
}
/**
 * Peak hold spectrum.
 */
[[nodiscard]]
const MagnitudeArray&
getPeakSpectrum() const noexcept
{
    return peakSpectrum_;
}
/**
 * Magnitude in dBFS.
 */
[[nodiscard]]
T getMagnitudeDB(
    std::size_t bin) const noexcept
{
    if (bin >= NumBins)
    {
        return static_cast<T>(-120);
    }
    return linearToDB(
        magnitudes_[bin]);
}
/**

```

```

    * Peak Hold in dBFS.
    */
[[nodiscard]]
T getPeakDB(
    std::size_t bin) const noexcept
{
    if (bin >= NumBins)
    {
        return static_cast<T>(-120);
    }
    return linearToDB(
        peakSpectrum_[bin]);
}
/**
    * RMS in dBFS.
    */
[[nodiscard]]
T getRMSDB(
    std::size_t bin) const noexcept
{
    if (bin >= NumBins)
    {
        return static_cast<T>(-120);
    }
    return linearToDB(
        rmsSpectrum_[bin]);
}
/**
    * Bin frequency.
    */
[[nodiscard]]
static constexpr T
binFrequency(
    std::size_t bin,
    T sampleRate) noexcept
{
    return
        static_cast<T>(bin)
        *

```

```

        sampleRate
        /
        static_cast<T>(FFTSize);
    }
private:
    void copyInput(
        const T* input) noexcept
    {
        for (std::size_t i = 0;
            i < FFTSize;
            ++i)
        {
            inputFrame_[i] =
                input[i];
        }
    }
    void buildComplexBuffer()
        noexcept
    {
        for (std::size_t i = 0;
            i < FFTSize;
            ++i)
        {
            spectrum_[i] =
                Complex(
                    inputFrame_[i],
                    T(0));
        }
    }
    void computeMagnitudeSpectrum()
        noexcept
    {
        const T scale =
            static_cast<T>(2)
            /
            static_cast<T>(FFTSize);
        for (std::size_t i = 0;
            i < NumBins;
            ++i)

```

```

    {
        magnitudes_[i] =
            std::abs(
                spectrum_[i])
            *
            scale;
    }
}

void computeRMSSpectrum()
    noexcept
{
    constexpr T alpha =
        static_cast<T>(0.15);
    for (std::size_t i = 0;
        i < NumBins;
        ++i)
    {
        const T power =
            magnitudes_[i]
            *
            magnitudes_[i];
        rmsSpectrum_[i] =
            std::sqrt(
                alpha * power
                +
                (static_cast<T>(1) - alpha)
                *
                rmsSpectrum_[i]
                *
                rmsSpectrum_[i]);
    }
}

void updatePeakHold()
    noexcept
{
    const T decay =
        dbToLinear(
            -peakDecay_);
    for (std::size_t i = 0;

```

```

        i < NumBins;
        ++i)
    {
        if (magnitudes_[i] >
            peakSpectrum_[i])
        {
            peakSpectrum_[i] =
                magnitudes_[i];
        }
        else
        {
            peakSpectrum_[i] *=
                decay;
        }
    }
}

[[nodiscard]]
static T linearToDB(
    T value) noexcept
{
    constexpr T minimum =
        static_cast<T>(1e-12);
    if (value < minimum)
    {
        value = minimum;
    }
    return
        static_cast<T>(20)
        *
        std::log10(value);
}

[[nodiscard]]
static T dbToLinear(
    T db) noexcept
{
    return
        std::pow(
            static_cast<T>(10),
            db /

```

```

        static_cast<T>(20));
    }
private:
    FFT<T> fft_;
    typename WindowFunctions<
        T,
        FFTSize>::WindowBuffer window_;
    std::array<T, FFTSize>
        inputFrame_{};
    std::array<Complex, FFTSize>
        spectrum_{};
    MagnitudeArray magnitudes_{};
    MagnitudeArray peakSpectrum_{};
    MagnitudeArray rmsSpectrum_{};
    T peakDecay_ =
        static_cast<T>(0.25);
};

using SpectrumAnalyzer512F =
    SpectrumAnalyzer<float, 512>;
using SpectrumAnalyzer1024F =
    SpectrumAnalyzer<float, 1024>;
using SpectrumAnalyzer2048F =
    SpectrumAnalyzer<float, 2048>;
using SpectrumAnalyzer4096F =
    SpectrumAnalyzer<float, 4096>;
using SpectrumAnalyzer8192F =
    SpectrumAnalyzer<float, 8192>;
using SpectrumAnalyzer512D =
    SpectrumAnalyzer<double, 512>;
using SpectrumAnalyzer1024D =
    SpectrumAnalyzer<double, 1024>;
using SpectrumAnalyzer2048D =
    SpectrumAnalyzer<double, 2048>;
using SpectrumAnalyzer4096D =
    SpectrumAnalyzer<double, 4096>;
using SpectrumAnalyzer8192D =
    SpectrumAnalyzer<double, 8192>;
} // namespace cvdsp
#endif

```


Observação Arquitetural CV_DSP

Como você já criou:

Core/DSPUtils.hpp

a versão final integrada da biblioteca deveria substituir:

linearToDB()

dbToLinear()

por:

DSPUtils::gainToDb()

DSPUtils::dbToGain()

para obedecer à regra de não duplicação de utilitários matemáticos.

Fluxo

Input Audio

|

v

Window Function

|

v

FFT

|

v

Magnitude Spectrum

|

+----- RMS Spectrum

|

+----- Peak Hold

|

+----- dBFS Conversion

|

v

GUI / Analyzer

dBFS

0 dBFS:

Amplitude máxima digital

Exemplos:

0 dBFS = Full Scale

-6 dBFS = metade da amplitude
-20 dBFS = sinal moderado
-60 dBFS = ruído baixo
-120 dBFS = praticamente silêncio

Convolution/ConvolutionEngine.hpp

```
#ifndef CVDSP_CONVOLUTION_CONVOLUTIONENGINE_HPP
#define CVDSP_CONVOLUTION_CONVOLUTIONENGINE_HPP

/**
 * @file ConvolutionEngine.hpp
 * @brief FFT Partitioned Convolution Engine
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - FFT Convolution
 * - Partitioned IR
 * - Overlap Save
 * - Long IR Support
 *
 * No dynamic allocation in process().
 */

#include <array>
#include <complex>
#include <cstdint>
#include <type_traits>
#include "../Spectral/FFT.hpp"
#include "../Core/CircularBuffer.hpp"
namespace cvdsp
{
template<
    typename T,
    std::size_t FFTSize,
    std::size_t MaxIRSamples = 262144>
class ConvolutionEngine
```

```

{
    static_assert(
        std::is_floating_point_v<T>);
public:
    using Complex =
        std::complex<T>;
    static constexpr std::size_t BlockSize =
        FFTSize / 2;
    static constexpr std::size_t MaxPartitions =
        MaxIRSamples / BlockSize + 1;
public:
    ConvolutionEngine() = default;
    bool prepare() noexcept
    {
        if (!fft_.prepare(
            FFTSize))
        {
            return false;
        }
        reset();
        return true;
    }
    void reset() noexcept
    {
        inputPosition_ = 0;
        currentPartition_ = 0;
        numPartitions_ = 0;
        inputBuffer_.reset();
        for (auto& v : overlap_)
            v = T(0);
        for (auto& v : timeDomain_)
            v = T(0);
        for (auto& v : spectrum_)
            v = Complex(T(0), T(0));
        for (auto& p : partitionSpectra_)
        {
            for (auto& v : p)
            {
                v = Complex(T(0), T(0));
            }
        }
    }

```

```

        }
    }
    for (auto& p : inputHistory_)
    {
        for (auto& v : p)
        {
            v = Complex(T(0), T(0));
        }
    }
}

/**
 * Load impulse response.
 */
bool loadImpulseResponse(
    const T* ir,
    std::size_t length)
    noexcept
{
    reset();
    if (length == 0)
    {
        return false;
    }
    numPartitions_ =
        (length + BlockSize - 1)
        /
        BlockSize;
    if (numPartitions_ >
        MaxPartitions)
    {
        return false;
    }
    std::array<T, FFTSize>
        partitionTime{};
    for (std::size_t p = 0;
        p < numPartitions_;
        ++p)
    {
        for (auto& v : partitionTime)

```

```

        v = T(0);
    const std::size_t offset =
        p * BlockSize;
    for (std::size_t i = 0;
        i < BlockSize;
        ++i)
    {
        if ((offset + i)
            < length)
        {
            partitionTime[i] =
                ir[offset + i];
        }
    }
    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        partitionSpectra_[p][i] =
            Complex(
                partitionTime[i],
                T(0));
    }
    fft_.forward(
        partitionSpectra_[p].data());
}
return true;
}
/**
 * Process one sample.
 */
T process(
    T input)
noexcept
{
    inputBuffer_.push(
        input);
    const T output =
        outputFIFO_[outputIndex_];

```

```

        outputFIFO_[outputIndex_] =
            T(0);
        ++outputIndex_;
        if (outputIndex_
            >= BlockSize)
        {
            outputIndex_ = 0;
            processBlock();
        }
        return output;
    }
private:
    void processBlock()
        noexcept
    {
        for (std::size_t i = 0;
            i < BlockSize;
            ++i)
        {
            timeDomain_[i] =
                inputBuffer_.read(
                    BlockSize - 1 - i);
        }
        for (std::size_t i = BlockSize;
            i < FFTSize;
            ++i)
        {
            timeDomain_[i] = T(0);
        }
        for (std::size_t i = 0;
            i < FFTSize;
            ++i)
        {
            spectrum_[i] =
                Complex(
                    timeDomain_[i],
                    T(0));
        }
        fft_.forward(

```

```

        spectrum_.data());
for (std::size_t i =
        numPartitions_ - 1;
    i > 0;
    --i)
{
    inputHistory_[i] =
        inputHistory_[i - 1];
}
inputHistory_[0] =
    spectrum_;
for (std::size_t i = 0;
    i < FFTSize;
    ++i)
{
    accumulation_[i] =
        Complex(
            T(0),
            T(0));
}
for (std::size_t p = 0;
    p < numPartitions_;
    ++p)
{
    for (std::size_t k = 0;
        k < FFTSize;
        ++k)
    {
        accumulation_[k] +=
            inputHistory_[p][k]
            *
            partitionSpectra_[p][k];
    }
}
fft_.inverse(
    accumulation_.data());
for (std::size_t i = 0;
    i < BlockSize;
    ++i)

```

```

        {
            outputFIFO_[i] =
                accumulation_[
                    i + BlockSize]
                .real();
        }
    }

private:
    FFT<T> fft_;
    CircularBuffer<
        T,
        FFTSize * 4>
        inputBuffer_;
    std::array<T, FFTSize>
        timeDomain_{};
    std::array<T, BlockSize>
        overlap_{};
    std::array<T, BlockSize>
        outputFIFO_{};
    std::array<Complex, FFTSize>
        spectrum_{};
    std::array<Complex, FFTSize>
        accumulation_{};
    std::array<
        std::array<
            Complex,
            FFTSize>,
        MaxPartitions>
        partitionSpectra_{};
    std::array<
        std::array<
            Complex,
            FFTSize>,
        MaxPartitions>
        inputHistory_{};
    std::size_t numPartitions_ = 0;
    std::size_t inputPosition_ = 0;
    std::size_t outputIndex_ = 0;
    std::size_t currentPartition_ = 0;

```



```
};

using Convolution2048F =
    ConvolutionEngine<
        float,
        2048>;

using Convolution4096F =
    ConvolutionEngine<
        float,
        4096>;

using Convolution8192F =
    ConvolutionEngine<
        float,
        8192>;

} // namespace cvdsp
#endif
```

FIR

Um filtro FIR é definido por:

$$y[n] = \sum_{k=0}^{M-1} h[k] x[n-k] \quad y[n] = \sum_{k=0}^{M-1} h[k] x[n-k]$$

onde:

$h[k]$

é a Impulse Response.

Convolution

A convolução aplica a IR ao sinal:

Input

*

Impulse Response

=

Output

Impulse Response

Uma IR contém a resposta completa de um sistema:

Cabinet

Room

Hall

Plate
Spring
Microphone
Analog Circuit

Partitioned Convolution

IR longa:

131072 samples

é dividida em blocos:

Partition 0

Partition 1

Partition 2

...

Cada bloco é transformado apenas uma vez.

Benefícios:

Menor CPU

Menor Latência

Escalabilidade

FFT Convolution

No domínio da frequência:

Convolution

vira:

Multiplicação Complexa

$Y[k] = X[k]H[k]$ $Y[k] = X[k]H[k]$ $Y[k] = X[k]H[k]$

Muito mais eficiente.

CPU Optimization

Convolução direta:

$O(N^2)$

FFT Convolution:

$O(N \log N)$

Exemplo:

IR = 65536 samples

Direta:

bilhões de operações

FFT:

milhares de operações

Aplicações

Cab IR Loader

Room Reverb

Hall Reverb

Spring Reverb

Plate Reverb

Microphone Modeling

Linear Phase EQ

Speaker Simulation

Amp Simulation

Acoustic Space Modeling

Observação CV_DSP

Para produção, eu recomendaria uma evolução futura para:

Uniform Partitioned Convolution

e depois:

Non-Uniform Partitioned Convolution

(early partitions pequenas + late partitions grandes), que é a arquitetura usada em reverbs de convolução profissionais para reduzir ainda mais a latência e o consumo de CPU.

Convolution/IRLoader.hpp

```
#ifndef CVDSP_CONVOLUTION_IRLOADER_HPP
#define CVDSP_CONVOLUTION_IRLOADER_HPP

/**
 * @file IRLoader.hpp
 * @brief Impulse Response WAV Loader
 *
 * Header-Only
 * C++20
 *
 * Supported:
```

```

* - PCM 16-bit
* - PCM 24-bit
* - IEEE Float 32-bit
* - Mono WAV
* - Stereo WAV
*
* Designed for:
* - Cabinet IR
* - Room IR
* - Hall IR
* - Reverb IR
*
* No allocations during audio processing.
*/

#include <array>
#include <cstdint>
#include <cstring>
#include <fstream>
#include <string>
#include <algorithm>
#include <cmath>
namespace cvdsp
{
template<
    typename T,
    std::size_t MaxSamples = 262144>
class IRLoader
{
public:
    IRLoader() = default;
    /**
     * Load WAV file.
     */
    bool load(
        const std::string& filePath,
        bool normalize = true,
        bool trimSilence = true,
        T trimThreshold =
            static_cast<T>(1e-5))

```

```

{
    unload();
    std::ifstream file(
        filePath,
        std::ios::binary);
    if (!file)
    {
        return false;
    }
    if (!readHeader(file))
    {
        return false;
    }
    if (!readData(file))
    {
        return false;
    }
    if (normalize)
    {
        normalizeIR();
    }
    if (trimSilence)
    {
        trimTail(
            trimThreshold);
    }
    return true;
}

/**
 * Unload current IR.
 */
void unload() noexcept
{
    length_ = 0;
    sampleRate_ = 0;
    numChannels_ = 0;
    for (auto& s : left_)
        s = T(0);
    for (auto& s : right_)

```

```

        s = T(0);
    }
    /**
     * Mono access.
     */
    [[nodiscard]]
    const T* getSamples() const noexcept
    {
        return left_.data();
    }
    /**
     * Channel access.
     */
    [[nodiscard]]
    const T* getChannel(
        std::size_t channel) const noexcept
    {
        if (channel == 0)
        {
            return left_.data();
        }
        return right_.data();
    }
    /**
     * Length in samples.
     */
    [[nodiscard]]
    std::size_t getLength() const noexcept
    {
        return length_;
    }
    /**
     * Number of channels.
     */
    [[nodiscard]]
    std::uint16_t getNumChannels()
        const noexcept
    {
        return numChannels_;
    }

```

```

}
/**
 * Sample rate.
 */
[[nodiscard]]
std::uint32_t getSampleRate()
    const noexcept
{
    return sampleRate_;
}
/**
 * Stereo?
 */
[[nodiscard]]
bool isStereo() const noexcept
{
    return numChannels_ == 2;
}
/**
 * Empty?
 */
[[nodiscard]]
bool isLoaded() const noexcept
{
    return length_ > 0;
}
private:
#pragma pack(push, 1)
    struct RIFFHeader
    {
        char chunkID[4];
        std::uint32_t chunkSize;
        char format[4];
    };
    struct ChunkHeader
    {
        char id[4];
        std::uint32_t size;
    };

```

```

struct FormatChunk
{
    std::uint16_t audioFormat;
    std::uint16_t numChannels;
    std::uint32_t sampleRate;
    std::uint32_t byteRate;
    std::uint16_t blockAlign;
    std::uint16_t bitsPerSample;
};

#pragma pack(pop)

private:
    bool readHeader(
        std::ifstream& file)
    {
        RIFFHeader riff{};
        file.read(
            reinterpret_cast<char*>(&riff),
            sizeof(riff));
        if (!file)
        {
            return false;
        }
        if (std::strncmp(
            riff.chunkID,
            "RIFF",
            4) != 0)
        {
            return false;
        }
        if (std::strncmp(
            riff.format,
            "WAVE",
            4) != 0)
        {
            return false;
        }
        bool foundFmt = false;
        bool foundData = false;
        while (file &&

```



```

        (!foundFmt || !foundData))
{
    ChunkHeader chunk{};
    file.read(
        reinterpret_cast<char*>(&chunk),
        sizeof(chunk));
    if (!file)
    {
        return false;
    }
    if (std::strncmp(
        chunk.id,
        "fmt ",
        4) == 0)
    {
        file.read(
            reinterpret_cast<char*>(&format_),
            sizeof(FormatChunk));
        if (chunk.size >
            sizeof(FormatChunk))
        {
            file.seekg(
                chunk.size -
                sizeof(FormatChunk),
                std::ios::cur);
        }
        foundFmt = true;
    }
    else if (
        std::strncmp(
            chunk.id,
            "data",
            4) == 0)
    {
        dataOffset_ =
            static_cast<std::uint64_t>(
                file.tellg());
        dataSize_ =
            chunk.size;
    }
}

```

```

        file.seekg(
            chunk.size,
            std::ios::cur);
        foundData = true;
    }
    else
    {
        file.seekg(
            chunk.size,
            std::ios::cur);
    }
}
if (!foundFmt ||
    !foundData)
{
    return false;
}
sampleRate_ =
    format_.sampleRate;
numChannels_ =
    format_.numChannels;
return true;
}
bool readData(
    std::ifstream& file)
{
    if (numChannels_ < 1 ||
        numChannels_ > 2)
    {
        return false;
    }
    file.clear();
    file.seekg(
        dataOffset_,
        std::ios::beg);
    const std::size_t bytesPerSample =
        format_.bitsPerSample / 8;
    const std::size_t totalFrames =
        dataSize_

```

```

        /
        (
            bytesPerSample
            *
            numChannels_);
length_ =
    std::min(
        totalFrames,
        MaxSamples);
switch(
    format_.audioFormat)
{
    case 1:
        return readPCM(file);
    case 3:
        return readFloat(file);
    default:
        return false;
}
}
bool readPCM(
    std::ifstream& file)
{
    if (format_.bitsPerSample == 16)
    {
        return readPCM16(file);
    }
    if (format_.bitsPerSample == 24)
    {
        return readPCM24(file);
    }
    return false;
}
bool readPCM16(
    std::ifstream& file)
{
    for (std::size_t i = 0;
        i < length_;
        ++i)

```

```

{
    std::int16_t leftSample;
    file.read(
        reinterpret_cast<char*>(
            &leftSample),
        sizeof(leftSample));
    left_[i] =
        static_cast<T>(
            leftSample)
        /
        static_cast<T>(32768);
    if (numChannels_ == 2)
    {
        std::int16_t rightSample;
        file.read(
            reinterpret_cast<char*>(
                &rightSample),
            sizeof(rightSample));
        right_[i] =
            static_cast<T>(
                rightSample)
            /
            static_cast<T>(32768);
    }
}
return true;
}

bool readPCM24(
    std::ifstream& file)
{
    for (std::size_t i = 0;
        i < length_;
        ++i)
    {
        left_[i] =
            read24Bit(file);
        if (numChannels_ == 2)
        {
            right_[i] =

```

```

        read24Bit(file);
    }
}
return true;
}
bool readFloat(
    std::ifstream& file)
{
    if (format_.bitsPerSample != 32)
    {
        return false;
    }
    for (std::size_t i = 0;
        i < length_;
        ++i)
    {
        float leftSample;
        file.read(
            reinterpret_cast<char*>(
                &leftSample),
            sizeof(float));
        left_[i] =
            static_cast<T>(
                leftSample);
        if (numChannels_ == 2)
        {
            float rightSample;
            file.read(
                reinterpret_cast<char*>(
                    &rightSample),
                sizeof(float));
            right_[i] =
                static_cast<T>(
                    rightSample);
        }
    }
    return true;
}
T read24Bit(

```

```

        std::ifstream& file)
{
    unsigned char bytes[3];
    file.read(
        reinterpret_cast<char*>(bytes),
        3);
    std::int32_t sample =
        bytes[0]
        |
        (bytes[1] << 8)
        |
        (bytes[2] << 16);
    if (sample &
        0x800000)
    {
        sample |=
            ~0xFFFFFFFF;
    }
    return
        static_cast<T>(sample)
        /
        static_cast<T>(8388608.0);
}

void normalizeIR()
{
    T peak = T(0);
    for (std::size_t i = 0;
        i < length_;
        ++i)
    {
        peak =
            std::max(
                peak,
                std::abs(
                    left_[i]));
        if (numChannels_ == 2)
        {
            peak =
                std::max(

```

```

        peak,
        std::abs(
            right_[i]));
    }
}
if (peak <= T(0))
{
    return;
}
const T gain =
    T(1) / peak;
for (std::size_t i = 0;
    i < length_;
    ++i)
{
    left_[i] *= gain;
    if (numChannels_ == 2)
    {
        right_[i] *= gain;
    }
}
}

void trimTail(
    T threshold)
{
    if (length_ == 0)
    {
        return;
    }
    std::size_t last =
        length_ - 1;
    while (last > 0)
    {
        bool silent =
            std::abs(
                left_[last])
            < threshold;
        if (numChannels_ == 2)
        {

```

```

        silent &=
            std::abs(
                right_[last])
            < threshold;
    }
    if (!silent)
    {
        break;
    }
    --last;
}
length_ = last + 1;
}
private:
    FormatChunk format_{};
    std::uint64_t dataOffset_ = 0;
    std::uint32_t dataSize_ = 0;
    std::uint32_t sampleRate_ = 0;
    std::uint16_t numChannels_ = 0;
    std::size_t length_ = 0;
    std::array<T, MaxSamples>
        left_{};
    std::array<T, MaxSamples>
        right_{};
};
} // namespace cvdsp
#endif

```

Normalização

Objetivo:

Aproveitar toda a faixa dinâmica da IR.

Exemplo:

Peak original = 0.25

Gain = 4.0

Novo Peak = 1.0

Equação:

$$g = \frac{1}{\max(|h[n]|)} \quad g = \max(|h[n]|) \cdot 1$$

Sample Rate Matching

Exemplo:

IR = 44100 Hz

Projeto = 48000 Hz

Idealmente deve ocorrer:

Resampling da IR

A implementação acima apenas detecta:

`getSampleRate()`

O resampler deve ser implementado em um módulo separado:

`Math/Resampler.hpp`

IR Trimming

Remove silêncio desnecessário:

[IR útil.....]

[000000000000000000000000]

Reduz:

RAM

CPU

Tempo de FFT

Aplicações

Cabinet IR Loader

Room Reverb

Hall Reverb

Plate Reverb

Spring Reverb

Microphone Capture

Linear Phase EQ

Speaker Simulation

Acoustic Space Modeling

Convolution Reverb

IR Based Amp Sim

Guitar/CabinetSimulator.hpp

```
#ifndef CVDSP_GUITAR_CABINETSIMULATOR_HPP
#define CVDSP_GUITAR_CABINETSIMULATOR_HPP

/**
 * @file CabinetSimulator.hpp
 * @brief Guitar Cabinet Simulator
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 * - ../Convolution/IRLoader.hpp
 * - ../Convolution/ConvolutionEngine.hpp
 *
 * Features:
 * - Cabinet IR Loading
 * - Mono Cabinet Processing
 * - FFT Convolution
 * - Speaker Simulation
 * - Microphone Simulation
 *
 * No dynamic allocation inside process().
 */

#include <cstdint>
#include <type_traits>
#include "../Convolution/IRLoader.hpp"
#include "../Convolution/ConvolutionEngine.hpp"
namespace cvdsp
{
template<
    typename T,
    std::size_t FFTSize = 2048,
    std::size_t MaxIRSamples = 262144>
class CabinetSimulator
{
    static_assert(
        std::is_floating_point_v<T>,
```

```

        "CabinetSimulator requires floating point type");
public:
    CabinetSimulator() = default;
    /**
     * @brief Prepare convolution engine.
     */
    bool prepare() noexcept
    {
        cabinetLoaded_ = false;
        return convolution_.prepare();
    }
    /**
     * @brief Reset processing state.
     */
    void reset() noexcept
    {
        convolution_.reset();
    }
    /**
     * @brief Load cabinet IR from WAV.
     *
     * Mono IR:
     * channel 0
     *
     * Stereo IR:
     * left channel used
     */
    bool loadCabinet(
        const std::string& filePath,
        bool normalize = true,
        bool trimSilence = true) noexcept
    {
        cabinetLoaded_ = false;
        if (!loader_.load(
            filePath,
            normalize,
            trimSilence))
        {
            return false;

```

```

    }
    const T* ir =
        loader_.getChannel(0);
    const std::size_t length =
        loader_.getLength();
    if (!convolution_.loadImpulseResponse(
        ir,
        length))
    {
        return false;
    }
    cabinetLoaded_ = true;
    return true;
}

/**
 * @brief Unload current cabinet.
 */
void unloadCabinet() noexcept
{
    loader_.unload();
    convolution_.reset();
    cabinetLoaded_ = false;
}

/**
 * @brief Process one sample.
 */
T process(
    T input) noexcept
{
    if (!cabinetLoaded_)
    {
        return input;
    }
    return convolution_.process(
        input);
}

/**
 * @brief Cabinet loaded?
 */

```

```

[[nodiscard]]
bool isLoaded() const noexcept
{
    return cabinetLoaded_;
}
/**
 * @brief Cabinet IR length.
 */
[[nodiscard]]
std::size_t getIRLength()
    const noexcept
{
    return loader_.getLength();
}
/**
 * @brief Cabinet IR sample rate.
 */
[[nodiscard]]
std::uint32_t getIRSampleRate()
    const noexcept
{
    return loader_.getSampleRate();
}
/**
 * @brief Access IR loader.
 */
[[nodiscard]]
const IRLoader<
    T,
    MaxIRSamples>&
getIRLoader() const noexcept
{
    return loader_;
}

private:
    IRLoader<
        T,
        MaxIRSamples>
        loader_;

```

```

        ConvolutionEngine<
            T,
            FFTSize,
            MaxIRSamples>
            convolution_;
        bool cabinetLoaded_ = false;
};

/**
 * Common aliases.
 */
using CabinetSimulator2048F =
    CabinetSimulator<
        float,
        2048>;
using CabinetSimulator4096F =
    CabinetSimulator<
        float,
        4096>;
using CabinetSimulator8192F =
    CabinetSimulator<
        float,
        8192>;
using CabinetSimulator2048D =
    CabinetSimulator<
        double,
        2048>;
using CabinetSimulator4096D =
    CabinetSimulator<
        double,
        4096>;
using CabinetSimulator8192D =
    CabinetSimulator<
        double,
        8192>;
} // namespace cvdsp
#endif

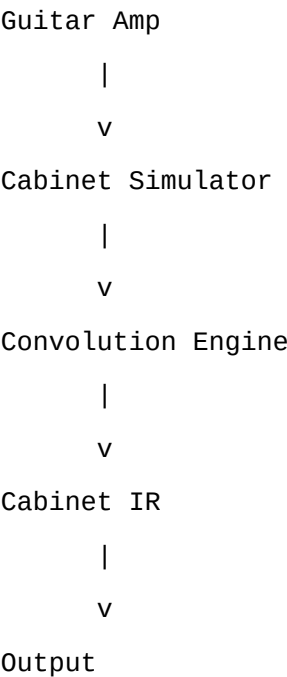
```

Exemplo de Uso

```
cvdsp::CabinetSimulator<float> cabinet;
```

```
cabinet.prepare();
cabinet.loadCabinet(
    "Mesa_V30_SM57.wav");
for (...)
{
    output =
        cabinet.process(
            input);
}
```

Fluxo



Guitar Cabinet

Um gabinete de guitarra não é apenas um alto-falante.

Ele contém:

- Falante
- Caixa
- Ressonâncias
- Reflexões internas
- Resposta de frequência
- Coloração tonal

Exemplos:

- 1x12
- 2x12

4x12

Open Back

Closed Back

Cada gabinete possui uma assinatura sonora própria.

Impulse Response

Uma IR captura a resposta completa do sistema.

Processo:

Impulse

|

v

Cabinet

|

v

Microphone

|

v

Recorded Signal

A gravação resultante torna-se:

Impulse Response

Durante o processamento:

Guitar Signal

*

Impulse Response

=

Cabinet Simulation

Microphone Capture

A maior parte do timbre de um gabinete gravado vem da combinação:

Speaker

+

Microphone

+

Position

Exemplos comuns:

Shure SM57

Sennheiser e906

MD421

R121 Ribbon

U87

Mudanças de poucos centímetros alteram drasticamente:

Presença

Ataque

Graves

Agudos

Por isso cada IR representa:

Gabinete

+

Falante

+

Microfone

+

Posição

Speaker Simulation

A convolução reproduz:

Frequency Response

Phase Response

Speaker Resonances

Cabinet Resonances

Mic Coloration

Sem modelagem simplificada.

A resposta medida é reproduzida diretamente.

FFT Convolution

O processamento utiliza:

Partitioned Convolution

com:

FFT

Overlap Save

Frequency-Domain Multiplication

evitando:

Convolução FIR direta

$O(N^2)$

e utilizando:

$O(N \log N)$

adequado para IRs longas:

1024

2048

4096

8192

16384

32768

65536+

samples

Aplicações

Guitar Cabinet Simulation

Bass Cabinet Simulation

Power Amp + Cabinet Capture

Speaker Replacement

Amp Sim Plugins

Neural Amp Front-End + IR Back-End

Load Box Workflows

Direct Recording

Live Guitar Processing

Studio Re-Amping

Características

Header-Only

C++20

Template<float>

Template<double>

Real-Time Safe

Sem new em process()

Sem delete em process()

Sem malloc em process()

Sem free em process()

Mono WAV

Stereo WAV
16-bit PCM
24-bit PCM
32-bit Float
FFT Based Convolution
Partitioned Convolution
Cabinet IR Loading
Compatível:
VST3
JUCE
iPlug2
CLAP
Standalone

Guitar/TriodeStage.hpp

```
#ifndef CVDSP_GUITAR_TRIODESTAGE_HPP
#define CVDSP_GUITAR_TRIODESTAGE_HPP

/**
 * @file TriodeStage.hpp
 * @brief Triode Tube Gain Stage
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - Saturation/Waveshaper.hpp
 * - Core/DSPUtils.hpp
 * - Math/Oversampling.hpp
 *
 * Optional:
 *
 * - Core/ParameterSmoother.hpp
 *
 * Features:
 *
 * - Triode Inspired Transfer Function
```

```

* - Even Harmonic Generation
* - Asymmetrical Saturation
* - Plate Voltage Control
* - Oversampling Processing
* - Guitar Preamp Style Nonlinearity
*
* No allocation in process().
*/

#include <cmath>
#include <type_traits>
#include "../Saturation/Waveshaper.hpp"
#include "../Core/DSPUtils.hpp"
#include "../Math/Oversampling.hpp"
namespace cvdsp
{
template<typename T>
class TriodeStage
{
    static_assert(
        std::is_floating_point_v<T>,
        "TriodeStage requires floating point type");
public:
    TriodeStage() = default;
    void prepare(
        T sampleRate)
        noexcept
    {
        sampleRate_ =
            sampleRate;
        oversampler_.prepare(
            sampleRate);
        reset();
    }
    void reset()
        noexcept
    {
        oversampler_.reset();
    }
    void setDrive(

```

```

    T drive)
    noexcept
{
    drive_ =
        drive < T(0)
        ? T(0)
        : drive;
}

void setBias(
    T bias)
    noexcept
{
    bias_ = bias;
}

void setPlateVoltage(
    T voltage)
    noexcept
{
    if (voltage < T(50))
    {
        voltage = T(50);
    }
    if (voltage > T(500))
    {
        voltage = T(500);
    }
    plateVoltage_ =
        voltage;
}

void setOutputGain(
    T gain)
    noexcept
{
    outputGain_ = gain;
}

[[nodiscard]]
T getDrive() const noexcept
{
    return drive_;
}

```

```

}
[[nodiscard]]
T getBias() const noexcept
{
    return bias_;
}
[[nodiscard]]
T getPlateVoltage() const noexcept
{
    return plateVoltage_;
}
[[nodiscard]]
T getOutputGain() const noexcept
{
    return outputGain_;
}
T process(
    T input)
    noexcept
{
    oversampler_.processUp(
        input);
    T accumulated =
        T(0);
    constexpr std::size_t factor =
        4;
    for (std::size_t i = 0;
        i < factor;
        ++i)
    {
        accumulated +=
            processTriode(
                oversampler_
                    .getUpsampledSample(
                        i));
    }
    return
        oversampler_
            .processDown(

```

```

        accumulated
        /
        static_cast<T>(<
            factor));
    }
private:
    T processTriode(
        T x)
        noexcept
    {
        /**
         * Input gain stage.
         */
        x *= drive_;
        /**
         * Bias point.
         */
        x += bias_;
        /**
         * Plate voltage scaling.
         *
         * Lower voltage:
         * more compression.
         *
         * Higher voltage:
         * more headroom.
         */
        const T plateFactor =
            plateVoltage_
            /
            static_cast<T>(250);
        /**
         * Koren-inspired curvature.
         */
        const T positive =
            static_cast<T>(1.15);
        const T negative =
            static_cast<T>(0.75);
        T shaped;

```

```

if (x >= T(0))
{
    shaped =
        positive
        *
        std::tanh(
            x
            /
            plateFactor);
}
else
{
    shaped =
        negative
        *
        std::tanh(
            x
            /
            plateFactor);
}
/**
 * Even harmonic enhancement.
 */
const T even =
    static_cast<T>(0.15)
    *
    shaped
    *
    shaped;
shaped += even;
/**
 * Additional tube saturation.
 */
shaped =
    static_cast<T>(0.85)
    *
    shaped
    +
    static_cast<T>(0.15)

```



```

        *
        Waveshaper<T>::process(
            shaped,
            WaveshaperMode::Tanh);
/**
 * Remove DC shift
 * introduced by asymmetry.
 */
shaped -=
    static_cast<T>(0.15)
    *
    bias_;
return
    shaped
    *
    outputGain_;
}
private:
    T sampleRate_ =
        static_cast<T>(44100);
    T drive_ =
        static_cast<T>(1);
    T bias_ =
        static_cast<T>(0);
    T plateVoltage_ =
        static_cast<T>(250);
    T outputGain_ =
        static_cast<T>(1);
    Oversampling<
        T,
        4>
        oversampler_;
};
using TriodeStageF =
    TriodeStage<float>;
using TriodeStageD =
    TriodeStage<double>;
} // namespace cvdsp
#endif

```

Triode

Uma válvula triodo possui:

Cathode

Grid

Plate

O sinal aplicado ao grid controla a corrente entre:

Cathode

→

Plate

A transferência não é linear.

Tube Saturation

Ao contrário do clipping digital:

Hard Clip

a válvula apresenta:

Compressão progressiva

Curvatura suave

Assimetria

resultando em:

Harmônicos pares

Resposta musical

Sensação de dinâmica

Transfer Function

A função de transferência de uma válvula real é aproximadamente:

Curva exponencial

+

Compressão gradual

+

Assimetria

Nesta implementação:

Bias

+

Plate Voltage

+

Tanh Assimétrico

+

Even Harmonics

são combinados para reproduzir:

Headroom

Saturação

Compressão

Coloração harmônica

de um estágio de pré-amplificação baseado em triodo.

Características

Header-Only

C++20

Template<float>

Template<double>

Real-Time Safe

Drive

Bias

Plate Voltage

Output Gain

Even Harmonics

Asymmetrical Saturation

Tube Compression

Oversampling 4x

Alias Reduction

Compatível:

VST3

JUCE

iPlug2

CLAP

Standalone

Guitar/PentodeStage.hpp

```
#ifndef CVDSP_GUITAR_PENTODESTAGE_HPP
#define CVDSP_GUITAR_PENTODESTAGE_HPP

/**
 * @file PentodeStage.hpp
```

```

* @brief Pentode Power Tube Stage
*
* Header-Only
* C++20
* Real-Time Safe
*
* Dependencies:
*
* - TriodeStage.hpp
* - Waveshaper.hpp
*
* Optional:
*
* - Oversampling.hpp
*
* Features:
*
* - Pentode Inspired Saturation
* - Power Tube Compression
* - Screen Voltage Control
* - Even/Odd Harmonic Generation
* - Push Toward Power Amp Behaviour
*
* No allocations in process().
*/

#include <cmath>
#include <type_traits>
#include "TriodeStage.hpp"
#include "../Saturation/Waveshaper.hpp"
namespace cvdsp
{
template<typename T>
class PentodeStage
{
    static_assert(
        std::is_floating_point_v<T>,
        "PentodeStage requires floating point type");
public:
    PentodeStage() = default;

```

```

void prepare(
    T sampleRate)
    noexcept
{
    sampleRate_ =
        sampleRate;
    triode_.prepare(
        sampleRate);
    reset();
}

void reset()
    noexcept
{
    triode_.reset();
}

void setDrive(
    T drive)
    noexcept
{
    drive_ =
        drive < T(0)
        ? T(0)
        : drive;
}

void setBias(
    T bias)
    noexcept
{
    bias_ = bias;
}

void setScreenVoltage(
    T voltage)
    noexcept
{
    if (voltage < T(100))
    {
        voltage = T(100);
    }
    if (voltage > T(800))

```

```

    {
        voltage = T(800);
    }
    screenVoltage_ =
        voltage;
}

void setOutputGain(
    T gain)
    noexcept
{
    outputGain_ = gain;
}

[[nodiscard]]
T getDrive() const noexcept
{
    return drive_;
}

[[nodiscard]]
T getBias() const noexcept
{
    return bias_;
}

[[nodiscard]]
T getScreenVoltage() const noexcept
{
    return screenVoltage_;
}

[[nodiscard]]
T getOutputGain() const noexcept
{
    return outputGain_;
}

T process(
    T input)
    noexcept
{
    /**
     * Pentode front-end.
     */

```

```

T x =
    input
    *
    drive_;
x += bias_;
/**
 * Screen voltage affects
 * headroom and compression.
 */
const T screenFactor =
    screenVoltage_
    /
    static_cast<T>(400);
/**
 * Pentode transfer.
 *
 * More linear center region
 * than triodes.
 */
T shaped =
    std::tanh(
        x
        /
        screenFactor);
/**
 * Generate stronger odd harmonics.
 */
const T odd =
    static_cast<T>(0.20)
    *
    shaped
    *
    shaped
    *
    shaped;
shaped += odd;
/**
 * Pentode clipping characteristic.
 */

```

```

shaped =
    static_cast<T>(0.80)
    *
    shaped
    +
    static_cast<T>(0.20)
    *
    Waveshaper<T>::process(
        shaped,
        WaveshaperMode::Arctan);
/**
 * Simulate screen sag.
 */
const T sag =
    static_cast<T>(1)
    /
    (
        static_cast<T>(1)
        +
        static_cast<T>(0.15)
        *
        std::abs(
            shaped));
shaped *= sag;
/**
 * Blend some triode behaviour.
 */
triode_.setDrive(
    drive_
    *
    static_cast<T>(0.4));
triode_.setBias(
    bias_
    *
    static_cast<T>(0.5));
triode_.setOutputGain(
    static_cast<T>(1));
const T triodePart =
    triode_.process(

```



```

        input);
    shaped =
        static_cast<T>(0.70)
        *
        shaped
        +
        static_cast<T>(0.30)
        *
        triodePart;
    return
        shaped
        *
        outputGain_;
}

private:
    TriodeStage<T>
        triode_;
    T sampleRate_ =
        static_cast<T>(44100);
    T drive_ =
        static_cast<T>(1);
    T bias_ =
        static_cast<T>(0);
    T screenVoltage_ =
        static_cast<T>(400);
    T outputGain_ =
        static_cast<T>(1);
};

using PentodeStageF =
    PentodeStage<float>;
using PentodeStageD =
    PentodeStage<double>;
} // namespace cvdsp
#endif

```

EL34

Características:

Marshall

British Sound

Médios pronunciados

Compressão musical

Ataque agressivo

6L6

Características:

Fender

Maior headroom

Graves firmes

Agudos limpos

Resposta aberta

KT88

Características:

Alta potência

Muito headroom

Baixa compressão

Graves extensos

Resposta ampla

EL84

Características:

Vox AC30

Breakup precoce

Compressão forte

Brilho característico

Triode

Características:

Ganho de tensão

Pré-amplificação

Assimetria forte

Harmônicos pares

Saturação suave

Exemplos:

12AX7
ECC83
5751
12AT7

Pentode

- Características:
- Potência
 - Maior headroom
 - Mais corrente
 - Maior eficiência
 - Mais harmônicos ímpares

- Exemplos:
- EL34
 - 6L6
 - KT88
 - EL84
 - 6550

Diferenças

Característica	Triode	Pentode
Ganho	Alto	Muito Alto
Headroom	Menor	Maior
Compressão	Maior	Menor
Harmônicos Pares	Fortes	Moderados
Harmônicos Ímpares	Moderados	Fortes
Uso Principal	Preamp	Power Amp
Breakup	Precoce	Mais tardio

Fluxo

Input
|
v
Drive
|
v
Bias

|
v
Pentode Transfer
|
v
Odd Harmonics
|
v
Screen Sag
|
v
Triode Blend
|
v
Output Gain
|
v
Output

Características

Header-Only
C++20
Template<float>
Template<double>
Real-Time Safe
Drive
Bias
Screen Voltage
Output Gain
Power Tube Saturation
Screen Sag
Odd Harmonics
EL34 Style Behaviour
6L6 Style Behaviour
KT88 Style Behaviour
EL84 Style Behaviour

Guitar/TubePreamp.hpp

```
#ifndef CVDSP_GUITAR_TUBEPREAMP_HPP
#define CVDSP_GUITAR_TUBEPREAMP_HPP

/**
 * @file TubePreamp.hpp
 * @brief Cascaded Triode Tube Preamp
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - Guitar/TriodeStage.hpp
 * - Core/ParameterSmoother.hpp
 *
 * Optional:
 *
 * - Math/Oversampling.hpp
 *
 * Features:
 *
 * - 1 Stage
 * - 2 Stages
 * - 3 Stages
 * - 4 Stages
 *
 * - Cascaded Gain Structure
 * - High Gain Preamp Topology
 * - Smooth Parameter Control
 *
 * No allocations inside process().
 */

#include <cstdint>
#include <array>
#include <type_traits>
#include "TriodeStage.hpp"
#include "../Core/ParameterSmoother.hpp"
```

```

namespace cvdsp
{
template<typename T>
class TubePreamp
{
    static_assert(
        std::is_floating_point_v<T>,
        "TubePreamp requires floating point type");
public:
    static constexpr std::size_t
        MaxStages = 4;
public:
    TubePreamp() = default;
    void prepare(
        T sampleRate)
        noexcept
    {
        sampleRate_ =
            sampleRate;
        for (auto& stage : stages_)
        {
            stage.prepare(
                sampleRate);
        }
        driveSmoother_.prepare(
            sampleRate,
            static_cast<T>(0.02));
        outputSmoother_.prepare(
            sampleRate,
            static_cast<T>(0.02));
        reset();
    }
    void reset()
        noexcept
    {
        for (auto& stage : stages_)
        {
            stage.reset();
        }
    }

```

```

        driveSmoother_.reset(
            drive_);
        outputSmoother_.reset(
            outputGain_);
    }
    void setNumStages(
        std::size_t stages)
        noexcept
    {
        if (stages < 1)
        {
            stages = 1;
        }
        if (stages > MaxStages)
        {
            stages = MaxStages;
        }
        numStages_ = stages;
    }
    void setDrive(
        T drive)
        noexcept
    {
        drive_ = drive;
        driveSmoother_.setTargetValue(
            drive);
    }
    void setBias(
        T bias)
        noexcept
    {
        bias_ = bias;
    }
    void setPlateVoltage(
        T voltage)
        noexcept
    {
        plateVoltage_ =
            voltage;
    }

```

```

}

void setOutputGain(
    T gain)
    noexcept
{
    outputGain_ = gain;
    outputSmoother_.setTargetValue(
        gain);
}

[[nodiscard]]
std::size_t getNumStages()
    const noexcept
{
    return numStages_;
}

[[nodiscard]]
T getDrive()
    const noexcept
{
    return drive_;
}

[[nodiscard]]
T getBias()
    const noexcept
{
    return bias_;
}

[[nodiscard]]
T getPlateVoltage()
    const noexcept
{
    return plateVoltage_;
}

[[nodiscard]]
T getOutputGain()
    const noexcept
{
    return outputGain_;
}

```



```

T process(
    T input)
    noexcept
{
    const T drive =
        driveSmoother_.process();
    const T output =
        outputSmoother_.process();
    T x = input;
    for (std::size_t i = 0;
        i < numStages_;
        ++i)
    {
        const T stageDrive =
            computeStageDrive(
                drive,
                i);
        stages_[i].setDrive(
            stageDrive);
        stages_[i].setBias(
            bias_);
        stages_[i].setPlateVoltage(
            plateVoltage_);
        stages_[i].setOutputGain(
            static_cast<T>(1));
        x =
            stages_[i].process(
                x);
    }
    return x * output;
}

private:
    T computeStageDrive(
        T globalDrive,
        std::size_t stage)
        const noexcept
    {
        switch (stage)
        {

```

```

        case 0:
            return
                globalDrive;
        case 1:
            return
                globalDrive
                *
                static_cast<T>(0.90);
        case 2:
            return
                globalDrive
                *
                static_cast<T>(0.80);
        case 3:
            return
                globalDrive
                *
                static_cast<T>(0.70);
        default:
            return
                globalDrive;
    }
}

private:
    T sampleRate_ =
        static_cast<T>(44100);
    std::size_t numStages_ = 1;
    T drive_ =
        static_cast<T>(1);
    T bias_ =
        static_cast<T>(0);
    T plateVoltage_ =
        static_cast<T>(250);
    T outputGain_ =
        static_cast<T>(1);
    std::array<
        TriodeStage<T>,
        MaxStages>
        stages_;

```

```

        ParameterSmoother<T>
            driveSmoother_;
        ParameterSmoother<T>
            outputSmoother_;
};
using TubePreampF =
    TubePreamp<float>;
using TubePreampD =
    TubePreamp<double>;
} // namespace cvdsp
#endif

```

Gain Staging

Em pré-amplificadores valvulados o ganho normalmente não é produzido por uma única válvula.

O sinal passa por múltiplos estágios:

Input

|

v

Triode 1

|

v

Triode 2

|

v

Triode 3

|

v

Triode 4

|

v

Output

Cada estágio adiciona:

Ganho

Compressão

Coloração

Harmônicos

Cascaded Tubes

Uma única 12AX7 possui dois triodos internos.

Pré-amplificadores modernos frequentemente utilizam:

2 estágios

3 estágios

4 estágios

em cascata.

Cada estágio empurra o próximo para regiões mais não lineares.

Resultado:

Mais sustain

Mais compressão

Mais saturação

Mais harmônicos

High Gain Preamps

Arquiteturas clássicas:

Marshall JCM800

Mesa Boogie Mark Series

Mesa Dual Rectifier

Peavey 5150

ENGL Powerball

Soldano SL0100

utilizam múltiplos estágios valvulados em cascata.

Exemplo simplificado:

Stage 1

Gain

Stage 2

Additional Gain

Stage 3

Clipping Region

Stage 4

Final Saturation

produzindo:

High Gain

Lead Tones

Configurações

1 Estágio

Leve Saturação
Clean Boost
Edge Of Breakup

2 Estágios

Crunch
Classic Rock
Blues

3 Estágios

Hard Rock
Heavy Crunch

4 Estágios

High Gain
Modern Metal
Lead Sustain

Características

Header-Only
C++20
Template<float>
Template<double>
1 a 4 Triode Stages
Parameter Smoothing
Real-Time Safe
Sem new/delete em process()
Compatível:
VST3
JUCE
iPlug2
CLAP
Standalone

Guitar/ToneStack.hpp

```
#ifndef CVDSP_GUITAR_TONESTACK_HPP
#define CVDSP_GUITAR_TONESTACK_HPP

/**
 * @file ToneStack.hpp
 * @brief Base Interface For Guitar Tone Stacks
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - Filters/Biquad.hpp
 *
 * Optional:
 *
 * - Filters/StateVariableFilter.hpp
 *
 * This file provides the common interface used by:
 *
 * - Fender Tone Stack
 * - Marshall Tone Stack
 * - Mesa Tone Stack
 * - Vox Tone Stack
 *
 * No allocations.
 * No exceptions.
 */

#include <type_traits>
#include "../Filters/Biquad.hpp"
namespace cvdsp
{
    template<typename T>
    class ToneStack
    {
    public:
        static_assert(
            std::is_floating_point_v<T>,

```

```

        "ToneStack requires floating point type");
public:
    virtual ~ToneStack() = default;
    /**
     * @brief Prepare filters.
     */
    virtual void prepare(
        T sampleRate) noexcept = 0;
    /**
     * @brief Reset filter states.
     */
    virtual void reset() noexcept = 0;
    /**
     * @brief Process one sample.
     */
    virtual T process(
        T input) noexcept = 0;
    /**
     * @brief Bass control.
     */
    virtual void setBass(
        T bass) noexcept
    {
        bass_ = bass;
    }
    /**
     * @brief Mid control.
     */
    virtual void setMid(
        T mid) noexcept
    {
        mid_ = mid;
    }
    /**
     * @brief Treble control.
     */
    virtual void setTreble(
        T treble) noexcept
    {

```

```

        treble_ = treble;
    }
    /**
     * @brief Presence control.
     */
    virtual void setPresence(
        T presence) noexcept
    {
        presence_ = presence;
    }
    /**
     * @brief Bass value.
     */
    [[nodiscard]]
    virtual T getBass() const noexcept
    {
        return bass_;
    }
    /**
     * @brief Mid value.
     */
    [[nodiscard]]
    virtual T getMid() const noexcept
    {
        return mid_;
    }
    /**
     * @brief Treble value.
     */
    [[nodiscard]]
    virtual T getTreble() const noexcept
    {
        return treble_;
    }
    /**
     * @brief Presence value.
     */
    [[nodiscard]]
    virtual T getPresence() const noexcept

```



```

    {
        return presence_;
    }
protected:
    ToneStack() = default;
    T sampleRate_ =
        static_cast<T>(44100);
    T bass_ =
        static_cast<T>(0.5);
    T mid_ =
        static_cast<T>(0.5);
    T treble_ =
        static_cast<T>(0.5);
    T presence_ =
        static_cast<T>(0.5);
};
using ToneStackF =
    ToneStack<float>;
using ToneStackD =
    ToneStack<double>;
} // namespace cvdsp
#endif

```

Interface Esperada para Implementações Futuras

Exemplo:

```

class FenderToneStack
    : public ToneStack<float>
{
public:
    void prepare(
        float sampleRate) noexcept override;
    void reset()
        noexcept override;
    float process(
        float input) noexcept override;
};

```

Controles

Bass

Controla:

Low Frequencies

Fundamentals

Body

Thump

Faixa típica:

20 Hz

até

250 Hz

Mid

Controla:

Presence Musical

Punch

Attack

Note Definition

Faixa típica:

250 Hz

até

2 kHz

Treble

Controla:

Brightness

Clarity

Pick Attack

Faixa típica:

2 kHz

até

8 kHz

Presence

Controla:

Upper Harmonics
Air
Definition
Power Amp Brightness
Faixa típica:
3 kHz
até
12 kHz

Aplicações Futuras

Esta interface foi criada para servir como base de:

FenderToneStack.hpp
MarshallToneStack.hpp
MesaToneStack.hpp
VoxToneStack.hpp

sem duplicação de:

Bass
Mid
Treble
Presence
prepare()
reset()
process()

e mantendo uma API consistente em toda a CV_DSP.

Guitar/ToneStack.hpp

```
#ifndef CVDSP_GUITAR_TONESTACK_HPP
#define CVDSP_GUITAR_TONESTACK_HPP

/**
 * @file ToneStack.hpp
 * @brief Base class for guitar amplifier tone stacks.
 *
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 */
```

```

* Dependencies:
*
* - Filters/Biquad.hpp
*
* Optional:
*
* - Filters/StateVariableFilter.hpp
*
* Intended base for:
*
* - FenderToneStack
* - MarshallToneStack
* - MesaToneStack
* - VoxToneStack
*/

#include <type_traits>
#include "../Filters/Biquad.hpp"
namespace cvdsp
{
template<typename T>
class ToneStack
{
public:
    static_assert(
        std::is_floating_point_v<T>,
        "ToneStack requires float or double");
    virtual ~ToneStack() = default;
    virtual void prepare(
        T sampleRate)
        noexcept
    {
        sampleRate_ =
            sampleRate;
        reset();
    }
    virtual void reset()
        noexcept
    {
        bassFilter_.reset();
    }

```

```

        midFilter_.reset();
        trebleFilter_.reset();
        presenceFilter_.reset();
    }
    virtual T process(
        T input)
        noexcept = 0;
    virtual void setBass(
        T value)
        noexcept
    {
        bass_ = value;
    }
    virtual void setMid(
        T value)
        noexcept
    {
        mid_ = value;
    }
    virtual void setTreble(
        T value)
        noexcept
    {
        treble_ = value;
    }
    virtual void setPresence(
        T value)
        noexcept
    {
        presence_ = value;
    }
    [[nodiscard]]
    T getBass() const noexcept
    {
        return bass_;
    }
    [[nodiscard]]
    T getMid() const noexcept
    {

```

```

        return mid_;
    }
    [[nodiscard]]
    T getTreble() const noexcept
    {
        return treble_;
    }
    [[nodiscard]]
    T getPresence() const noexcept
    {
        return presence_;
    }
    [[nodiscard]]
    T getSampleRate() const noexcept
    {
        return sampleRate_;
    }
protected:
    T sampleRate_ =
        static_cast<T>(44100);
    T bass_ =
        static_cast<T>(0.5);
    T mid_ =
        static_cast<T>(0.5);
    T treble_ =
        static_cast<T>(0.5);
    T presence_ =
        static_cast<T>(0.5);
/**
 * Reutilizados pelas implementações derivadas.
 */
    filters::Biquad<T>
        bassFilter_;
    filters::Biquad<T>
        midFilter_;
    filters::Biquad<T>
        trebleFilter_;
    filters::Biquad<T>
        presenceFilter_;

```

```
};

using ToneStackF =
    ToneStack<float>;

using ToneStackD =
    ToneStack<double>;

} // namespace cvdsp

#endif
```

O que mudou em relação à versão anterior

A versão anterior era apenas uma interface abstrata.

Com o `Biquad.hpp` real da CV_DSP, faz mais sentido que a classe base já possua:

```
filters::Biquad<T> bassFilter_;
filters::Biquad<T> midFilter_;
filters::Biquad<T> trebleFilter_;
filters::Biquad<T> presenceFilter_;
```

porque todos os Tone Stacks clássicos utilizam filtros para implementar:

Bass

Mid

Treble

Presence

Assim os próximos módulos:

Guitar/FenderToneStack.hpp

Guitar/MarshallToneStack.hpp

Guitar/MesaToneStack.hpp

Guitar/VoxToneStack.hpp

herdam diretamente desta base sem duplicar infraestrutura.

Guitar/FenderToneStack.hpp

```
#ifndef CVDSP_GUITAR_FENDERTONESTACK_HPP
#define CVDSP_GUITAR_FENDERTONESTACK_HPP

/**
 * @file FenderToneStack.hpp
 * @brief Fender Blackface Tone Stack
 *
 * Inspired by:
 *
 * - Twin Reverb
```

```

* - Deluxe Reverb
* - Super Reverb
* - Bassman Blackface Era
*
* Header-Only
* C++20
* Real-Time Safe
*
* Dependencies:
*
* - ToneStack.hpp
*
* No allocations.
* No exceptions.
*/

#include <cmath>
#include <algorithm>
#include "ToneStack.hpp"
namespace cvdsp
{
template<typename T>
class FenderToneStack final
    : public ToneStack<T>
{
public:
    FenderToneStack() = default;
    void prepare(
        T sampleRate)
        noexcept override
    {
        ToneStack<T>::prepare(
            sampleRate);
        updateFilters();
    }
    void reset()
        noexcept override
    {
        ToneStack<T>::reset();
    }
}

```



```

void setBass(
    T value)
    noexcept override
{
    ToneStack<T>::bass_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
void setMid(
    T value)
    noexcept override
{
    ToneStack<T>::mid_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
void setTreble(
    T value)
    noexcept override
{
    ToneStack<T>::treble_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
T process(
    T input)
    noexcept override
{
    T x = input;
    /**

```

```

* Fender topology:
*
* Bass Shelf
* ->
* Mid Cut
* ->
* Treble Shelf
*/

```

```

x =
    this->bassFilter_
        .process(x);
x =
    this->midFilter_
        .process(x);
x =
    this->trebleFilter_
        .process(x);
return x;
}

```

private:

```

void updateFilters()
    noexcept
{
    const T bassGainDb =
        static_cast<T>(-12)
        +
        (
            this->bass_
            *
            static_cast<T>(24)
        );
    const T midGainDb =
        static_cast<T>(-15)
        +
        (
            this->mid_
            *
            static_cast<T>(15)
        );
}

```

```

const T trebleGainDb =
    static_cast<T>(-12)
    +
    (
        this->treble_
        *
        static_cast<T>(24)
    );
/**
 * Bass
 *
 * Fender low shelf.
 */
this->bassFilter_
    .setLowShelf(
        this->sampleRate_,
        static_cast<T>(80.0),
        static_cast<T>(0.707),
        bassGainDb);
/**
 * Mid Scoop Region.
 */
this->midFilter_
    .setPeak(
        this->sampleRate_,
        static_cast<T>(450.0),
        static_cast<T>(0.7),
        midGainDb);
/**
 * Treble Shelf.
 */
this->trebleFilter_
    .setHighShelf(
        this->sampleRate_,
        static_cast<T>(4000.0),
        static_cast<T>(0.707),
        trebleGainDb);

```

```

}

```

```

};

```

```
using FenderToneStackF =  
    FenderToneStack<float>;  
using FenderToneStackD =  
    FenderToneStack<double>;  
} // namespace cvdsp  
#endif
```

Blackface Tone Stack

A topologia Blackface foi utilizada nos amplificadores Fender da década de 1960.

Exemplos clássicos:

Twin Reverb

Deluxe Reverb

Super Reverb

Pro Reverb

Bandmaster

Twin Reverb

Características:

Grande headroom

Graves profundos

Agudos brilhantes

Médios recuados

Clean cristalino

Conhecido pelo clássico:

Fender Clean

Deluxe Reverb

Características:

Breakup mais cedo

Médios um pouco mais presentes

Agudos doces

Resposta dinâmica

Muito utilizado em:

Blues

Country

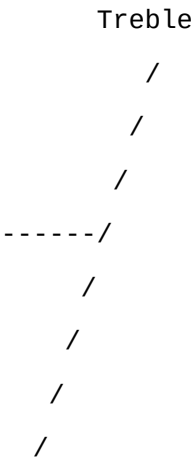
Surf
Classic Rock

Resposta Clássica Fender

Assinatura típica:

Bass
↑
Mid
↓
Treble
↑

Curva aproximada:



Mid Scoop
Bass Rise

Características sonoras:

- Som aberto
- Brilhante
- Espacial
- Limpo
- Grande sensação de headroom

Fluxo

Input
|
v
Low Shelf
(Bass)
|

v
Mid Peak/Cut
(Mid)
|
v
High Shelf
(Treble)
|
v
Output

Características

Header-Only
C++20
Template<float>
Template<double>
Real-Time Safe
Sem alocação em process()
Reutiliza ToneStack
Reutiliza Biquad

Guitar/MarshallToneStack.hpp

```
#ifndef CVDSP_GUITAR_MARSHALLTONESTACK_HPP
#define CVDSP_GUITAR_MARSHALLTONESTACK_HPP

/**
 * @file MarshallToneStack.hpp
 * @brief Marshall Tone Stack
 *
 * Inspired by:
 *
 * - Plexi 1959
 * - JMP
 * - JCM800
 * - JCM900
 *
 * Header-Only
 * C++20
```

```

* Real-Time Safe
*
* Dependencies:
*
* - ToneStack.hpp
*
* No allocations.
* No exceptions.
*/

#include <algorithm>
#include <cmath>
#include "ToneStack.hpp"
namespace cvdsp
{
template<typename T>
class MarshallToneStack final
    : public ToneStack<T>
{
public:
    MarshallToneStack() = default;
    void prepare(
        T sampleRate)
        noexcept override
    {
        ToneStack<T>::prepare(
            sampleRate);
        updateFilters();
    }
    void reset()
        noexcept override
    {
        ToneStack<T>::reset();
    }
    void setBass(
        T value)
        noexcept override
    {
        this->bass_ =
            std::clamp(

```

```

        value,
        T(0),
        T(1));
    updateFilters();
}
void setMid(
    T value)
    noexcept override
{
    this->mid_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
void setTreble(
    T value)
    noexcept override
{
    this->treble_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
void setPresence(
    T value)
    noexcept override
{
    this->presence_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
T process(

```



```

    T input)
    noexcept override
{
    T x = input;
    /**
     * Marshall topology.
     *
     * Bass
     * →
     * Mid
     * →
     * Treble
     * →
     * Presence
     */
    x =
        this->bassFilter_
            .process(x);
    x =
        this->midFilter_
            .process(x);
    x =
        this->trebleFilter_
            .process(x);
    x =
        this->presenceFilter_
            .process(x);
    return x;
}

private:
    void updateFilters()
    noexcept
    {
        /**
         * Marshall stacks generally
         * exhibit:
         *
         * - tighter bass
         * - stronger mids

```

```

* - aggressive upper mids
* - bright presence region
*/
const T bassGainDb =
    static_cast<T>(-10)
    +
    (
        this->bass_
        *
        static_cast<T>(20)
    );
const T midGainDb =
    static_cast<T>(-8)
    +
    (
        this->mid_
        *
        static_cast<T>(24)
    );
const T trebleGainDb =
    static_cast<T>(-10)
    +
    (
        this->treble_
        *
        static_cast<T>(20)
    );
const T presenceGainDb =
    static_cast<T>(0)
    +
    (
        this->presence_
        *
        static_cast<T>(12)
    );
/**
* Bass
*
* Marshall low-end is tighter

```

```

    * than Fender.
    */
this->bassFilter_
    .setLowShelf(
        this->sampleRate_,
        static_cast<T>(120.0),
        static_cast<T>(0.707),
        bassGainDb);
/**
    * Midrange emphasis.
    *
    * Classic Marshall growl.
    */
this->midFilter_
    .setPeak(
        this->sampleRate_,
        static_cast<T>(700.0),
        static_cast<T>(0.8),
        midGainDb);
/**
    * Upper-mid / treble attack.
    */
this->trebleFilter_
    .setHighShelf(
        this->sampleRate_,
        static_cast<T>(2500.0),
        static_cast<T>(0.707),
        trebleGainDb);
/**
    * Presence region.
    *
    * Power amp feedback style
    * brightness control.
    */
this->presenceFilter_
    .setHighShelf(
        this->sampleRate_,
        static_cast<T>(4500.0),
        static_cast<T>(0.707),

```

```
        presenceGainDb);  
  
    }  
};  
  
using MarshallToneStackF =  
    MarshallToneStack<float>;  
  
using MarshallToneStackD =  
    MarshallToneStack<double>;  
} // namespace cvdsp  
#endif
```

Plexi

Referência:

Marshall Super Lead 1959

Características:

Graves controlados

Médios pronunciados

Agudos abertos

Grande dinâmica

Assinatura:

British Crunch

JCM800

Características:

Mais ganho

Médios agressivos

Ataque forte

Excelente definição

Região característica:

600 Hz

até

1.5 kHz

Muito utilizado em:

Hard Rock

Heavy Metal Clássico

NWOBHM

JCM900

Características:

- Mais saturação
- Mais compressão
- Agudos mais pronunciados
- Maior ganho disponível

Aplicações:

- Hard Rock
- Metal
- Modern Rock

Resposta Clássica Marshall

Comparado ao Fender:

- Mais médios
- Menos scoop
- Mais ataque
- Mais presença

Curva aproximada:

Bass

↑

Mid

↑↑

Treble

↑

Presence

↑↑

Fluxo

Input

|

v

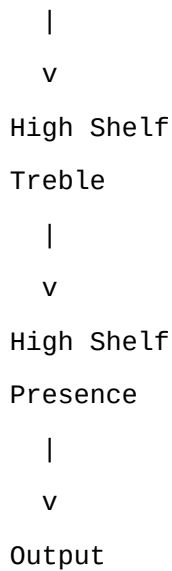
Low Shelf

Bass

|

v

Mid Peak



Características

- Header-Only
- C++20
- Template<float>
- Template<double>
- Real-Time Safe
- Sem alocação em process()
- Reutiliza ToneStack
- Reutiliza Biquad

Guitar/VoxToneStack.hpp

```
#ifndef CVDSP_GUITAR_VOXTONESTACK_HPP
#define CVDSP_GUITAR_VOXTONESTACK_HPP

/**
 * @file VoxToneStack.hpp
 * @brief Vox AC15 / AC30 Top Boost Tone Stack
 *
 * Inspired by:
 *
 * - Vox AC15
 * - Vox AC30
 * - Top Boost Channel
 *
 * Header-Only
 * C++20
 */
```

```

* Real-Time Safe
*
* Dependencies:
*
* - ToneStack.hpp
*
* No allocations.
* No exceptions.
*/

#include <algorithm>
#include <cmath>
#include "ToneStack.hpp"
namespace cvdsp
{
template<typename T>
class VoxToneStack final
    : public ToneStack<T>
{
public:
    VoxToneStack() = default;
    void prepare(
        T sampleRate)
        noexcept override
    {
        ToneStack<T>::prepare(
            sampleRate);
        updateFilters();
    }
    void reset()
        noexcept override
    {
        ToneStack<T>::reset();
    }
    void setBass(
        T value)
        noexcept override
    {
        this->bass_ =
            std::clamp(

```

```

        value,
        T(0),
        T(1));
    updateFilters();
}
void setMid(
    T value)
    noexcept override
{
    this->mid_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
void setTreble(
    T value)
    noexcept override
{
    this->treble_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
void setPresence(
    T value)
    noexcept override
{
    this->presence_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}
T process(

```



```

T input)
noexcept override
{
    T x = input;
    /**
     * Vox Top Boost topology.
     *
     * Bass
     * ->
     * Mid shaping
     * ->
     * Treble
     * ->
     * Presence
     */
    x =
        this->bassFilter_
            .process(x);
    x =
        this->midFilter_
            .process(x);
    x =
        this->trebleFilter_
            .process(x);
    x =
        this->presenceFilter_
            .process(x);
    return x;
}

private:
void updateFilters()
noexcept
{
    /**
     * AC15 / AC30 Top Boost
     *
     * Character:
     *
     * - Chime

```

```

* - Sparkle
* - Upper-mid emphasis
* - Bright top end
* - Less low-end than Fender
*/

const T bassGainDb =
    static_cast<T>(-10)
    +
    (
        this->bass_
        *
        static_cast<T>(20)
    );

const T midGainDb =
    static_cast<T>(-6)
    +
    (
        this->mid_
        *
        static_cast<T>(12)
    );

const T trebleGainDb =
    static_cast<T>(-12)
    +
    (
        this->treble_
        *
        static_cast<T>(24)
    );

const T presenceGainDb =
    this->presence_
    *
    static_cast<T>(10);

/**
* Tight bass response.
*/

this->bassFilter_
    .setLowShelf(
        this->sampleRate_,

```

```

        static_cast<T>(100.0),
        static_cast<T>(0.707),
        bassGainDb);
/**
 * Upper-mid chime region.
 */
this->midFilter_
    .setPeak(
        this->sampleRate_,
        static_cast<T>(1200.0),
        static_cast<T>(0.8),
        midGainDb);
/**
 * Top Boost treble section.
 */
this->trebleFilter_
    .setHighShelf(
        this->sampleRate_,
        static_cast<T>(3500.0),
        static_cast<T>(0.707),
        trebleGainDb);
/**
 * Air and brilliance.
 */
this->presenceFilter_
    .setHighShelf(
        this->sampleRate_,
        static_cast<T>(7000.0),
        static_cast<T>(0.707),
        presenceGainDb);
    }
};

using VoxToneStackF =
    VoxToneStack<float>;
using VoxToneStackD =
    VoxToneStack<double>;
} // namespace cvdsp
#endif

```

AC15

Características:

Breakup precoce

Compressão natural

Médios articulados

Agudos brilhantes

Aplicações clássicas:

Blues

Classic Rock

British Pop

AC30

Características:

Maior headroom

Mais volume

Chime característico

Resposta aberta

Associado a:

The Beatles

Queen

U2

The Edge

Brian May

Top Boost

O circuito Top Boost foi introduzido para adicionar:

Treble

Brilliance

Presence

criando a assinatura clássica Vox:

Chime

Sparkle

Air

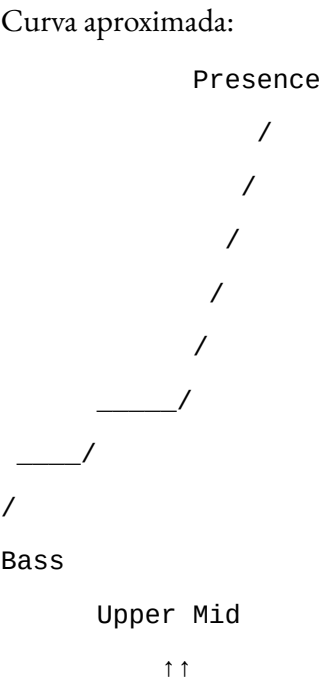
Regiões enfatizadas:

1 kHz

até
4 kHz
+
6 kHz
até
10 kHz

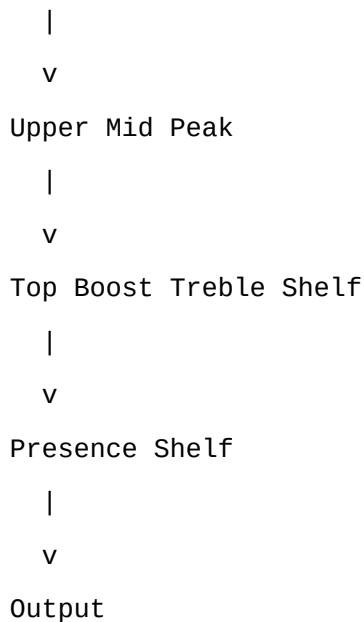
Resposta Típica Vox

Bass
↑
Low Mid
↓
Upper Mid
↑↑
Treble
↑↑
Presence
↑



Fluxo

Input
|
v
Bass Shelf



Características

- Header-Only
- C++20
- Template<float>
- Template<double>
- Real-Time Safe
- Sem alocação em process()
- Reutiliza ToneStack
- Reutiliza Biquad
- Compatível:
 - VST3
 - JUCE
 - iPlug2
 - CLAP
 - Standalone

Guitar/MesaToneStack.hpp

```
#ifndef CVDSP_GUITAR_MESATONESTACK_HPP
#define CVDSP_GUITAR_MESATONESTACK_HPP

/**
 * @file MesaToneStack.hpp
 * @brief Mesa Boogie Mark Series Tone Stack
 *
 * Inspired by:
```

```

*

* - Mesa Mark I
* - Mesa Mark II
* - Mesa Mark III
* - Mesa Mark IV
* - Mesa Mark V
*

* Header-Only
* C++20
* Real-Time Safe
*

* Dependencies:
*

* - ToneStack.hpp
*

* No allocations.
* No exceptions.
*/

#include <algorithm>
#include <cmath>
#include "ToneStack.hpp"
namespace cvdsp
{
template<typename T>
class MesaToneStack final
    : public ToneStack<T>
{
public:
    MesaToneStack() = default;
    void prepare(
        T sampleRate)
        noexcept override
    {
        ToneStack<T>::prepare(
            sampleRate);
        updateFilters();
    }
    void reset()
        noexcept override

```

```

{
    ToneStack<T>::reset();
}

void setBass(
    T value)
    noexcept override
{
    this->bass_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}

void setMid(
    T value)
    noexcept override
{
    this->mid_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}

void setTreble(
    T value)
    noexcept override
{
    this->treble_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}

void setPresence(
    T value)
    noexcept override

```



```

{
    this->presence_ =
        std::clamp(
            value,
            T(0),
            T(1));
    updateFilters();
}

T process(
    T input)
    noexcept override
{
    T x = input;
    /**
     * Mesa Mark topology.
     *
     * Bass
     * ->
     * Mid
     * ->
     * Treble
     * ->
     * Presence
     */
    x =
        this->bassFilter_
            .process(x);
    x =
        this->midFilter_
            .process(x);
    x =
        this->trebleFilter_
            .process(x);
    x =
        this->presenceFilter_
            .process(x);
    return x;
}

private:

```

```

void updateFilters()
    noexcept
{
    /**
     * Mesa Mark Series characteristics:
     *
     * Tight low end
     * Aggressive upper mids
     * Strong treble interaction
     * Wide presence range
     *
     * Tone stack positioned
     * before significant gain stages
     * in traditional Mark circuits.
     */
    const T bassGainDb =
        static_cast<T>(-12)
        +
        (
            this->bass_
            *
            static_cast<T>(18)
        );
    const T midGainDb =
        static_cast<T>(-18)
        +
        (
            this->mid_
            *
            static_cast<T>(24)
        );
    const T trebleGainDb =
        static_cast<T>(-10)
        +
        (
            this->treble_
            *
            static_cast<T>(26)
        );
};

```

```

const T presenceGainDb =
    (
        this->presence_
        *
        static_cast<T>(14)
    );
/**
 * Bass.
 *
 * Mesa low-end remains tight
 * and controlled.
 */
this->bassFilter_
    .setLowShelf(
        this->sampleRate_,
        static_cast<T>(90.0),
        static_cast<T>(0.707),
        bassGainDb);
/**
 * Characteristic mid control.
 *
 * Mark amplifiers often produce
 * the famous V-shaped response
 * when mids are reduced.
 */
this->midFilter_
    .setPeak(
        this->sampleRate_,
        static_cast<T>(750.0),
        static_cast<T>(0.9),
        midGainDb);
/**
 * Treble control is extremely
 * influential in Mark amplifiers.
 *
 * It affects both brightness
 * and perceived gain structure.
 */
this->trebleFilter_

```

```
        .setHighShelf(  
            this->sampleRate_,  
            static_cast<T>(2200.0),  
            static_cast<T>(0.707),  
            trebleGainDb);  
    /**  
     * Power amp presence region.  
     */  
    this->presenceFilter_  
        .setHighShelf(  
            this->sampleRate_,  
            static_cast<T>(5000.0),  
            static_cast<T>(0.707),  
            presenceGainDb);  
}  
};  
  
using MesaToneStackF =  
    MesaToneStack<float>;  
using MesaToneStackD =  
    MesaToneStack<double>;  
} // namespace cvdsp  
#endif
```

Mesa Boogie Mark Series

Modelos clássicos:

Mark I

Mark II

Mark III

Mark IV

Mark V

Características:

Alto ganho

Grande definição

Ataque rápido

Resposta articulada

Baixos firmes

Assinatura Sonora

Comparado a um Marshall:

Mais precisão

Mais foco

Mais ataque

Mais controle nos graves

Comparado a um Fender:

Muito mais ganho

Menos headroom limpo

Mais compressão

Mais médios agressivos

Resposta Típica Mark Series

Bass

↑

Low Mid

↓

Upper Mid

↑

Treble

↑↑

Presence

↑

Quando combinado com o clássico Graphic EQ de 5 bandas:

80 Hz ↑

240 Hz ↓

750 Hz ↓↓

2200 Hz ↑

6600 Hz ↑

produzindo a famosa curva:

V Shape

utilizada em:

Metal

Progressive Metal

Fusion

Lead Guitar

Fluxo

Input

|

v

Bass Shelf

|

v

Mid Peak/Cut

|

v

Treble Shelf

|

v

Presence Shelf

|

v

Output

Características

Header-Only

C++20

Template<float>

Template<double>

Real-Time Safe

Sem alocação em process()

Reutiliza ToneStack

Reutiliza Biquad

Guitar/PowerAmp.hpp

```
#ifndef CVDSP_GUITAR_POWERAMP_HPP
#define CVDSP_GUITAR_POWERAMP_HPP

/**
 * @file PowerAmp.hpp
 * @brief Guitar Power Amplifier Simulation
```

```

*

* Models:

*
* - EL34
* - 6L6
* - KT88
* - EL84
*

* Header-Only
* C++20
* Real-Time Safe
*

* Dependencies:
*
* - PentodeStage.hpp
*

* Optional:
*
* - Presence Filter
* - Resonance Filter
*

* No allocations.
* No exceptions.
*/

#include <cmath>
#include <type_traits>
#include "PentodeStage.hpp"
namespace cvdsp
{
template<typename T>
class PowerAmp
{
public:
    static_assert(
        std::is_floating_point_v<T>,
        "PowerAmp requires floating point type");
    enum class Model
    {
        EL34,

```

```

        L6L6,
        KT88,
        EL84
    };

public:
    PowerAmp() = default;

    void prepare(
        T sampleRate)
        noexcept
    {
        sampleRate_ =
            sampleRate;
        stageA_.prepare(
            sampleRate);
        stageB_.prepare(
            sampleRate);
        configureModel();
        reset();
    }

    void reset()
        noexcept
    {
        stageA_.reset();
        stageB_.reset();
        sagState_ =
            T(0);
    }

    void setModel(
        Model model)
        noexcept
    {
        model_ =
            model;
        configureModel();
    }

    Model getModel() const noexcept
    {
        return model_;
    }

```



```

T process(
    T input)
    noexcept
{
    /**
     * Dynamic supply sag.
     */
    const T envelope =
        std::abs(
            input);
    sagState_ +=
        sagCoefficient_
        *
        (
            envelope
            -
            sagState_
        );
    const T sagGain =
        T(1)
        -
        (
            sagAmount_
            *
            sagState_
        );
    T x =
        input
        *
        sagGain;
    /**
     * Push-Pull approximation.
     */
    x =
        stageA_.process(
            x);
    x =
        stageB_.process(
            x);
}

```

```

/**
 * Dynamic compression.
 */
const T compression =
    T(1)
    /
    (
        T(1)
        +
        compressionAmount_
        *
        std::abs(x)
    );
x *= compression;
return x;
}

private:
void configureModel()
    noexcept
{
    switch (model_)
    {
        case Model::EL34:
        {
            stageA_.setDrive(
                static_cast<T>(3.0));
            stageB_.setDrive(
                static_cast<T>(2.8));
            stageA_.setBias(
                static_cast<T>(0.05));
            stageB_.setBias(
                static_cast<T>(-0.05));
            stageA_.setScreenVoltage(
                static_cast<T>(420));
            stageB_.setScreenVoltage(
                static_cast<T>(420));
            sagAmount_ =
                static_cast<T>(0.12);
            compressionAmount_ =

```

```

        static_cast<T>(0.30);
    break;
}
case Model::L6L6:
{
    stageA_.setDrive(
        static_cast<T>(2.2));
    stageB_.setDrive(
        static_cast<T>(2.2));
    stageA_.setScreenVoltage(
        static_cast<T>(460));
    stageB_.setScreenVoltage(
        static_cast<T>(460));
    sagAmount_ =
        static_cast<T>(0.08);
    compressionAmount_ =
        static_cast<T>(0.18);
    break;
}
case Model::KT88:
{
    stageA_.setDrive(
        static_cast<T>(1.8));
    stageB_.setDrive(
        static_cast<T>(1.8));
    stageA_.setScreenVoltage(
        static_cast<T>(520));
    stageB_.setScreenVoltage(
        static_cast<T>(520));
    sagAmount_ =
        static_cast<T>(0.04);
    compressionAmount_ =
        static_cast<T>(0.10);
    break;
}
case Model::EL84:
{
    stageA_.setDrive(
        static_cast<T>(4.0));

```

```

        stageB_.setDrive(
            static_cast<T>(3.8));
        stageA_.setScreenVoltage(
            static_cast<T>(310));
        stageB_.setScreenVoltage(
            static_cast<T>(310));
        sagAmount_ =
            static_cast<T>(0.18);
        compressionAmount_ =
            static_cast<T>(0.40);
        break;
    }
}
stageA_.setOutputGain(
    static_cast<T>(1));
stageB_.setOutputGain(
    static_cast<T>(1));
}

private:
    T sampleRate_ =
        static_cast<T>(44100);
    Model model_ =
        Model::EL34;
    PentodeStage<T>
        stageA_;
    PentodeStage<T>
        stageB_;
    T sagState_ =
        T(0);
    T sagCoefficient_ =
        static_cast<T>(0.001);
    T sagAmount_ =
        static_cast<T>(0.10);
    T compressionAmount_ =
        static_cast<T>(0.20);
};

using PowerAmpF =
    PowerAmp<float>;
using PowerAmpD =

```

```
PowerAmp<double>;  
} // namespace cvdsp  
#endif
```

EL34

Marshall Plexi

JCM800

JCM900

Características:

Médios fortes

Breakup musical

Compressão moderada

Rock clássico

6L6

Fender Twin

Bassman

Mesa Mark

Características:

Headroom elevado

Graves firmes

Agudos limpos

Menos compressão

KT88

Hiwatt

Orange

Ampeg

Características:

Alta potência

Grande headroom

Resposta ampla

Pouca compressão

EL84

Vox AC15

Vox AC30

Características:

Breakup precoce

Compressão elevada

Chime característico

Power Saturation

Diferente do pré-amplificador:

Preamp

=

Clipping principal

Power Amp

=

Compressão dinâmica

+

Sag

+

Harmônicos

O power amp responde à intensidade da execução.

Sag

Sag representa a queda momentânea da fonte de alimentação.

Quando o sinal aumenta:

Fonte ↓

Headroom ↓

Compressão ↑

Sensação:

Amplificador "respira"

Muito presente em:

EL84

EL34

Dynamic Compression

A compressão surge naturalmente devido a:

Fonte

Transformador

Válvulas

E produz:

Sustain

Punch

Controle de transientes

Fluxo

Input

|

v

Sag Model

|

v

Pentode A

|

v

Pentode B

|

v

Dynamic Compression

|

v

Output

Características

Header-Only

C++20

Template<float>

Template<double>

Real-Time Safe

Sem alocação em process()

Modelos:

EL34

6L6

KT88

EL84

Power Saturation

Dynamic Compression

Supply Sag

Push-Pull Approximation


```
=====
FILE: LICENSE
=====
```

MIT License

Copyright (c) 2026 Cloves Rodrigues

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

```
=====
FILE: CV_DSP/Convolution/ConvolutionEngine.hpp
=====
```

```
#ifndef CVDSP_CONVOLUTION_CONVOLUTIONENGINE_HPP
#define CVDSP_CONVOLUTION_CONVOLUTIONENGINE_HPP
```

```
/**
 * @file ConvolutionEngine.hpp
 * @brief FFT Partitioned Convolution Engine
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - FFT Convolution
 * - Partitioned IR
 * - Overlap Save
 * - Long IR Support
 *
 * No dynamic allocation in process().
 */
```

```
#include <array>
#include <complex>
#include <cstdint>
#include <type_traits>
```

```
#include "../Spectral/FFT.hpp"
#include "../Core/CircularBuffer.hpp"
```

```
namespace cvdsp
{
```

```
template<
    typename T,
    std::size_t FFTSize,
```

```

    std::size_t MaxIRSamples = 262144>
class ConvolutionEngine
{
    static_assert(
        std::is_floating_point_v<T>);

public:

    using Complex =
        std::complex<T>;

    static constexpr std::size_t BlockSize =
        FFTSize / 2;

    static constexpr std::size_t MaxPartitions =
        MaxIRSamples / BlockSize + 1;

public:

    ConvolutionEngine() = default;

    bool prepare() noexcept
    {
        if (!fft_.prepare(
            FFTSize))
        {
            return false;
        }

        reset();

        return true;
    }

    void reset() noexcept
    {
        inputPosition_ = 0;
        currentPartition_ = 0;
        numPartitions_ = 0;

        inputBuffer_.reset();

        for (auto& v : overlap_)
            v = T(0);

        for (auto& v : timeDomain_)
            v = T(0);

        for (auto& v : spectrum_)
            v = Complex(T(0), T(0));

        for (auto& p : partitionSpectra_)
        {
            for (auto& v : p)
            {
                v = Complex(T(0), T(0));
            }
        }

        for (auto& p : inputHistory_)
        {
            for (auto& v : p)
            {
                v = Complex(T(0), T(0));
            }
        }
    }

```

```

    }
}

/**
 * Load impulse response.
 */
bool loadImpulseResponse(
    const T* ir,
    std::size_t length)
    noexcept
{
    reset();

    if (length == 0)
    {
        return false;
    }

    numPartitions_ =
        (length + BlockSize - 1)
        /
        BlockSize;

    if (numPartitions_ >
        MaxPartitions)
    {
        return false;
    }

    std::array<T, FFTSize>
        partitionTime{};

    for (std::size_t p = 0;
        p < numPartitions_;
        ++p)
    {
        for (auto& v : partitionTime)
            v = T(0);

        const std::size_t offset =
            p * BlockSize;

        for (std::size_t i = 0;
            i < BlockSize;
            ++i)
        {
            if ((offset + i)
                < length)
            {
                partitionTime[i] =
                    ir[offset + i];
            }
        }

        for (std::size_t i = 0;
            i < FFTSize;
            ++i)
        {
            partitionSpectra_[p][i] =
                Complex(
                    partitionTime[i],
                    T(0));
        }
    }
}

```

```

        fft_.forward(
            partitionSpectra_[p].data());
    }

    return true;
}

/**
 * Process one sample.
 */
T process(
    T input)
    noexcept
{
    inputBuffer_.push(
        input);

    const T output =
        outputFIFO_[outputIndex_];

    outputFIFO_[outputIndex_] =
        T(0);

    ++outputIndex_;

    if (outputIndex_
        >= BlockSize)
    {
        outputIndex_ = 0;

        processBlock();
    }

    return output;
}

```

private:

```

void processBlock()
    noexcept
{
    for (std::size_t i = 0;
        i < BlockSize;
        ++i)
    {
        timeDomain_[i] =
            inputBuffer_.read(
                BlockSize - 1 - i);
    }

    for (std::size_t i = BlockSize;
        i < FFTSize;
        ++i)
    {
        timeDomain_[i] = T(0);
    }

    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        spectrum_[i] =
            Complex(

```

```

        timeDomain_[i],
        T(0));
    }

    fft_.forward(
        spectrum_.data());

    for (std::size_t i =
        numPartitions_ - 1;
        i > 0;
        --i)
    {
        inputHistory_[i] =
            inputHistory_[i - 1];
    }

    inputHistory_[0] =
        spectrum_;

    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        accumulation_[i] =
            Complex(
                T(0),
                T(0));
    }

    for (std::size_t p = 0;
        p < numPartitions_;
        ++p)
    {
        for (std::size_t k = 0;
            k < FFTSize;
            ++k)
        {
            accumulation_[k] +=
                inputHistory_[p][k]
                *
                partitionSpectra_[p][k];
        }
    }

    fft_.inverse(
        accumulation_.data());

    for (std::size_t i = 0;
        i < BlockSize;
        ++i)
    {
        outputFIFO_[i] =
            accumulation_[
                i + BlockSize]
                .real();
    }
}

```

private:

```

    FFT<T> fft_;

    CircularBuffer<
        T,

```

```

        FFTSize * 4>
        inputBuffer_;

    std::array<T, FFTSize>
        timeDomain_{};

    std::array<T, BlockSize>
        overlap_{};

    std::array<T, BlockSize>
        outputFIFO_{};

    std::array<Complex, FFTSize>
        spectrum_{};

    std::array<Complex, FFTSize>
        accumulation_{};

    std::array<
        std::array<
            Complex,
            FFTSize>,
        MaxPartitions>
        partitionSpectra_{};

    std::array<
        std::array<
            Complex,
            FFTSize>,
        MaxPartitions>
        inputHistory_{};

    std::size_t numPartitions_ = 0;

    std::size_t inputPosition_ = 0;

    std::size_t outputIndex_ = 0;

    std::size_t currentPartition_ = 0;
};

using Convolution2048F =
    ConvolutionEngine<
        float,
        2048>;

using Convolution4096F =
    ConvolutionEngine<
        float,
        4096>;

using Convolution8192F =
    ConvolutionEngine<
        float,
        8192>;

} // namespace cvdsp

#endif

=====
FILE: CV_DSP/Convolution/IRLoader.hpp

```

```

=====
#ifndef CVDSP_CONVOLUTION_IRLOADER_HPP
#define CVDSP_CONVOLUTION_IRLOADER_HPP

/**
 * @file IRLoader.hpp
 * @brief Impulse Response WAV Loader
 *
 * Header-Only
 * C++20
 *
 * Supported:
 * - PCM 16-bit
 * - PCM 24-bit
 * - IEEE Float 32-bit
 * - Mono WAV
 * - Stereo WAV
 *
 * Designed for:
 * - Cabinet IR
 * - Room IR
 * - Hall IR
 * - Reverb IR
 *
 * No allocations during audio processing.
 */

#include <array>
#include <cstdint>
#include <cstring>
#include <fstream>
#include <string>
#include <algorithm>
#include <cmath>

namespace cvdsp
{
template<
    typename T,
    std::size_t MaxSamples = 262144>
class IRLoader
{
public:
    IRLoader() = default;

    /**
     * Load WAV file.
     */
    bool load(
        const std::string& filePath,
        bool normalize = true,
        bool trimSilence = true,
        T trimThreshold =
            static_cast<T>(1e-5))
    {
        unload();

        std::ifstream file(
            filePath,
            std::ios::binary);

        if (!file)

```

```

    {
        return false;
    }

    if (!readHeader(file))
    {
        return false;
    }

    if (!readData(file))
    {
        return false;
    }

    if (normalize)
    {
        normalizeIR();
    }

    if (trimSilence)
    {
        trimTail(
            trimThreshold);
    }

    return true;
}

/**
 * Unload current IR.
 */
void unload() noexcept
{
    length_ = 0;

    sampleRate_ = 0;

    numChannels_ = 0;

    for (auto& s : left_)
        s = T(0);

    for (auto& s : right_)
        s = T(0);
}

/**
 * Mono access.
 */
[[nodiscard]]
const T* getSamples() const noexcept
{
    return left_.data();
}

/**
 * Channel access.
 */
[[nodiscard]]
const T* getChannel(
    std::size_t channel) const noexcept
{
    if (channel == 0)
    {

```



```

        return left_.data();
    }

    return right_.data();
}

/**
 * Length in samples.
 */
[[nodiscard]]
std::size_t getLength() const noexcept
{
    return length_;
}

/**
 * Number of channels.
 */
[[nodiscard]]
std::uint16_t getNumChannels()
    const noexcept
{
    return numChannels_;
}

/**
 * Sample rate.
 */
[[nodiscard]]
std::uint32_t getSampleRate()
    const noexcept
{
    return sampleRate_;
}

/**
 * Stereo?
 */
[[nodiscard]]
bool isStereo() const noexcept
{
    return numChannels_ == 2;
}

/**
 * Empty?
 */
[[nodiscard]]
bool isLoaded() const noexcept
{
    return length_ > 0;
}

```

private:

```
#pragma pack(push, 1)
```

```

struct RIFFHeader
{
    char chunkID[4];
    std::uint32_t chunkSize;
    char format[4];
};

```

```

struct ChunkHeader
{
    char id[4];
    std::uint32_t size;
};

struct FormatChunk
{
    std::uint16_t audioFormat;
    std::uint16_t numChannels;
    std::uint32_t sampleRate;
    std::uint32_t byteRate;
    std::uint16_t blockAlign;
    std::uint16_t bitsPerSample;
};

#pragma pack(pop)

private:

    bool readHeader(
        std::ifstream& file)
    {
        RIFFHeader riff{};

        file.read(
            reinterpret_cast<char*>(&riff),
            sizeof(riff));

        if (!file)
        {
            return false;
        }

        if (std::strncmp(
            riff.chunkID,
            "RIFF",
            4) != 0)
        {
            return false;
        }

        if (std::strncmp(
            riff.format,
            "WAVE",
            4) != 0)
        {
            return false;
        }

        bool foundFmt = false;
        bool foundData = false;

        while (file &&
            (!foundFmt || !foundData))
        {
            ChunkHeader chunk{};

            file.read(
                reinterpret_cast<char*>(&chunk),
                sizeof(chunk));

            if (!file)
            {

```

```

        return false;
    }

    if (std::strncmp(
        chunk.id,
        "fmt ",
        4) == 0)
    {
        file.read(
            reinterpret_cast<char*>(&format_),
            sizeof(FormatChunk));

        if (chunk.size >
            sizeof(FormatChunk))
        {
            file.seekg(
                chunk.size -
                sizeof(FormatChunk),
                std::ios::cur);
        }

        foundFmt = true;
    }
    else if (
        std::strncmp(
            chunk.id,
            "data",
            4) == 0)
    {
        dataOffset_ =
            static_cast<std::uint64_t>(
                file.tellg());

        dataSize_ =
            chunk.size;

        file.seekg(
            chunk.size,
            std::ios::cur);

        foundData = true;
    }
    else
    {
        file.seekg(
            chunk.size,
            std::ios::cur);
    }
}

if (!foundFmt ||
    !foundData)
{
    return false;
}

sampleRate_ =
    format_.sampleRate;

numChannels_ =
    format_.numChannels;

return true;
}

```

```

bool readData(
    std::ifstream& file)
{
    if (numChannels_ < 1 ||
        numChannels_ > 2)
    {
        return false;
    }

    file.clear();

    file.seekg(
        dataOffset_,
        std::ios::beg);

    const std::size_t bytesPerSample =
        format_.bitsPerSample / 8;

    const std::size_t totalFrames =
        dataSize_
        /
        (
            bytesPerSample
            *
            numChannels_);

    length_ =
        std::min(
            totalFrames,
            MaxSamples);

    switch(
        format_.audioFormat)
    {
    case 1:
        return readPCM(file);

    case 3:
        return readFloat(file);

    default:
        return false;
    }
}

bool readPCM(
    std::ifstream& file)
{
    if (format_.bitsPerSample == 16)
    {
        return readPCM16(file);
    }

    if (format_.bitsPerSample == 24)
    {
        return readPCM24(file);
    }

    return false;
}

bool readPCM16(
    std::ifstream& file)

```

```

{
    for (std::size_t i = 0;
        i < length_;
        ++i)
    {
        std::int16_t leftSample;

        file.read(
            reinterpret_cast<char*>(
                &leftSample),
            sizeof(leftSample));

        left_[i] =
            static_cast<T>(
                leftSample)
            /
            static_cast<T>(32768);

        if (numChannels_ == 2)
        {
            std::int16_t rightSample;

            file.read(
                reinterpret_cast<char*>(
                    &rightSample),
                sizeof(rightSample));

            right_[i] =
                static_cast<T>(
                    rightSample)
                /
                static_cast<T>(32768);
        }
    }

    return true;
}

bool readPCM24(
    std::ifstream& file)
{
    for (std::size_t i = 0;
        i < length_;
        ++i)
    {
        left_[i] =
            read24Bit(file);

        if (numChannels_ == 2)
        {
            right_[i] =
                read24Bit(file);
        }
    }

    return true;
}

bool readFloat(
    std::ifstream& file)
{
    if (format_.bitsPerSample != 32)
    {
        return false;
    }
}

```

```

    }

    for (std::size_t i = 0;
         i < length_;
         ++i)
    {
        float leftSample;

        file.read(
            reinterpret_cast<char*>(
                &leftSample),
            sizeof(float));

        left_[i] =
            static_cast<T>(
                leftSample);

        if (numChannels_ == 2)
        {
            float rightSample;

            file.read(
                reinterpret_cast<char*>(
                    &rightSample),
                sizeof(float));

            right_[i] =
                static_cast<T>(
                    rightSample);
        }
    }

    return true;
}

T read24Bit(
    std::ifstream& file)
{
    unsigned char bytes[3];

    file.read(
        reinterpret_cast<char*>(bytes),
        3);

    std::int32_t sample =
        bytes[0]
        |
        (bytes[1] << 8)
        |
        (bytes[2] << 16);

    if (sample &
        0x800000)
    {
        sample |=
            ~0xFFFFF;
    }

    return
        static_cast<T>(sample)
        /
        static_cast<T>(8388608.0);
}

```

```

void normalizeIR()
{
    T peak = T(0);

    for (std::size_t i = 0;
         i < length_;
         ++i)
    {
        peak =
            std::max(
                peak,
                std::abs(
                    left_[i]));

        if (numChannels_ == 2)
        {
            peak =
                std::max(
                    peak,
                    std::abs(
                        right_[i]));
        }
    }

    if (peak <= T(0))
    {
        return;
    }

    const T gain =
        T(1) / peak;

    for (std::size_t i = 0;
         i < length_;
         ++i)
    {
        left_[i] *= gain;

        if (numChannels_ == 2)
        {
            right_[i] *= gain;
        }
    }
}

void trimTail(
    T threshold)
{
    if (length_ == 0)
    {
        return;
    }

    std::size_t last =
        length_ - 1;

    while (last > 0)
    {
        bool silent =
            std::abs(
                left_[last])
            < threshold;

        if (numChannels_ == 2)

```

```

        {
            silent &=
                std::abs(
                    right_[last])
                < threshold;
        }

        if (!silent)
        {
            break;
        }

        --last;
    }

    length_ = last + 1;
}

```

private:

```

    FormatChunk format_{};

    std::uint64_t dataOffset_ = 0;

    std::uint32_t dataSize_ = 0;

    std::uint32_t sampleRate_ = 0;

    std::uint16_t numChannels_ = 0;

    std::size_t length_ = 0;

    std::array<T, MaxSamples>
        left_{};

    std::array<T, MaxSamples>
        right_{};
};

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Core/AudioBufferView.hpp
=====
#ifndef CVDSP_CORE_AUDIOBUFFERVIEW_HPP
#define CVDSP_CORE_AUDIOBUFFERVIEW_HPP

/**
 * @file AudioBufferView.hpp
 * @brief Non-owning Audio Buffer View
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Non-owning
 * - No allocation
 * - No resizing
 * - No ownership

```



```

* - Zero-copy access
*
* Compatible with:
* - VST3
* - iPlug2
* - JUCE
* - CLAP
*
* Inspired by:
* - std::span
* - gsl::span
*/

#include <cassert>
#include <cstddef>
#include <type_traits>

namespace cvdsp
{
/**
 * @brief Non-owning audio buffer view.
 *
 * Layout:
 *
 * channels_[channel][sample]
 *
 * No memory ownership.
 *
 * No allocation.
 *
 * No resizing.
 */
template<typename T>
class AudioBufferView
{
    static_assert(
        std::is_floating_point_v<T>,
        "AudioBufferView requires floating point type");

public:
    using value_type      = T;
    using pointer         = T*;
    using const_pointer   = const T*;
    using size_type       = std::size_t;

public:
    /**
     * @brief Default constructor.
     */
    constexpr AudioBufferView() noexcept = default;

    /**
     * @brief Construct from channel array.
     *
     * channels must remain valid during
     * the entire lifetime of the view.
     */
    constexpr AudioBufferView(
        T* const* channels,
        size_type numChannels,
        size_type numSamples) noexcept

```

```

        :
        channels_(channels),
        numChannels_(numChannels),
        numSamples_(numSamples)
    {
    }

/**
 * @brief Construct from mutable channel pointers.
 */
template<std::size_t NumChannels>
constexpr AudioBufferView(
    T* (&channels)[NumChannels],
    size_type numSamples) noexcept
    :
    channels_(channels),
    numChannels_(NumChannels),
    numSamples_(numSamples)
{
}

/**
 * @brief Returns channel pointer.
 */
[[nodiscard]]
constexpr T* getChannel(
    size_type channel) const noexcept
{
    assert(channel < numChannels_);

    return channels_[channel];
}

/**
 * @brief Returns number of channels.
 */
[[nodiscard]]
constexpr size_type getNumChannels() const noexcept
{
    return numChannels_;
}

/**
 * @brief Returns number of samples.
 */
[[nodiscard]]
constexpr size_type getNumSamples() const noexcept
{
    return numSamples_;
}

/**
 * @brief Channel access.
 *
 * Example:
 *
 * buffer[0][100]
 */
[[nodiscard]]
constexpr T* operator[](
    size_type channel) const noexcept
{
    return getChannel(channel);
}

```

```

/**
 * @brief Check if view is valid.
 */
[[nodiscard]]
constexpr bool isValid() const noexcept
{
    return
        channels_ != nullptr
        &&
        numChannels_ > 0
        &&
        numSamples_ > 0;
}

```

```

/**
 * @brief Check if empty.
 */
[[nodiscard]]
constexpr bool empty() const noexcept
{
    return
        numChannels_ == 0
        ||
        numSamples_ == 0;
}

```

private:

```

/**
 * Non-owning array of channel pointers.
 */
T* const* channels_ = nullptr;

/**
 * Channel count.
 */
size_type numChannels_ = 0;

/**
 * Samples per channel.
 */
size_type numSamples_ = 0;
};

```

```

/**
 * @brief Const version.
 */
template<typename T>
class ConstAudioBufferView
{
    static_assert(
        std::is_floating_point_v<T>,
        "ConstAudioBufferView requires floating point type");

```

public:

```

    using value_type      = T;
    using pointer         = const T*;
    using size_type       = std::size_t;

```

public:

```

constexpr ConstAudioBufferView() noexcept = default;

constexpr ConstAudioBufferView(
    const T* const* channels,
    size_type numChannels,
    size_type numSamples) noexcept
    :
    channels_(channels),
    numChannels_(numChannels),
    numSamples_(numSamples)
{
}

template<std::size_t NumChannels>
constexpr ConstAudioBufferView(
    const T* (&channels)[NumChannels],
    size_type numSamples) noexcept
    :
    channels_(channels),
    numChannels_(NumChannels),
    numSamples_(numSamples)
{
}

[[nodiscard]]
constexpr const T* getChannel(
    size_type channel) const noexcept
{
    assert(channel < numChannels_);

    return channels_[channel];
}

[[nodiscard]]
constexpr size_type getNumChannels() const noexcept
{
    return numChannels_;
}

[[nodiscard]]
constexpr size_type getNumSamples() const noexcept
{
    return numSamples_;
}

[[nodiscard]]
constexpr const T* operator[](
    size_type channel) const noexcept
{
    return getChannel(channel);
}

[[nodiscard]]
constexpr bool isValid() const noexcept
{
    return
        channels_ != nullptr
        &&
        numChannels_ > 0
        &&
        numSamples_ > 0;
}

```

```

private:

    const T* const* channels_ = nullptr;

    size_type numChannels_ = 0;

    size_type numSamples_ = 0;
};

} // namespace cvdsp

#endif

=====
FILE: CV_DSP/Core/CircularBuffer.hpp
=====
#ifndef CVDSP_CORE_CIRCULARBUFFER_HPP
#define CVDSP_CORE_CIRCULARBUFFER_HPP

#include <array>
#include <cassert>
#include <cstdlib>
#include <cstring>
#include <type_traits>
#include <limits>

/**
 * @namespace cvdsp
 * @brief CV_DSP - Biblioteca profissional de processamento de Áudio em tempo
real
 *
 * Namespace central contendo todos os módulos da biblioteca CV_DSP.
 * Todos os componentes seguem princípios de Real-Time Safe Audio Processing.
 */
namespace cvdsp
{
/**
 * @class CircularBuffer
 * @brief Buffer circular genérico para processamento de Áudio em tempo real
 *
 * Uma estrutura de dados essencial para aplicações de DSP que necessitam de:
 *
 * - **Delay Lines**: Armazenamento de amostras para síntese de reverberação,
chorus, echo
 * - **Buffers de Modulação**: Histórico de sinal para cálculos de
coeficientes variáveis no tempo
 * - **Buffers para FFT**: Acumulação de frames para processamento via
transformadas de Fourier
 * - **Look-ahead Windows**: Necessário para processadores com lookahead
(compressores, limitadores)
 *
 * **Características técnicas**:
 *
 * - Header-Only: Sem dependências externas, compilação rápida
 * - Real-Time Safe: Nenhuma alocação dinâmica durante processamento
 * - Zero-Copy: Operações em O(1) sem cópias desnecessárias
 * - SIMD-Friendly: Alinhamento e layout otimizados para vetorização
 *
 * **Matemática do Buffer Circular**:
 *
 * Um buffer circular com tamanho N é definido como:

```

```

*   ~~~
*   Índice Lógico: i (0 ≤ i < N)
*   Índice Físico: i % N
*   Posição Atual: head (0 ≤ head < N)
*   ~~~
*
*   Operações fundamentais:
*   - Write: armazena valor em posição (head + offset) % N
*   - Read: recupera valor de posição (head + offset) % N
*   - Advance: move head para (head + 1) % N
*
*   **Exemplo de uso - Delay Line (100ms @ 48kHz = 4800 amostras)**:
*   ~~~cpp
*   cvdsp::CircularBuffer<float, 4800> delayLine;
*   delayLine.prepare();
*
*   // Durante processamento (real-time safe)
*   float input = getInputSample();
*   float delayed = delayLine.read(0); // Lê a amostra mais antiga
*   delayLine.write(0, input);         // Sobrescreve com entrada atual
*   delayLine.advance();               // Move para próxima posição
*   ~~~
*
*   **Exemplo de uso - Buffer de Modulação (LFO history)**:
*   ~~~cpp
*   cvdsp::CircularBuffer<double, 256> lfoHistory;
*   lfoHistory.prepare();
*
*   // Lê histórico dos últimos 256 valores LFO
*   for (size_t i = 0; i < lfoHistory.capacity(); ++i) {
*       double historicalLFO = lfoHistory.read(i);
*   }
*   lfoHistory.write(0, currentLFO);
*   lfoHistory.advance();
*   ~~~
*
*   **Exemplo de uso - Buffer para FFT Overlap-Add (frame acumulada)**:
*   ~~~cpp
*   const size_t FRAME_SIZE = 512;
*   const size_t FFT_SIZE = 2048;
*   cvdsp::CircularBuffer<float, FFT_SIZE> fftBuffer;
*   fftBuffer.prepare();
*
*   for (size_t n = 0; n < FRAME_SIZE; ++n) {
*       fftBuffer.write(n, audioFrame[n]);
*   }
*   // Realizar FFT em fftBuffer...
*   fftBuffer.advance(); // Não usado neste caso, apenas exemplo
*   ~~~
*
*   @tparam T Tipo de dado (float ou double). Deve ser compatível com operações
aritméticas.
*   @tparam Capacity Tamanho fixo do buffer em amostras. Deve ser > 0.
*
*   @note Este buffer é **estritamente real-time safe**:
*   - Nenhuma alocação dinâmica
*   - Nenhuma exceção lançada em process()
*   - Nenhuma operação O(n) no caminho crítico
*/
template <typename T, size_t Capacity>
class CircularBuffer
{
    static_assert(std::is_arithmetic_v<T>,
                  "CircularBuffer requires arithmetic type (float, double, int,

```

```

etc.)");
    static_assert(Capacity > 0,
                  "CircularBuffer capacity must be greater than 0");
    static_assert((Capacity & (Capacity - 1)) == 0,
                  "CircularBuffer capacity must be a power of 2 for efficient
modulo via bitmask");

public:
    // =====
    // TYPEDEFS E CONSTANTES
    // =====

    /// @brief Tipo de dado armazenado no buffer
    using value_type = T;

    /// @brief Tipo para Índices e offsets
    using size_type = size_t;

    /// @brief Máscara de bits para operação de modulo eficiente
    /// Pré-calculada em tempo de compilação: (Capacity - 1)
    /// Permite: index % Capacity â†’ index & MODULO_MASK (uma operação
bitwise!)
    static constexpr size_type MODULO_MASK = Capacity - 1;

    /// @brief Capacidade do buffer em amostras
    static constexpr size_type BUFFER_CAPACITY = Capacity;

    // =====
    // CONSTRUTOR E DESTRUTOR
    // =====

    /**
     * @brief Construtor padrão. Inicializa o buffer com zero-initialization.
     *
     * **Comportamento**:
     * - Buffer: preenchido com zeros (amostras iniciais = 0)
     * - Head: posicionado em Índice 0
     * - Pronto para processamento após prepare()
     *
     * **Tempo de compilação**: O(1) amortizado via aggregate initialization
     *
     * @note Nenhuma alocação dinâmica ocorre.
     */
    constexpr CircularBuffer() noexcept
        : m_head(0)
        , m_buffer{} // Zero-initialize all elements
    {
    }

    /// @brief Destrutor padrão (trivial, sem recursos dinâmicos)
    ~CircularBuffer() noexcept = default;

    /// @brief Cópia explicitamente deletada (buffer é recurso único)
    CircularBuffer(const CircularBuffer&) = delete;

    /// @brief Atribuição explicitamente deletada
    CircularBuffer& operator=(const CircularBuffer&) = delete;

    /// @brief Move constructor explicitamente deletado (semanticamente não faz
sentido)
    CircularBuffer(CircularBuffer&&) = delete;

    /// @brief Move assignment explicitamente deletado
    CircularBuffer& operator=(CircularBuffer&&) = delete;

```

```

// =====
// MÃTODOS DE INICIALIZAÃ§Ã£o E RESET
// =====

/**
 * @brief Prepara o buffer para processamento.
 *
 * **PropÃ³sito**: InicializaÃ§Ã£o determinÃstica e sincronizaÃ§Ã£o com
host.
 *
 * **OperaÃ§ÃÃes**:
 * - Zera todas as amostras no buffer
 * - Posiciona head em Ãndice 0
 * - Reseta estado interno para condiÃ§ÃÃes conhecidas
 *
 * **Complexidade**:  $O(N)$  onde  $N = \text{Capacity}$ 
 *
 * **Contexto de chamada**:
 * - VST3: chamado em onActivate() do plugin
 * - iPlug2: chamado em OnReset() do plugin
 * - JUCE: chamado em prepareToPlay()
 * - CLAP: chamado em activate()
 * - Standalone: chamado durante inicializaÃ§Ã£o de Ãudio
 *
 * **Thread safety**: Deve ser chamado em thread de configuraÃ§Ã£o (nÃ£o em
audio thread)
 *
 * @return void
 *
 * @note Esta operaÃ§Ã£o **NÃ£o Ã© real-time safe** (realiza limpeza em
 $O(N)$ ).
 *      Nunca chamar durante processamento de Ãudio.
 */
void prepare() noexcept
{
    // Zera o buffer inteiro via memset otimizado pelo compilador
    // Para tipos aritmÃticos, isso Ã© seguro e eficiente
    std::memset(m_buffer.data(), 0, sizeof(m_buffer));

    // Reseta posiÃ§Ã£o de leitura/escrita para inÃcio
    m_head = 0;
}

/**
 * @brief Reseta o estado do buffer sem realocaÃ§Ã£o.
 *
 * **PropÃ³sito**: Limpeza rÃpida entre processamentos (ex: bypass, mute).
 *
 * **OperaÃ§ÃÃes**:
 * - Zera todas as amostras
 * - Posiciona head em Ãndice 0
 *
 * **Complexidade**:  $O(N)$  onde  $N = \text{Capacity}$ 
 *
 * **DiferenÃas de prepare()**:
 * - reset() pode ser chamado com mais frequÃncia
 * - Semanticamente equivalente a prepare() nesta implementaÃ§Ã£o
 *
 * **Contexto de chamada**:
 * - Bypass de efeito ativado
 * - Nota MIDI Note-Off sem sustain
 * - Reset de processador via UI
 */

```



```

* **Thread safety**: Deve ser chamado fora do audio thread
*
* @return void
*
* @note Esta operação é **NÃO real-time safe** (realiza limpeza em
O(N)).
*/
void reset() noexcept
{
    std::memset(m_buffer.data(), 0, sizeof(m_buffer));
    m_head = 0;
}

// =====
// OPERAÇÕES DE ESCRITA
// =====

/**
* @brief Escreve um valor no buffer em posição com offset relativo.
*
* **Matemática**:
* ```
* posição_física = (head + offset) & MODULO_MASK
* buffer[posição_física] = value
* ```
*
* **Complexidade**: O(1) constante
*
* **Uso típico - Delay Line**:
* ```cpp
* delayLine.write(0, inputSample); // Escreve entrada atual
* ```
*
* **Uso típico - Multi-tap delay**:
* ```cpp
* for (size_t tap = 0; tap < numTaps; ++tap) {
*     size_t delayIndex = (tap * delayTimeSamples) / numTaps;
*     buffer.write(delayIndex, processedSample);
* }
* ```
*
* **Validação de offset**:
* - offset >= Capacity: comportamento indefinido (DEBUG: assert falha)
* - A máscara de bits garante wrap automático para offsets válidos
*
* **Real-time safety**: NÃO SEGURO - O(1), sem alocação, sem exceção
*
* @param[in] offset Deslocamento relativo do head (0 ≤ offset < Capacity)
* @param[in] value Valor a ser escrito
*
* @return void
*
* @pre offset < Capacity (não verificado em Release, apenas em Debug via
assert)
*
* @post buffer[(head + offset) & MODULO_MASK] == value
*/
inline void write(size_type offset, const value_type value) noexcept
{
    // Validação em Debug (Zero Cost em Release via NDEBUG)
    // Garante que offset está dentro dos limites válidos
    assert(offset < Capacity && "CircularBuffer write offset out of
bounds");
}

```

```

    // Calcula Índice físico no buffer
    // & mais rápido que % para potências de 2
    const size_type physical_index = (m_head + offset) & MODULO_MASK;

    // Escreve valor
    m_buffer[physical_index] = value;
}

/**
 * @brief Escreve múltiplos valores consecutivos começando em offset.
 *
 * **Matemática**:
 * Para cada i em [0, count):
 * ```
 * posição_física[i] = (head + offset + i) & MODULO_MASK
 * buffer[posição_física[i]] = source[i]
 * ```
 *
 * **Características**:
 * - Otimizado para câpias contínuas
 * - Pode fazer até 2 memcpy internamente (wrap around)
 * - Usa memcpy quando disponível (mais rápido)
 *
 * **Complexidade**: O(count) linear
 *
 * **Uso típico - Frame de áudio**:
 * ```cpp
 * float inputFrame[512];
 * fftBuffer.write(0, inputFrame, 512); // Escreve frame inteiro
 * ```
 *
 * **Uso com wrap-around**:
 * ```cpp
 * // Se head + offset + count > Capacity, copia em duas partes:
 * // Parte 1: from head+offset to end
 * // Parte 2: from start to remainder
 * buffer.write(lastPos, data, 256); // Pode cruzar limite!
 * ```
 *
 * **Validação**:
 * - offset + count <= Capacity: garantido por assert em Debug
 *
 * **Real-time safety**: " SEGURO - O(count), sem alocação, sem
exceção
 *
 * @param[in] offset Deslocamento inicial relativo ao head
 * @param[in] source Ponteiro para dados a serem copiados
 * @param[in] count Número de amostras a copiar
 *
 * @return void
 *
 * @pre offset + count <= Capacity (não verificado em Release)
 * @pre source != nullptr
 *
 * @post Buffer contém count amostras de source a partir de offset
 */
inline void write(size_type offset, const value_type* source, size_type
count) noexcept
{
    // Validação em Debug
    assert(source != nullptr && "CircularBuffer write source pointer is
null");
    assert((offset + count) <= Capacity &&
"CircularBuffer write would exceed buffer bounds");

```

```

// Calcula Índice físico inicial
size_type physical_index = (m_head + offset) & MODULO_MASK;

// Verifica se há wrap-around
if (physical_index + count <= Capacity)
{
    // Caso simples: dados contíguos no buffer
    // Usa memcpy para eficiência máxima
    std::memcpy(&m_buffer[physical_index], source, count *
sizeof(value_type));
}
else
{
    // Caso complexo: dados cruzam limite circular
    // Copia em duas partes:
    // Parte 1: do physical_index até final do buffer
    const size_type part1_size = Capacity - physical_index;
    std::memcpy(&m_buffer[physical_index],
source,
part1_size * sizeof(value_type));

    // Parte 2: do início do buffer até Índice final
    const size_type part2_size = count - part1_size;
    std::memcpy(&m_buffer[0],
&source[part1_size],
part2_size * sizeof(value_type));
}
}

// =====
// OPERAÇÕES DE LEITURA
// =====

/**
 * @brief Lê um valor do buffer com offset relativo ao head.
 *
 * **Matemática**:
 * ```
 * posição_física = (head + offset) & MODULO_MASK
 * retorna buffer[posição_física]
 * ```
 *
 * **Complexidade**:  $O(1)$  constante
 *
 * **Uso típico - Delay Line (Tapped Delay)**:
 * ```cpp
 * // Lê amostras em diferentes pontos do delay
 * float oldest = delayLine.read(0);           // Amostra mais antiga
 * float middle = delayLine.read(Capacity/2); // Meio do buffer
 * float recent = delayLine.read(Capacity-1);  // Mais recente
 * ```
 *
 * **Uso com interpolação linear**:
 * ```cpp
 * // Delay fracionário: 2.5 amostras
 * float delayFrac = 2.5f;
 * size_t delayInt = static_cast<size_t>(delayFrac);
 * float frac = delayFrac - delayInt;
 *
 * float sample0 = delayLine.read(delayInt);
 * float sample1 = delayLine.read(delayInt + 1);
 * float interpolated = sample0 + frac * (sample1 - sample0);
 * ```

```

```

*
* **Offset out-of-bounds**:
* - offset >= Capacity: comportamento indefinido (DEBUG: assert falha)
* - A máscara garante wrap automático para offsets válidos
*
* **Real-time safety**: " SEGURO - O(1), sem alocação, sem exceção
*
* @param[in] offset Deslocamento relativo ao head (0 ≤ offset < Capacity)
*
* @return value_type Valor lido do buffer
*
* @pre offset < Capacity (não verificado em Release)
*/
inline value_type read(size_type offset) const noexcept
{
    // Validação em Debug
    assert(offset < Capacity && "CircularBuffer read offset out of bounds");

    // Calcula índice físico
    const size_type physical_index = (m_head + offset) & MODULO_MASK;

    // Lê e retorna valor
    return m_buffer[physical_index];
}

/**
* @brief Lê múltiplos valores consecutivos começando em offset.
*
* **Matemática**:
* Para cada i em [0, count):
* ```
* posição_física[i] = (head + offset + i) & MODULO_MASK
* destination[i] = buffer[posição_física[i]]
* ```
*
* **Características**:
* - Otimizado para leituras contínuas
* - Maneja automaticamente wrap-around
* - Usa memcpy quando possível
*
* **Complexidade**: O(count) linear
*
* **Uso típico - FFT Readout**:
* ```cpp
* float fftFrame[2048];
* fftBuffer.read(0, fftFrame, 2048); // Lê frame inteiro para
processamento FFT
* ```
*
* **Uso com deslocamento**:
* ```cpp
* // Lê histórico de amostras antigas
* float history[256];
* delayLine.read(100, history, 256); // Começa 100 amostras atrás
*
* **Validação**:
* - offset + count ≤ Capacity: garantido por assert em Debug
*
* **Real-time safety**: " SEGURO - O(count), sem alocação, sem
exceção
*
* @param[in] offset Deslocamento inicial relativo ao head
* @param[out] destination Ponteiro para buffer destino

```

```

* @param[in] count Número de amostras a ler
*
* @return void
*
* @pre offset + count <= Capacity (não verificado em Release)
* @pre destination != nullptr
*
* @post destination contém count amostras do buffer a partir de offset
*/
inline void read(size_type offset, value_type* destination, size_type count)
const noexcept
{
    // Validações em Debug
    assert(destination != nullptr && "CircularBuffer read destination
pointer is null");
    assert((offset + count) <= Capacity &&
           "CircularBuffer read would exceed buffer bounds");

    // Calcula índice físico inicial
    size_type physical_index = (m_head + offset) & MODULO_MASK;

    // Verifica se há wrap-around
    if (physical_index + count <= Capacity)
    {
        // Caso simples: dados contíguos no buffer
        std::memcpy(destination, &m_buffer[physical_index], count *
sizeof(value_type));
    }
    else
    {
        // Caso complexo: dados cruzam limite circular
        // Copia em duas partes:
        // Parte 1: do physical_index até final do buffer
        const size_type part1_size = Capacity - physical_index;
        std::memcpy(destination,
                    &m_buffer[physical_index],
                    part1_size * sizeof(value_type));

        // Parte 2: do início do buffer até índice final
        const size_type part2_size = count - part1_size;
        std::memcpy(&destination[part1_size],
                    &m_buffer[0],
                    part2_size * sizeof(value_type));
    }
}

// =====
// OPERAÇÕES DE POSICIONAMENTO
// =====

/**
* @brief Avança o head para a próxima posição (incremento cíclico).
*
* **Matemática**:
* ...
* head = (head + 1) & MODULO_MASK
* ...
*
* **Propósito**: Move o "cursor" de escrita para a próxima posição
circular.
*
* **Complexidade**: O(1) constante
*
* **Semântica**:

```

```

* - Após escrever uma amostra em write(0, sample), deve-se chamar
advance()
* - Próxima chamada write(0, sample) sobrescreverá; posição anterior
mais nova
* - Offset 0 sempre referencia a posição do head
*
* **Uso típico - Delay Line Sample-by-Sample**:
* ```cpp
* for (size_t n = 0; n < blockSize; ++n) {
*     float input = inputBuffer[n];
*     float delayed = delayLine.read(delaySamples); // Lê amostra
atrasada
*     outputBuffer[n] = delayed;
*     delayLine.write(0, input); // Escreve entrada
atual
*     delayLine.advance(); // Move para próxima
* }
* ...
*
* **Uso em feedback loop**:
* ```cpp
* // Schroeder reverberator (comb filter em paralelo)
* float output = 0.0f;
* for (size_t i = 0; i < numCombFilters; ++i) {
*     float delayed = combFilters[i].read(0);
*     float fed = delayed * feedbackGain;
*     output += delayed;
*     combFilters[i].write(0, input + fed);
*     combFilters[i].advance();
* }
* ...
*
* **Real-time safety**: " SEGURO - O(1), sem alocação, sem exceção
*
* @return void
*
* @post head = (head_anterior + 1) & MODULO_MASK
*/
inline void advance() noexcept
{
    // Incremento cíclico usando máscara de bits
    // Equivalente a: head = (head + 1) % Capacity
    // Mas muito mais eficiente (uma operação bitwise!)
    m_head = (m_head + 1) & MODULO_MASK;
}

/**
* @brief Escreve uma amostra na posição atual e avança o head.
*
* Conveniência real-time safe para delay lines e motores de convolução
que
* precisam inserir uma amostra por chamada de processamento. Equivale a
* write(0, value) seguido de advance().
*
* @param value Amostra a inserir.
*/
inline void push(value_type value) noexcept
{
    write(0, value);
    advance();
}

/**
* @brief Avança o head por múltiplas posições.

```

```

*
* **Matemática**:
* ```
* head = (head + count) & MODULO_MASK
* ```
*
* **Propósito**: Avançar múltiplas amostras de uma só vez.
*
* **Complexidade**: O(1) constante
*
* **Uso típico - Frame-based Processing**:
* ```cpp
* // Processa frame de 512 amostras
* for (size_t n = 0; n < 512; ++n) {
*     outputFrame[n] = fftBuffer.read(n);
*     fftBuffer.write(n, processedFrame[n]);
* }
* fftBuffer.advance(512); // Avançar frame inteiro
* ```
*
* **Uso com múltiplos taps de delay**:
* ```cpp
* // Pipelined delay lines
* for (size_t tap = 0; tap < numTaps; ++tap) {
*     tapLines[tap].advance(delayIncrementSamples);
* }
* ```
*
* **Validação**:
* - count pode ser qualquer valor (wrap automático)
* - count > Capacity é válido (wraps múltiplas vezes)
*
* **Real-time safety**: " SEGURO - O(1), sem alocação, sem exceção
*
* @param[in] count Número de posições a avançar
*
* @return void
*
* @post head = (head_anterior + count) & MODULO_MASK
*/
inline void advance(size_type count) noexcept
{
    // Incremento múltiplo com wrap automático
    m_head = (m_head + count) & MODULO_MASK;
}

// =====
// OPERAÇÕES DE CONSULTA
// =====

/**
* @brief Retorna o tamanho (capacidade) do buffer em amostras.
*
* **Retorna**: Capacity (constante conhecida em tempo de compilação)
*
* **Complexidade**: O(1) constante
*
* **Uso típico - Alocação de buffers auxiliares**:
* ```cpp
* size_t bufSize = delayLine.size();
* float* tempBuffer = new float[bufSize]; // NÃO fazer em Real-Time!
* ```
*
* **Melhor prática - Pré-alocar**:

```

```

* ``cpp
* // Stack-allocated buffer com tamanho conhecido
* float tempBuffer[CircularBuffer<float, 4800>::BUFFER_CAPACITY];
*
*
* **Real-time safety**: " SEGURO - O(1), constexpr
*
* @return size_type Tamanho do buffer (= Capacity)
*/
static constexpr size_type size() noexcept
{
    return Capacity;
}

/**
* @brief Retorna a capacidade do buffer (alias para size()).
*
* **Retorna**: Capacity (constante conhecida em tempo de compilação)
*
* **Diferença de size()**:
* - Semanticamente equivalentes nesta implementação
* - capacity() segue convenção de STL containers
*
* **Complexidade**: O(1) constante
*
* **Uso típico - Verificação de limites**:
* ``cpp
* if (offset < buffer.capacity()) {
*     value = buffer.read(offset); // Seguro
* }
*
*
* **Real-time safety**: " SEGURO - O(1), constexpr
*
* @return size_type Capacidade do buffer (= Capacity)
*/
static constexpr size_type capacity() noexcept
{
    return Capacity;
}

/**
* @brief Retorna a posição atual do head no buffer.
*
* **Retorna**: Índice físico do head (0 ≤ head < Capacity)
*
* **Propósito**: Introspection para debugging e sincronização.
*
* **Complexidade**: O(1) constante
*
* **Uso típico - Debugging em Development**:
* ``cpp
* if (delayLine.head() == 0) {
*     // Buffer completamente rotacionado uma vez
*     logDebug("Delay line wrapped");
* }
*
*
* **Uso com resampling (exemplo avançado)**:
* ``cpp
* // Sincroniza múltiplos buffers circulares
* if (bufferA.head() != bufferB.head()) {
*     size_t headDiff = (bufferA.head() - bufferB.head()) & MODULO_MASK;
*     // Adjust bufferB to match bufferA...

```



```

* }
* \...
*
* **Real-time safety**: " SEGURO - O(1), sem alocação, sem exceção
*
* @return size_type Posição atual do head (0 ≤ valor < Capacity)
*/
inline size_type head() const noexcept
{
    return m_head;
}

/**
* @brief Define manualmente a posição do head.
*
* **Propósito**: Controle explícito de posicionamento (casos avançados).
*
* **Complexidade**: O(1) constante
*
* **Uso cuidadoso - Sincronização de sessão**:
* ```cpp
* // DAW fornece posição de transportador
* size_t daw_position = host->getTransportPosition();
* size_t buffer_position = daw_position % buffer.capacity();
* buffer.set_head(buffer_position);
* ```
*
* **Aviso**: Pode causar desconexão em processamento contínuo!
*
* **Real-time safety**: " CUIDADO - O(1), mas pode quebrar continuidade
de sinal
*
* @param[in] new_head Nova posição do head (0 ≤ new_head < Capacity)
*
* @return void
*
* @warning Use apenas quando sincronização é necessária. Pode
introduzir cliques!
*/
inline void set_head(size_type new_head) noexcept
{
    // Validação em Debug
    assert(new_head < Capacity && "CircularBuffer set_head index out of
bounds");
    m_head = new_head;
}

// =====
// ACESSO DIRETO (BAIXO NÍVEL)
// =====

/**
* @brief Retorna ponteiro para buffer interno (acesso de baixo nível).
*
* **Propósito**: Otimizações avançadas e operações SIMD diretas.
*
* **Complexidade**: O(1) constante
*
* **Uso com SIMD (SSE/AVX)**:
* ```cpp
* // Processamento vetorizado de sequência contínua
* float* bufPtr = buffer.data();
* size_t headPhysical = buffer.head();
*

```

```

* // Processa 16 amostras com AVX
* for (size_t i = 0; i < Capacity / 16; ++i) {
*     __m256 vec = _mm256_loadu_ps(&bufPtr[headPhysical]);
*     // ... SIMD operations ...
*     _mm256_storeu_ps(&bufPtr[headPhysical], vec);
*     headPhysical = (headPhysical + 16) & MODULO_MASK;
* }
* ...
*
* **Aviso**: Operações em bufPtr podem desorganizar lógica circular!
*
* **Real-time safety**: " SEGURO - O(1), apenas leitura de ponteiro
*
* @return value_type* Ponteiro para início do buffer interno
*
* @warning Não modifique estrutura de dados circular via ponteiro
retornado!
*/
inline value_type* data() noexcept
{
    return m_buffer.data();
}

/**
* @brief Retorna ponteiro const para buffer interno (acesso de baixo nível).
*
* **Propósito**: Operações de leitura SIMD e introspection.
*
* **Complexidade**: O(1) constante
*
* **Uso com análise FFT**:
* ```cpp
* const float* bufPtr = buffer.data();
* // Analisa espectro sem copiar dados
* fft.process(bufPtr, bufPtr + buffer.capacity());
* ```
*
* **Real-time safety**: " SEGURO - O(1), apenas leitura
*
* @return const value_type* Ponteiro const para buffer interno
*/
inline const value_type* data() const noexcept
{
    return m_buffer.data();
}

private:
// =====
// MEMBROS PRIVADOS
// =====

/// @brief Posição atual do head (cursor de escrita)
/// Valor: 0 ≤ m_head < Capacity
/// Incrementa com advance() e wraps via máscara de bits
size_type m_head;

/// @brief Buffer circular de armazenamento
/// Tipo: std::array para alocação em stack, sem dynamicallocation
/// Tamanho: Capacity amostras (Capacity deve ser potência de 2)
/// Inicialização: Zero-initialized na construção
std::array<value_type, Capacity> m_buffer;
};

```

```

} // namespace cvdsp

#endif // CVDSP_CORE_CIRCULARBUFFER_HPP

=====
FILE: CV_DSP/Core/Config.hpp
=====
#ifndef CVDSP_CORE_CONFIG_HPP
#define CVDSP_CORE_CONFIG_HPP

/**
 * @file Config.hpp
 * @brief Global compile-time configuration flags for CV_DSP.
 *
 * The values in this header describe default library policy and are expressed
 * as `inline constexpr` variables to avoid preprocessor-heavy configuration.
 * Projects may include this header from plug-in framework code, offline tools,
 * or tests without triggering allocation, exceptions, RTTI, or runtime setup.
 *
 * @note No DSP processing is implemented in this header.
 */

namespace cvdsp
{
/**
 * @brief Enables internal assertion checks in CV_DSP components.
 *
 * The foundational library defaults to enabled assertions so future debug-time
 * validation can be compiled from a single policy value without requiring
 * framework-specific macros. Assertion mechanisms must remain outside real-time
 * audio paths unless they are proven real-time safe.
 */
inline constexpr bool EnableAssertions = true;

/**
 * @brief Enables denormal-number protection policy for future DSP components.
 *
 * Denormal protection is enabled by default because subnormal floating-point
 * values can cause severe CPU spikes in real-time audio callbacks. Concrete DSP
 * modules should implement this policy without heap allocation and without
 * throwing exceptions inside `process()`.
 */
inline constexpr bool EnableDenormalProtection = true;
} // namespace cvdsp

#endif // CVDSP_CORE_CONFIG_HPP

=====
FILE: CV_DSP/Core/Constants.hpp
=====
#ifndef CVDSP_CORE_CONSTANTS_HPP
#define CVDSP_CORE_CONSTANTS_HPP

/**
 * @file Constants.hpp
 * @brief Compile-time mathematical constants for CV_DSP.
 *
 * This header provides typed mathematical constants for single-precision and
 * double-precision DSP code. Constants are exposed as variable templates so
 * call sites can request the required scalar type without implicit narrowing.

```

```

*
* @note Constants are `inline constexpr` and therefore require no storage with
*       external linkage beyond normal C++20 inline variable semantics.
* @note This header is real-time safe: it performs no allocation, contains no
*       executable runtime work, and introduces no exceptions or RTTI usage.
*/

#include "Types.hpp"

namespace cvdsp
{
/**
 * @brief Archimedes' constant  $\pi$ .
 * @tparam T Floating-point scalar type, typically cvdsp::f32 or cvdsp::f64.
 */
template <typename T>
inline constexpr T pi = static_cast<T>(3.141592653589793238462643383279502884L);

/**
 * @brief Full-cycle radian constant  $2\pi$ .
 * @tparam T Floating-point scalar type, typically cvdsp::f32 or cvdsp::f64.
 */
template <typename T>
inline constexpr T twoPi =
static_cast<T>(6.283185307179586476925286766559005768L);

/**
 * @brief Half-cycle quadrant radian constant  $\pi/2$ .
 * @tparam T Floating-point scalar type, typically cvdsp::f32 or cvdsp::f64.
 */
template <typename T>
inline constexpr T halfPi =
static_cast<T>(1.570796326794896619231321691639751442L);

/**
 * @brief Multiplicative inverse of  $\pi$ .
 * @tparam T Floating-point scalar type, typically cvdsp::f32 or cvdsp::f64.
 */
template <typename T>
inline constexpr T invPi =
static_cast<T>(0.318309886183790671537767526745028724L);

/**
 * @brief Principal square root of 2.
 * @tparam T Floating-point scalar type, typically cvdsp::f32 or cvdsp::f64.
 */
template <typename T>
inline constexpr T sqrt2 =
static_cast<T>(1.414213562373095048801688724209698079L);

/**
 * @brief Multiplicative inverse of the principal square root of 2.
 * @tparam T Floating-point scalar type, typically cvdsp::f32 or cvdsp::f64.
 */
template <typename T>
inline constexpr T invSqrt2 =
static_cast<T>(0.707106781186547524400844362104849039L);

/**
 * @brief Euler's number e, the base of the natural logarithm.
 * @tparam T Floating-point scalar type, typically cvdsp::f32 or cvdsp::f64.
 */
template <typename T>
inline constexpr T euler =

```

```
static_cast<T>(2.718281828459045235360287471352662498L);
} // namespace cvdsp
```

```
#endif // CVDSP_CORE_CONSTANTS_HPP
```

```
=====
FILE: CV_DSP/Core/DSPUtils.hpp
=====
```

```
#ifndef CVDSP_CORE_DSPUTILS_HPP
#define CVDSP_CORE_DSPUTILS_HPP
```

```
/**
 * @file DSPUtils.hpp
 * @brief Common DSP Utility Functions
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Centralized utility functions for the entire CV_DSP library.
 *
 * All generic reusable DSP math should live here.
 *
 * Dependencies:
 * - Core/Constants.hpp
 * - Core/Types.hpp
 */
```

```
#include <algorithm>
#include <cmath>
#include <limits>
#include <type_traits>
```

```
#include "Constants.hpp"
#include "Types.hpp"
```

```
namespace cvdsp
{
```

```
/**
 * @brief Utility DSP functions.
 *
 * Static-only utility class.
 */
```

```
class DSPUtils
{
public:
```

```
    DSPUtils() = delete;
```

```
    DSPUtils(const DSPUtils&) = delete;
    DSPUtils& operator=(const DSPUtils&) = delete;
```

```
/**
 * @brief Convert dB to linear gain.
 *
 * gain = 10^(dB/20)
 */
```

```
template<typename T>
[[nodiscard]]
static inline T dbToGain(
    T dB) noexcept
```

```

{
    static_assert(
        std::is_floating_point_v<T>);

    return std::pow(
        static_cast<T>(10),
        dB / static_cast<T>(20));
}

/**
 * @brief Convert linear gain to dB.
 *
 * dB = 20*log10(gain)
 */
template<typename T>
[[nodiscard]]
static inline T gainToDb(
    T gain) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);

    constexpr T minGain =
        static_cast<T>(1e-20);

    gain =
        std::max(
            gain,
            minGain);

    return
        static_cast<T>(20)
        *
        std::log10(gain);
}

/**
 * @brief Clamp value.
 */
template<typename T>
[[nodiscard]]
static constexpr T clamp(
    T value,
    T minimum,
    T maximum) noexcept
{
    return
        value < minimum
        ? minimum
        : (
            value > maximum
            ? maximum
            : value
        );
}

/**
 * @brief Linear interpolation.
 *
 * t = 0 -> a
 * t = 1 -> b
 */
template<typename T>
[[nodiscard]]

```

```

static constexpr T lerp(
    T a,
    T b,
    T t) noexcept
{
    return
        a
        +
        (
            b - a
        )
        *
        t;
}

/**
 * @brief Linear range mapping.
 */
template<typename T>
[[nodiscard]]
static constexpr T mapLinear(
    T value,
    T inMin,
    T inMax,
    T outMin,
    T outMax) noexcept
{
    if (inMax == inMin)
    {
        return outMin;
    }

    const T normalized =
        (
            value - inMin
        )
        /
        (
            inMax - inMin
        );

    return
        outMin
        +
        normalized
        *
        (
            outMax - outMin
        );
}

/**
 * @brief Logarithmic mapping.
 *
 * Useful for:
 * - Frequency controls
 * - Time controls
 * - Audio parameters
 */
template<typename T>
[[nodiscard]]
static inline T mapLog(
    T normalized,
    T minimum,

```

```

    T maximum) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);

    normalized =
        clamp(
            normalized,
            static_cast<T>(0),
            static_cast<T>(1));

    minimum =
        std::max(
            minimum,
            static_cast<T>(1e-12));

    const T ratio =
        maximum
        /
        minimum;

    return
        minimum
        *
        std::pow(
            ratio,
            normalized);
}

```

```

/**
 * @brief Wrap value into interval.
 *
 * Example:
 *
 * wrap(1.2,0,1) -> 0.2
 * wrap(-0.2,0,1) -> 0.8
 */

```

```

template<typename T>
[[nodiscard]]
static inline T wrap(
    T value,
    T minimum,
    T maximum) noexcept
{
    const T range =
        maximum
        -
        minimum;

    if (range <= static_cast<T>(0))
    {
        return minimum;
    }

    while (value >= maximum)
    {
        value -= range;
    }

    while (value < minimum)
    {
        value += range;
    }
}

```



```

        return value;
    }

/**
 * @brief Return sign of value.
 *
 * Returns:
 *
 * -1
 *  0
 * +1
 */
template<typename T>
[[nodiscard]]
static constexpr int sign(
    T value) noexcept
{
    return
        (T(0) < value)
        -
        (value < T(0));
}

/**
 * @brief Detect denormal number.
 *
 * Denormals can severely impact CPU usage.
 */
template<typename T>
[[nodiscard]]
static inline bool isDenormal(
    T value) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);

    return
        std::fpclassify(value)
        ==
        FP_SUBNORMAL;
}

/**
 * @brief Remove denormal values.
 *
 * Returns zero if value is denormal.
 */
template<typename T>
[[nodiscard]]
static inline T killDenormal(
    T value) noexcept
{
    static_assert(
        std::is_floating_point_v<T>);

    return
        isDenormal(value)
        ? static_cast<T>(0)
        : value;
}

/**
 * @brief Remove extremely small values.
 *

```

```

    * Faster alternative to fpclassify.
    */
template<typename T>
[[nodiscard]]
static constexpr T killTiny(
    T value,
    T threshold =
        static_cast<T>(1e-30)) noexcept
{
    return
        (
            value > -threshold
            &&
            value < threshold
        )
        ? static_cast<T>(0)
        : value;
}

/**
 * @brief Normalize value to 0..1 range.
 */
template<typename T>
[[nodiscard]]
static constexpr T normalize(
    T value,
    T minimum,
    T maximum) noexcept
{
    if (maximum == minimum)
    {
        return static_cast<T>(0);
    }

    return
        (
            value - minimum
        )
        /
        (
            maximum - minimum
        );
}

/**
 * @brief Check finite number.
 */
template<typename T>
[[nodiscard]]
static inline bool isFinite(
    T value) noexcept
{
    return std::isfinite(value);
}

/**
 * @brief Safe reciprocal.
 */
template<typename T>
[[nodiscard]]
static inline T reciprocal(
    T value,
    T epsilon =
        static_cast<T>(1e-20)) noexcept

```

```

{
    if (std::abs(value) < epsilon)
    {
        return static_cast<T>(0);
    }

    return
        static_cast<T>(1)
        /
        value;
}

/**
 * @brief Safe division.
 */
template<typename T>
[[nodiscard]]
static inline T safeDivide(
    T numerator,
    T denominator,
    T epsilon =
        static_cast<T>(1e-20)) noexcept
{
    if (std::abs(denominator) < epsilon)
    {
        return static_cast<T>(0);
    }

    return
        numerator
        /
        denominator;
}

/**
 * @brief Convert milliseconds to samples.
 */
template<typename T>
[[nodiscard]]
static constexpr T msToSamples(
    T milliseconds,
    T sampleRate) noexcept
{
    return
        milliseconds
        *
        sampleRate
        *
        static_cast<T>(0.001);
}

/**
 * @brief Convert samples to milliseconds.
 */
template<typename T>
[[nodiscard]]
static constexpr T samplesToMs(
    T samples,
    T sampleRate) noexcept
{
    if (sampleRate <= static_cast<T>(0))
    {
        return static_cast<T>(0);
    }
}

```

```

        return
            samples
            *
            static_cast<T>(1000)
            /
            sampleRate;
    }
};

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Core/Namespace.hpp
=====

```

```

#ifndef CVDSP_CORE_NAMESPACE_HPP
#define CVDSP_CORE_NAMESPACE_HPP

```

```

/**
 * @file Namespace.hpp
 * @brief Minimal portability annotations for CV_DSP declarations.
 *
 * This header intentionally defines only the smallest macro surface needed by
 * the foundation layer. The macros are limited to compiler attributes that are
 * useful for real-time DSP code while keeping CV_DSP compatible with VST3 SDK,
 * iPlug2, JUCE, CLAP, and standalone C++20 builds.
 *
 * @note No namespace-opening or namespace-closing macros are provided; use the
 * explicit `namespace cvdsp { ... }` form in source code for clarity.
 */

```

```

/**
 * @def CVDSP_FORCE_INLINE
 * @brief Requests aggressive inlining for small real-time safe functions.
 *
 * The macro maps to compiler-specific force-inline attributes when available
 * and falls back to standard `inline` elsewhere. It should be used sparingly on
 * short functions that are expected to sit on the audio processing hot path.
 */

```

```

#if defined(_MSC_VER)
#define CVDSP_FORCE_INLINE __forceinline
#elif defined(__GNUC__) || defined(__clang__)
#define CVDSP_FORCE_INLINE inline __attribute__((always_inline))
#else
#define CVDSP_FORCE_INLINE inline
#endif

```

```

/**
 * @def CVDSP_NODISCARD
 * @brief Marks return values that should not be ignored by callers.
 *
 * The macro maps directly to the C++20 `[[nodiscard]]` attribute and exists as
 * a stable annotation point for future compiler-specific policy if required.
 */

```

```

#define CVDSP_NODISCARD [[nodiscard]]

```

```

namespace cvdsp
{
} // namespace cvdsp

```

```
#endif // CVDSP_CORE_NAMESPACE_HPP
```

```
=====
FILE: CV_DSP/Core/ParameterSmoother.hpp
=====
```

```
#ifndef CVDSP_CORE_PARAMETER_SMOOTHER_HPP
```

```
#define CVDSP_CORE_PARAMETER_SMOOTHER_HPP
```

```
/**
```

```
 * @file ParameterSmoother.hpp
```

```
 * @brief Suavizadores de parâmetros em tempo real para automação sample accurate.
```

```
 *
```

```
 * O arquivo implementa três estratégias sem alocação dinâmica:
```

```
 * - LinearSmoother: rampa linear com incremento constante.
```

```
 * - ExponentialSmoother: rampa exponencial normalizada, com chegada exata ao alvo.
```

```
 * - OnePoleSmoother: filtro passa-baixas de primeira ordem aplicado ao parâmetro.
```

```
 *
```

```
 * Compatibilidade com automação VST3 sample accurate:
```

```
 * os suavizadores são independentes de bloco. Chame setTarget() exatamente no sample
```

```
 * indicado pelo evento de automação do host e chame process() uma vez por sample.
```

```
 * Nenhuma classe aloca memória, lança exceções ou depende de estado global.
```

```
 *
```

```
 * Exemplo genérico de uso com eventos sample accurate:
```

```
 * @code{.cpp}
```

```
 * cvdsp::LinearSmoother<float> gain;
```

```
 * gain.prepare(48000.0f, 0.005f); // 5 ms
```

```
 * gain.reset(0.0f);
```

```
 *
```

```
 * for (uint32_t i = 0; i < numSamples; ++i)
```

```
 * {
```

```
 *     if (automationEventAt(i))
```

```
 *         gain.setTarget(eventValueAt(i)); // aplicado no offset exato do host
```

```
 *
```

```
 *     const float g = gain.process();
```

```
 *     left[i] *= g;
```

```
 *     right[i] *= g;
```

```
 * }
```

```
 * @endcode
```

```
 *
```

```
 * Exemplo com duração explícita em samples, até quando o host ou plugin decide
```

```
 * o tamanho da transição por evento:
```

```
 * @code{.cpp}
```

```
 * cvdsp::ExponentialSmoother<double> cutoff;
```

```
 * cutoff.prepare(48000.0, 0.020, 6.0); // padrão: 20 ms, curva mais acentuada
```

```
 * cutoff.reset(1000.0);
```

```
 * cutoff.setTarget(8000.0, 128); // rampa exatamente por 128 samples
```

```
 * const double hz = cutoff.process();
```

```
 * @endcode
```

```
 *
```

```
 * Exemplo de one-pole para parâmetros que podem perseguir continuamente o alvo:
```

```
 * @code{.cpp}
```

```
 * cvdsp::OnePoleSmoother<float> pan;
```

```
 * pan.prepare(48000.0f, 0.010f); // constante de tempo tau = 10 ms
```

```
 * pan.reset(0.0f);
```

```
 * pan.setTarget(1.0f);
```

```

* const float smoothPan = pan.process();
* @endcode
*
* Matemática:
*
* Linear Ramp:
* dado valor inicial x0, alvo xT e N samples, o incremento é
*  $d = (xT - x0) / N$ .
* A sequência é
*  $y[n] = x0 + n d$ ,  $1 \leq n \leq N$ ,
* com y[N] forçado para xT para evitar erro acumulado de ponto flutuante.
* A velocidade da mudança é constante; a primeira derivada é descontínua nos
* pontos de início/fim da rampa.
*
* Exponential Ramp:
* esta implementação usa uma curva exponencial normalizada para funcionar com
* valores positivos, zero, negativos e cruzamentos de sinal:
*  $p[n] = (1 - \exp(-k n / N)) / (1 - \exp(-k))$ ,  $1 \leq n \leq N$ ,
*  $y[n] = x0 + (xT - x0) p[n]$ .
* k controla a curvatura; k próximo de zero se aproxima de uma rampa linear,
* k maior gera movimento inicial mais rápido e aproxima-se do final mais lenta.
* Como  $p[N] = 1$ , a rampa termina exatamente em xT.
*
* One Pole Filter:
* o parâmetro é filtrado por um passa-baixas de primeira ordem:
*  $y[n] = y[n-1] + a (x[n] - y[n-1])$ ,
*  $a = 1 - \exp(-1 / (\tau Fs))$ .
* tau é a constante de tempo em segundos e Fs é a taxa de amostragem. Após
tau
* segundos, a resposta ao degrau percorre aproximadamente 63,2% da distância
* até o alvo. Diferente das rampas de N samples, o one-pole se aproxima
* assintoticamente do alvo.
*/

```

```

#include "Namespace.hpp"
#include "Types.hpp"

```

```

#include <cmath>
#include <limits>
#include <type_traits>

```

```

namespace cvdsp
{
namespace detail
{
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T clampToNonNegative(const T value)
noexcept
{
    return value > static_cast<T>(0) ? value : static_cast<T>(0);
}
}

```

```

template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr u32 secondsToSamples(const T
sampleRate,
                                                                    const T
seconds) noexcept
{
    const T safeSampleRate = sampleRate > static_cast<T>(0) ? sampleRate :
static_cast<T>(0);
    const T safeSeconds = clampToNonNegative(seconds);
    const T samples = safeSampleRate * safeSeconds;

    if (samples <= static_cast<T>(0))

```

```

        return 0;

constexpr T maxU32 = static_cast<T>(std::numeric_limits<u32>::max());
if (samples >= maxU32)
    return std::numeric_limits<u32>::max();

    return static_cast<u32>(samples + static_cast<T>(0.5));
}
} // namespace detail

/**
 * @brief Rampa linear de par metro com dura  o configur vel.
 *
 * @tparam T Tipo de ponto flutuante, normalmente float ou double.
 */
template <typename T = f32>
class LinearSmoother
{
    static_assert(std::is_floating_point_v<T>, "LinearSmoother requires a
floating-point type");

public:
    using value_type = T;

    /**
     * @brief Configura a taxa de amostragem e a dura  o padr o da rampa.
     */
    CVDSP_FORCE_INLINE void prepare(const T sampleRate, const T rampTimeSeconds)
    noexcept
    {
        sampleRate_ = sampleRate > static_cast<T>(0) ? sampleRate :
static_cast<T>(0);
        defaultRampSamples_ = detail::secondsToSamples(sampleRate_,
rampTimeSeconds);
    }

    /**
     * @brief Reinicia o estado imediatamente para value.
     */
    CVDSP_FORCE_INLINE void reset(const T value = static_cast<T>(0)) noexcept
    {
        current_ = value;
        target_ = value;
        step_ = static_cast<T>(0);
        samplesRemaining_ = 0;
    }

    /**
     * @brief Define o alvo usando a dura  o padr o configurada em prepare().
     */
    CVDSP_FORCE_INLINE void setTarget(const T target) noexcept
    {
        setTarget(target, defaultRampSamples_);
    }

    /**
     * @brief Define o alvo usando uma dura  o expl cita em samples.
     */
    CVDSP_FORCE_INLINE void setTarget(const T target, const u32 rampSamples)
    noexcept
    {
        target_ = target;
        samplesRemaining_ = rampSamples;
    }

```

```

        if (samplesRemaining_ == 0)
        {
            current_ = target_;
            step_ = static_cast<T>(0);
            return;
        }

        step_ = (target_ - current_) / static_cast<T>(samplesRemaining_);
    }

    /**
     * @brief Processa um sample de suaviza  o e retorna o valor atual.
     */
    CVDSP_NODISCARD CVDSP_FORCE_INLINE T process() noexcept
    {
        if (samplesRemaining_ == 0)
            return current_;

        --samplesRemaining_;
        if (samplesRemaining_ == 0)
        {
            current_ = target_;
            step_ = static_cast<T>(0);
            return current_;
        }

        current_ += step_;
        return current_;
    }

    CVDSP_NODISCARD CVDSP_FORCE_INLINE T getCurrentValue() const noexcept
    { return current_; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE T getTargetValue() const noexcept
    { return target_; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE bool isSmoothing() const noexcept
    { return samplesRemaining_ != 0; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE u32 getSamplesRemaining() const noexcept
    { return samplesRemaining_; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE T getSampleRate() const noexcept { return
sampleRate_; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE u32 getDefaultRampSamples() const
noexcept { return defaultRampSamples_; }

private:
    T sampleRate_ = static_cast<T>(0);
    T current_ = static_cast<T>(0);
    T target_ = static_cast<T>(0);
    T step_ = static_cast<T>(0);
    u32 defaultRampSamples_ = 0;
    u32 samplesRemaining_ = 0;
};

/**
 * @brief Rampa exponencial normalizada com chegada exata ao alvo.
 *
 * A forma normalizada evita as restri  es da rampa exponencial multiplicativa
 * cl  ssica, portanto    segura para par  metros que podem ser zero, negativos
ou
 * cruzar zero. O custo    uma chamada a std::exp por sample enquanto h   rampa.
 */
template <typename T = f32>
class ExponentialSmoother
{
    static_assert(std::is_floating_point_v<T>, "ExponentialSmoother requires a

```



```

floating-point type");

public:
    using value_type = T;

    /**
     * @brief Configura taxa de amostragem, duração padrão e curvatura.
     *
     * @param curve Valor positivo. Valores próximos de zero aproximam uma
linha;      valores maiores deixam o começo mais rápido e o fim mais suave.
     */
    CVDSP_FORCE_INLINE void prepare(const T sampleRate,
                                    const T rampTimeSeconds,
                                    const T curve = static_cast<T>(5)) noexcept
    {
        sampleRate_ = sampleRate > static_cast<T>(0) ? sampleRate :
static_cast<T>(0);
        defaultRampSamples_ = detail::secondsToSamples(sampleRate_,
rampTimeSeconds);
        setCurve(curve);
    }

    CVDSP_FORCE_INLINE void reset(const T value = static_cast<T>(0)) noexcept
    {
        start_ = value;
        current_ = value;
        target_ = value;
        totalSamples_ = 0;
        samplesRemaining_ = 0;
        position_ = 0;
    }

    CVDSP_FORCE_INLINE void setTarget(const T target) noexcept
    {
        setTarget(target, defaultRampSamples_);
    }

    CVDSP_FORCE_INLINE void setTarget(const T target, const u32 rampSamples)
noexcept
    {
        target_ = target;
        start_ = current_;
        totalSamples_ = rampSamples;
        samplesRemaining_ = rampSamples;
        position_ = 0;

        if (samplesRemaining_ == 0)
        {
            current_ = target_;
            start_ = target_;
        }
    }

    CVDSP_NODISCARD CVDSP_FORCE_INLINE T process() noexcept
    {
        if (samplesRemaining_ == 0)
            return current_;

        ++position_;
        --samplesRemaining_;

        if (samplesRemaining_ == 0 || position_ >= totalSamples_)
        {

```

```

        current_ = target_;
        start_ = target_;
        return current_;
    }

    const T phase = static_cast<T>(position_) /
static_cast<T>(totalSamples_);
    const T progress = isNearlyLinear_
                        ? phase
                        : (static_cast<T>(1) - std::exp(-curve_ * phase))
* normalization_;

    current_ = start_ + (target_ - start_) * progress;
    return current_;
}

/**
 * @brief Atualiza apenas a curvatura das próximas rampas.
 */
CVDSP_FORCE_INLINE void setCurve(const T curve) noexcept
{
    curve_ = curve > static_cast<T>(0) ? curve : static_cast<T>(0);
    isNearlyLinear_ = curve_ <= static_cast<T>(1.0e-6);
    normalization_ = isNearlyLinear_
                    ? static_cast<T>(1)
                    : static_cast<T>(1) / (static_cast<T>(1) -
std::exp(-curve_));
}

CVDSP_NODISCARD CVDSP_FORCE_INLINE T getCurrentValue() const noexcept
{ return current_; }
CVDSP_NODISCARD CVDSP_FORCE_INLINE T getTargetValue() const noexcept
{ return target_; }
CVDSP_NODISCARD CVDSP_FORCE_INLINE T getCurve() const noexcept { return
curve_; }
CVDSP_NODISCARD CVDSP_FORCE_INLINE bool isSmoothing() const noexcept
{ return samplesRemaining_ != 0; }
CVDSP_NODISCARD CVDSP_FORCE_INLINE u32 getSamplesRemaining() const noexcept
{ return samplesRemaining_; }
CVDSP_NODISCARD CVDSP_FORCE_INLINE T getSampleRate() const noexcept { return
sampleRate_; }
CVDSP_NODISCARD CVDSP_FORCE_INLINE u32 getDefaultRampSamples() const
noexcept { return defaultRampSamples_; }

private:
    T sampleRate_ = static_cast<T>(0);
    T start_ = static_cast<T>(0);
    T current_ = static_cast<T>(0);
    T target_ = static_cast<T>(0);
    T curve_ = static_cast<T>(5);
    T normalization_ = static_cast<T>(1) / (static_cast<T>(1) -
std::exp(static_cast<T>(-5)));
    u32 defaultRampSamples_ = 0;
    u32 totalSamples_ = 0;
    u32 samplesRemaining_ = 0;
    u32 position_ = 0;
    bool isNearlyLinear_ = false;
};

/**
 * @brief Suavizador one-pole: filtro passa-baixas de primeira ordem.
 *
 * % ideal para parâmetros continuamente atualizados. Como a convergência é
 * assintótica, use LinearSmoother ou ExponentialSmoother quando for

```

```

necessário
* alcançar exatamente o alvo após um número fixo de samples.
*/
template <typename T = f32>
class OnePoleSmoother
{
    static_assert(std::is_floating_point_v<T>, "OnePoleSmoother requires a
floating-point type");

public:
    using value_type = T;

    /**
     * @brief Configura a taxa de amostragem e a constante de tempo tau.
     */
    CVDSP_FORCE_INLINE void prepare(const T sampleRate, const T
timeConstantSeconds) noexcept
    {
        sampleRate_ = sampleRate > static_cast<T>(0) ? sampleRate :
static_cast<T>(0);
        timeConstantSeconds_ = detail::clampToNonNegative(timeConstantSeconds);
        updateCoefficient();
    }

    CVDSP_FORCE_INLINE void reset(const T value = static_cast<T>(0)) noexcept
    {
        current_ = value;
        target_ = value;
    }

    CVDSP_FORCE_INLINE void setTarget(const T target) noexcept
    {
        target_ = target;
    }

    CVDSP_NODISCARD CVDSP_FORCE_INLINE T process() noexcept
    {
        current_ += coefficient_ * (target_ - current_);
        return current_;
    }

    CVDSP_NODISCARD CVDSP_FORCE_INLINE T getCurrentValue() const noexcept
    { return current_; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE T getTargetValue() const noexcept
    { return target_; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE T getCoefficient() const noexcept
    { return coefficient_; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE T getSampleRate() const noexcept { return
sampleRate_; }
    CVDSP_NODISCARD CVDSP_FORCE_INLINE T getTimeConstantSeconds() const noexcept
    { return timeConstantSeconds_; }

private:
    CVDSP_FORCE_INLINE void updateCoefficient() noexcept
    {
        if (sampleRate_ <= static_cast<T>(0) || timeConstantSeconds_ <=
static_cast<T>(0))
        {
            coefficient_ = static_cast<T>(1);
            return;
        }

        coefficient_ = static_cast<T>(1) -
std::exp(static_cast<T>(-1) / (timeConstantSeconds_ *

```

```

sampleRate_));
    }

    T sampleRate_ = static_cast<T>(0);
    T timeConstantSeconds_ = static_cast<T>(0);
    T current_ = static_cast<T>(0);
    T target_ = static_cast<T>(0);
    T coefficient_ = static_cast<T>(1);
};
} // namespace cvdsp

#endif // CVDSP_CORE_PARAMETER_SMOOTHER_HPP

```

```

=====
FILE: CV_DSP/Core/ProcessContext.hpp
=====
#ifndef CVDSP_CORE_PROCESSCONTEXT_HPP
#define CVDSP_CORE_PROCESSCONTEXT_HPP

/**
 * @file ProcessContext.hpp
 * @brief Unified DSP Processing Context
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Provides a common processing context
 * for all CV_DSP modules.
 *
 * Compatible with:
 * - VST3
 * - iPlug2
 * - JUCE
 * - CLAP
 * - Standalone
 */

#include <cstdint>
#include <cstdint>
#include <algorithm>
#include <type_traits>

namespace cvdsp
{
/**
 * @brief Processing context shared by all DSP modules.
 *
 * Contains transport, timing and
 * audio engine information.
 *
 * This structure is intentionally
 * POD-like for maximum efficiency.
 */
template<typename T>
struct ProcessContext
{
    static_assert(
        std::is_floating_point_v<T>,
        "ProcessContext requires floating point type");

```

```

/**
 * Audio sample rate.
 *
 * Example:
 *
 * 44100
 * 48000
 * 96000
 */
T sampleRate =
    static_cast<T>(44100);

/**
 * Current processing block size.
 */
std::size_t blockSize =
    0;

/**
 * Number of active channels.
 */
std::size_t numChannels =
    0;

/**
 * Host tempo in BPM.
 */
T tempo =
    static_cast<T>(120);

/**
 * Time signature numerator.
 *
 * Example:
 *
 * 4/4 -> 4
 * 3/4 -> 3
 * 7/8 -> 7
 */
std::uint32_t timeSignatureNumerator =
    4;

/**
 * Time signature denominator.
 *
 * Example:
 *
 * 4/4 -> 4
 * 7/8 -> 8
 */
std::uint32_t timeSignatureDenominator =
    4;

/**
 * Host transport state.
 */
bool isPlaying =
    false;

/**
 * Host recording state.
 */
bool isRecording =
    false;

```

```

/**
 * Sample position.
 */
* Optional host information.
*/
std::uint64_t samplePosition =
    0;

/**
 * PPQ position.
 */
* Optional host information.
*/
T ppqPosition =
    static_cast<T>(0);

/**
 * Bar start PPQ.
 */
* Optional host information.
*/
T barStartPPQ =
    static_cast<T>(0);

/**
 * Seconds since transport start.
 */
T timeInSeconds =
    static_cast<T>(0);

/**
 * Reset to defaults.
 */
constexpr void reset() noexcept
{
    sampleRate =
        static_cast<T>(44100);

    blockSize =
        0;

    numChannels =
        0;

    tempo =
        static_cast<T>(120);

    timeSignatureNumerator =
        4;

    timeSignatureDenominator =
        4;

    isPlaying =
        false;

    isRecording =
        false;

    samplePosition =
        0;

    ppqPosition =

```

```

        static_cast<T>(0);

    barStartPPQ =
        static_cast<T>(0);

    timeInSeconds =
        static_cast<T>(0);
}

/**
 * Returns samples per beat.
 */
[[nodiscard]]
constexpr T samplesPerBeat() const noexcept
{
    return
        (
            static_cast<T>(60)
            *
            sampleRate
        )
        /
        tempo;
}

/**
 * Returns beats per second.
 */
[[nodiscard]]
constexpr T beatsPerSecond() const noexcept
{
    return
        tempo
        /
        static_cast<T>(60);
}

/**
 * Returns seconds per beat.
 */
[[nodiscard]]
constexpr T secondsPerBeat() const noexcept
{
    return
        static_cast<T>(60)
        /
        tempo;
}

/**
 * Returns duration of one bar.
 */
[[nodiscard]]
constexpr T secondsPerBar() const noexcept
{
    return
        secondsPerBeat()
        *
        static_cast<T>(
            timeSignatureNumerator);
}

/**
 * Returns samples per bar.

```

```

    */
[[nodiscard]]
constexpr T samplesPerBar() const noexcept
{
    return
        samplesPerBeat()
        *
        static_cast<T>(
            timeSignatureNumerator);
}

/**
 * Returns duration of current block.
 */
[[nodiscard]]
constexpr T blockDurationSeconds() const noexcept
{
    if (sampleRate <= static_cast<T>(0))
    {
        return static_cast<T>(0);
    }

    return
        static_cast<T>(blockSize)
        /
        sampleRate;
}

/**
 * Returns Nyquist frequency.
 */
[[nodiscard]]
constexpr T nyquist() const noexcept
{
    return
        sampleRate
        *
        static_cast<T>(0.5);
}

/**
 * Returns true if transport running.
 */
[[nodiscard]]
constexpr bool playing() const noexcept
{
    return isPlaying;
}

/**
 * Returns true if recording.
 */
[[nodiscard]]
constexpr bool recording() const noexcept
{
    return isRecording;
}

/**
 * Returns true if transport stopped.
 */
[[nodiscard]]
constexpr bool stopped() const noexcept
{

```



```

        return !isPlaying;
    }

    /**
     * Returns true if context appears valid.
     */
    [[nodiscard]]
    constexpr bool isValid() const noexcept
    {
        return
            sampleRate >
            static_cast<T>(0)
            &&
            blockSize > 0
            &&
            numChannels > 0
            &&
            tempo >
            static_cast<T>(0)
            &&
            timeSignatureNumerator > 0
            &&
            timeSignatureDenominator > 0;
    }

    /**
     * Returns current beat position.
     */
    [[nodiscard]]
    constexpr T beatPosition() const noexcept
    {
        return ppqPosition;
    }

    /**
     * Returns beat position within bar.
     */
    [[nodiscard]]
    constexpr T beatInBar() const noexcept
    {
        return
            ppqPosition
            -
            barStartPPQ;
    }

    /**
     * Returns current measure length in beats.
     */
    [[nodiscard]]
    constexpr T beatsPerBar() const noexcept
    {
        return static_cast<T>(
            timeSignatureNumerator);
    }
};

using ProcessContextF =
    ProcessContext<float>;

using ProcessContextD =
    ProcessContext<double>;

} // namespace cvdsp

```

```
#endif
```

```
=====
FILE: CV_DSP/Core/Types.hpp
=====
```

```
#ifndef CVDSP_CORE_TYPES_HPP
```

```
#define CVDSP_CORE_TYPES_HPP
```

```
/**
```

```
 * @file Types.hpp
```

```
 * @brief Fundamental fixed-width type aliases for the CV_DSP library.
```

```
 *
```

```
 * This header defines the minimal set of scalar aliases used by CV_DSP.
```

```
 * The aliases intentionally map directly to standard fixed-width integer
```

```
 * types and built-in floating-point types so the library remains portable
```

```
 * across VST3 SDK, iPlug2, JUCE, CLAP, and standalone host integrations.
```

```
 *
```

```
 * @note This header is real-time safe: it performs no allocation, contains no  
 *       executable runtime work, and introduces no exceptions or RTTI usage.
```

```
 */
```

```
#include <cstdint>
```

```
namespace cvdsp
```

```
{
```

```
/**
```

```
 * @brief 32-bit IEEE-754 floating-point scalar alias.
```

```
 *
```

```
 * Used for single-precision DSP paths and host-facing sample buffers where the  
 * framework exposes `float` audio samples.
```

```
 */
```

```
using f32 = float;
```

```
/**
```

```
 * @brief 64-bit IEEE-754 floating-point scalar alias.
```

```
 *
```

```
 * Used for double-precision DSP paths and host-facing sample buffers where the  
 * framework exposes `double` audio samples.
```

```
 */
```

```
using f64 = double;
```

```
/**
```

```
 * @brief Signed 32-bit integer alias.
```

```
 *
```

```
 * Suitable for sample counts, channel indices, and bounded integer parameters  
 * where a fixed 32-bit representation is required.
```

```
 */
```

```
using i32 = std::int32_t;
```

```
/**
```

```
 * @brief Unsigned 32-bit integer alias.
```

```
 *
```

```
 * Suitable for bit fields, packed identifiers, and non-negative counters where  
 * a fixed 32-bit representation is required.
```

```
 */
```

```
using u32 = std::uint32_t;
```

```
/**
```

```
 * @brief Signed 64-bit integer alias.
```

```
 *
```

```
 * Suitable for large sample positions, timeline values, and fixed-width signed
```

```

    * counters used outside the audio callback.
    */
using i64 = std::int64_t;

/**
 * @brief Unsigned 64-bit integer alias.
 *
 * Suitable for large monotonic counters, packed identifiers, and fixed-width
 * unsigned values used by host integration layers.
 */
using u64 = std::uint64_t;
} // namespace cvdsp

#endif // CVDSP_CORE_TYPES_HPP

```

```

=====
FILE: CV_DSP/Core/Version.hpp
=====

```

```

#ifndef CVDSP_CORE_VERSION_HPP
#define CVDSP_CORE_VERSION_HPP

/**
 * @file Version.hpp
 * @brief Semantic version metadata for CV_DSP.
 *
 * This header centralizes version information for the header-only CV_DSP
 * foundation. Version values are compile-time constants so plug-in wrappers,
 * diagnostics, documentation generators, and tests can query the library
 * version without runtime initialization.
 *
 * @note This header is real-time safe: it performs no allocation, contains no
 *       executable runtime work, and introduces no exceptions or RTTI usage.
 */

namespace cvdsp
{
    /** @brief Major semantic version component. */
    inline constexpr int Major = 0;

    /** @brief Minor semantic version component. */
    inline constexpr int Minor = 1;

    /** @brief Patch semantic version component. */
    inline constexpr int Patch = 0;

    /**
     * @brief Null-terminated semantic version string.
     *
     * The string is intentionally represented as a string literal with static
     * storage duration. It requires no allocation and can be referenced from host
     * metadata code or diagnostic output.
     */
    inline constexpr const char* VersionString = "0.1.0";
} // namespace cvdsp

#endif // CVDSP_CORE_VERSION_HPP

```

```

=====
FILE: CV_DSP/Delay/DelayLine.hpp
=====

```

```
#ifndef CVDSP_DELAY_DELAYLINE_HPP
#define CVDSP_DELAY_DELAYLINE_HPP

#include <array>
#include <cmath>
#include <cstdint>
#include <cstdint>
#include <cstring>
#include <type_traits>
#include <algorithm>
#include <cassert>

/**
 * @file DelayLine.hpp
 * @namespace cvdsp::delay
 * @brief ImplementaÃ§Ã£o profissional de delay line com suporte a delay
        fracionÃ¡rio
 *
 * Este mÃ³dulo implementa delay lines de alta qualidade para processamento de
        Ã¡udio
 * em tempo real, com suporte a:
 * - Delay inteiro (amostras discretas)
 * - Delay fracionÃ¡rio (interpolaÃ§Ã£o entre amostras)
 * - InterpolaÃ§Ã£o linear (qualidade boa, baixa latÃªncia)
 * - InterpolaÃ§Ã£o cÃºbica (qualidade excelente, mais CPU)
 *
 * AplicaÃ§Ãµes:
 * - Echo/ReverberaÃ§Ã£o
 * - Chorus (modulaÃ§Ã£o de delay)
 * - Flanger (modulaÃ§Ã£o rÃ¡pida)
 * - Vibrato (modulaÃ§Ã£o de pitch)
 * - Doppler effect (deslocamento de frequÃªncia)
 * - Look-ahead para compressores/limitadores
 *
 * **Estrutura da Biblioteca CV_DSP**:
 * ...
 * CV_DSP/
 *   "core"€ Core/
 *   "cb", "circularbuffer"€ CircularBuffer.hpp
 *   "a", "algorithm"€ ...
 *   "a"€ Delay/
 *   "a"€ DelayLine.hpp  â†“ Este arquivo
 *   ...
 */

namespace cvdsp::delay
{
/**
 * @enum InterpolationType
 * @brief Tipos de interpolaÃ§Ã£o suportados para delay fracionÃ¡rio
 *
 * Define qual algoritmo de interpolaÃ§Ã£o serÃ¡ utilizado durante o
        processamento
 * para melhorar a qualidade do delay fracionÃ¡rio.
 */
enum class InterpolationType : std::uint8_t
{
    /// Sem interpolaÃ§Ã£o (delay inteiro apenas)
    /// LatÃªncia: 0 amostras
    /// CPU: MÃnimo
    /// Qualidade: Baixa em delays modulados
    None = 0,

```

```

    /// Interpolação linear (2 pontos)
    /// Latência: ~0.5 amostras
    /// CPU: Baixo
    /// Qualidade: Boa (erro ~0.3dB @ Nyquist)
    /// Uso: Chorus, vibrato, flanger
    Linear = 1,

    /// Interpolação cúbica Hermite (4 pontos)
    /// Latência: ~1.5 amostras
    /// CPU: ~4x linear
    /// Qualidade: Excelente (erro ~0.02dB @ Nyquist)
    /// Uso: Qualidade profissional, broadcast
    Cubic = 2
};

```

```

/**
 * @class DelayLine
 * @brief Delay line profissional com interpolação para DSP em tempo real
 *
 * **Propósito Geral**:
 *
 * A DelayLine é um componente fundamental em qualquer biblioteca DSP. Ela
 * armazena histórico de sinal e permite leitura em pontos variáveis no
 * passado,
 * possibilitando efeitos que dependem de delay temporal.
 *
 *

```

```

=====
==

```

```

 * MATEMÁTICA DO DELAY INTEIRO
 *

```

```

=====
==

```

```

 *
 * Um delay inteiro simples é dado por:
 *
 *  $y[n] = x[n - D]$ 
 *
 * onde D é o delay em amostras (número inteiro positivo).
 *
 * Implementação com buffer circular:
 *
 * Buffer:      [s0][s1][s2][s3][s4][s5] ... [sN-1]
 * WriteIndex:  5 (próxima posição a escrever)
 * Delay (D):   2 amostras
 *
 * Índice de leitura: readIndex = (writeIndex - D) % N
 *                      = (5 - 2) % N = 3
 *
 * Então:  $y[n] = \text{buffer}[3]$  (a amostra 2 passos atrás)
 *
 *
 * Vantagem: Sem aliasing, sem artefatos
 * Desvantagem: Delay preso a valores inteiros (granularidade de 1 amostra)
 *

```

```

=====
==

```

```

 * MATEMÁTICA DO DELAY FRACIONÁRIO (FRACTIONAL DELAY)
 *

```

```

=====
==

```

```

 *
 * Quando D não é inteiro ( $D = D_{\text{int}} + D_{\text{frac}}$ , onde  $0 \leq D_{\text{frac}} < 1$ ):

```

```

* Necessário interpolar entre duas amostras adjacentes.
*
* **INTERPOLAÇÃO LINEAR**:
*
* Fórmula:
* ```
* y[n] = (1 - D_frac) * x[n - D_int] + D_frac * x[n - D_int - 1]
* ```
*
* Interpretação geométrica:
* ```
* sample_0 (em D_int)          sample_1 (em D_int+1)
*   â—†
*   â—†'
*   â—†â—†â—† Interpolação linear aqui (D_frac = 0.3)
*
* output = 0.7 * sample_0 + 0.3 * sample_1
* ```
*
* Características:
* - Erro de fase:  $\hat{\alpha} \approx 0.3$  dB @ Nyquist (aceitável para muitas aplicações)
* - Latência introduzida:  $\sim 0.5$  amostras (estimativa média)
* - CPU: Mínimo (2 multiplicações, 1 adição)
* - Qualidade: Boa para efeitos modulados
*
* Resposta em frequência:
* ```
* |H(f)|  $\hat{\alpha} \approx \text{sinc}(f/f_c)$  (com aliasing pequeno)
* Bandwidth: até  $\sim 15$  kHz mantendo qualidade aceitável
* ```
*
* **INTERPOLAÇÃO CUBICA HERMITE**:
*
* Usa 4 pontos adjacentes para construir polinômio cúbico suave:
* ```
* y[0] = x[n - D_int]
* y[1] = x[n - D_int - 1]
* y[2] = x[n - D_int - 2]
* y[3] = x[n - D_int - 3]
* ```
*
* Fórmulas Hermite (baseadas em  $t = D\_frac$ ):
* ```
* h00(t) =  $2t^3 - 3t^2 + 1$           (influência de y[0])
* h10(t) =  $t^3 - 2t^2 + t$           (influência derivada y[0])
* h01(t) =  $-2t^3 + 3t^2$           (influência de y[1])
* h11(t) =  $t^3 - t^2$           (influência derivada y[1])
* ```
*
* Derivadas aproximadas (diferenças finitas centralizadas):
* ```
* m0 = (y[2] - y[1]) / 2          (slope em y[0])
* m1 = (y[3] - y[0]) / 2          (slope em y[1])
* ```
*
* Interpolação final:
* ```
* output = h00(t)*y[0] + h10(t)*m0 + h01(t)*y[1] + h11(t)*m1
* ```
*
* Visualização:
* ```

```

```

* CÃbica (suave):      â•±â•²â•±â•²â•±â•²
* Linear (degraus):   â•±â•²€â•²â•²â•²€â•²â•²
* Sinal real:        i½ži½ži½ži½ž
*
*
* CaracterÃsticas:
* - Erro de fase: â%ˆ 0.02 dB @ Nyquist (excelente!)
* - LatÃncia introduzida: ~1.5 amostras
* - CPU: ~4x interpolaÃÃo linear
* - Qualidade: Profissional (banda passante atÃ© ~20 kHz)
*
* Resposta em frequÃncia:
*
* |H(f)| â%ˆ sinc³(f/fc) (muito superior)
* Bandwidth: atÃ© ~20 kHz com qualidade excelente
*
*
*

```

```

=====
==
* DOPPLER EFFECT (Efeito Doppler)
*
=====

```

```

==
*
* **FenÃmeno FÃsico**:
* Quando uma fonte sonora se move em relaÃÃo ao observador, a frequÃncia muda.
* Este efeito ocorre porque a distÃncia (delay) entre fonte e observador varia.
*
* **ImplementaÃÃo em DSP**:
* Delay variÃvel no tempo causa mudanÃa de frequÃncia (pitch shifting):
*
* Delay variÃvel: D[n] = D_base + A * sin(2π * f_lfo * n / Fs)
*
* Taxa de mudanÃa: dD/dn
* Desvio de frequÃncia: f' = -f_original * (dD/dt) * Fs
*
* onde:
* D_base: delay central em amostras
* A: amplitude de modulaÃÃo (amostras)
* f_lfo: frequÃncia do LFO (Hz)
* Fs: taxa de amostragem (Hz)
* f_original: frequÃncia da amostra original (Hz)
*
*
* **Exemplo NumÃrico - AmbulÃncia**:
*
* ParÃmetros:
* - FrequÃncia original: 1000 Hz (tom de ambulÃncia)
* - Delay varia: Â±5 ms @ 2 Hz
* - Sample rate: 48 kHz
*
* Delay em amostras: Â±5ms * 48 = Â±240 amostras
* Taxa mÃxima dD/dn: 2 * 240 = 480 amostras/perÃodo
*
* Desvio mÃximo: f' = 1000 * (480 / 48000) * 48000 = Â±10 Hz
*
* Resultado: FrequÃncia varia entre 990 Hz e 1010 Hz (efeito audÃvel!)
*
*
* **AplicaÃÃes PrÃticas**:
* - Sirene de ambulÃncia/polÃcia (passando)

```

```

* - Zumbido de abelha (movimento)
* - Efeito "Wow" em vintage tape
* - RotaÃ§Ã£o Leslie (Ã³rgÃ£o Hammond)
*
* **CÃ³digo Exemplo**:
* ```.cpp
* DelayLine<float, 16384, InterpolationType::Cubic> doppler;
* doppler.prepare(48000);
*
* float phase = 0.0f;
* for (size_t n = 0; n < blockSize; ++n) {
*     // Delay oscila entre 5-15 ms
*     float lfo = std::sin(phase);
*     doppler.setDelayMilliseconds(10.0f + lfo * 5.0f);
*
*     outputBuffer[n] = doppler.process(inputBuffer[n]);
*
*     phase += 2.0f * M_PI * 2.0f / 48000.0f; // 2 Hz LFO
* }
* ```
*
*
=====
==
* CHORUS (Engrossamento de Som)
*
=====
==
*
* **Objetivo Sonoro**: Fazer som parecer mÃºltiplas vozes tocando
simultaneamente.
*
* **FÃ³rmula BÃ¡sica**:
* ```
* y[n] = x[n] + g * x[n - D(n)]
*
* D(n) = D_center + A * sin(2Ï€ * f_lfo * n / Fs)
* ```
*
* onde:
* x[n]: sinal de entrada (seco)
* g: ganho do sinal atrasado (tÃ­pico: 0.5-0.9)
* D_center: delay central em amostras (tÃ­pico: 25-50 ms)
* A: amplitude de modulaÃ§Ã£o (tÃ­pico: 5-15 ms)
* f_lfo: frequÃªncia do LFO (tÃ­pico: 1-5 Hz)
*
* **ParÃ¢metros TÃ­picos**:
* - Vocal: D_center=40ms, A=10ms, f_lfo=2.5Hz, g=0.7
* - Instrumento: D_center=30ms, A=8ms, f_lfo=1.5Hz, g=0.6
* - Synth: D_center=50ms, A=15ms, f_lfo=3Hz, g=0.8
*
* **Mecanismo TÃ©cnico**:
* ```
* Delay modulado cria desvios de frequÃªncia pequenos (+/- alguns Hz).
* Esses desvios causam batimentos leves (combinaÃ§Ã£o de duas frequÃªncias
prÃ³ximas).
* Resultado auditivo: som "mais largo" e "mais natural".
*
* MudanÃ§a de pitch:  $\hat{f} \hat{=} \hat{f} \pm 1\text{-}5\text{ Hz}$  (imperceptÃ­vel como pitch shift)
* Mas perceptÃ­vel como "thickening" do som
* ```
*
* **Diagrama de Sinal**:
* ```

```



```
* Entrada  $\hat{a}''$ ,  $\hat{a}''^2$  Saída  
*  $\hat{a}''$ ,  $\hat{a}'' + 0.7$   
*  $\hat{a}''$ ,  $\hat{a}''$   
*  $\hat{a}''' \hat{a}'' - \Delta$  Delay  $\hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}''$   
* modulado  
* ( $40\pm 10\text{ms}$ )  
  
* Espectro (exemplo, 1kHz entrada):  
* Sinal seco: 1000 Hz  $\hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}''$   
* Sinal atrasado: ~998-1002 Hz (modulado)  $\hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}'' \hat{a}''$   
* Resultado: Batimento leve em ~2Hz (largura  $\Delta$ - profundidade)  
* ````  
*  
* **Aplicações Práticas**:  
* - Vocal enhancement (fazer voz parecer dupla)  
* - Instrumento enriquecido (violão, piano)  
* - Sintetizador (som mais "vivo")  
*  
* **Código Exemplo**:  
* ```cpp  
* DelayLine<float, 16384, InterpolationType::Linear> chorus;  
* chorus.prepare(48000);  
*  
* float phase = 0.0f;  
* const float delay_center = 40.0f; // 40 ms  
* const float delay_mod = 10.0f; //  $\Delta \pm 10$  ms  
* const float gain = 0.7f;  
* const float lfo_freq = 2.5f; // 2.5 Hz  
*  
* for (size_t n = 0; n < blockSize; ++n) {  
* float lfo = std::sin(phase);  
* float delay_ms = delay_center + delay_mod * lfo;  
* chorus.setDelayMilliseconds(delay_ms);  
*  
* float input = inputBuffer[n];  
* float delayed = chorus.process(input);  
*  
* outputBuffer[n] = input + gain * delayed;  
*  
* phase += 2.0f * M_PI * lfo_freq / 48000.0f;  
* }  
* ```  
*  
*  
=====
```

[illegible]

```

* **Código Exemplo - Flanger Clássico**:
```

```

*   ``cpp
*   DelayLine<float, 4096, InterpolationType::Linear> flanger;
*   flanger.prepare(48000);
*
*   float phase = 0.0f;
*   const float delay_center = 2.0f; // 2 ms (curto!)
*   const float delay_mod = 1.5f;    // ±1.5 ms
*   const float gain = 0.8f;
*   const float lfo_freq = 2.0f;    // 2 Hz
*
*   for (size_t n = 0; n < blockSize; ++n) {
*       float lfo = std::sin(phase);
*       float delay_ms = delay_center + delay_mod * lfo;
*       flanger.setDelayMilliseconds(delay_ms);
*
*       float input = inputBuffer[n];
*       float delayed = flanger.process(input);
*
*       outputBuffer[n] = input + gain * delayed; // Sem feedback = simples
*
*       phase += 2.0f * M_PI * lfo_freq / 48000.0f;
*   }
*   ``
*
*

```

```

=====
==

```

```

* VIBRATO (Modulação de Pitch)
*

```

```

=====
==

```

```

*
* **Objetivo Sonoro**: Variaçāo natural de altura (pitch) - como violinista
tocando.
*

```

```

* **Fórmula Básica**:
*   ``

```

```

*    $y[n] = x[n - D(n)]$   ⚠ APENAS delay, sem sinal seco!
*

```

```

*    $D(n) = D_{center} + A * \sin(2\pi * f_{lfo} * n / F_s)$ 
*   ``
*

```

```

* **DIFERENÇA CRÍTICA DE CHORUS/FLANGER**:

```

```

* - Chorus/Flanger:  $y[n] = x[n] + g * x[n-D(n)]$  (sinal seco + atrasado)
* - Vibrato:  $y[n] = x[n - D(n)]$  (APENAS atrasado, SEM seco!)
*

```

```

* **Parâmetros Típicos**:

```

```

* -  $D_{center}$ : 1-5 ms (delay base)
* -  $A$ : 0.5-2 ms (amplitude)
* -  $f_{lfo}$ : 5-10 Hz (natural em instrumentos)
*

```

```

* **Mecanismo Técnico - Pitch Shift**:
*   ``

```

```

* Delay variável causa mudança de frequência instantânea:
*

```

```

* Frequência instantânea:  $f_{inst}[n] = f_{orig} * (1 - dD/dn)$ 
*

```

```

* Exemplo:

```

```

*    $f_{orig} = 440$  Hz (A4)
*    $D[n]$  oscila ±2ms
*   Resultado: frequência varia entre ~420-460 Hz (áudio pitch shift)
*   ``
*

```



```

* | Flanger      | 0.5-5ms | 0.5-2ms | 0.5-10Hz | y=x[n]+g*x[n-D] | Excelente|
* | Vibrato      | 1-5ms  | 0.5-2ms | 5-10Hz  | y=x[n-D(n)]      | Muito Bom|
*
*

```

```

=====
==

```

```

* IMPLEMENTAÇÃO REAL-TIME SAFE
*

```

```

=====
==

```

```

*
* **Ciclo de Configuração (fora do audio thread)**:
* ```cpp
* // Apenas uma vez ou em thread de controle
* DelayLine<float, 16384, InterpolationType::Linear> delay;
* delay.prepare(48000); // O(N) - aceitável fora do loop
* delay.setDelaySamples(4800); // O(1) - rápido
* ```
*
* **Ciclo de Áudio (audio thread - real-time)**:
* ```cpp
* // Chamado frequentemente (cada amostra ou bloco)
* for (size_t n = 0; n < blockSize; ++n) {
*     float output = delay.process(inputBuffer[n]); // O(1) GARANTIDO
*     outputBuffer[n] = output;
*     // - Sem alocação
*     // - Sem exception
*     // - Sem RTTI
*     // - Determinístico (mesma latência sempre)
* }
* ```
*
* **Garantias Real-Time Safe**:
* - process(): O(1) constante, sem alocação, sem exception
* - setDelaySamples(): O(1) constante (mudança suave de delay)
* - setDelayMilliseconds(): O(1) constante
* - Mudanças de delay não causam clicks ou artifacts
*
* @tparam T Tipo aritmético (float ou double) para amostras
* @tparam MaxDelaySamples Número máximo de amostras de delay (potência de 2)
* @tparam Interpolation Tipo de interpolação (None, Linear, Cubic)
*
* @note DelayLine usa CircularBuffer internamente:
* - Zero alocações dinâmicas
* - O(1) por amostra processada
* - Real-time safe em todos os métodos de processo
* - Suporta delay de 0 até MaxDelaySamples amostras
*
* @see cvdsp::CircularBuffer para buffer circular base
*/

```

```

template <typename T, size_t MaxDelaySamples = 65536, InterpolationType
Interpolation = InterpolationType::Linear>

```

```

class DelayLine
{

```

```

    // =====
    // STATIC ASSERTIONS - VALIDAÇÕES EM TEMPO DE COMPILAÇÃO
    // =====

```

```

    static_assert(std::is_floating_point_v<T>,
                  "DelayLine requires floating-point type (float or double)");

```

```

    static_assert(MaxDelaySamples > 0,
                  "DelayLine max delay must be greater than 0");

```

```

        static_assert((MaxDelaySamples & (MaxDelaySamples - 1)) == 0,
                      "DelayLine max delay must be power of 2 for efficient circular
wrapping");

public:
    // =====
    // TYPEDEFS E CONSTANTES
    // =====

    /// @brief Tipo de dado (float ou double)
    using value_type = T;

    /// @brief Tipo para tamanhos e Índices
    using size_type = size_t;

    /// @brief Capacidade máxima de delay em amostras
    static constexpr size_type MAX_DELAY_SAMPLES = MaxDelaySamples;

    /// @brief Máscara de bits para modulo circular eficiente
    static constexpr size_type MODULO_MASK = MaxDelaySamples - 1;

    /// @brief Tipo de interpolação configurado
    static constexpr InterpolationType INTERPOLATION_TYPE = Interpolation;

    // =====
    // CONSTRUTOR E DESTRUTOR
    // =====

    /**
     * @brief Construtor padrão. Inicializa delay line em estado neutro.
     *
     * **Comportamento**:
     * - Buffer: preenchido com zeros
     * - Delay: 0 amostras (sem efeito)
     * - Sample rate: 48000 Hz (padrão)
     * - WriteIndex: 0
     * - Pronto para prepare()
     *
     * **Complexidade**: O(1)
     *
     * @note Não aloca memória (buffer é estático em stack)
     */
    constexpr DelayLine() noexcept
        : m_buffer{}
        , m_writeIndex(0)
        , m_delaySamples(0.0)
        , m_sampleRate(48000)
        , m_initialized(false)
    {
    }

    /// @brief Destrutor padrão (trivial)
    ~DelayLine() noexcept = default;

    /// @brief Cópia explicitamente deletada
    DelayLine(const DelayLine&) = delete;

    /// @brief Atribuição explicitamente deletada
    DelayLine& operator=(const DelayLine&) = delete;

    /// @brief Move constructor explicitamente deletado
    DelayLine(DelayLine&&) = delete;

    /// @brief Move assignment explicitamente deletado

```

```
DelayLine& operator=(DelayLine&&) = delete;
```

```
// =====  
// MÃ%TODOS DE INICIALIZAÃf0  
// =====
```

```
/**  
 * @brief Prepara a delay line para processamento.  
 *  
 * **OperaÃsÃes**:  
 * - Zera o buffer circular completo  
 * - Define taxa de amostragem (necessÃrio para setDelayMilliseconds())  
 * - Reseta Ãndice de escrita para 0  
 * - Reseta delay para 0 amostras (sem efeito)  
 * - Marca como inicializado  
 *  
 * **Complexidade**:  
 * O(MaxDelaySamples)  
 *  
 * **Contexto de Chamada** (fora do audio thread):  
 * - VST3: onActivate() do plugin  
 * - iPlug2: OnReset() do plugin  
 * - JUCE: prepareToPlay()  
 * - CLAP: activate()  
 * - Standalone: na inicializaÃsÃo de Ãudio  
 *  
 * **Thread Safety**:  
 * Deve ser chamado apenas em thread de configuraÃsÃo  
 *  
 * @param[in] sampleRate FrequÃncia de amostragem em Hz (ex: 48000, 96000)  
 *  
 * @return void  
 *  
 * @post m_initialized = true  
 * @post m_sampleRate = sampleRate  
 * @post m_writeIndex = 0  
 * @post m_delaySamples = 0  
 * @post buffer completamente zerado  
 *  
 * @note **NÃo Ã real-time safe** (realiza limpeza em O(N))  
 *  
 * @warning Chamar antes de qualquer process()  
 */
```

```
void prepare(size_type sampleRate) noexcept  
{  
    // Zera o buffer circular via memset otimizado  
    std::memset(m_buffer.data(), 0, sizeof(m_buffer));  
  
    // Define sample rate para conversÃo ms/samples  
    m_sampleRate = sampleRate;  
  
    // Reseta Ãndice de escrita (volta ao inÃcio)  
    m_writeIndex = 0;  
  
    // Reseta delay para 0 (sem efeito inicial)  
    m_delaySamples = static_cast<value_type>(0);  
  
    // Marca como pronto  
    m_initialized = true;  
}
```

```
/**  
 * @brief Prepara a delay line mantendo compatibilidade com mÃ³dulos que  
 * informam tambÃm um limite mÃximo em milissegundos.  
 *  
 * O armazenamento mÃximo continua definido pelo parÃmetro de template
```

```

    * MaxDelaySamples; maxDelayMilliseconds Ã© validado indiretamente pelos
    * setters de delay, que fazem clamp para a capacidade real do buffer.
    */
void prepare(size_type sampleRate, value_type /*maxDelayMilliseconds*/)
noexcept
{
    prepare(sampleRate);
}

/**
 * @brief Reseta o estado do buffer sem reconfigurÃ£o.
 *
 * **OperaÃµes**:
 * - Zera o buffer circular completo
 * - Reseta Ãndice de escrita
 * - MantÃm delay atual e sample rate
 *
 * **Complexidade**:  $O(\text{MaxDelaySamples})$ 
 *
 * **Uso**:
 * - Bypass de efeito ativado
 * - Mute/Note-Off sem sustain
 * - Reset rÃpido entre notas
 * - SincronizaÃ£o de transporte
 *
 * **Thread Safety**: Deve ser chamado fora do audio thread
 *
 * @return void
 *
 * @post buffer completamente zerado
 * @post m_writeIndex = 0
 * @post m_delaySamples mantido
 * @post m_sampleRate mantido
 *
 * @note **NÃo Ã© real-time safe** (realiza limpeza em  $O(N)$ )
 */
void reset() noexcept
{
    std::memset(m_buffer.data(), 0, sizeof(m_buffer));
    m_writeIndex = 0;
}

// =====
// MÃTODOS DE CONFIGURAÃO - REAL-TIME SAFE
// =====

/**
 * @brief Define o delay em amostras (inteiro ou fracionÃrio).
 *
 * **MatemÃtica**:
 * ``
 * delayInSamples = D (pode ser float para fracionÃrio)
 *
 * Delay inteiro: D_int = floor(delayInSamples)
 * FraÃo: D_frac = delayInSamples - floor(delayInSamples)
 *
 * Ãndice de leitura: readIndex = (writeIndex - D_int) & MODULO_MASK
 * InterpolaÃo: usa D_frac para interpolar entre readIndex e readIndex-1
 * ``
 *
 * **Complexidade**:  $O(1)$  constante
 *
 * **ValidaÃo**:
 * - 0 â‰¤ delayInSamples â‰¤ MaxDelaySamples

```



```

* - Valores fora deste range sÃ£o clampados automaticamente
*
* **MudanÃ§as de Delay**:
* - MudanÃ§as suaves nÃ£o causam clicks audÃveis
* - InterpolaÃ§Ã£o cÃbica garante continuidade
* - Linear interpolation tambÃm Ã suave
*
* **Uso TÃpico**:
* ```cpp
* // Delay fixo de 1 segundo @ 48kHz
* delay.setDelaySamples(48000.0f);
*
* // Delay fracionÃrio (100.5 amostras)
* delay.setDelaySamples(100.5f);
*
* // Delay modulado (varia continuamente) - REAL-TIME SAFE
* float lfoValue = lfo.process(); // -1 a 1
* float modulatedDelay = baseDelay + lfoValue * modAmount;
* delay.setDelaySamples(modulatedDelay); // Suave, sem clicks
* ```
*
* **Real-Time Safety**: â€ GARANTIDO
* - O(1) constante
* - Sem alocaÃÃo
* - Sem exception
* - DeterminÃstico
*
* @param[in] delayInSamples Delay em amostras (pode ser fracionÃrio)
*
* @return void
*
* @post m_delaySamples = clamp(delayInSamples, 0, MaxDelaySamples)
*
* @note Ideal para:
* - ModulaÃÃo contÃnua (chorus, flanger, vibrato)
* - Controle via parÃmetro
* - Delays variÃveis em tempo real
*/
inline void setDelaySamples(value_type delayInSamples) noexcept
{
    // Clamp delay para range vÃlido
    // MÃnimo: 0 (sem delay, bypass)
    // MÃximo: MaxDelaySamples (buffer completo)
    m_delaySamples = std::clamp(delayInSamples,
                                static_cast<value_type>(0),
                                static_cast<value_type>(MaxDelaySamples));
}

/**
* @brief Define o delay em milissegundos.
*
* **MatemÃtica**:
* ```
* delay_ms = D_ms
* delay_samples = D_ms * Fs / 1000
*
* Exemplo: 100 ms @ 48 kHz
* delay_samples = 100 * 48000 / 1000 = 4800 amostras
* ```
*
* **Complexidade**: O(1) constante
*
* **ValidaÃÃo**:
* - Internamente usa setDelaySamples()

```

```

* - Valores fora de range sÃ£o clampados automaticamente
* - ConversÃ£o precisa (exato para delays fracionÃ¡rios)
*
* **Uso TÃpico**:
* ```cpp
* // Delay de 250 ms (clÃssico para chorus)
* delay.setDelayMilliseconds(250.0f);
*
* // Delay curto para flanger (2 ms)
* delay.setDelayMilliseconds(2.0f);
*
* // ModulaÃ§Ã£o em milissegundos (intuitivo!)
* float lfoValue = lfo.process(); // -1 a 1
* float modulatedDelay = 50.0f + lfoValue * 10.0f; // 40-60 ms
* delay.setDelayMilliseconds(modulatedDelay);
* ```
*
* **Real-Time Safety**: âœ“ GARANTIDO
* - O(1) constante
* - Sem alocaÃ§Ã£o
* - Sem exception
*
* @param[in] delayInMilliseconds Delay em milissegundos
*
* @return void
*
* @pre prepare() deve ter sido chamado para definir sampleRate
*
* @note **Requer prepare()** ter sido chamado antes
*       ConversÃ£o Ã© automÃ¡tica baseada no sample rate definido
*/
inline void setDelayMilliseconds(value_type delayInMilliseconds) noexcept
{
    // Converte milissegundos para amostras
    // delay_samples = delay_ms * Fs / 1000
    value_type delaySamples = delayInMilliseconds *
                             static_cast<value_type>(m_sampleRate) /
                             static_cast<value_type>(1000);
    setDelaySamples(delaySamples);
}

/**
* @brief Define o delay em segundos.
*
* **MatemÃ¡tica**:
* ```
* delay_s = D_s
* delay_samples = D_s * Fs
*
* Exemplo: 0.5 s @ 48 kHz
* delay_samples = 0.5 * 48000 = 24000 amostras
* ```
*
* **Complexidade**: O(1) constante
*
* **Uso TÃpico**:
* ```cpp
* // Delay de 1 segundo (longo para reverb)
* delay.setDelaySeconds(1.0f);
*
* // Delay muito curto (50 ms)
* delay.setDelaySeconds(0.05f);
* ```
*

```

```

* **Real-Time Safety**: " GARANTIDO
*
* @param[in] delayInSeconds Delay em segundos
*
* @return void
*/
inline void setDelaySeconds(value_type delayInSeconds) noexcept
{
    setDelayMilliseconds(delayInSeconds * static_cast<value_type>(1000));
}

// =====
// MÃ%TODOS DE PROCESSAMENTO - REAL-TIME SAFE
// =====

/**
* @brief Processa uma amostra atravÃs da delay line.
*
* **OperaÃÃes**:
* 1. Escreve amostra de entrada na posiÃÃo de escrita atual
* 2. Calcula Ãndice de leitura com delay (inteiro + fracionÃrio)
* 3. Interpola valor lido com base no tipo configurado
* 4. AvanaÃ Ãndice de escrita para prÃxima posiÃÃo
*
* **Complexidade**: O(1) constante
*
* **Fluxo Detalhado**:
* ``
* 1. Escrita:
*    buffer[writeIndex] = inputSample
*
* 2. CÃlculo de Ãndice de leitura:
*    D_int = floor(m_delaySamples)
*    readIndex = (writeIndex - D_int) & MODULO_MASK
*
* 3. InterpolaÃÃo (depende de tipo):
*    - None: output = buffer[readIndex]
*    - Linear: output = lerp(buffer[readIndex], buffer[readIndex-1], frac)
*    - Cubic: output = hermite(4 pontos adjacentes, frac)
*
* 4. AvanaÃ:
*    writeIndex = (writeIndex + 1) & MODULO_MASK
*    ``
*
* **MatemÃtica - Sem InterpolaÃÃo (Delay Inteiro)**:
* ``
* D_int = floor(m_delaySamples)
* readIndex = (writeIndex - D_int) % MaxDelaySamples
* output = buffer[readIndex]
* ``
*
* **MatemÃtica - InterpolaÃÃo Linear**:
* ``
* D_int = floor(m_delaySamples)
* D_frac = m_delaySamples - D_int
*
* readIndex_0 = (writeIndex - D_int) % MaxDelaySamples
* readIndex_1 = (readIndex_0 - 1 + MaxDelaySamples) % MaxDelaySamples
*
* sample_0 = buffer[readIndex_0]
* sample_1 = buffer[readIndex_1]
*
* output = (1 - D_frac) * sample_0 + D_frac * sample_1
* ``

```

```

*
* **Matemática - Interpolação Cúbica Hermite**
* ```
* D_int = floor(m_delaySamples)
* D_frac = m_delaySamples - D_int
*
* Lã 4 pontos:
* y[0] = buffer[(writeIndex - D_int + 0) % MaxDelaySamples]
* y[1] = buffer[(writeIndex - D_int - 1) % MaxDelaySamples]
* y[2] = buffer[(writeIndex - D_int - 2) % MaxDelaySamples]
* y[3] = buffer[(writeIndex - D_int - 3) % MaxDelaySamples]
*
* Funções Hermite (t = D_frac):
* h00 = 2t³ - 3t² + 1
* h10 = t³ - 2t² + t
* h01 = -2t³ + 3t²
* h11 = t³ - t²
*
* Derivadas (diferenças finitas):
* m0 = (y[2] - y[1]) / 2
* m1 = (y[3] - y[0]) / 2
*
* output = h00*y[0] + h10*m0 + h01*y[1] + h11*m1
* ```
*
* **Real-Time Safety**: ã GARANTIDO
* - O(1) constante, sem alocação
* - Sem exception
* - Sem RTTI
* - Determinístico (mesma latência sempre)
*
* **Uso em Loop de Áudio - Echo Simples**:
* ```cpp
* for (size_t n = 0; n < blockSize; ++n) {
*     float input = inputBuffer[n];
*     float delayed = delay.process(input);
*     outputBuffer[n] = input + 0.7f * delayed; // Echo com feedback
* }
* ```
*
* **Uso - Chorus (Modulação)**:
* ```cpp
* cvdsp::Oscillator<float> lfo;
* lfo.prepare(48000);
* lfo.setFrequency(2.0f); // 2 Hz
*
* for (size_t n = 0; n < blockSize; ++n) {
*     float lfoValue = lfo.process(); // -1 a 1
*     float baseDelay = 40.0f; // 40 ms
*     float modAmount = 10.0f; // ±10 ms
*     float modulatedDelay = baseDelay + lfoValue * modAmount;
*
*     delay.setDelaySamples(modulatedDelay * 48.0f); // 48 samples/ms
*
*     float input = inputBuffer[n];
*     float delayed = delay.process(input);
*     outputBuffer[n] = input + 0.5f * delayed; // Chorus
* }
* ```
*
* **Uso - Flanger (Efeito Sweep)**:
* ```cpp
* float phase = 0.0f;
* for (size_t n = 0; n < blockSize; ++n) {

```

```

*     float lfo = std::sin(phase);
*     delay.setDelayMilliseconds(2.0f + lfo * 1.5f); // 0.5-3.5ms
*
*     float input = inputBuffer[n];
*     float delayed = delay.process(input);
*     outputBuffer[n] = input + 0.8f * delayed; // Flanger
*
*     phase += 2.0f * M_PI * 2.0f / 48000.0f; // 2 Hz LFO
* }
* ...
*
* **Uso - Vibrato (Pitch Modulado)**:
* ```cpp
* for (size_t n = 0; n < blockSize; ++n) {
*     float lfo = std::sin(2.0f * M_PI * 6.0f * n / 48000.0f);
*     delay.setDelayMilliseconds(2.5f + lfo * 1.0f);
*
*     // IMPORTANTE: apenas delay, sem sinal seco!
*     outputBuffer[n] = delay.process(inputBuffer[n]);
* }
* ...
*
* @param[in] inputSample Amostra de entrada a processar
*
* @return value_type Amostra atrasada (interpolada se aplicável)
*
* @note Delay mínimo: 0 (bypass)
*       Delay máximo: MaxDelaySamples
*       Mudanças suaves de delay não causam clicks audíveis
*
* @warning Deve ser chamado exatamente uma vez por amostra processada
*/
inline value_type process(const value_type inputSample) noexcept
{
    // ===== ESCRITA =====
    // Escreve entrada no buffer circular
    m_buffer[m_writeIndex] = inputSample;

    // ===== LEITURA COM INTERPOLAÇÃO =====
    value_type outputSample = value_type(0);

    if constexpr (Interpolation == InterpolationType::None)
    {
        // Sem interpolação: delay inteiro
        outputSample = processNoInterpolation();
    }
    else if constexpr (Interpolation == InterpolationType::Linear)
    {
        // Interpolação linear (2 pontos)
        outputSample = processLinearInterpolation();
    }
    else if constexpr (Interpolation == InterpolationType::Cubic)
    {
        // Interpolação cúbica (4 pontos)
        outputSample = processCubicInterpolation();
    }

    // ===== AVANÇO DO PONTA DE ESCRITA =====
    // Move para próxima posição no buffer circular
    m_writeIndex = (m_writeIndex + 1) & MODULO_MASK;

    return outputSample;
}

```

```

/**
 * @brief Processa um bloco de amostras (vectorizável).
 *
 * **Operações**:
 * - Processa cada amostra do bloco via process()
 * - Sem dependência de dados entre iterações (exceto índices internos)
 *
 * **Complexidade**: O(blockSize)
 *
 * **Otimização do Compilador**:
 * - Loop pode ser vetorizado parcialmente
 * - Dependência de writeIndex impede vetorização total
 * - Moderno compilador (clang, gcc) pode usar SIMD parcialmente
 *
 * **Uso Típico**:
 * ```cpp
 * float inputBlock[512];
 * float outputBlock[512];
 * delay.processBlock(inputBlock, outputBlock, 512);
 * ```
 *
 * **Real-Time Safety**: "GARANTIDO"
 *
 * @param[in] inputBlock Ponteiro para bloco de entrada
 * @param[out] outputBlock Ponteiro para bloco de saída
 * @param[in] blockSize Número de amostras a processar
 *
 * @return void
 *
 * @pre inputBlock != nullptr
 * @pre outputBlock != nullptr
 * @pre blockSize > 0
 */
inline void processBlock(const value_type* inputBlock,
                        value_type* outputBlock,
                        size_type blockSize) noexcept
{
    assert(inputBlock != nullptr);
    assert(outputBlock != nullptr);
    assert(blockSize > 0);

    for (size_type n = 0; n < blockSize; ++n)
    {
        outputBlock[n] = process(inputBlock[n]);
    }
}

// =====
// MÃTODOS DE CONSULTA - REAL-TIME SAFE
// =====

/**
 * @brief Lê uma amostra atrasada por um número explícito de amostras.
 *
 * Este método separa leitura e escrita para blocos como all-pass, chorus e
 * flanger que precisam calcular feedback antes de gravar a próxima
amostra.
 */
[[nodiscard]]
inline value_type readSamples(value_type delayInSamples) const noexcept
{
    return readDelaySamples(delayInSamples);
}

```

```

/**
 * @brief Lê uma amostra atrasada por um número inteiro de amostras.
 *
 * Nome separado evita ambiguidade com literais inteiros em readSamples().
 */
[[nodiscard]]
inline value_type readIntegerSamples(size_type delayInSamples) const
noexcept
{
    return readDelaySamples(static_cast<value_type>(delayInSamples));
}

/**
 * @brief Lê uma amostra atrasada por milissegundos, usando interpolação.
 */
[[nodiscard]]
inline value_type readInterpolated(value_type delayInMilliseconds) const
noexcept
{
    const value_type delayInSamples =
        delayInMilliseconds * static_cast<value_type>(m_sampleRate) /
        static_cast<value_type>(1000);

    return readDelaySamples(delayInSamples);
}

/**
 * @brief Escreve uma amostra e avança o cursor circular.
 */
inline void write(value_type inputSample) noexcept
{
    m_buffer[m_writeIndex] = inputSample;
    m_writeIndex = (m_writeIndex + 1) & MODULO_MASK;
}

/**
 * @brief Retorna o delay atual em amostras.
 *
 * **Complexidade**: O(1)
 *
 * @return value_type Delay em amostras (pode ser fracionário)
 */
inline value_type getDelaySamples() const noexcept
{
    return m_delaySamples;
}

/**
 * @brief Retorna o delay atual em milissegundos.
 *
 * **Matemática**:  $\text{delay\_ms} = \text{delay\_samples} * 1000 / F_s$ 
 *
 * **Complexidade**: O(1)
 *
 * @return value_type Delay em ms
 */
inline value_type getDelayMilliseconds() const noexcept
{
    return m_delaySamples * static_cast<value_type>(1000) /
        static_cast<value_type>(m_sampleRate);
}

/**
 * @brief Retorna o delay atual em segundos.

```

```

*
* **Matemática**: delay_s = delay_samples / Fs
*
* **Complexidade**: O(1)
*
* @return value_type Delay em segundos
*/
inline value_type getDelaySeconds() const noexcept
{
    return m_delaySamples / static_cast<value_type>(m_sampleRate);
}

/**
* @brief Retorna a taxa de amostragem configurada.
*
* **Complexidade**: O(1)
*
* @return size_type Sample rate em Hz
*/
inline size_type getSampleRate() const noexcept
{
    return m_sampleRate;
}

/**
* @brief Retorna se a delay line foi inicializada via prepare().
*
* **Complexidade**: O(1)
*
* @return bool true se prepare() foi chamado
*/
inline bool isInitialized() const noexcept
{
    return m_initialized;
}

/**
* @brief Retorna a capacidade máxima de delay.
*
* **Complexidade**: O(1) compile-time
*
* @return size_type Máximo de amostras de delay
*/
static constexpr size_type getMaxDelaySamples() noexcept
{
    return MAX_DELAY_SAMPLES;
}

/**
* @brief Retorna o tipo de interpolação utilizado.
*
* **Complexidade**: O(1) compile-time
*
* @return InterpolationType Tipo configurado (None, Linear, Cubic)
*/
static constexpr InterpolationType getInterpolationType() noexcept
{
    return INTERPOLATION_TYPE;
}

```

private:

```

[[nodiscard]]
inline value_type readDelaySamples(value_type delayInSamples) const noexcept

```



```

{
    const value_type clampedDelay = std::clamp(delayInSamples,
                                                static_cast<value_type>(0),
static_cast<value_type>(MaxDelaySamples));

    if constexpr (Interpolation == InterpolationType::None)
    {
        const size_type delayInt = static_cast<size_type>(clampedDelay);
        const size_type readIndex = (m_writeIndex + MaxDelaySamples -
delayInt) & MODULO_MASK;
        return m_buffer[readIndex];
    }
    else if constexpr (Interpolation == InterpolationType::Linear)
    {
        const value_type delayFrac = clampedDelay -
std::floor(clampedDelay);
        const size_type delayInt = static_cast<size_type>(clampedDelay);
        const size_type readIndex0 = (m_writeIndex + MaxDelaySamples -
delayInt) & MODULO_MASK;
        const size_type readIndex1 = (readIndex0 + MaxDelaySamples - 1) &
MODULO_MASK;

        const value_type sample0 = m_buffer[readIndex0];
        const value_type sample1 = m_buffer[readIndex1];

        return sample0 + delayFrac * (sample1 - sample0);
    }
    else
    {
        const value_type t = clampedDelay - std::floor(clampedDelay);
        const size_type delayInt = static_cast<size_type>(clampedDelay);

        const size_type idx0 = (m_writeIndex + MaxDelaySamples - delayInt +
1) & MODULO_MASK;
        const size_type idx1 = (m_writeIndex + MaxDelaySamples - delayInt) &
MODULO_MASK;
        const size_type idx2 = (idx1 + MaxDelaySamples - 1) & MODULO_MASK;
        const size_type idx3 = (idx1 + MaxDelaySamples - 2) & MODULO_MASK;

        const value_type y0 = m_buffer[idx0];
        const value_type y1 = m_buffer[idx1];
        const value_type y2 = m_buffer[idx2];
        const value_type y3 = m_buffer[idx3];

        const value_type m0 = (y2 - y1) * static_cast<value_type>(0.5);
        const value_type m1 = (y3 - y0) * static_cast<value_type>(0.5);

        const value_type t2 = t * t;
        const value_type t3 = t2 * t;

        const value_type h00 = static_cast<value_type>(2.0) * t3 -
                                static_cast<value_type>(3.0) * t2 +
                                static_cast<value_type>(1.0);
        const value_type h10 = t3 - static_cast<value_type>(2.0) * t2 + t;
        const value_type h01 = -static_cast<value_type>(2.0) * t3 +
                                static_cast<value_type>(3.0) * t2;
        const value_type h11 = t3 - t2;

        return h00 * y0 + h10 * m0 + h01 * y1 + h11 * m1;
    }
}

private:

```

```

// =====
// MEMBROS PRIVADOS
// =====

/// @brief Buffer circular para armazenar hist³rico de amostras
/// Tamanho: MaxDelaySamples amostras
/// InicializaÃ§Ã£o: Zero-initialized na construÃ§Ã£o
std::array<value_type, MaxDelaySamples> m_buffer;

/// @brief Ãndice de escrita (cursor de inserÃ§Ã£o)
/// Incrementa com cada process(), wraps automaticamente via MODULO_MASK
/// Faixa: 0 â‰¤ m_writeIndex < MaxDelaySamples
size_type m_writeIndex;

/// @brief Delay em amostras (pode ser fracionÃrio para interpolaÃ§Ã£o)
/// Faixa: 0 â‰¤ m_delaySamples â‰¤ MaxDelaySamples
/// Atualizado por setDelaySamples(), setDelayMilliseconds(),
setDelaySeconds()
value_type m_delaySamples;

/// @brief Taxa de amostragem em Hz (necessÃrio para conversÃ£o ms)
/// TÃpicos valores: 44100, 48000, 96000, 192000
size_type m_sampleRate;

/// @brief Flag de inicializaÃ§Ã£o (track se prepare() foi chamado)
bool m_initialized;

// =====
// MÃTODOS PRIVADOS - INTERPOLAÃÃO
// =====

/**
 * @brief LÃa amostra com delay inteiro (sem interpolaÃ§Ã£o).
 *
 * Apenas lÃa da posiÃ§Ã£o circular, sem suavizaÃ§Ã£o.
 * Usado quando InterpolationType = None.
 *
 * **Complexidade**: O(1)
 *
 * @return value_type Amostra atrasada (inteira, sem interpolaÃ§Ã£o)
 */
inline value_type processNoInterpolation() const noexcept
{
    // Calcula delay inteiro
    const size_type delayInt = static_cast<size_type>(m_delaySamples);

    // Calcula Ãndice de leitura (atual - delay)
    // Nota: writeIndex jÃ foi incrementado antes do return, entÃo
jÃ aponta
    // para pr³xima posiÃ§Ã£o. Compensar com -1.
    const size_type readIndex = (m_writeIndex - delayInt - 1 +
MaxDelaySamples) & MODULO_MASK;

    // LÃa e retorna
    return m_buffer[readIndex];
}

/**
 * @brief LÃa amostra com interpolaÃ§Ã£o linear.
 *
 * Interpola entre duas amostras usando fraÃ§Ã£o de delay.
 * Usado quando InterpolationType = Linear.
 *
 * **F³rmula**:

```

```

*   *
*   output = (1 - frac) * sample_0 + frac * sample_1
*   *
*
*   **Complexidade**: O(1)
*
*   @return value_type Amostra interpolada linearmente
*/
inline value_type processLinearInterpolation() const noexcept
{
    // Separa delay em parte inteira e fracionária
    const value_type delayFrac = m_delaySamples -
std::floor(m_delaySamples);
    const size_type delayInt = static_cast<size_type>(m_delaySamples);

    // Calcula dois índices de leitura
    const size_type readIndex0 = (m_writeIndex - delayInt - 1 +
MaxDelaySamples) & MODULO_MASK;
    const size_type readIndex1 = (readIndex0 - 1 + MaxDelaySamples) &
MODULO_MASK;

    // Lê as duas amostras
    const value_type sample0 = m_buffer[readIndex0];
    const value_type sample1 = m_buffer[readIndex1];

    // Interpolação linear: output = (1 - frac) * sample0 + frac * sample1
    return sample0 + delayFrac * (sample1 - sample0);
}

/**
*   @brief Lê amostra com interpolação cúbica Hermite.
*
*   Interpola entre 4 pontos usando polinômio cúbico Hermite.
*   Produz qualidade superior com apenas ~4x o custo de uma leitura.
*   Usado quando InterpolationType = Cubic.
*
*   **Fórmula de interpolação cúbica Hermite**:
*   *
*   
$$h_{00}(t) = 2t^3 - 3t^2 + 1$$

*   
$$h_{10}(t) = t^3 - 2t^2 + t$$

*   
$$h_{01}(t) = -2t^3 + 3t^2$$

*   
$$h_{11}(t) = t^3 - t^2$$

*
*   Derivadas aproximadas (diferenças finitas):
*   
$$m_0 = (y[2] - y[1]) / 2$$

*   
$$m_1 = (y[3] - y[0]) / 2$$

*
*   
$$\text{interpolated} = h_{00}(t)y[0] + h_{10}(t)m_0 + h_{01}(t)y[1] + h_{11}(t)m_1$$

*   *
*
*   **Complexidade**: O(1)
*
*   @return value_type Amostra interpolada cubicamente
*/
inline value_type processCubicInterpolation() const noexcept
{
    // Separa delay em parte inteira e fracionária
    const value_type t = m_delaySamples - std::floor(m_delaySamples);
    const size_type delayInt = static_cast<size_type>(m_delaySamples);

    // Calcula quatro índices de leitura
    const size_type idx0 = (m_writeIndex - delayInt - 0 + MaxDelaySamples) &
MODULO_MASK;
    const size_type idx1 = (m_writeIndex - delayInt - 1 + MaxDelaySamples) &

```

```

MODULO_MASK;
    const size_type idx2 = (m_writeIndex - delayInt - 2 + MaxDelaySamples) &
MODULO_MASK;
    const size_type idx3 = (m_writeIndex - delayInt - 3 + MaxDelaySamples) &
MODULO_MASK;

    // Lê as quatro amostras
    const value_type y0 = m_buffer[idx0];
    const value_type y1 = m_buffer[idx1];
    const value_type y2 = m_buffer[idx2];
    const value_type y3 = m_buffer[idx3];

    // Calcula derivadas usando diferenças finitas centralizadas
    // m0 = (y[2] - y[1]) / 2
    // m1 = (y[3] - y[0]) / 2
    const value_type m0 = (y2 - y1) * static_cast<value_type>(0.5);
    const value_type m1 = (y3 - y0) * static_cast<value_type>(0.5);

    // Prê-calcula potências de t para eficiência
    const value_type t2 = t * t;
    const value_type t3 = t2 * t;

    // Calcula funções Hermite base
    // h00(t) = 2t³ - 3t² + 1
    const value_type h00 = static_cast<value_type>(2.0) * t3 -
        static_cast<value_type>(3.0) * t2 +
        static_cast<value_type>(1.0);

    // h10(t) = t³ - 2t² + t
    const value_type h10 = t3 - static_cast<value_type>(2.0) * t2 + t;

    // h01(t) = -2t³ + 3t²
    const value_type h01 = -static_cast<value_type>(2.0) * t3 +
        static_cast<value_type>(3.0) * t2;

    // h11(t) = t³ - t²
    const value_type h11 = t3 - t2;

    // Interpolação cúbica final: combinação linear ponderada
    return h00 * y0 + h10 * m0 + h01 * y1 + h11 * m1;
}
};

} // namespace cvdsp::delay

#endif // CVDSP_DELAY_DELAYLINE_HPP

```

```

=====
FILE: CV_DSP/Dynamics/Compressor.hpp
=====
#ifndef CVDSP_DYNAMICS_COMPRESSOR_HPP
#define CVDSP_DYNAMICS_COMPRESSOR_HPP

#include <cmath>
#include <cstdint>
#include <type_traits>
#include <algorithm>

#include "../Core/Types.hpp"
#include "EnvelopeFollower.hpp"

/**

```

[illegible]

```

* entre @f$ T - W/2 @f$ e @f$ T + W/2 @f$:
*
* @f[
*   Y_{dB} =
*   \begin{cases}
*     X_{dB}, & \& X_{dB} < T - W/2 \\[6pt]
*     X_{dB} + \dfrac{(1/R - 1)\,(X_{dB} - T + W/2)^2}{2W}, \\
*     & \& T - W/2 \leq X_{dB} \leq T + W/2 \\[6pt]
*     T + \dfrac{X_{dB} - T}{R}, & \& X_{dB} > T + W/2
*   \end{cases}
* @f]
*
* Nota: quando @f$ W = 0 @f$, o comportamento se reduz ao hard knee exato.
*
* @section gr Gain Reduction
*
* A redu  o de ganho em dB   simplesmente:
*
* @f[
*   GR_{dB} = Y_{dB} - X_{dB} \;\leq\; 0
* @f]
*
* e o ganho linear aplicado   amostra  :
*
* @f[
*   g = 10^{(GR_{dB} + \text{makeupDB})/20}
* @f]
*
* @section envelopes Envelope Detection
*
* A detec  o do n vel   delegada ao m dulo @ref EnvelopeFollower (com
* ataque/release assim tricos). Isso desacopla a estimativa de amplitude da
* curva de ganho, permitindo uso com detector Peak (reage a picos/transientes)
* ou RMS (compress o mais "musical" baseada em energia percebida).
*
* @section utilities Fun  es utilit rias
*
* - @ref dbToGain : converte @f$ dB \to @f$ ganho linear @f$ = 10^{dB/20} @f$.
* - @ref gainToDb : converte ganho linear @f$ \to @f$ dB @f$ = 20\log_{10}(g)
* @f$.
*
* @section rt Garantias de tempo real
*
* - `process()`   0(1), `noexcept`, sem aloca  o/exce  es/RTTI.
* - Usa `std::log10` e `std::pow` (ou equivalente `exp`/`log`) por amostra
*   para as convers es linear dB. Para contextos SIMD-cr ticos recomenda-se
*   usar lookup tables ou aproxima  es polinomiais (ex.: FastMath.hpp).
* - Prote  o contra n vel zero (clampa para um piso antes de log).
*/
namespace cvdsp::dynamics
{
/**
* @brief Converte decib is em ganho linear.
*
* @f$ g = 10^{dB/20} @f$.
*
* @tparam T Tipo de ponto flutuante.
* @param dB N vel em decib is.
* @return Ganho linear correspondente.
*/
template <typename T>
[[nodiscard]] inline T dbToGain(T dB) noexcept
{

```

```

    return std::pow(static_cast<T>(10), dB / static_cast<T>(20));
}

/**
 * @brief Converte ganho linear em decib is.
 *
 * @f$ dB = 20 \cdot \log_{10}(g) @f$.
 *
 * @tparam T Tipo de ponto flutuante.
 * @param gain Ganho linear (> 0; valores <= 0 produzem resultado indefinido).
 * @return N vel em dB.
 */
template <typename T>
[[nodiscard]] inline T gainToDb(T gain) noexcept
{
    return static_cast<T>(20) * std::log10(gain);
}

/**
 * @class Compressor
 * @brief Compressor feed-forward com soft/hard knee e makeup gain.
 *
 * @tparam T Tipo de ponto flutuante do processamento (`float` ou `double`).
 *
 * @par Exemplo    compressor vocal leve (soft knee):
 * @code
 * cvdsp::dynamics::Compressor<float> comp;
 * comp.prepare(48000.0f);
 * comp.setThresholdDB(-18.0f);
 * comp.setRatio(3.0f);
 * comp.setAttackMs(10.0f);
 * comp.setReleaseMs(100.0f);
 * comp.setKneeDB(6.0f); // soft knee de 6 dB
 * comp.setMakeupGainDB(4.0f);
 * for (std::size_t n = 0; n < numSamples; ++n)
 *     buffer[n] = comp.process(buffer[n]);
 * @endcode
 *
 * @par Exemplo    limiter agressivo (hard knee, ratio alto):
 * @code
 * cvdsp::dynamics::Compressor<double> lim;
 * lim.prepare(96000.0);
 * lim.setThresholdDB(-1.0);
 * lim.setRatio(100.0); // ratio muito alto    quase limiter
 * lim.setAttackMs(0.1);
 * lim.setReleaseMs(50.0);
 * lim.setKneeDB(0.0); // hard knee (sem suaviza  o)
 * lim.setMakeupGainDB(0.0);
 * double y = lim.process(x);
 * @endcode
 *
 * @par Exemplo    compressor RMS para bus (gentil):
 * @code
 * cvdsp::dynamics::Compressor<float> bus;
 * bus.prepare(48000.0f);
 * bus.setDetectionMode(cvdsp::dynamics::EnvelopeMode::RMS);
 * bus.setThresholdDB(-12.0f);
 * bus.setRatio(2.0f);
 * bus.setAttackMs(30.0f);
 * bus.setReleaseMs(200.0f);
 * bus.setKneeDB(10.0f);
 * bus.setMakeupGainDB(3.0f);
 * float y = bus.process(x);
 * @endcode

```

```

*/
template <typename T>
class Compressor
{
public:
    static_assert(std::is_floating_point_v<T>,
                  "Compressor requires a floating point type (float or
double)");

    /// @brief Tipo escalar de ponto flutuante usado pelo compressor.
    using value_type = T;
    /// @brief Tipo inteiro sem sinal para contagens e Índices.
    using size_type = std::size_t;

    /**
     * @brief Constrói um compressor com parâmetros padrão estabelecidos.
     *
     * Parâmetros: threshold 0 dB (sem compressão), ratio 1:1 (unity), attack 10
ms,
     * release 100 ms, knee 0 dB (hard), makeup 0 dB. Não aloca.
     */
    constexpr Compressor() noexcept = default;

    /// @brief Destrutor trivial; nada alocado.
    ~Compressor() noexcept = default;

    Compressor(const Compressor&) noexcept = default;
    Compressor& operator=(const Compressor&) noexcept = default;
    Compressor(Compressor&&) noexcept = default;
    Compressor& operator=(Compressor&&) noexcept = default;

    /**
     * @brief Prepara o compressor para uma taxa de amostragem.
     *
     * Deve ser chamado fora do audio thread. Delega ao EnvelopeFollower e
     * zera o estado de redução de ganho.
     *
     * @param sampleRate Taxa de amostragem em Hz (> 0).
     */
    inline void prepare(value_type sampleRate) noexcept
    {
        m_envelope.prepare(sampleRate);
        m_currentGainReductionDB = static_cast<value_type>(0);
    }

    /**
     * @brief Zera o estado interno (envelope + gain reduction).
     *
     * Real-time safe: O(1), sem alocação/exceções.
     */
    inline void reset() noexcept
    {
        m_envelope.reset();
        m_currentGainReductionDB = static_cast<value_type>(0);
    }

    /**
     * @brief Define o threshold em dB (não pode ser maior do que ocorre compressão).
     * @param thresholdDB Threshold em dBFS (tipicamente negativo).
     */
    inline void setThresholdDB(value_type thresholdDB) noexcept
    {
        m_thresholdDB = thresholdDB;
    }

```



```

/**
 * @brief Define o ratio de compressÃ£o (N:1).
 *
 * Ratio = 1 â†’ sem compressÃ£o. Ratio = âˆž (ou valor muito grande) â†’
limiter.
 * @param ratio Ratio (>= 1; clamped para min 1).
 */
inline void setRatio(value_type ratio) noexcept
{
    m_ratio = std::max(ratio, static_cast<value_type>(1));
}

/**
 * @brief Define o tempo de ataque do detector em milissegundos.
 * @param attackMs Tempo de ataque (ms, >= 0).
 */
inline void setAttackMs(value_type attackMs) noexcept
{
    m_envelope.setAttackMs(attackMs);
}

/**
 * @brief Define o tempo de release do detector em milissegundos.
 * @param releaseMs Tempo de release (ms, >= 0).
 */
inline void setReleaseMs(value_type releaseMs) noexcept
{
    m_envelope.setReleaseMs(releaseMs);
}

/**
 * @brief Define a largura do knee em dB.
 *
 * @f$ W = 0 @f$ â†’ hard knee (transiÃ§Ã£o abrupta);
 * @f$ W > 0 @f$ â†’ soft knee (transiÃ§Ã£o quadrÃ¡tica suave).
 * @param kneeDB Largura do knee em dB (>= 0; clamped para min 0).
 */
inline void setKneeDB(value_type kneeDB) noexcept
{
    m_kneeDB = std::max(kneeDB, static_cast<value_type>(0));
}

/**
 * @brief Define o ganho de compensaÃ§Ã£o (makeup) em dB.
 *
 * Adicionado apÃ³s a reduÃ§Ã£o para compensar a perda de volume.
 * @param makeupDB Ganho de makeup em dB.
 */
inline void setMakeupGainDB(value_type makeupDB) noexcept
{
    m_makeupGainDB = makeupDB;
}

/**
 * @brief Define o modo de detecÃ§Ã£o do envelope (Peak ou RMS).
 *
 * Peak reage a transientes individuais; RMS comprime baseado em energia
 * percebida (mais "musical" para bus/master). PadrÃ£o: Peak.
 * @param mode Modo de detecÃ§Ã£o (ver @ref EnvelopeMode).
 */
inline void setDetectionMode(EnvelopeMode mode) noexcept
{
    m_envelope.setMode(mode);
}

```

```

}

/**
 * @brief Processa uma amostra e retorna o sinal comprimido.
 *
 * Fluxo:
 * 1. EnvelopeFollower â†’ nÃvel linear @f$ e @f$.
 * 2. @f$  $X_{\text{dB}} = 20 \log_{10}(\max(e, \epsilon))$  @f$.
 * 3. Gain Computer â†’ @f$  $Y_{\text{dB}}$  @f$ (com knee).
 * 4. @f$  $GR_{\text{dB}} = Y_{\text{dB}} - X_{\text{dB}}$  @f$.
 * 5. @f$  $g = 10^{\{(GR_{\text{dB}} + \text{makeup})/20\}}$  @f$.
 * 6. @f$  $y = x \cdot g$  @f$.
 *
 * @param x Amostra de entrada @f$  $x[n]$  @f$.
 * @return Amostra comprimida @f$  $y[n]$  @f$.
 *
 * Real-time safe:  $O(1)$ , `noexcept`, sem alocaÃ§Ã£o/exceÃ§Ãµes/RTTI.
 */
[[nodiscard]] inline value_type process(value_type x) noexcept
{
    const value_type twenty = static_cast<value_type>(20);

    // 1. DetecÃ§Ã£o de envelope (linear,  $\geq 0$ ).
    const value_type env = m_envelope.process(x);

    // 2. Converter envelope para dB (com piso para evitar  $\log(0)$ ).
    const value_type envClamped = std::max(env, kFloorLinear);
    const value_type inputDB = twenty * std::log10(envClamped);

    // 3. Gain Computer â€” calcula nÃvel de saÃda desejado em dB.
    const value_type outputDB = computeGain(inputDB);

    // 4. ReduÃ§Ã£o de ganho em dB (sempre  $\leq 0$  para compressÃ£o).
    const value_type grDB = outputDB - inputDB;
    m_currentGainReductionDB = grDB;

    // 5. Converter reduÃ§Ã£o + makeup para ganho linear.
    const value_type totalDB = grDB + m_makeupGainDB;
    const value_type gainLin =
        std::pow(static_cast<value_type>(10), totalDB / twenty);

    // 6. Aplicar ganho Ã amostra original.
    return x * gainLin;
}

/**
 * @brief Processa um bloco de amostras in-place.
 * @param buffer Ponteiro para as amostras (nÃo-nulo se @p numSamples > 0).
 * @param numSamples Quantidade de amostras a processar.
 *
 * Real-time safe:  $O(N)$ , sem alocaÃ§Ã£o/exceÃ§Ãµes.
 */
inline void processBlock(value_type* buffer, size_type numSamples) noexcept
{
    for (size_type n = 0; n < numSamples; ++n)
        buffer[n] = process(buffer[n]);
}

/**
 * @brief Retorna a reduÃ§Ã£o de ganho atual em dB ( $\leq 0$ ).
 *
 * Ãštil para medidores de GR em UIs.
 */
[[nodiscard]] constexpr value_type getGainReductionDB() const noexcept

```

```

{
    return m_currentGainReductionDB;
}

/// @brief Threshold configurado (dB).
[[nodiscard]] constexpr value_type getThresholdDB() const noexcept { return
m_thresholdDB; }
/// @brief Ratio configurado (N:1).
[[nodiscard]] constexpr value_type getRatio() const noexcept { return
m_ratio; }
/// @brief Knee configurado (dB).
[[nodiscard]] constexpr value_type getKneeDB() const noexcept { return
m_kneeDB; }
/// @brief Makeup gain configurado (dB).
[[nodiscard]] constexpr value_type getMakeupGainDB() const noexcept { return
m_makeupGainDB; }

private:
/**
 * @brief Gain Computer com suporte a soft knee.
 *
 * Dado @f$ X_{dB} @f$, retorna @f$ Y_{dB} @f$ conforme a curva de
 * compress o (threshold, ratio, knee).
 *
 * @param inputDB N vel de entrada em dB.
 * @return N vel de sa da em dB (nunca maior que inputDB para ratio >= 1).
 */
[[nodiscard]] inline value_type computeGain(value_type inputDB) const
noexcept
{
    const value_type thresh = m_thresholdDB;
    const value_type R = m_ratio;
    const value_type W = m_kneeDB;
    const value_type half = static_cast<value_type>(0.5);
    const value_type two = static_cast<value_type>(2);

    if (W <= static_cast<value_type>(0))
    {
        // Hard knee.
        if (inputDB < thresh)
            return inputDB;
        return thresh + (inputDB - thresh) / R;
    }

    // Soft knee   tr s regi es.
    const value_type lowerBound = thresh - W * half;
    const value_type upperBound = thresh + W * half;

    if (inputDB < lowerBound)
    {
        // Abaixo do knee: sem compress o.
        return inputDB;
    }
    if (inputDB > upperBound)
    {
        // Acima do knee: compress o total.
        return thresh + (inputDB - thresh) / R;
    }

    // Dentro do knee: transi  o quadr tica suave.
    //  $Y = X + (1/R - 1) * (X - thresh + W/2)^2 / (2W)$ 
    const value_type diff = inputDB - thresh + W * half;
    const value_type slope = (static_cast<value_type>(1) / R)
        - static_cast<value_type>(1);

```

```

        return inputDB + slope * (diff * diff) / (two * W);
    }

    /**
     * @brief Piso linear mínimo para evitar  $\log_{10}(0) = -\infty$ .
     *
     * Equivale a aproximadamente -200 dBFS, muito abaixo de qualquer sinal
     * audível; garante que std::log10 nunca receba zero ou negativo.
     */
    static constexpr value_type kFloorLinear =
        static_cast<value_type>(1e-10);

    EnvelopeFollower<value_type> m_envelope{};          ///< Detector de nível.
    value_type m_thresholdDB{static_cast<value_type>(0)}; ///< Threshold (dB).
    value_type m_ratio{static_cast<value_type>(1)};      ///< Ratio (N:1).
    value_type m_kneeDB{static_cast<value_type>(0)};     ///< Knee width (dB).
    value_type m_makeupGainDB{static_cast<value_type>(0)}; ///< Makeup (dB).
    value_type m_currentGainReductionDB{static_cast<value_type>(0)}; ///< GR
    atual.
};

} // namespace cvdsp::dynamics

#endif // CVDSP_DYNAMICS_COMPRESSOR_HPP

```

```

=====
FILE: CV_DSP/Dynamics/EnvelopeFollower.hpp
=====

```

```

#ifndef CVDSP_DYNAMICS_ENVELOPEFOLLOWER_HPP
#define CVDSP_DYNAMICS_ENVELOPEFOLLOWER_HPP

```

```

#include <cmath>
#include <cstdint>
#include <type_traits>
#include <algorithm>

```

```

#include "../Core/Types.hpp"

```

```

/**
 * @file EnvelopeFollower.hpp
 * @brief Peak / RMS envelope follower (detector) for CV_DSP.
 * @namespace cvdsp::dynamics
 *
 * @section overview Visão geral
 *
 * Um *envelope follower* (detector de envelope) estima a amplitude
 * instantânea a "energia" de um sinal de áudio ao longo do tempo. É
 * o
 * bloco de medição que alimenta processadores de dinâmica (compressores,
 * limiters, gates, expanders), moduladores controlados por amplitude (auto-wah,
 * envelope filters) e medidores de nível.
 *
 * Esta implementação oferece dois modos de detecção @b Peak e @b RMS
 * com
 * tempos de @b ataque e @b release independentes, todos parametrizados em
 * milissegundos e convertidos em coeficientes de suavização de um polo a
 * partir
 * da taxa de amostragem.
 *
 * Segue as regras do CV_DSP: header-only, C++20, sem dependências externas,
 * real-time safe (sem alocação, exceto ou RTTI em process()),
 * template<typename T> para float e double, com Doxygen completo.

```

```

*
* @section peak Peak detection (detecção de pico)
*
* A detecção de pico retifica o sinal (valor absoluto) e suaviza o resultado
* com um filtro de um pólo assimétrico (ataque rápido, release lento):
*
* @f[
*   d[n] = |x[n]|
* @f]
* @f[
*   e[n] =
*   \begin{cases}
*     \alpha_a, e[n-1] + (1-\alpha_a), d[n], & d[n] > e[n-1] \\
*     \alpha_r, e[n-1] + (1-\alpha_r), d[n], & d[n] \leq e[n-1]
*   \end{cases}
* @f]
*
* O envelope sobe rápido quando o sinal cresce (coeficiente de ataque) e
* desce devagar quando o sinal cai (coeficiente de release). Isso captura
* transientes sem "soltar" o nível bruscamente.
*
* @section rms RMS detection (detecção RMS)
*
* A detecção RMS estima a raiz do valor quadrático médio, que corresponde
* à energia percebida (mais próxima da audição que o pico). Suaviza-se o
* quadrado do sinal e extrai-se a raiz:
*
* @f[
*   d[n] = x^2[n]
* @f]
* @f[
*   m[n] =
*   \begin{cases}
*     \alpha_a, m[n-1] + (1-\alpha_a), d[n], & d[n] > m[n-1] \\
*     \alpha_r, m[n-1] + (1-\alpha_r), d[n], & d[n] \leq m[n-1]
*   \end{cases}
* @f]
* @f[
*   e[n] = \sqrt{m[n]}
* @f]
*
* O estado interno  $m[n]$  é a média móvel exponencial do sinal ao
* quadrado (o "mean square"); a saída é a sua raiz (o "root mean square").
*
* @section coeffs Conversão tempo → coeficiente
*
* Cada coeficiente vem da resposta de um filtro de um pólo de primeira ordem.
* Para uma constante de tempo  $\tau$  (em segundos), o coeficiente é:
*
* @f[
*   \alpha = e^{-1/(\tau f_s)}, \quad \tau = \text{timeMs} \times 10^{-3}
* @f]
*
* Interpretação: após  $\tau$  segundos, o envelope percorre
*  $1 - 1/e \approx 63.2\%$  da distância até o novo valor. Esta é a
* definição clássica de "tempo de ataque/release" por constante de tempo.
* Tempos menores →  $\alpha$  menor → resposta mais rápida;
*  $\tau = 0$  →  $\alpha = 0$  → resposta instantânea.
*
* @section tracking Envelope tracking (gráficos conceituais)

```

```
* Comportamento ataque rápido / release lento sobre uma rajada de sinal:
```

```
* @verbatim
```

$|x[n]|$ (retificado)

t

envelope e[n]

```
* Peak : segue a crista ( $|x|$ ), reage a transientes individuais.
```

```
* RMS : segue a energia ( $\sqrt{\text{mean}(x^2)}$ ), mais suave e "musical".
```

```
* @endverbatim
```

```
* @section rt Garantias de tempo real
```

```
* - `process()` Ã© O(1), `noexcept`, sem alocaÃ§Ã£o/exceÃ§Ãµes/RTTI.
```

```
* - Apenas uma comparaÃ§Ã£o, duas multiplicaÃ§Ãµes e uma soma por amostra (mais um `sqrt` no modo RMS).
```

```
* - ProteÃ§Ã£o contra denormais no estado do envelope.
```

```
*/
```

```
namespace cvdsp::dynamics
```

```
{
```

```
/**
```

```
* @brief Modo de detecÃ§Ã£o do @ref EnvelopeFollower.
```

```
*/
```

```
enum class EnvelopeMode
```

```
{
```

```
    Peak, ///< DetecÃ§Ã£o de pico: suaviza  $|x[n]|$ .
```

```
    RMS, ///< DetecÃ§Ã£o RMS: suaviza  $x^2[n]$  e retorna a raiz.
```

```
};
```

```
/**
```

```
* @class EnvelopeFollower
```

```
* @brief Detector de envelope Peak/RMS com ataque e release independentes.
```

```
* @tparam T Tipo de ponto flutuante do processamento (`float` ou `double`).
```

```
* @par Exemplo â€” detector de pico para um VU/mecedor:
```

```
* @code
```

```
cvsdp::dynamics::EnvelopeFollower<float> env;
```

```
env.prepare(48000.0f);
```

```
env.setMode(cvsdp::dynamics::EnvelopeMode::Peak);
```

```
env.setAttackMs(1.0f); // ataque r apido (1 ms)
```

```
env.setReleaseMs(100.0f); // release suave (100 ms)
```

```
for (std::size_t n = 0; n < numSamples; ++n)
```

```
{
```

```
    float level = env.process(buffer[n]); // 0..~1 (linear)
```

```
// usar 'level' para iluminar um medidor, etc.
```

```
}
```

```
@endcode
```

```
* @par Exemplo â€” detector RMS alimentando um ganho de compressor:
```

```
* @code
```

```
cvsdp::dynamics::EnvelopeFollower<double> det;
```

```
det.prepare(96000.0);
```

```
det.setMode(cvsdp::dynamics::EnvelopeMode::RMS);
```

```
det.setAttackMs(10.0);
```

```
det.setReleaseMs(200.0);
```

```
double rms = det.process(x);
```

```

*   double rmsDB = 20.0 * std::log10(std::max(rms, 1e-9));
*   // computar redu  o de ganho a partir de rmsDB...
* @endcode
*
* @par Exemplo    auto-wah (envelope controla a cutoff):
* @code
*   float e = env.process(guitarSample);    // envelope da guitarra
*   float cutoff = 300.0f + 4000.0f * e;    // mapeia envelope -> Hz
*   // alimentar 'cutoff' em um Biquad/SVF...
* @endcode
*/
template <typename T>
class EnvelopeFollower
{
public:
    static_assert(std::is_floating_point_v<T>,
                  "EnvelopeFollower requires a floating point type (float or
double)");

    /// @brief Tipo escalar de ponto flutuante usado pelo detector.
    using value_type = T;
    /// @brief Tipo inteiro sem sinal para contagens e  ndices.
    using size_type = std::size_t;

    /**
     * @brief Constr i um detector com tempos padr o e estado zerado.
     *
     * Padr es: modo Peak, ataque 10 ms, release 100 ms. Os coeficientes s o
s o
     * v lidos ap s @ref prepare definir a taxa de amostragem. N o aloca.
     */
    constexpr EnvelopeFollower() noexcept = default;

    /// @brief Destrutor trivial; nada   alocado.
    ~EnvelopeFollower() noexcept = default;

    EnvelopeFollower(const EnvelopeFollower&) noexcept = default;
    EnvelopeFollower& operator=(const EnvelopeFollower&) noexcept = default;
    EnvelopeFollower(EnvelopeFollower&&) noexcept = default;
    EnvelopeFollower& operator=(EnvelopeFollower&&) noexcept = default;

    /**
     * @brief Prepara o detector para uma taxa de amostragem.
     *
     * Deve ser chamado fora do audio thread. Armazena @f$f_s@f$, zera o
     * estado e recalcula os coeficientes de ataque e release a partir dos
     * tempos (ms) atuais.
     *
     * @param sampleRate Taxa de amostragem em Hz (> 0; inv lidos viram 48
kHz).
     */
    inline void prepare(value_type sampleRate) noexcept
    {
        m_sampleRate = (sampleRate > static_cast<value_type>(0))
            ? sampleRate
            : static_cast<value_type>(48000);

        reset();
        m_attackCoeff = coeffFromMs(m_attackMs);
        m_releaseCoeff = coeffFromMs(m_releaseMs);
    }

    /**
     * @brief Zera o estado interno do envelope.
     */

```

```

    * Real-time safe: O(1), sem alocação/exceções.
    */
inline void reset() noexcept
{
    m_state = static_cast<value_type>(0);
}

/**
 * @brief Define o tempo de ataque em milissegundos.
 *
 * Recalcula apenas o coeficiente de ataque. @f$ 0 @f$ = instantâneo.
 * @param attackMs Tempo de ataque (ms, >= 0).
 */
inline void setAttackMs(value_type attackMs) noexcept
{
    m_attackMs = std::max(attackMs, static_cast<value_type>(0));
    m_attackCoeff = coeffFromMs(m_attackMs);
}

/**
 * @brief Define o tempo de release em milissegundos.
 *
 * Recalcula apenas o coeficiente de release. @f$ 0 @f$ = instantâneo.
 * @param releaseMs Tempo de release (ms, >= 0).
 */
inline void setReleaseMs(value_type releaseMs) noexcept
{
    m_releaseMs = std::max(releaseMs, static_cast<value_type>(0));
    m_releaseCoeff = coeffFromMs(m_releaseMs);
}

/**
 * @brief Define o modo de detecção (Peak ou RMS).
 *
 * @param mode Novo modo (ver @ref EnvelopeMode).
 *
 * @note Os estados internos de Peak e RMS têm escalas distintas ( $|x|$  vs  $x^2$ ). Trocar de modo em tempo de execução pode produzir um
degrau;
 *
 * chame @ref reset se desejar reiniciar o seguimento.
 */
inline void setMode(EnvelopeMode mode) noexcept { m_mode = mode; }

/**
 * @brief Processa uma amostra e retorna o valor atual do envelope (linear).
 *
 * No modo Peak retorna o envelope de @f$  $|x|$  @f$; no modo RMS retorna
 * @f$  $\sqrt{m[n]}$  @f$.
 *
 * @param x Amostra de entrada @f$  $x[n]$  @f$.
 * @return Valor do envelope @f$  $e[n]$  @f$ (escala linear).
 *
 * Real-time safe: O(1), `noexcept`, sem alocação/exceções/RTTI.
 */
[[nodiscard]] inline value_type process(value_type x) noexcept
{
    // Sinal do detector:  $|x|$  (Peak) ou  $x^2$  (RMS).
    const value_type d = (m_mode == EnvelopeMode::Peak)
        ? std::abs(x)
        : x * x;

    // Coeficiente assimétrico: ataque ao subir, release ao descer.
    const value_type coeff = (d > m_state) ? m_attackCoeff
        : m_releaseCoeff;

```



```

    // Suaviza o valor de um pulso:  $e = \text{coeff} * e + (1 - \text{coeff}) * d$ .
    m_state = coeff * m_state +
        (static_cast<value_type>(1) - coeff) * d;

    // Protege contra denormais no estado.
    m_state += kDenormalGuard;
    m_state -= kDenormalGuard;

    return (m_mode == EnvelopeMode::Peak) ? m_state
        : std::sqrt(m_state);
}

/**
 * @brief Retorna o valor atual do envelope sem processar nova amostra.
 * @return Envelope corrente (linear), já com a raiz aplicada no modo RMS.
 */
[[nodiscard]] inline value_type getEnvelope() const noexcept
{
    return (m_mode == EnvelopeMode::Peak) ? m_state
        : std::sqrt(m_state);
}

/// @brief Modo de detecção atual.
[[nodiscard]] constexpr EnvelopeMode getMode() const noexcept { return
m_mode; }
/// @brief Tempo de ataque configurado (ms).
[[nodiscard]] constexpr value_type getAttackMs() const noexcept { return
m_attackMs; }
/// @brief Tempo de release configurado (ms).
[[nodiscard]] constexpr value_type getReleaseMs() const noexcept { return
m_releaseMs; }
/// @brief Coeficiente de ataque em uso.
[[nodiscard]] constexpr value_type getAttackCoeff() const noexcept { return
m_attackCoeff; }
/// @brief Coeficiente de release em uso.
[[nodiscard]] constexpr value_type getReleaseCoeff() const noexcept { return
m_releaseCoeff; }
/// @brief Taxa de amostragem configurada (Hz).
[[nodiscard]] constexpr value_type getSampleRate() const noexcept { return
m_sampleRate; }

private:
/**
 * @brief Converte um tempo (ms) em coeficiente de um pulso.
 *
 * @f$ \alpha = e^{\{-1/(\tau f_s)\}} @f$, com @f$ \tau = \text{timeMs}/1000
@f$.
 * Para @f$ \text{timeMs} \leq 0 @f$ retorna 0 (resposta instantânea),
 * evitando divisão por zero.
 *
 * @param timeMs Tempo em milissegundos ( $\geq 0$ ).
 * @return Coeficiente @f$ \alpha \in [0, 1) @f$.
 */
[[nodiscard]] inline value_type coeffFromMs(value_type timeMs) const
noexcept
{
    if (timeMs <= static_cast<value_type>(0))
        return static_cast<value_type>(0);

    const value_type tauSamples =
        (timeMs * static_cast<value_type>(0.001)) * m_sampleRate;
    //  $\alpha = \exp(-1 / (\tau_{\text{in samples}}))$ .
    return std::exp(static_cast<value_type>(-1) / tauSamples);
}

```

```

}

/// @brief Constante anti-denormal para o estado do envelope.
static constexpr value_type kDenormalGuard =
    static_cast<value_type>(1e-20);

EnvelopeMode m_mode{EnvelopeMode::Peak};           ///< Modo atual.
value_type m_attackMs{static_cast<value_type>(10)}; ///< Ataque (ms).
value_type m_releaseMs{static_cast<value_type>(100)}; ///< Release (ms).
value_type m_attackCoeff{static_cast<value_type>(0)}; ///< Coef. ataque.
value_type m_releaseCoeff{static_cast<value_type>(0)}; ///< Coef. release.
value_type m_state{static_cast<value_type>(0)};      ///< Estado e/m[n].
value_type m_sampleRate{static_cast<value_type>(48000)}; ///< fs (Hz).
};

} // namespace cvdsp::dynamics

#endif // CVDSP_DYNAMICS_ENVELOPEFOLLOWER_HPP

=====
FILE: CV_DSP/Dynamics/Expander.hpp
=====
#ifndef CVDSP_DYNAMICS_EXPANDER_HPP
#define CVDSP_DYNAMICS_EXPANDER_HPP

/**
 * @file Expander.hpp
 * @brief Downward Expander
 *
 * Header-only
 * Real-Time Safe
 * C++20
 */

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <type_traits>

namespace cvdsp::dynamics
{
/**
 * @brief Downward Expander
 *
 * Threshold em dBFS
 * Ratio >= 1
 *
 * Exemplo:
 *
 * threshold = -40 dB
 * ratio = 4
 *
 * Sinais abaixo do threshold
 * serão progressivamente atenuados.
 */
template<typename T>
class Expander
{
    static_assert(
        std::is_floating_point_v<T>,
        "Expander requires floating point type");

```

```

public:

    constexpr Expander() noexcept = default;

    /**
     * @brief Inicializa coeficientes.
     *
     * @param sampleRate Taxa de amostragem
     * @param attackMs Ataque em ms
     * @param releaseMs Release em ms
     * @param thresholdDb Threshold em dBFS
     * @param ratio Ratio de expansÃ£o
     */
    void prepare(
        T sampleRate,
        T attackMs,
        T releaseMs,
        T thresholdDb,
        T ratio) noexcept
    {
        sampleRate_ = sampleRate;

        attackMs_ = std::max(
            attackMs,
            static_cast<T>(0.001));

        releaseMs_ = std::max(
            releaseMs,
            static_cast<T>(0.001));

        thresholdDb_ = thresholdDb;

        ratio_ = std::max(
            ratio,
            static_cast<T>(1));

        attackCoeff_ =
            computeTimeCoefficient(
                attackMs_);

        releaseCoeff_ =
            computeTimeCoefficient(
                releaseMs_);

        reset();
    }

    /**
     * @brief Reinicia estado interno.
     */
    void reset() noexcept
    {
        envelopeDb_ = static_cast<T>(-120);
        gainDb_ = static_cast<T>(0);
    }

    /**
     * @brief Processa uma amostra.
     *
     * @param input Entrada
     * @return Saída expandida
     */
    inline T process(T input) noexcept

```

```

{
    constexpr T kMinDb =
        static_cast<T>(-120);

    constexpr T kEpsilon =
        static_cast<T>(1e-30);

    const T absInput =
        std::abs(input);

    const T levelDb =
        static_cast<T>(20) *
        std::log10(
            std::max(
                absInput,
                kEpsilon));

    const T detectorCoeff =
        (levelDb > envelopeDb_)
        ? attackCoeff_
        : releaseCoeff_;

    envelopeDb_ =
        detectorCoeff * envelopeDb_
        +
        (static_cast<T>(1) - detectorCoeff)
        * levelDb;

    T targetGainDb =
        static_cast<T>(0);

    if (envelopeDb_ < thresholdDb_)
    {
        const T delta =
            thresholdDb_ - envelopeDb_;

        targetGainDb =
            -delta *
            (ratio_ - static_cast<T>(1));
    }

    const T gainCoeff =
        (targetGainDb < gainDb_)
        ? attackCoeff_
        : releaseCoeff_;

    gainDb_ =
        gainCoeff * gainDb_
        +
        (static_cast<T>(1) - gainCoeff)
        * targetGainDb;

    gainDb_ =
        std::clamp(
            gainDb_,
            kMinDb,
            static_cast<T>(0));

    const T linearGain =
        dbToLinear(
            gainDb_);

    return input * linearGain;
}

```

```

private:

    /**
     * @brief Converte dB para ganho linear.
     */
    static constexpr T dbToLinear(
        T db) noexcept
    {
        return std::pow(
            static_cast<T>(10),
            db / static_cast<T>(20));
    }

    /**
     * @brief Coeficiente exponencial.
     */
    T computeTimeCoefficient(
        T timeMs) const noexcept
    {
        const T timeSeconds =
            timeMs *
            static_cast<T>(0.001);

        return std::exp(
            static_cast<T>(-1)
            /
            (
                sampleRate_
                *
                timeSeconds
            ));
    }

private:

    T sampleRate_ = static_cast<T>(44100);

    T thresholdDb_ = static_cast<T>(-40);

    T ratio_ = static_cast<T>(4);

    T attackMs_ = static_cast<T>(5);

    T releaseMs_ = static_cast<T>(100);

    T attackCoeff_ = static_cast<T>(0);

    T releaseCoeff_ = static_cast<T>(0);

    T envelopeDb_ = static_cast<T>(-120);

    T gainDb_ = static_cast<T>(0);
};

} // namespace cvdsp::dynamics

namespace cvdsp
{
    template<typename T>
    using Expander = dynamics::Expander<T>;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Dynamics/Limiter.hpp
=====
#ifndef CVDSP_DYNAMICS_LIMITER_HPP
#define CVDSP_DYNAMICS_LIMITER_HPP

#include <cmath>
#include <cstdint>
#include <type_traits>
#include <algorithm>

#include "../Core/Types.hpp"
#include "EnvelopeFollower.hpp"

/**
 * @file Limiter.hpp
 * @brief Brickwall limiter (sem lookahead) para CV_DSP.
 * @namespace cvdsp::dynamics
 *
 * @section overview Visão geral
 *
 * Implementa um brickwall limiter que garante que nenhum sample na saída
da
 * exceda o threshold configurado em valor absoluto. Opera com ataque
instantâneo (zero) e release ajustável, reutilizando o módulo
 * @ref EnvelopeFollower (modo Peak) para estimativa de nível.
 *
 * Segue as regras do CV_DSP: header-only, C++20, sem dependências externas,
 * real-time safe (sem alocação, exceto em process()),
 * template<typename T> para float e double, com Doxygen completo.
 *
 * @section brickwall Brickwall Limiting
 *
 * Um "brickwall" limiter é o processador de dinâmica mais agressivo possível:
 * garante que a saída nunca exceda uma amplitude máxima (o
ceiling/threshold).
 * Diferentemente de um compressor com ratio finito, o limiter efetivamente
impõe
 * ratio infinito (∞:1) acima do threshold.
 *
 * O princípio é:
 *
 * @f[
 * g[n] = \min\{1, \frac{T_{\text{lin}}}{|x[n]|}\}
 * @f[
 * @f[
 * y[n] = x[n] \cdot g[n]
 * @f[
 *
 * onde @f$ T_{\text{lin}} = 10^{\{T_{\text{dB}}/20\}} @f$ é o threshold em escala linear
e
 * @f$ e[n] @f$ é o envelope instantâneo de pico.
 *
 * @subsection guarantee Prova da garantia brickwall
 *
 * Usamos o @ref EnvelopeFollower com ataque zero (instantâneo). Isso
garante:
 *
 * - Caso ataque (@f$ |x[n]| > e[n-1] @f$):
 *   @f$ e[n] = |x[n]| @f$ (snap instantâneo).
 *   @f$ g[n] = T_{\text{lin}} / |x[n]| @f$.

```

[illegible]

```

* Esta implementa  o opera apenas em **sample-peak** (cada amostra discreta).
* Ela garante que nenhum sample digital exceda o threshold; true-peaks
* inter-amostra podem ainda existir ~0.5  3 dB acima. Para prote  o contra
* true-peak, aplicar oversampling (ex.: 4  ) antes deste limiter ou usar um
* limiter com lookahead + oversampled peak detection.
*
* @section rt Garantias de tempo real
*
* - `process()`    O(1), `noexcept`, sem aloca  o/exce  es/RTTI.
* - Zero lat  ncia (sem lookahead buffer).
* - Nenhum sample na sa  da excede o threshold (garantia brickwall).
* - Prote  o contra denormais no estado do envelope.
*/
namespace cvdsp::dynamics
{
/**
* @class Limiter
* @brief Brickwall limiter sem lookahead com ataque instant  neo.
*
* @tparam T Tipo de ponto flutuante do processamento (`float` ou `double`).
*
* @par Exemplo   " limiter de sa  da (ceiling -0.3 dBFS):
* @code
*   cvdsp::dynamics::Limiter<float> lim;
*   lim.prepare(48000.0f);
*   lim.setThresholdDB(-0.3f);
*   lim.setReleaseMs(50.0f);
*   for (std::size_t n = 0; n < numSamples; ++n)
*       buffer[n] = lim.process(buffer[n]);
*   // Garantia: |buffer[n]| <= 10^(-0.3/20)   ^ 0.966 para todo n.
* @endcode
*
* @par Exemplo   " safety limiter antes do DAC:
* @code
*   cvdsp::dynamics::Limiter<double> safety;
*   safety.prepare(96000.0);
*   safety.setThresholdDB(0.0);    // ceiling = 1.0 (0 dBFS)
*   safety.setReleaseMs(100.0);
*   double y = safety.process(x); // |y| <= 1.0 sempre
* @endcode
*
* @par Exemplo   " uso em cadeia (ap  s compressor):
* @code
*   // Compressor reduz a din  mica; limiter garante ceiling absoluto.
*   float y = comp.process(x);    // pode ter overshoot residual
*   y = limiter.process(y);       // brickwall garante |y| <= threshold
* @endcode
*/
template <typename T>
class Limiter
{
public:
    static_assert(std::is_floating_point_v<T>,
                  "Limiter requires a floating point type (float or double)");

    /// @brief Tipo escalar de ponto flutuante usado pelo limiter.
    using value_type = T;
    /// @brief Tipo inteiro sem sinal para contagens e   ndices.
    using size_type = std::size_t;

/**
* @brief Constr  i um limiter com threshold 0 dBFS (unity) e release 50 ms.
*

```



```

    * Estado zerado; coeficientes válidos após @ref prepare. Não aloca.
    */
constexpr Limiter() noexcept = default;

/// @brief Destrutor trivial; nada alocado.
~Limiter() noexcept = default;

Limiter(const Limiter&) noexcept = default;
Limiter& operator=(const Limiter&) noexcept = default;
Limiter(Limiter&&) noexcept = default;
Limiter& operator=(Limiter&&) noexcept = default;

/**
 * @brief Prepara o limiter para uma taxa de amostragem.
 *
 * Deve ser chamado fora do audio thread. Configura o EnvelopeFollower
 * interno (Peak, attack=0, release conforme setReleaseMs), zera estado e
 * recalcula o threshold linear.
 *
 * @param sampleRate Taxa de amostragem em Hz (> 0).
 */
inline void prepare(value_type sampleRate) noexcept
{
    m_sampleRate = (sampleRate > static_cast<value_type>(0))
        ? sampleRate
        : static_cast<value_type>(48000);
    m_envelope.prepare(m_sampleRate);
    m_envelope.setMode(EnvelopeMode::Peak);
    m_envelope.setAttackMs(static_cast<value_type>(0)); // instantâneo
    m_envelope.setReleaseMs(m_releaseMs);
    updateThresholdLinear();
}

/**
 * @brief Zera o estado interno do limiter.
 *
 * Real-time safe: O(1), sem alocação/exceções.
 */
inline void reset() noexcept
{
    m_envelope.reset();
}

/**
 * @brief Define o threshold (ceiling) em dBFS.
 *
 * Nenhum sample de saída terá valor absoluto acima de
 *  $10^{\{T_{dB}\}/20}$  @f$. Tipicamente 0 dB (unity) ou ligeiramente
 * negativo (-0.1 a -1 dBFS) para headroom em conversores.
 *
 * @param thresholdDB Threshold em dBFS (<= 0 para não amplificar).
 */
inline void setThresholdDB(value_type thresholdDB) noexcept
{
    m_thresholdDB = thresholdDB;
    updateThresholdLinear();
}

/**
 * @brief Define o tempo de release em milissegundos.
 *
 * Controla quão rápido o ganho retorna a 1.0 após um transiente ser
 * limitado. Valores menores = release mais rápido (mais distorção);
 * maiores = release mais suave (mais bombeamento/ducking).
 */

```

```

*
* @param releaseMs Tempo de release (ms, >= 0; 0 = instantâneo).
*/
inline void setReleaseMs(value_type releaseMs) noexcept
{
    m_releaseMs = std::max(releaseMs, static_cast<value_type>(0));
    m_envelope.setReleaseMs(m_releaseMs);
}

/**
* @brief Processa uma amostra e retorna o sinal limitado.
*
* Garante @f$ |y[n]| \le T_{\text{lin}} @f$ para toda amostra.
*
* Fluxo:
* 1. EnvelopeFollower(Peak, attack=0) â†’ @f$ e[n] \ge |x[n]| @f$
(provado).
* 2. @f$ g[n] = \min(1, T_{\text{lin}} / \max(e[n], \epsilon)) @f$.
* 3. @f$ y[n] = x[n] \cdot g[n] @f$.
*
* @param x Amostra de entrada @f$ x[n] @f$.
* @return Amostra limitada @f$ y[n] @f$ com @f$ |y[n]| \le T_{\text{lin}}
@f$.
*
* Real-time safe: O(1), `noexcept`, sem alocação/exceções/RTTI.
*/
[[nodiscard]] inline value_type process(value_type x) noexcept
{
    // 1. Envelope instantâneo de pico (env >= |x| garantido).
    const value_type env = m_envelope.process(x);

    // 2. Ganho necessário para não exceder threshold.
    const value_type envSafe = std::max(env, kFloorLinear);
    const value_type gain = std::min(static_cast<value_type>(1),
                                     m_thresholdLinear / envSafe);

    // 3. Aplicar ganho.
    return x * gain;
}

/**
* @brief Processa um bloco de amostras in-place.
* @param buffer Ponteiro para as amostras (não-nulo se @p numSamples > 0).
* @param numSamples Quantidade de amostras a processar.
*
* Real-time safe: O(N), sem alocação/exceções.
*/
inline void processBlock(value_type* buffer, size_type numSamples) noexcept
{
    for (size_type n = 0; n < numSamples; ++n)
        buffer[n] = process(buffer[n]);
}

/// @brief Threshold configurado (dBFS).
[[nodiscard]] constexpr value_type getThresholdDB() const noexcept { return
m_thresholdDB; }
/// @brief Threshold linear em uso.
[[nodiscard]] constexpr value_type getThresholdLinear() const noexcept
{ return m_thresholdLinear; }
/// @brief Tempo de release configurado (ms).
[[nodiscard]] constexpr value_type getReleaseMs() const noexcept { return
m_releaseMs; }
/// @brief Taxa de amostragem (Hz).
[[nodiscard]] constexpr value_type getSampleRate() const noexcept { return

```

```

m_sampleRate; }

/**
 * @brief Retorna a redu  o de ganho atual em dB (<= 0).
 *
 * Calculada a partir do  ltimo envelope/threshold: quanto o limiter est ;
 * "segurando" o sinal neste instante.
 */
[[nodiscard]] inline value_type getGainReductionDB() const noexcept
{
    const value_type env = m_envelope.getEnvelope();
    if (env <= m_thresholdLinear)
        return static_cast<value_type>(0);
    // GR = 20*log10(threshold/env)
    return static_cast<value_type>(20) *
        std::log10(m_thresholdLinear / std::max(env, kFloorLinear));
}

private:
/**
 * @brief Recalcula o threshold linear a partir de m_thresholdDB.
 */
inline void updateThresholdLinear() noexcept
{
    m_thresholdLinear =
        std::pow(static_cast<value_type>(10),
            m_thresholdDB / static_cast<value_type>(20));
}

/// @brief Piso linear para evitar divis o por zero.
static constexpr value_type kFloorLinear =
    static_cast<value_type>(1e-10);

EnvelopeFollower<value_type> m_envelope{};          ///< Detector
Peak.
value_type m_thresholdDB{static_cast<value_type>(0)};  ///< Threshold
(dB).
value_type m_thresholdLinear{static_cast<value_type>(1)}; ///< Threshold
linear.
value_type m_releaseMs{static_cast<value_type>(50)};    ///< Release (ms).
value_type m_sampleRate{static_cast<value_type>(48000)}; ///< fs (Hz).
};

} // namespace cvdsp::dynamics

#endif // CVDSP_DYNAMICS_LIMITER_HPP

```

```

=====
FILE: CV_DSP/Dynamics/NoiseGate.hpp
=====
#ifndef CVDSP_DYNAMICS_NOISEGATE_HPP
#define CVDSP_DYNAMICS_NOISEGATE_HPP

#include <cmath>
#include <cstdint>
#include <type_traits>
#include <algorithm>

#include "../Core/Types.hpp"
#include "EnvelopeFollower.hpp"

```

```

/**
 * @file NoiseGate.hpp
 * @brief Noise gate com hysteresis, hold e ataque/release para CV_DSP.
 * @namespace cvdsp::dynamics
 *
 * @section overview Visão geral
 *
 * Um **noise gate** silencia o sinal quando ele cai abaixo de um limiar,
 * permitindo apenas passagem quando o sinal é "forte o suficiente". É
 * fundamental para instrumentos como guitarra elétrica, onde captadores de
 * alta impedância captam ruído eletromagnético constante que deve ser
 * eliminado nos silêncios.
 *
 * Esta implementação usa **dois limiares** (open / close) configurados
 * independentemente, criando uma banda de **hysteresis** que impede
 * oscilação rápida (chattering) ao redor de um único threshold.
 *
 * Segue as regras do CV_DSP: header-only, C++20, sem dependências externas,
 * real-time safe (sem alocação, excetões ou RTTI em `process()`),
 * `template<typename T>` para `float` e `double`, com Doxygen completo.
 *
 * @section guitar Gates para guitarra
 *
 * Em uma cadeia de efeitos de guitarra, o noise gate tipicamente fica
 * **antes** dos efeitos de distorção/overdrive ou como primeiro efeito:
 *
 * @verbatim
 *   Guitarra â†' [NoiseGate] â†' Overdrive â†' EQ â†' Amp
 * @endverbatim
 *
 * Por quê? A distorção amplifica o ruído de fundo dramaticamente. Um gate
 * antes da distorção elimina o ruído **antes** de ser amplificado, resultando
 * em silêncios limpos entre notas. Em setups de metal/high-gain, o gate é
 * essencial para "tight stops" (paradas bruscas entre riffs).
 *
 * Parâmetros típicos para guitarra:
 * - Threshold Open: -40 a -30 dBFS (acima do nível de ruído dos captadores)
 * - Threshold Close: 3 a 6 dB abaixo do Open (hysteresis)
 * - Attack: 0.1 a 2 ms (muito rápido para preservar transientes de palhetada)
 * - Hold: 20 a 100 ms (sustém notas curtas sem cortar)
 * - Release: 10 a 50 ms (decai suavemente sem "click" abrupto)
 *
 * @section pickups Ruído de captadores
 *
 * Captadores magnéticos de guitarra (single-coil e humbucker) são
 * susceptíveis a:
 * - **Hum 50/60 Hz**: interferência de rede elétrica (mais forte em
 *   single-coils).
 * - **Ruído térmico**: gerado pela resistência do fio do captador
 *   (~5 a 20 kΩ); aparece como um chiado (hiss) de banda larga.
 * - **EMI/RFI**: interferência de fontes externas (lâmpadas fluorescentes,
 *   dimmers, aparelhos digitais, Wi-Fi).
 *
 * Níveis típicos de ruído de captadores em silêncio:
 * - Single-coil: -50 a -40 dBFS (alto, especialmente hum)
 * - Humbucker: -60 a -50 dBFS (o modo diferencial cancela o hum)
 * - Captadores ativos (EMG, Fishman): -70 a -60 dBFS (pré-amp interno,
 *   baixa impedância)
 *
 * O noise gate deve ter threshold ajustado **acima** deste piso de ruído
 * para silenciar o sinal nos trechos sem toque.
 *
 * @section hysteresis Hysteresis
 *

```

[illegible]

```

*      g[n] =
*      \begin{cases}
*          \alpha_a, & g[n-1] + (1-\alpha_a)\cdot 1, & \& \text{abrindo}\\
*          \alpha_r, & g[n-1] + (1-\alpha_r)\cdot 0, & \& \text{fechando}
*      \end{cases}
*  @f]
*
*  onde @f$ \alpha_a = e^{\{-1/(\tau_a f_s)\}} @f$ e
*  @f$ \alpha_r = e^{\{-1/(\tau_r f_s)\}} @f$.
*
*  @section rt Garantias de tempo real
*
*  - `process()` Ã© O(1), `noexcept`, sem alocaÃ§Ã£o/exceÃ§Ãµes/RTTI.
*  - DetecÃ§Ã£o de nÃvel via @ref EnvelopeFollower (Peak, attack e release
*    rÃpidos) para evitar chattering em amostras individuais.
*  - Zero latÃancia (sem lookahead).
*/
namespace cvdsp::dynamics
{
/**
 * @brief Estado interno do noise gate.
 */
enum class GateState
{
    Closed,    ///< Gate fechado â€ ganho tendendo a 0 (silenciando).
    Open,      ///< Gate aberto â€ ganho tendendo a 1 (passando sinal).
    Holding    ///< Gate em hold â€ ganho em 1, aguardando timer expirar.
};

/**
 * @class NoiseGate
 * @brief Noise gate com hysteresis, hold e suavizaÃ§Ã£o de ataque/release.
 *
 * @tparam T Tipo de ponto flutuante do processamento (`float` ou `double`).
 *
 * @par Exemplo â€ gate para guitarra high-gain:
 * @code
 *     cvdsp::dynamics::NoiseGate<float> gate;
 *     gate.prepare(48000.0f);
 *     gate.setThresholdOpenDB(-35.0f);
 *     gate.setThresholdCloseDB(-40.0f);    // 5 dB hysteresis
 *     gate.setAttackMs(0.5f);
 *     gate.setHoldMs(50.0f);
 *     gate.setReleaseMs(30.0f);
 *     for (std::size_t n = 0; n < numSamples; ++n)
 *         buffer[n] = gate.process(buffer[n]);
 * @endcode
 *
 * @par Exemplo â€ gate suave para vocal (reduÃ§Ã£o de bleed):
 * @code
 *     cvdsp::dynamics::NoiseGate<double> voxGate;
 *     voxGate.prepare(96000.0);
 *     voxGate.setThresholdOpenDB(-45.0);
 *     voxGate.setThresholdCloseDB(-50.0);
 *     voxGate.setAttackMs(2.0);
 *     voxGate.setHoldMs(100.0);
 *     voxGate.setReleaseMs(80.0);
 *     double y = voxGate.process(x);
 * @endcode
 *
 * @par Exemplo â€ gate em cadeia com compressor:
 * @code
 *     // Gate remove ruÃdo, compressor controla dinÃmica.

```

```

*   float y = gate.process(guitarSample);    // silencia ruído
*   y = compressor.process(y);               // comprime picos
* @endcode
*/
template <typename T>
class NoiseGate
{
public:
    static_assert(std::is_floating_point_v<T>,
                  "NoiseGate requires a floating point type (float or double)");

    /// @brief Tipo escalar de ponto flutuante.
    using value_type = T;
    /// @brief Tipo inteiro sem sinal para contagens e índices.
    using size_type = std::size_t;

    /**
     * @brief Constrói um noise gate com parâmetros padrão.
     *
     * Padrões: threshOpen -40 dB, threshClose -45 dB (5 dB hysteresis),
     * attack 1 ms, hold 50 ms, release 30 ms, estado Closed. Não aloca.
     */
    constexpr NoiseGate() noexcept = default;

    /// @brief Destrutor trivial; nada alocado.
    ~NoiseGate() noexcept = default;

    NoiseGate(const NoiseGate&) noexcept = default;
    NoiseGate& operator=(const NoiseGate&) noexcept = default;
    NoiseGate(NoiseGate&&) noexcept = default;
    NoiseGate& operator=(NoiseGate&&) noexcept = default;

    /**
     * @brief Prepara o gate para uma taxa de amostragem.
     *
     * Deve ser chamado fora do audio thread. Configura o detector de nível
     * interno, recalcula coeficientes e limiares lineares.
     *
     * @param sampleRate Taxa de amostragem em Hz (> 0).
     */
    inline void prepare(value_type sampleRate) noexcept
    {
        m_sampleRate = (sampleRate > static_cast<value_type>(0))
            ? sampleRate
            : static_cast<value_type>(48000);

        // Detector de nível: Peak com attack muito rápido e release curto
        // para detecção responsiva sem chattering.
        m_detector.prepare(m_sampleRate);
        m_detector.setMode(EnvelopeMode::Peak);
        m_detector.setAttackMs(static_cast<value_type>(0.05));
        m_detector.setReleaseMs(static_cast<value_type>(1));

        updateThresholdsLinear();
        updateAttackCoeff();
        updateReleaseCoeff();
        updateHoldSamples();

        reset();
    }

    /**
     * @brief Zera o estado interno do gate (fecha, zera ganho e detector).
     */

```

```

    * Real-time safe: O(1), sem alocação/exceções.
    */
inline void reset() noexcept
{
    m_detector.reset();
    m_state = GateState::Closed;
    m_gain = static_cast<value_type>(0);
    m_holdCounter = 0;
}

/**
 * @brief Define o threshold de abertura em dBFS.
 *
 * O gate abre quando o nível sobe acima deste limiar.
 * @param thresholdDB Threshold open (dB).
 */
inline void setThresholdOpenDB(value_type thresholdDB) noexcept
{
    m_thresholdOpenDB = thresholdDB;
    updateThresholdsLinear();
}

/**
 * @brief Define o threshold de fechamento em dBFS.
 *
 * O gate fecha (após hold) quando o nível cai abaixo deste limiar.
 * Deve ser <= thresholdOpen para hysteresis efetiva.
 * @param thresholdDB Threshold close (dB).
 */
inline void setThresholdCloseDB(value_type thresholdDB) noexcept
{
    m_thresholdCloseDB = thresholdDB;
    updateThresholdsLinear();
}

/**
 * @brief Define o tempo de ataque (abertura) em milissegundos.
 *
 * Controla quão rápido o ganho sobe de 0 a 1 quando o gate abre.
 * Valores muito altos podem cortar o transiente; muito baixos podem
 * causar click.
 * @param attackMs Tempo de ataque (ms, >= 0; 0 = instantâneo).
 */
inline void setAttackMs(value_type attackMs) noexcept
{
    m_attackMs = std::max(attackMs, static_cast<value_type>(0));
    updateAttackCoeff();
}

/**
 * @brief Define o tempo de release (fechamento) em milissegundos.
 *
 * Controla quão rápido o ganho cai de 1 a 0 quando o gate fecha
 * (após o hold expirar).
 * @param releaseMs Tempo de release (ms, >= 0; 0 = instantâneo).
 */
inline void setReleaseMs(value_type releaseMs) noexcept
{
    m_releaseMs = std::max(releaseMs, static_cast<value_type>(0));
    updateReleaseCoeff();
}

/**
 * @brief Define o tempo de hold em milissegundos.

```



```

*
* O gate permanece aberto por este período até o nível cair abaixo
* de thresholdClose, antes de iniciar o release. Previne cortes
* prematuros de notas sustentadas.
* @param holdMs Tempo de hold (ms, >= 0).
*/
inline void setHoldMs(value_type holdMs) noexcept
{
    m_holdMs = std::max(holdMs, static_cast<value_type>(0));
    updateHoldSamples();
}

/**
* @brief Processa uma amostra e retorna o sinal com gate aplicado.
*
* Fluxo:
* 1. EnvelopeFollower(Peak) → nível suavizado.
* 2. Máquina de estados (Closed/Open/Holding) com hysteresis.
* 3. Suaviza o ganho (ataque/release one-pole).
* 4. Saída = entrada - ganho.
*
* @param x Amostra de entrada @f$x[n] @f$.
* @return Amostra com gate @f$y[n] = x[n] \cdot g[n] @f$.
*
* Real-time safe: O(1), `noexcept`, sem alocação/exceções/RTTI.
*/
[[nodiscard]] inline value_type process(value_type x) noexcept
{
    const value_type one = static_cast<value_type>(1);
    const value_type zero = static_cast<value_type>(0);

    // 1. Detecção de nível suavizada.
    const value_type env = m_detector.process(x);

    // 2. Máquina de estados com hysteresis.
    switch (m_state)
    {
    case GateState::Closed:
        if (env > m_thresholdOpenLinear)
            m_state = GateState::Open;
        break;

    case GateState::Open:
        if (env < m_thresholdCloseLinear)
        {
            m_state = GateState::Holding;
            m_holdCounter = m_holdSamples;
        }
        break;

    case GateState::Holding:
        if (env > m_thresholdCloseLinear)
        {
            // Re-abre: sinal voltou acima do threshold de fechamento.
            m_state = GateState::Open;
        }
        else if (m_holdCounter > 0)
        {
            --m_holdCounter;
        }
        else
        {
            // Hold expirado: fecha o gate.
            m_state = GateState::Closed;
        }
    }
}

```

```

        }
        break;
    }

    // 3. Ganho-alvo e suaviza  o.
    const value_type targetGain = (m_state == GateState::Closed) ? zero :
one;

    if (targetGain > m_gain)
    {
        // Abrindo: rampa de ataque.
        m_gain = m_attackCoeff * m_gain +
            (one - m_attackCoeff) * targetGain;
    }
    else
    {
        // Fechando (ou j   fechado): rampa de release.
        m_gain = m_releaseCoeff * m_gain +
            (one - m_releaseCoeff) * targetGain;
    }

    // Prote  o contra denormais.
    m_gain += kDenormalGuard;
    m_gain -= kDenormalGuard;

    // 4. Aplicar ganho.
    return x * m_gain;
}

/**
 * @brief Processa um bloco de amostras in-place.
 * @param buffer Ponteiro para as amostras (n  o-nulo se @p numSamples > 0).
 * @param numSamples Quantidade de amostras a processar.
 *
 * Real-time safe: O(N), sem aloca  o/exce  es.
 */
inline void processBlock(value_type* buffer, size_type numSamples) noexcept
{
    for (size_type n = 0; n < numSamples; ++n)
        buffer[n] = process(buffer[n]);
}

/// @brief Estado atual do gate.
[[nodiscard]] constexpr GateState getState() const noexcept { return
m_state; }
/// @brief Ganho atual aplicado (0..1 linear).
[[nodiscard]] constexpr value_type getGain() const noexcept { return m_gain;
}
/// @brief Threshold open (dB).
[[nodiscard]] constexpr value_type getThresholdOpenDB() const noexcept
{ return m_thresholdOpenDB; }
/// @brief Threshold close (dB).
[[nodiscard]] constexpr value_type getThresholdCloseDB() const noexcept
{ return m_thresholdCloseDB; }
/// @brief Tempo de ataque (ms).
[[nodiscard]] constexpr value_type getAttackMs() const noexcept { return
m_attackMs; }
/// @brief Tempo de release (ms).
[[nodiscard]] constexpr value_type getReleaseMs() const noexcept { return
m_releaseMs; }
/// @brief Tempo de hold (ms).
[[nodiscard]] constexpr value_type getHoldMs() const noexcept { return
m_holdMs; }
/// @brief Taxa de amostragem (Hz).

```

```

[[nodiscard]] constexpr value_type getSampleRate() const noexcept { return
m_sampleRate; }

private:
/**
 * @brief Converte um tempo (ms) em coeficiente de um pÃ³lo.
 */
[[nodiscard]] inline value_type coeffFromMs(value_type timeMs) const
noexcept
{
    if (timeMs <= static_cast<value_type>(0))
        return static_cast<value_type>(0);
    const value_type tauSamples =
        (timeMs * static_cast<value_type>(0.001)) * m_sampleRate;
    return std::exp(static_cast<value_type>(-1) / tauSamples);
}

inline void updateThresholdsLinear() noexcept
{
    m_thresholdOpenLinear =
        std::pow(static_cast<value_type>(10),
            m_thresholdOpenDB / static_cast<value_type>(20));
    m_thresholdCloseLinear =
        std::pow(static_cast<value_type>(10),
            m_thresholdCloseDB / static_cast<value_type>(20));
}

inline void updateAttackCoeff() noexcept
{
    m_attackCoeff = coeffFromMs(m_attackMs);
}

inline void updateReleaseCoeff() noexcept
{
    m_releaseCoeff = coeffFromMs(m_releaseMs);
}

inline void updateHoldSamples() noexcept
{
    m_holdSamples = static_cast<std::int32_t>(
        m_holdMs * static_cast<value_type>(0.001) * m_sampleRate +
        static_cast<value_type>(0.5));
}

/// @brief Constante anti-denormal.
static constexpr value_type kDenormalGuard =
    static_cast<value_type>(1e-20);

EnvelopeFollower<value_type> m_detector{};          ///<
Detector.
value_type m_thresholdOpenDB{static_cast<value_type>(-40)};    ///< Open
(dB).
value_type m_thresholdCloseDB{static_cast<value_type>(-45)};    ///< Close
(dB).
value_type m_thresholdOpenLinear{static_cast<value_type>(0.01)}; ///< Open
(lin).
value_type m_thresholdCloseLinear{static_cast<value_type>(0.0056234)}; ///<
Close (lin).
value_type m_attackMs{static_cast<value_type>(1)};            ///< Attack
(ms).
value_type m_releaseMs{static_cast<value_type>(30)};          ///<
Release (ms).
value_type m_holdMs{static_cast<value_type>(50)};            ///< Hold
(ms).

```

```

        value_type m_attackCoeff{static_cast<value_type>(0)};          ///< Coef.
    attack.
        value_type m_releaseCoeff{static_cast<value_type>(0)};        ///< Coef.
    release.
        std::int32_t m_holdSamples{0};                                  ///< Hold
    (samples).
        std::int32_t m_holdCounter{0};                                  ///<
    Contador hold.
        GateState m_state{GateState::Closed};                          ///< Estado
    atual.
        value_type m_gain{static_cast<value_type>(0)};                 ///< Ganho
    (0..1).
        value_type m_sampleRate{static_cast<value_type>(48000)};      ///< fs
    (Hz).
    };

} // namespace cvdsp::dynamics

#endif // CVDSP_DYNAMICS_NOISEGATE_HPP

```

```

=====
FILE: CV_DSP/Effects/Chorus.hpp
=====
#ifndef CVDSP_EFFECTS_CHORUS_HPP
#define CVDSP_EFFECTS_CHORUS_HPP

/**
 * @file Chorus.hpp
 * @brief Mono Chorus Effect
 *
 * Dependencies:
 * - DelayLine.hpp
 * - LFO.hpp
 *
 * Header-only
 * C++20
 * Real-Time Safe
 */

#include <algorithm>
#include <type_traits>

#include "../Delay/DelayLine.hpp"
#include "../Modulation/LFO.hpp"

namespace cvdsp
{
    /**
     * @brief Mono Chorus.
     *
     * Parameters:
     *
     * Rate (Hz)
     * Depth (0..1)
     * Mix (0..1)
     *
     * Delay Modulation:
     *
     * Base Delay:
     * 15 ms
     */
}

```

```

* Modulation:
*  $\hat{\pm}10$  ms
*/
template<typename T>
class Chorus
{
    static_assert(
        std::is_floating_point_v<T>,
        "Chorus requires floating point type");

public:

    constexpr Chorus() noexcept = default;

    /**
     * @brief Initialize chorus.
     *
     * @param sampleRate Processing sample rate.
     * @param maxDelayMs Maximum delay buffer size.
     */
    void prepare(
        T sampleRate,
        T maxDelayMs = static_cast<T>(50.0)) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));

        maxDelayMs_ =
            std::clamp(
                maxDelayMs,
                static_cast<T>(1),
                delayMaxMilliseconds());

        delay_.prepare(
            static_cast<typename delay::DelayLine<T>::size_type>(sampleRate_));

        lfo_.prepare(
            sampleRate_,
            rateHz_,
            static_cast<T>(1),
            LFOWaveform::Sine);

        reset();
    }

    /**
     * @brief Reset effect state.
     */
    void reset() noexcept
    {
        delay_.reset();
        lfo_.reset();
    }

    /**
     * @brief Set modulation rate.
     *
     * Range:
     *
     * 0.01 Hz
     * to
     * 20 Hz
     */

```

```

    */
void setRate(
    T rateHz) noexcept
{
    rateHz_ =
        std::clamp(
            rateHz,
            static_cast<T>(0.01),
            static_cast<T>(20.0));

    lfo_.setRate(rateHz_);
}

/**
 * @brief Set modulation depth.
 *
 * Range:
 *
 * 0.0 .. 1.0
 */
void setDepth(
    T depth) noexcept
{
    depth_ =
        std::clamp(
            depth,
            static_cast<T>(0),
            static_cast<T>(1));
}

/**
 * @brief Set wet/dry mix.
 *
 * Range:
 *
 * 0.0 .. 1.0
 */
void setMix(
    T mix) noexcept
{
    mix_ =
        std::clamp(
            mix,
            static_cast<T>(0),
            static_cast<T>(1));
}

/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    const T modulation =
        lfo_.process();

    const T delayMs =
        std::clamp(
            baseDelayMs_
            +
            (
                modulationDepthMs_
                *
                depth_

```

```

        *
        modulation
    ),
    static_cast<T>(0),
    maxDelayMs_);

const T delayed =
    delay_.readInterpolated(
        delayMs);

delay_.write(input);

const T dry =
    input;

const T wet =
    delayed;

return
    (
        dry
        *
        (
            static_cast<T>(1)
            -
            mix_
        )
    )
    +
    (
        wet
        *
        mix_
    );
}

```

private:

```

[[nodiscard]]
T delayMaxMilliseconds() const noexcept
{
    return static_cast<T>(delay_::DelayLine<T>::getMaxDelaySamples())
        * static_cast<T>(1000)
        / sampleRate_;
}

delay_::DelayLine<T> delay_;

LFO<T> lfo_;

T sampleRate_ =
    static_cast<T>(44100);

T rateHz_ =
    static_cast<T>(0.5);

T depth_ =
    static_cast<T>(0.5);

T mix_ =
    static_cast<T>(0.5);

/**
 * Typical chorus values.

```

```

    */
    T baseDelayMs_ =
        static_cast<T>(15.0);

    T modulationDepthMs_ =
        static_cast<T>(10.0);

    T maxDelayMs_ =
        static_cast<T>(50.0);
};

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Effects/Flanger.hpp
=====

```

```

#ifndef CVDSP_EFFECTS_FLANGER_HPP
#define CVDSP_EFFECTS_FLANGER_HPP

```

```

/**
 * @file Flanger.hpp
 * @brief Feedback Flanger
 *
 * Dependencies:
 * - DelayLine.hpp
 * - LFO.hpp
 *
 * Header-only
 * C++20
 * Real-Time Safe
 */

```

```

#include <algorithm>
#include <type_traits>

```

```

#include "../Delay/DelayLine.hpp"
#include "../Modulation/LFO.hpp"

```

```

namespace cvdsp
{

```

```

/**
 * @brief Feedback Flanger.
 *
 * Signal Path:
 *
 * Input
 *   |
 *   +-----+
 *   |               |
 *   v               |
 * Delay Line         |
 *   |               |
 *   v               |
 * Delayed Signal -----+
 *   |               |
 *   v               |
 * Wet/Dry Mix        |
 *   |               |
 * Output

```



```

*
* Parameters:
*
* Rate
* Depth
* Feedback
* Mix
*/
template<typename T>
class Flanger
{
    static_assert(
        std::is_floating_point_v<T>,
        "Flanger requires floating point type");

public:
    constexpr Flanger() noexcept = default;

    /**
     * @brief Initialize flanger.
     *
     * @param sampleRate Processing sample rate.
     * @param maxDelayMs Delay buffer size.
     */
    void prepare(
        T sampleRate,
        T maxDelayMs = static_cast<T>(20.0)) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));

        maxDelayMs_ =
            std::clamp(
                maxDelayMs,
                static_cast<T>(1),
                delayMaxMilliseconds());

        delay_.prepare(
            static_cast<typename delay::DelayLine<T>::size_type>(sampleRate_));

        lfo_.prepare(
            sampleRate_,
            rateHz_,
            static_cast<T>(1),
            LFOWaveform::Sine);

        reset();
    }

    /**
     * @brief Reset internal state.
     */
    void reset() noexcept
    {
        delay_.reset();
        lfo_.reset();

        feedbackSample_ =
            static_cast<T>(0);
    }
}

```

```

/**
 * @brief Set modulation rate.
 *
 * Range:
 * 0.01 Hz .. 20 Hz
 */
void setRate(
    T rateHz) noexcept
{
    rateHz_ =
        std::clamp(
            rateHz,
            static_cast<T>(0.01),
            static_cast<T>(20.0));

    lfo_.setRate(
        rateHz_);
}

/**
 * @brief Set modulation depth.
 *
 * Range:
 * 0.0 .. 1.0
 */
void setDepth(
    T depth) noexcept
{
    depth_ =
        std::clamp(
            depth,
            static_cast<T>(0),
            static_cast<T>(1));
}

/**
 * @brief Set feedback amount.
 *
 * Range:
 * -0.99 .. +0.99
 */
void setFeedback(
    T feedback) noexcept
{
    feedback_ =
        std::clamp(
            feedback,
            static_cast<T>(-0.99),
            static_cast<T>(0.99));
}

/**
 * @brief Set wet/dry mix.
 *
 * Range:
 * 0.0 .. 1.0
 */
void setMix(
    T mix) noexcept
{
    mix_ =
        std::clamp(
            mix,
            static_cast<T>(0),

```

```

        static_cast<T>(1));
}

/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    const T modulation =
        lfo_.process();

    const T delayMs =
        std::clamp(
            baseDelayMs_
            +
            (
                maxModulationMs_
                *
                depth_
                *
                modulation
            ),
            static_cast<T>(0),
            maxDelayMs_);

    const T delayed =
        delay_.readInterpolated(
            delayMs);

    const T delayInput =
        input
        +
        (
            delayed
            *
            feedback_
        );

    delay_.write(
        delayInput);

    feedbackSample_ =
        delayed;

    const T dry =
        input;

    const T wet =
        delayed;

    return
        (
            dry
            *
            (
                static_cast<T>(1)
                -
                mix_
            )
        )
        +
        (
            wet

```

```

        *
        mix_
    );
}

private:

[[nodiscard]]
T delayMaxMilliseconds() const noexcept
{
    return static_cast<T>(delay::DelayLine<T>::getMaxDelaySamples())
        * static_cast<T>(1000)
        / sampleRate_;
}

delay::DelayLine<T> delay_;

LFO<T> lfo_;

T sampleRate_ =
    static_cast<T>(44100);

T rateHz_ =
    static_cast<T>(0.25);

T depth_ =
    static_cast<T>(0.5);

T feedback_ =
    static_cast<T>(0.5);

T mix_ =
    static_cast<T>(0.5);

/**
 * Typical flanger delay.
 *
 * Smaller than chorus.
 */
T baseDelayMs_ =
    static_cast<T>(2.0);

/**
 * Maximum modulation range.
 */
T maxModulationMs_ =
    static_cast<T>(3.0);

T maxDelayMs_ =
    static_cast<T>(20.0);

T feedbackSample_ =
    static_cast<T>(0);
};

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Filters/AllPassFilter.hpp
=====

```

```

#ifndef CVDSP_FILTERS_ALLPASSFILTER_HPP
#define CVDSP_FILTERS_ALLPASSFILTER_HPP

/**
 * @file AllPassFilter.hpp
 * @brief All-Pass Filter
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Phase Rotation
 * - Diffusion
 * - Schroeder Reverb Building Block
 * - Phaser Building Block
 */

#include <algorithm>
#include <cstdint>
#include <type_traits>

#include "../Delay/DelayLine.hpp"

namespace cvdsp::filters
{
/**
 * @brief Feedback All-Pass Filter.
 *
 * Transfer Function:
 *
 * 
$$H(z) = \frac{-g + z^{-N}}{1 - g * z^{-N}}$$

 *
 * where:
 *
 * g = feedback coefficient
 * N = delay length
 *
 * Magnitude Response:
 *
 *  $|H(z)| = 1$ 
 *
 * Only phase is modified.
 */
template<typename T>
class AllPassFilter
{
    static_assert(
        std::is_floating_point_v<T>,
        "AllPassFilter requires floating point type");

public:
    constexpr AllPassFilter() noexcept = default;

    /**
     * @brief Initialize filter.
     *
     * @param sampleRate Processing sample rate.
     * @param maxDelaySamples Maximum delay size.
     */

```

```

void prepare(
    T sampleRate,
    std::size_t maxDelaySamples) noexcept
{
    sampleRate_ =
        std::max(
            sampleRate,
            static_cast<T>(1));

    maxDelaySamples_ =
        std::clamp<std::size_t>(
            maxDelaySamples,
            1u,
            delay::DelayLine<T>::getMaxDelaySamples());

    delay_.prepare(
        static_cast<typename delay::DelayLine<T>::size_type>(sampleRate_));

    reset();
}

/**
 * @brief Reset internal state.
 */
void reset() noexcept
{
    delay_.reset();

    delaySamples_ = 1;
}

/**
 * @brief Set delay length.
 *
 * Range:
 *
 * 1 .. maxDelaySamples
 */
void setDelaySamples(
    std::size_t delaySamples) noexcept
{
    delaySamples_ =
        std::clamp(
            delaySamples,
            static_cast<std::size_t>(1),
            maxDelaySamples_);
}

/**
 * @brief Set feedback coefficient.
 *
 * Range:
 *
 * -0.999 .. +0.999
 */
void setFeedback(
    T feedback) noexcept
{
    feedback_ =
        std::clamp(
            feedback,
            static_cast<T>(-0.999),
            static_cast<T>(0.999));
}

```

```

/**
 * @brief Process one sample.
 *
 * Canonical Schroeder all-pass:
 *
 * 
$$y[n] = -g \cdot x[n] + d[n]$$

 *
 * 
$$write = x[n] + g \cdot y[n]$$

 */
inline T process(
    T input) noexcept
{
    const T delayed =
        delay_.readIntegerSamples(
            delaySamples_);

    const T output =
        (-feedback_ * input)
        +
        delayed;

    const T writeSample =
        input
        +
        (
            feedback_
            *
            output
        );

    delay_.write(
        writeSample);

    return output;
}

private:

    delay_::DelayLine<T> delay_;

    T sampleRate_ =
        static_cast<T>(44100);

    T feedback_ =
        static_cast<T>(0.5);

    std::size_t delaySamples_ =
        1;

    std::size_t maxDelaySamples_ =
        1;
};

} // namespace cvdsp::filters

namespace cvdsp
{

```

```
template<typename T>
using AllPassFilter = filters::AllPassFilter<T>;
} // namespace cvdsp
```

```
#endif
```

```
=====
FILE: CV_DSP/Filters/Biquad.hpp
=====
```

```
#ifndef CVDSP_FILTERS_BIQUAD_HPP
#define CVDSP_FILTERS_BIQUAD_HPP
```

```
#include <cmath>
#include <cstdint>
#include <type_traits>
#include <algorithm>
```

```
#include "../Core/Types.hpp"
#include "../Core/Constants.hpp"
```

```
/**
```

```
 * @file Biquad.hpp
 * @brief Second-order (biquad) IIR filter â€” RBJ Audio EQ Cookbook, Direct
Form
```

```
 * II Transposed â€” for CV_DSP.
 * @namespace cvdsp::filters
```

```
 *
 * @section overview VisÃ£o geral
```

```
 * Este mÃ³dulo implementa um filtro biquad (IIR de 2ª ordem) configurÃ¡vel,
 * cobrindo os oito tipos clÃ¡ssicos do *Audio EQ Cookbook* de Robert
 * Bristow-Johnson (RBJ): passa-baixas, passa-altas, passa-banda, rejeita-banda
 * (notch), passa-tudo, peaking EQ e shelving (low/high). O processamento usa a
 * estrutura **Direct Form II Transposed (DF2T)**, que oferece excelente
 * comportamento numÃ©rico em ponto flutuante.
```

```
 *
 * Projetado conforme as regras do CV_DSP: header-only, C++20, sem dependÃªncias
 * externas, real-time safe (sem alocaÃ§Ã£o, exceÃ§Ãµes ou RTTI em `process()`),
 * `template<typename T>` para `float` e `double`, com Doxygen completo.
```

```
 *
 * @section diffeq EquaÃ§Ã£o geral do biquad
```

```
 * Um biquad realiza a equaÃ§Ã£o de diferenÃ§as (coeficientes jÃ¡ normalizados
por
```

```
 * @f$ a_0 @f$):
```

```
 *
 * @f[
 * \quad y[n] = b_0 x[n] + b_1 x[n-1] + b_2 x[n-2]
 * \quad \quad - a_1 y[n-1] - a_2 y[n-2]
 * @f]
```

```
 * cuja funÃ§Ã£o de transferÃªncia Ã©:
```

```
 *
 * @f[
 * \quad H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}
 * @f]
```

```
 *
 * @section df2t Direct Form II Transposed
```

```
 * A forma direta II transposta calcula a mesma @f$ H(z) @f$ usando apenas dois
 * estados (@f$ s_1, s_2 @f$) e a seguinte recorrÃªncia por amostra:
```



```

*
* @f[
* \begin{aligned}
* y[n] &= b_0 \backslash, x[n] + s_1 \backslash \backslash
* s_1 &= b_1 \backslash, x[n] - a_1 \backslash, y[n] + s_2 \backslash \backslash
* s_2 &= b_2 \backslash, x[n] - a_2 \backslash, y[n]
* \end{aligned}
* @f]
*
* Vantagens da DF2T frente Ã Direct Form I/II:
* - Usa o mÃnimo de elementos de atraso (2 estados).
* - Ã numericamente robusta com coeficientes em ponto flutuante: o erro de
* quantizaÃÃo/arredondamento Ã moldado favoravelmente, reduzindo ruÃdo e
* risco de oscilaÃÃes de ciclo-limite em precisÃo simples (`float`).
* - MantÃm os estados em escala compatÃvel com a entrada, evitando overflow
* interno em estÃgios de alto Q.
*
* @section design Projeto dos coeficientes (RBJ Cookbook)
*
* As variÃveis intermediÃrias comuns a todos os tipos sÃo:
*
* @f[
* \begin{aligned}
* A &= 10^{\backslash, \text{gainDB}/40}
* &\quad \backslash \text{quad}(\backslash \text{apenas peaking/shelf}) \backslash \backslash
* \omega_0 &= \frac{2\pi f_0}{f_s} \backslash \backslash
* \cos \omega_0, \backslash; \sin \omega_0 &\backslash; \backslash \text{do } \omega_0 \backslash \text{do } \backslash \backslash
* \alpha &= \frac{\sin \omega_0}{2Q}
* \end{aligned}
* @f]
*
* onde @f$ f_0 @f$ Ã a frequÃncia central/de corte, @f$ f_s @f$ a taxa de
* amostragem, @f$ Q @f$ o fator de qualidade e @f$ A @f$ a amplitude linear do
* ganho (para peaking/shelf). Os coeficientes nÃo-normalizados por tipo sÃo
* dados nas seÃÃes de @ref Type. Em seguida normaliza-se dividindo todos por
* @f$ a_0 @f$, de modo que a recorrÃncia DF2T use @f$ a_0 = 1 @f$.
*
* @section interp Significado de Q por tipo
*
* - LowPass/HighPass: @f$ Q @f$ controla o pico de ressonÃncia na frequÃncia
de
* corte (@f$ Q = 1/\sqrt{2} \approx 0.707 @f$ Ã Butterworth, sem pico).
* - BandPass/Notch/AllPass: @f$ Q = f_0/\text{BW} @f$ relaciona-se Ã largura
de
* banda; maior @f$ Q @f$ Ã banda mais estreita.
* - PeakingEQ: @f$ Q @f$ define a largura do realce/atenuaÃÃo no centro.
* - Low/HighShelf: a inclinaÃÃo Ã controlada por @f$ Q @f$ (aqui usamos a
forma
* com @f$ \alpha = \sin \omega_0 / (2Q) @f$; @f$ Q = 1/\sqrt{2} @f$ dÃ; uma
shelf
* sem overshoot Ã equivalente a `S = 1` no cookbook).
*
* @section stability Estabilidade numÃrica
*
* - @f$ f_0 @f$ Ã fixado em @f$ [\backslash, \epsilon, \backslash; 0.5 f_s - \epsilon, \backslash] @f$ para
* manter o pÃlo dentro do cÃrculo unitÃrio e evitar @f$ \sin/\cos @f$ em
* Nyquist exato.
* - @f$ Q @f$ Ã fixado a um mÃnimo positivo para evitar divisÃo por zero.
* - `updateCoefficients()` calcula @f$ \sin/\cos/\sqrt{\ } @f$ uma Ãnica vez e
* normaliza por @f$ a_0 @f$; `process()` usa apenas multiplicaÃÃes/somas.
* - ProteÃÃo contra denormais Ã aplicada aos estados realimentados.
*
* @section rt Garantias de tempo real
*

```

```

* - `process()` Ã© 0(1), `noexcept`, sem alocaÃ§Ã£o/exceÃ§Ãµes/RTTI.
* - `updateCoefficients()` deve ser chamado fora do audio thread quando
*   possÃvel (usa funÃ§Ãµes transcendentais); ainda assim nÃ£o aloca nem
lanÃ§a.
*/
namespace cvdsp::filters
{
/**
 * @brief Tipos de filtro biquad suportados (RBJ Audio EQ Cookbook).
 *
 * Coeficientes nÃ£o-normalizados (antes de dividir por @f$a_0 @f$, com
 * @f$c = \cos\omega_0 @f$, @f$s = \sin\omega_0 @f$,
 * @f$\alpha = s/(2Q) @f$ e @f$A = 10^{\text{gainDB}/40} @f$:
 *
 * - @b LowPass:
 *   @f$b_0=\frac{1-c}{2}, \backslash; b_1=1-c, \backslash; b_2=\frac{1-c}{2} @f$;
 *   @f$a_0=1+\alpha, \backslash; a_1=-2c, \backslash; a_2=1-\alpha @f$.
 * - @b HighPass:
 *   @f$b_0=\frac{1+c}{2}, \backslash; b_1=-(1+c), \backslash; b_2=\frac{1+c}{2} @f$;
 *   @f$a_0=1+\alpha, \backslash; a_1=-2c, \backslash; a_2=1-\alpha @f$.
 * - @b BandPass (ganho de pico 0 dB):
 *   @f$b_0=\alpha, \backslash; b_1=0, \backslash; b_2=-\alpha @f$;
 *   @f$a_0=1+\alpha, \backslash; a_1=-2c, \backslash; a_2=1-\alpha @f$.
 * - @b Notch:
 *   @f$b_0=1, \backslash; b_1=-2c, \backslash; b_2=1 @f$;
 *   @f$a_0=1+\alpha, \backslash; a_1=-2c, \backslash; a_2=1-\alpha @f$.
 * - @b AllPass:
 *   @f$b_0=1-\alpha, \backslash; b_1=-2c, \backslash; b_2=1+\alpha @f$;
 *   @f$a_0=1+\alpha, \backslash; a_1=-2c, \backslash; a_2=1-\alpha @f$.
 * - @b PeakingEQ:
 *   @f$b_0=1+\alpha A, \backslash; b_1=-2c, \backslash; b_2=1-\alpha A @f$;
 *   @f$a_0=1+\alpha/A, \backslash; a_1=-2c, \backslash; a_2=1-\alpha/A @f$.
 * - @b LowShelf (com @f$\beta = 2\sqrt{A}\backslash, \backslash\alpha @f$):
 *   @f$b_0=A[(A+1)-(A-1)c+\beta], \backslash;
 *   @f$b_1=2A[(A-1)-(A+1)c], \backslash;
 *   @f$b_2=A[(A+1)-(A-1)c-\beta] @f$;
 *   @f$a_0=(A+1)+(A-1)c+\beta, \backslash;
 *   @f$a_1=-2[(A-1)+(A+1)c], \backslash;
 *   @f$a_2=(A+1)+(A-1)c-\beta @f$.
 * - @b HighShelf (com @f$\beta = 2\sqrt{A}\backslash, \backslash\alpha @f$):
 *   @f$b_0=A[(A+1)+(A-1)c+\beta], \backslash;
 *   @f$b_1=-2A[(A-1)+(A+1)c], \backslash;
 *   @f$b_2=A[(A+1)+(A-1)c-\beta] @f$;
 *   @f$a_0=(A+1)-(A-1)c+\beta, \backslash;
 *   @f$a_1=2[(A-1)-(A+1)c], \backslash;
 *   @f$a_2=(A+1)-(A-1)c-\beta @f$.
 */
enum class BiquadType
{
    LowPass, ///< Passa-baixas de 2ª ordem.
    HighPass, ///< Passa-altas de 2ª ordem.
    BandPass, ///< Passa-banda (ganho de pico 0 dB).
    Notch, ///< Rejeita-banda (band-reject).
    AllPass, ///< Passa-tudo (resposta de fase, magnitude plana).
    PeakingEQ, ///< Realce/atenuaÃ§Ã£o em sino (bell), usa gainDB.
    LowShelf, ///< Prateleira grave, usa gainDB.
    HighShelf, ///< Prateleira aguda, usa gainDB.
};

/**
 * @class Biquad
 * @brief Filtro biquad RBJ com processamento em Direct Form II Transposed.
 */

```

```

* @tparam T Tipo de ponto flutuante do processamento (`float` ou `double`).
*
* @par Exemplo "passa-baixas ressonante":
* @code
*   cvdsp::filters::Biquad<float> lp;
*   lp.prepare(48000.0f);
*   lp.setType(cvdsp::filters::BiquadType::LowPass);
*   lp.setFrequency(1000.0f);
*   lp.setQ(2.0f);
*   lp.updateCoefficients();           // recalcula os 5 coeficientes
*   for (std::size_t n = 0; n < numSamples; ++n)
*       buffer[n] = lp.process(buffer[n]);
* @endcode
*
* @par Exemplo "peaking EQ (+6 dB em 3 kHz):"
* @code
*   cvdsp::filters::Biquad<double> peak;
*   peak.prepare(96000.0);
*   peak.setType(cvdsp::filters::BiquadType::PeakingEQ);
*   peak.setFrequency(3000.0);
*   peak.setQ(1.0);
*   peak.setGainDB(6.0);
*   peak.updateCoefficients();
*   double y = peak.process(x);
* @endcode
*
* @par Exemplo "alterar parâmetros em tempo real:"
* @code
*   // No control thread / bloco: atualize parâmetros e recoeficientes.
*   filter.setFrequency(newFreq);
*   filter.setQ(newQ);
*   filter.updateCoefficients();       // seguro fora do hot loop
*   // No audio thread: apenas process().
* @endcode
*/
template <typename T>
class Biquad
{
public:
    static_assert(std::is_floating_point_v<T>,
                  "Biquad requires a floating point type (float or double)");

    /// @brief Tipo escalar de ponto flutuante usado pelo filtro.
    using value_type = T;
    /// @brief Tipo inteiro sem sinal para contagens e índices.
    using size_type = std::size_t;

    /**
     * @brief Constrói um biquad neutro (passthrough) estável.
     *
     * Estado zerado e coeficientes de identidade (@f$b_0=1 @f$, demais 0),
     * atÃ que @ref prepare / @ref updateCoefficients sejam chamados. NÃO
aloca.
     */
    constexpr Biquad() noexcept = default;

    /// @brief Destrutor trivial; nada Ã alocado.
    ~Biquad() noexcept = default;

    Biquad(const Biquad&) noexcept = default;
    Biquad& operator=(const Biquad&) noexcept = default;
    Biquad(Biquad&&) noexcept = default;
    Biquad& operator=(Biquad&&) noexcept = default;

```

```

/**
 * @brief Prepara o filtro para uma taxa de amostragem e recalcula coefs.
 *
 * Deve ser chamado fora do audio thread. Armazena @f$ f_s @f$, zera o
 * estado interno e chama @ref updateCoefficients com os parâmetros atuais.
 *
 * @param sampleRate Taxa de amostragem em Hz (> 0; inválidos viram 48
kHz).
 */
inline void prepare(value_type sampleRate) noexcept
{
    m_sampleRate = (sampleRate > static_cast<value_type>(0))
        ? sampleRate
        : static_cast<value_type>(48000);
    reset();
    updateCoefficients();
}

/**
 * @brief Zera o estado interno (@f$ s_1, s_2 @f$) do filtro.
 *
 * Preserva os coeficientes. Real-time safe: O(1), sem
alocação/exceções.
 */
inline void reset() noexcept
{
    m_s1 = static_cast<value_type>(0);
    m_s2 = static_cast<value_type>(0);
}

/**
 * @brief Define o tipo de filtro (ver @ref BiquadType).
 *
 * Apenas armazena o parâmetro; chame @ref updateCoefficients para aplicar.
 * @param type Novo tipo de filtro.
 */
inline void setType(BiquadType type) noexcept { m_type = type; }

/**
 * @brief Define a frequência central/de corte @f$ f_0 @f$ em Hz.
 *
 * O valor é fixado a @f$ [\epsilon, \, 0.5 f_s - \epsilon] @f$ em
 * @ref updateCoefficients. Apenas armazena; chame @ref updateCoefficients.
 * @param frequencyHz Frequência em Hz.
 */
inline void setFrequency(value_type frequencyHz) noexcept
{
    m_frequency = frequencyHz;
}

/**
 * @brief Define o fator de qualidade @f$ Q @f$.
 *
 * Fixado a um mínimo positivo em @ref updateCoefficients para evitar
divisão
 * por zero. Apenas armazena; chame @ref updateCoefficients.
 * @param q Fator de qualidade (> 0).
 */
inline void setQ(value_type q) noexcept { m_q = q; }

/**
 * @brief Define o ganho em decibéis (apenas Peaking/LowShelf/HighShelf).
 *
 * Ignorado pelos demais tipos. Apenas armazena; chame

```

```

* @ref updateCoefficients.
* @param gainDB Ganho em dB (positivo = realce, negativo = atenua  o).
*/
inline void setGainDB(value_type gainDB) noexcept { m_gainDB = gainDB; }

/**
* @brief Configura e aplica um filtro peaking EQ em uma chamada.
*/
inline void setPeak(
    value_type sampleRate,
    value_type frequencyHz,
    value_type q,
    value_type gainDB) noexcept
{
    prepare(sampleRate);
    setType(BiquadType::PeakingEQ);
    setFrequency(frequencyHz);
    setQ(q);
    setGainDB(gainDB);
    updateCoefficients();
}

/**
* @brief Configura e aplica uma low-shelf em uma chamada.
*/
inline void setLowShelf(
    value_type sampleRate,
    value_type frequencyHz,
    value_type q,
    value_type gainDB) noexcept
{
    prepare(sampleRate);
    setType(BiquadType::LowShelf);
    setFrequency(frequencyHz);
    setQ(q);
    setGainDB(gainDB);
    updateCoefficients();
}

/**
* @brief Configura e aplica uma high-shelf em uma chamada.
*/
inline void setHighShelf(
    value_type sampleRate,
    value_type frequencyHz,
    value_type q,
    value_type gainDB) noexcept
{
    prepare(sampleRate);
    setType(BiquadType::HighShelf);
    setFrequency(frequencyHz);
    setQ(q);
    setGainDB(gainDB);
    updateCoefficients();
}

/**
* @brief Recalcula os coeficientes a partir dos par metros atuais.
*
* Implementa as f rmulas do RBJ Audio EQ Cookbook para o @ref BiquadType
* selecionado, normalizando por @f$a_0 @f$. Usa @f$\sin/\cos/\sqrt{\ } @f$;
* portanto prefira chamar fora do hot loop de  udio. N o aloca, n o
lan sa.

```

```

*/
inline void updateCoefficients() noexcept
{
    const value_type one = static_cast<value_type>(1);
    const value_type two = static_cast<value_type>(2);
    const value_type half = static_cast<value_type>(0.5);

    // Clamp de estabilidade para f0 e Q.
    const value_type nyquist = m_sampleRate * half;
    const value_type eps = static_cast<value_type>(1e-6);
    const value_type f0 =
        std::clamp(m_frequency, eps, nyquist - eps);
    const value_type q =
        std::max(m_q, static_cast<value_type>(1e-4));

    // Variáveis intermediárias do cookbook.
    const value_type w0 = cvdsp::twoPi<value_type> * f0 / m_sampleRate;
    const value_type cosw0 = std::cos(w0);
    const value_type sinw0 = std::sin(w0);
    const value_type alpha = sinw0 / (two * q);
    //  $A = 10^{(\text{gainDB}/40)} = \sqrt{10^{(\text{gainDB}/20)}}$ .
    const value_type A =
        std::pow(static_cast<value_type>(10),
            m_gainDB / static_cast<value_type>(40));

    // Coeficientes não-normalizados.
    value_type b0, b1, b2, a0, a1, a2;

    switch (m_type)
    {
    case BiquadType::LowPass:
    {
        b0 = (one - cosw0) * half;
        b1 = one - cosw0;
        b2 = (one - cosw0) * half;
        a0 = one + alpha;
        a1 = -two * cosw0;
        a2 = one - alpha;
        break;
    }
    case BiquadType::HighPass:
    {
        b0 = (one + cosw0) * half;
        b1 = -(one + cosw0);
        b2 = (one + cosw0) * half;
        a0 = one + alpha;
        a1 = -two * cosw0;
        a2 = one - alpha;
        break;
    }
    case BiquadType::BandPass: // pico de 0 dB (constant peak gain)
    {
        b0 = alpha;
        b1 = static_cast<value_type>(0);
        b2 = -alpha;
        a0 = one + alpha;
        a1 = -two * cosw0;
        a2 = one - alpha;
        break;
    }
    case BiquadType::Notch:
    {
        b0 = one;
        b1 = -two * cosw0;
    }
    }
}

```

```

        b2 = one;
        a0 = one + alpha;
        a1 = -two * cosw0;
        a2 = one - alpha;
        break;
    }
    case BiquadType::AllPass:
    {
        b0 = one - alpha;
        b1 = -two * cosw0;
        b2 = one + alpha;
        a0 = one + alpha;
        a1 = -two * cosw0;
        a2 = one - alpha;
        break;
    }
    case BiquadType::PeakingEQ:
    {
        b0 = one + alpha * A;
        b1 = -two * cosw0;
        b2 = one - alpha * A;
        a0 = one + alpha / A;
        a1 = -two * cosw0;
        a2 = one - alpha / A;
        break;
    }
    case BiquadType::LowShelf:
    {
        const value_type sqrtA = std::sqrt(A);
        const value_type beta = two * sqrtA * alpha;
        b0 = A * ((A + one) - (A - one) * cosw0 + beta);
        b1 = two * A * ((A - one) - (A + one) * cosw0);
        b2 = A * ((A + one) - (A - one) * cosw0 - beta);
        a0 = (A + one) + (A - one) * cosw0 + beta;
        a1 = -two * ((A - one) + (A + one) * cosw0);
        a2 = (A + one) + (A - one) * cosw0 - beta;
        break;
    }
    case BiquadType::HighShelf:
    {
        const value_type sqrtA = std::sqrt(A);
        const value_type beta = two * sqrtA * alpha;
        b0 = A * ((A + one) + (A - one) * cosw0 + beta);
        b1 = -two * A * ((A - one) + (A + one) * cosw0);
        b2 = A * ((A + one) + (A - one) * cosw0 - beta);
        a0 = (A + one) - (A - one) * cosw0 + beta;
        a1 = two * ((A - one) - (A + one) * cosw0);
        a2 = (A + one) - (A - one) * cosw0 - beta;
        break;
    }
    default:
    {
        // Passthrough seguro (identidade) "nunca alcançado".
        b0 = one; b1 = static_cast<value_type>(0);
        b2 = static_cast<value_type>(0);
        a0 = one; a1 = static_cast<value_type>(0);
        a2 = static_cast<value_type>(0);
        break;
    }
}

// Normaliza por a0 (DF2T usa a0 = 1).
const value_type invA0 = one / a0;
m_b0 = b0 * invA0;

```

```

        m_b1 = b1 * invA0;
        m_b2 = b2 * invA0;
        m_a1 = a1 * invA0;
        m_a2 = a2 * invA0;
    }

/**
 * @brief Processa uma única amostra (Direct Form II Transposed).
 *
 * @param x Amostra de entrada @f$ x[n] @f$.
 * @return Amostra de saída @f$ y[n] @f$.
 *
 * Real-time safe: O(1), `noexcept`, sem alocação/exceções/RTTI. Aplica
 * proteção contra denormais aos estados realimentados.
 */
[[nodiscard]] inline value_type process(value_type x) noexcept
{
    // y = b0*x + s1
    const value_type y = m_b0 * x + m_s1;
    // s1 = b1*x - a1*y + s2
    m_s1 = m_b1 * x - m_a1 * y + m_s2 + kDenormalGuard - kDenormalGuard;
    // s2 = b2*x - a2*y
    m_s2 = m_b2 * x - m_a2 * y + kDenormalGuard - kDenormalGuard;
    return y;
}

/**
 * @brief Processa um bloco de amostras in-place.
 * @param buffer Ponteiro para as amostras (não-nulo se @p numSamples > 0).
 * @param numSamples Quantidade de amostras a processar.
 *
 * Real-time safe: O(N), sem alocação/exceções.
 */
inline void processBlock(value_type* buffer, size_type numSamples) noexcept
{
    for (size_type n = 0; n < numSamples; ++n)
        buffer[n] = process(buffer[n]);
}

/// @brief Tipo de filtro atual.
[[nodiscard]] constexpr BiquadType getType() const noexcept { return m_type;
}

/// @brief Frequência central/de corte configurada (Hz).
[[nodiscard]] constexpr value_type getFrequency() const noexcept { return
m_frequency; }
/// @brief Fator de qualidade configurado.
[[nodiscard]] constexpr value_type getQ() const noexcept { return m_q; }
/// @brief Ganho configurado em dB.
[[nodiscard]] constexpr value_type getGainDB() const noexcept { return
m_gainDB; }
/// @brief Taxa de amostragem configurada (Hz).
[[nodiscard]] constexpr value_type getSampleRate() const noexcept { return
m_sampleRate; }

private:
    /// @brief Constante anti-denormal para os estados realimentados.
    static constexpr value_type kDenormalGuard =
        static_cast<value_type>(1e-20);

    // Coeficientes normalizados (a0 = 1).
    value_type m_b0{static_cast<value_type>(1)}; ///< Feedforward b0.
    value_type m_b1{static_cast<value_type>(0)}; ///< Feedforward b1.
    value_type m_b2{static_cast<value_type>(0)}; ///< Feedforward b2.
    value_type m_a1{static_cast<value_type>(0)}; ///< Feedback a1.

```



```

value_type m_a2{static_cast<value_type>(0)}; ///< Feedback a2.

// Estados DF2T.
value_type m_s1{static_cast<value_type>(0)}; ///< Estado s1.
value_type m_s2{static_cast<value_type>(0)}; ///< Estado s2.

// Parâmetros.
BiquadType m_type{BiquadType::LowPass};           ///< Tipo de filtro.
value_type m_frequency{static_cast<value_type>(1000)}; ///< f0 (Hz).
value_type m_q{static_cast<value_type>(0.70710678118654752440)}; ///< Q.
value_type m_gainDB{static_cast<value_type>(0)};    ///< Ganho (dB).
value_type m_sampleRate{static_cast<value_type>(48000)}; ///< fs (Hz).
};

} // namespace cvdsp::filters

#endif // CVDSP_FILTERS_BIQUAD_HPP

```

```

=====
FILE: CV_DSP/Filters/DCBlocker.hpp
=====

```

```

#ifndef CVDSP_FILTERS_DCBLOCKER_HPP
#define CVDSP_FILTERS_DCBLOCKER_HPP

```

```

#include <cmath>
#include <cstdint>
#include <type_traits>
#include <algorithm>

```

```

#include "../Core/Types.hpp"
#include "../Core/Constants.hpp"

```

```

/**
 * @file DCBlocker.hpp
 * @brief First-order DC-blocking (DC offset removal) filter for CV_DSP.
 * @namespace cvdsp::filters
 *
 * @section overview Visão geral
 *
 * Este módulo implementa um filtro removedor de componente DC (corrente
 * contínua), também conhecido como "DC blocker" ou "DC trap". É um filtro
 * passa-altas de primeira ordem extremamente leve, projetado para rodar dentro
 * do callback de áudio em tempo real, sem alocação dinâmica, sem exceções
 * e sem RTTI.
 *
 * @section equation Equação de diferenças
 *
 * A equação é implementada @c:
 *
 * @f[
 * y[n] = x[n] - x[n-1] + R \cdot y[n-1]
 * @f]
 *
 * onde:
 * - @f$ x[n] @f$ é a amostra de entrada atual;
 * - @f$ x[n-1] @f$ é a amostra de entrada anterior;
 * - @f$ y[n-1] @f$ é a amostra de saída anterior;
 * - @f$ R @f$ é o coeficiente do pólo (real, @f$ 0 \leq R < 1 @f$),
 * tipicamente próximo de 1 (ex.: 0.995 a 0.9995).
 *
 * @section transfer Função de transferência

```

```

*
* Aplicando a transformada Z Ã equaÃ§Ã£o de diferenÃ§as:
*
* @f[
*   Y(z) = X(z) - z^{-1} X(z) + R z^{-1} Y(z)
* @f]
*
* @f[
*   H(z) = \frac{Y(z)}{X(z)} = \frac{1 - z^{-1}}{1 - R z^{-1}}
* @f]
*
* Observamos:
* - Um **zero** em @f$ z = 1 @f$ (DC, @f$ \omega = 0 @f$). Como o ganho do
*   zero anula exatamente a frequÃncia zero, qualquer offset constante (DC)
*   Ã levado a 0 no regime permanente. Esse Ã o mecanismo central do filtro.
* - Um **pÃlo** em @f$ z = R @f$ sobre o eixo real. Quanto mais prÃximo de 1,
*   mais estreita Ã a banda rejeitada em torno de DC (corte mais baixo) e mais
*   "transparente" o filtro fica para o Ãudio audÃvel.
*
* Em @f$ z = 1 @f$ (DC): @f$ H(1) = \frac{1-1}{1-R} = 0 @f$ â†’ DC totalmente
* removido. Em altas frequÃncias (@f$ z = -1 @f$, Nyquist):
* @f$ H(-1) = \frac{2}{1+R} \approx 1 @f$ â†’ sinal de Ãudio praticamente
* inalterado.
*
* @section cutoff FrequÃncia de corte (-3 dB)
*
* A frequÃncia de corte aproximada onde a resposta cai 3 dB relaciona-se com
* @f$ R @f$ por:
*
* @f[
*   f_c \approx \frac{f_s}{2\pi} \ln(1 - R)
*   \quad \Longleftrightarrow \quad
*   R \approx 1 - \frac{2\pi f_c}{f_s}
* @f]
*
* para @f$ f_c \ll f_s @f$. Esta aproximaÃÃo Ã usada por
* @ref DCBlocker::setCutoffHz para oferecer uma interface em Hz, enquanto
* @ref DCBlocker::setCoefficient permite controlar @f$ R @f$ diretamente.
*
* @section stability Estabilidade numÃrica
*
* O filtro Ã estÃvel enquanto @f$ |R| < 1 @f$ (pÃlo dentro do cÃrculo
* unitÃrio). A implementaÃÃo faz clamp de @f$ R @f$ em
* @f$ [0, R_{max}] @f$ com @f$ R_{max} < 1 @f$ para garantir estabilidade
* mesmo diante de entradas invÃlidas, e aplica proteÃÃo contra denormais no
* estado de realimentaÃÃo para evitar picos de CPU no callback de Ãudio.
*
* @section why Por que remover DC? â€” impacto na cadeia DSP
*
* Um offset DC Ã uma componente constante somada ao sinal de Ãudio. Ele Ã
* inaudÃvel por si sÃ (0 Hz estÃ abaixo da audiÃÃo), mas causa estragos
sÃrios
* quando o sinal passa por estÃgios nÃo-lineares ou dependentes de energia,
* muito comuns em modelagem de amplificadores de guitarra:
*
* - @b DC @b Offset: desloca a forma de onda para cima ou para baixo em torno
*   de zero. Reduz o headroom disponÃvel (a onda "encosta" mais cedo em um dos
*   trilhos), pode produzir cliques em ediÃÃes/automaÃÃo e desperdiÃa
faixa
*   dinÃmica.
*
* - @b SaturatÃÃo (waveshaping / clipping): funÃÃes nÃo-lineares como
*   @f$ \tanh(x) @f$, clipping suave/duro etc. sÃo assimÃtricas em torno de
um

```

```

*   sinal deslocado. Com DC presente, a saturação gera componentes harmô-
nicas
*   pares espúrias e um novo offset, "bombeando" a forma de onda. Colocar um
*   DC blocker @b antes e/ou @b depois de cada estágio de distorção mantém
O
*   sinal centrado em zero e a saturação simétrica e previsível.
*
* - @b Convolution (IRs de gabinete/reverb): a convolução com uma resposta ao
*   impulso preserva (e pode amplificar) o DC presente na entrada. Pior: muitas
*   IRs de cabinet têm ganho não-nulo em DC, podendo acumular offset ao longo
*   do tempo. Remover DC antes da convolução evita acúmulo de offset e
mantém
*   a IR operando na sua faixa linear.
*
* - @b Compressão (dinâmica): detectores de envelope (peak/RMS) medem
energia.
*   Um offset DC infla a leitura do detector, fazendo o compressor "agarrar"
*   incorretamente, alterar o ganho de redução e bombear. Um DC blocker no
*   início da cadeia garante medição de envelope honesta.
*
* @b Posição @b típica @b na @b cadeia: o DC blocker é normalmente o
primeiro
*   elo (limpeza da entrada) e também é inserido entre estágios não-lineares
*   (distorção → DC blocker → próximo estágio) num pedalboard/amp sim:
*
* @code
*   entrada → [DC Blocker] → ganho/drive → [wavershaper] → [DC Blocker]
*   → tone stack → [convolution IR] → compressor → saída
* @endcode
*
* @section rt Garantias de tempo real
*
* - Header-only, C++20, sem dependências externas.
* - @ref DCBlocker::process é O(1): apenas multiplicações e somas por
amostra.
* - Sem `new`/`delete`/`malloc`/`free`, sem exceções, sem RTTI.
* - `noexcept` e `constexpr` aplicados onde possível.
*/
namespace cvdsp::filters
{
/**
* @class DCBlocker
* @brief Filtro removedor de DC de primeira ordem (passa-altas leve).
*
* Implementa @f$ y[n] = x[n] - x[n-1] + R \cdot y[n-1] @f$.
*
* @tparam T Tipo de ponto flutuante do processamento (`float` ou `double`).
*
* Fluxo de uso:
* @code
*   cvdsp::filters::DCBlocker<float> dc;
*   dc.prepare(48000.0f);           // fora do audio thread
*   dc.setCutoffHz(20.0f);         // ou dc.setCoefficient(0.9995f);
*   // ... no callback de áudio:
*   for (std::size_t n = 0; n < numSamples; ++n)
*       buffer[n] = dc.process(buffer[n]);
* @endcode
*/
template <typename T>
class DCBlocker
{
public:
    static_assert(std::is_floating_point_v<T>,

```

```

        "DCBlocker requires a floating point type (float or double)");

/// @brief Tipo escalar de ponto flutuante usado pelo filtro.
using value_type = T;
/// @brief Tipo inteiro sem sinal para contagens e Índices.
using size_type = std::size_t;

/**
 * @brief Constrói um DC blocker com coeficiente padrão estável.
 *
 * O estado interno é zerado e @f$ R @f$ recebe um valor padrão
conservador
 * (@ref kDefaultCoefficient), adequado a um corte muito baixo. Nenhuma
 * alocação é realizada.
 */
constexpr DCBlocker() noexcept = default;

/// @brief Destrutor trivial; não libera recursos (nada é alocado).
~DCBlocker() noexcept = default;

DCBlocker(const DCBlocker&) noexcept = default;
DCBlocker& operator=(const DCBlocker&) noexcept = default;
DCBlocker(DCBlocker&&) noexcept = default;
DCBlocker& operator=(DCBlocker&&) noexcept = default;

/**
 * @brief Prepara o filtro para uma dada taxa de amostragem.
 *
 * Deve ser chamado fora do audio thread (ex.: em @c prepareToPlay /
 * @c setupProcessing). Armazena a taxa de amostragem (usada por
 * @ref setCutoffHz para converter Hz em coeficiente) e zera o estado
 * interno para evitar transientes residuais.
 *
 * @param sampleRate Taxa de amostragem em Hz. Deve ser > 0; valores
 * inválidos são substituídos por um padrão seguro.
 *
 * @note Real-time safe não é obrigatório aqui (uso fora do callback),
mas
 * a função ainda assim não aloca, não libera e é O(1).
 */
inline void prepare(value_type sampleRate) noexcept
{
    m_sampleRate = (sampleRate > static_cast<value_type>(0))
        ? sampleRate
        : static_cast<value_type>(48000);

    reset();
}

/**
 * @brief Zera completamente o estado interno do filtro.
 *
 * Limpa @f$ x[n-1] @f$ e @f$ y[n-1] @f$. Os coeficientes (R) são
 * preservados. Real-time safe: O(1), sem alocação, sem exceções.
 */
inline void reset() noexcept
{
    m_x1 = static_cast<value_type>(0);
    m_y1 = static_cast<value_type>(0);
}

/**
 * @brief Define diretamente o coeficiente do pólo @f$ R @f$.
 *
 * @param coefficient Valor desejado de @f$ R @f$. Para estabilidade, o

```

```

*      valor Ã© fixado (clamp) ao intervalo @f$ [0, R_{max}] @f$, com
*      @f$ R_{max} = @f$ @ref kMaxCoefficient @f$ < 1 @f$.
*
* InterpretaÃ§Ã£o: quanto mais prÃ³ximo de 1, mais baixa a frequÃªncia de
corte
* e mais transparente o filtro fica para o Ã¡udio; valores menores afastam
o
* corte para cima, removendo tambÃ©m graves muito baixos.
*
* Real-time safe: O(1), sem alocaÃ§Ã£o, sem exceÃ§Ães.
*/
inline void setCoefficient(value_type coefficient) noexcept
{
    m_R = std::clamp(coefficient,
                     static_cast<value_type>(0),
                     kMaxCoefficient);
}

/**
* @brief Define o coeficiente a partir de uma frequÃªncia de corte em Hz.
*
* Usa a aproximaÃ§Ã£o de primeira ordem
* @f$ R \approx 1 - \frac{2\pi f_c}{f_s} @f$, vÃ¡lida para
* @f$ f_c \ll f_s @f$. O resultado Ã© fixado ao intervalo estÃ¡vel via
* @ref setCoefficient.
*
* @param cutoffHz FrequÃªncia de corte (-3 dB) desejada, em Hz. Valores
*      nÃ£o-positivos resultam em @f$ R = R_{max} @f$ (corte mÃ¡ximo).
*
* @pre @ref prepare deve ter sido chamado para que @c m_sampleRate seja
*      vÃ¡lido. Caso contrÃ¡rio, usa-se o padrÃ£o de 48 kHz.
*
* Real-time safe: O(1), sem alocaÃ§Ã£o, sem exceÃ§Ães.
*/
inline void setCutoffHz(value_type cutoffHz) noexcept
{
    if (cutoffHz <= static_cast<value_type>(0))
    {
        setCoefficient(kMaxCoefficient);
        return;
    }
    const value_type r =
        static_cast<value_type>(1) -
        (cvdsp::twoPi<value_type> * cutoffHz) / m_sampleRate;
    setCoefficient(r);
}

/**
* @brief Retorna o coeficiente do pÃ³lo @f$ R @f$ atualmente em uso.
* @return Valor de @f$ R @f$ no intervalo @f$ [0, R_{max}] @f$.
*/
[[nodiscard]] constexpr value_type getCoefficient() const noexcept
{
    return m_R;
}

/**
* @brief Retorna a taxa de amostragem configurada (Hz).
* @return Taxa de amostragem atual.
*/
[[nodiscard]] constexpr value_type getSampleRate() const noexcept
{
    return m_sampleRate;
}

```

```

/**
 * @brief Processa uma única amostra, removendo a componente DC.
 *
 * Implementa @f$ y[n] = x[n] - x[n-1] + R \cdot y[n-1] @f$ e atualiza o estado
 * interno (@f$ x[n-1] \rightarrow x[n] @f$, @f$ y[n-1] \rightarrow y[n] @f$).
 *
 * @param x Amostra de entrada @f$ x[n] @f$.
 * @return Amostra de saída @f$ y[n] @f$ com DC removido.
 *
 * Real-time safe: O(1), sem alocação, sem exceções, sem RTTI. Aplica
 * proteção contra denormais ao estado realimentado para evitar picos de
CPU.
 */
[[nodiscard]] inline value_type process(value_type x) noexcept
{
    // y[n] = x[n] - x[n-1] + R * y[n-1]
    value_type y = x - m_x1 + m_R * m_y1;

    // Proteção contra denormais: um sinal de áudio que decai a zero pode
    // levar y[n-1] a valores subnormais, causando lentidão extrema em
    // algumas CPUs dentro do callback. Somar e subtrair uma constante
    // muito pequena ("DC anti-denormal") empurra o estado para zero exato.
    y += kDenormalGuard;
    y -= kDenormalGuard;

    // Atualiza o histórico (estado).
    m_x1 = x;
    m_y1 = y;
    return y;
}

/**
 * @brief Processa um bloco de amostras in-place.
 *
 * Conveniência para hosts que entregam buffers contíguos. Equivale a
 * chamar @ref process para cada amostra, na ordem.
 *
 * @param buffer Ponteiro para as amostras (não-nulo se @p numSamples > 0).
 * @param numSamples Quantidade de amostras a processar.
 *
 * Real-time safe: O(N), sem alocação, sem exceções.
 */
inline void processBlock(value_type* buffer, size_type numSamples) noexcept
{
    for (size_type n = 0; n < numSamples; ++n)
        buffer[n] = process(buffer[n]);
}

/**
 * @brief Coeficiente padrão de @f$ R @f$ (corte muito baixo, ~ alguns Hz).
 */
static constexpr value_type kDefaultCoefficient =
    static_cast<value_type>(0.9995);

/**
 * @brief Limite superior de @f$ R @f$ para garantir estabilidade
 * (pelo estritamente dentro do círculo unitário).
 */
static constexpr value_type kMaxCoefficient =
    static_cast<value_type>(0.99999);

private:
/**

```

```

    * @brief Constante anti-denormal somada/subtraída ao estado realimentado.
    *
    * Pequena o bastante para não alterar audivelmente o sinal, grande o
    * bastante para escapar da faixa subnormal de @c T.
    */
    static constexpr value_type kDenormalGuard =
        static_cast<value_type>(1e-20);

    value_type m_R{kDefaultCoefficient}; ///< Coeficiente do pólo R.
    value_type m_x1{static_cast<value_type>(0)}; ///< Estado x[n-1].
    value_type m_y1{static_cast<value_type>(0)}; ///< Estado y[n-1].
    value_type m_sampleRate{static_cast<value_type>(48000)}; ///< Taxa (Hz).
};

} // namespace cvdsp::filters

#endif // CVDSP_FILTERS_DCBLOCKER_HPP

```

```

=====
FILE: CV_DSP/Filters/LadderFilter.hpp
=====
#ifndef CVDSP_FILTERS_LADDERFILTER_HPP
#define CVDSP_FILTERS_LADDERFILTER_HPP

/**
 * @file LadderFilter.hpp
 * @brief Moog Inspired Ladder Filter
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - 4 Pole LowPass
 * - Resonance Feedback
 * - Drive Control
 * - Soft Saturation
 * - Real-Time Modulation
 */

#include <algorithm>
#include <cmath>
#include <numbers>
#include <type_traits>

namespace cvdsp::filters
{
    /**
     * @brief Moog-inspired Ladder Filter.
     *
     * 24 dB/oct LowPass
     *
     * Four cascaded one-pole stages.
     *
     * Resonance feedback path.
     *
     * Optional nonlinear drive.
     */
    template<typename T>
    class LadderFilter
    {

```

```

static_assert(
    std::is_floating_point_v<T>,
    "LadderFilter requires floating point type");

public:

    constexpr LadderFilter() noexcept = default;

    /**
     * @brief Initialize filter.
     */
    void prepare(
        T sampleRate) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));

        updateCoefficients();

        reset();
    }

    /**
     * @brief Reset internal states.
     */
    void reset() noexcept
    {
        z1_ =
            static_cast<T>(0);

        z2_ =
            static_cast<T>(0);

        z3_ =
            static_cast<T>(0);

        z4_ =
            static_cast<T>(0);
    }

    /**
     * @brief Set cutoff frequency.
     *
     * Safe for real-time modulation.
     */
    void setCutoff(
        T cutoffHz) noexcept
    {
        const T maxCutoff =
            sampleRate_
            *
            static_cast<T>(0.495);

        cutoffHz_ =
            std::clamp(
                cutoffHz,
                static_cast<T>(5),
                maxCutoff);

        updateCoefficients();
    }
}

```



```

/**
 * @brief Set resonance amount.
 *
 * Range:
 *
 * 0.0 .. 4.0
 */
void setResonance(
    T resonance) noexcept
{
    resonance_ =
        std::clamp(
            resonance,
            static_cast<T>(0),
            static_cast<T>(4));

    updateFeedback();
}

/**
 * @brief Set input drive.
 *
 * Range:
 *
 * 1.0 .. 20.0
 */
void setDrive(
    T drive) noexcept
{
    drive_ =
        std::clamp(
            drive,
            static_cast<T>(1),
            static_cast<T>(20));
}

/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    T x =
        input
        -
        (
            feedbackGain_
            *
            z4_
        );

    x *= drive_;

    x = saturate(x);

    z1_ += g_ * (x - z1_);
    z1_ = saturate(z1_);

    z2_ += g_ * (z1_ - z2_);
    z2_ = saturate(z2_);

    z3_ += g_ * (z2_ - z3_);
    z3_ = saturate(z3_);
}

```

```

        z4_ += g_ * (z3_ - z4_);
        z4_ = saturate(z4_);

        return z4_;
    }

```

private:

```

/**
 * @brief Soft saturation.
 *
 * tanh approximation.
 */
static inline T saturate(
    T x) noexcept
{
    return std::tanh(x);
}

/**
 * @brief Update cutoff coefficient.
 */
void updateCoefficients() noexcept
{
    const T normalized =
        cutoffHz_
        /
        sampleRate_;

    const T omega =
        static_cast<T>(2)
        *
        std::numbers::pi_v<T>
        *
        normalized;

    g_ =
        static_cast<T>(1)
        -
        std::exp(
            -omega);

    g_ =
        std::clamp(
            g_,
            static_cast<T>(0),
            static_cast<T>(1));

    updateFeedback();
}

/**
 * @brief Resonance feedback scaling.
 */
void updateFeedback() noexcept
{
    feedbackGain_ =
        resonance_;
}

```

private:

```

    T sampleRate_ =
        static_cast<T>(44100);

```

```

    T cutoffHz_ =
        static_cast<T>(1000);

    T resonance_ =
        static_cast<T>(0);

    T drive_ =
        static_cast<T>(1);

    T g_ =
        static_cast<T>(0);

    T feedbackGain_ =
        static_cast<T>(0);

    /**
     * Four ladder stages.
     */

    T z1_ =
        static_cast<T>(0);

    T z2_ =
        static_cast<T>(0);

    T z3_ =
        static_cast<T>(0);

    T z4_ =
        static_cast<T>(0);
};

} // namespace cvdsp::filters

namespace cvdsp
{
template<typename T>
using LadderFilter = filters::LadderFilter<T>;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Filters/OnePoleFilter.hpp
=====

```

```

#ifndef CVDSP_FILTERS_ONEPOLEFILTER_HPP
#define CVDSP_FILTERS_ONEPOLEFILTER_HPP

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <type_traits>
#include <cassert>

/**
 * @file OnePoleFilter.hpp
 * @brief One-pole lowpass and highpass filters for CV_DSP
 * @namespace cvdsp::filters
 *
 * ImplementaÃ§Ã£o simples e numericamente estÃ¡vel de filtros de um pÃ³lo
 * (first-order) adequados para uso em tempo real. Projetados para serem

```

```

* header-only, C++20, sem dependências externas, e real-time safe.
*
* Design decisions:
* - Usamos o modelo discreto de "exponential smoothing" para o filtro
*   passa-baixas:  $y[n] = (1 - \alpha) * x[n] + \alpha * y[n-1]$ ,
*   com  $\alpha = \exp(-2 * \pi * f_c / f_s)$ .
* - O passa-altas é implementado como complemento:  $hp[n] = x[n] - lp[n]$ .
*
* Razões técnicas:
* -  $\alpha$  obtido por discretização exponencial (equivalente à solução
*   do diferencial  $y' + w_0 * y = w_0 * x$ , com  $w_0 = 2 * \pi * f_c$ ) produz um filtro
*   uniparamétrico estável e eficiente.
* - A implementação evita divisions caras no loop crítico: apenas usa
*   multiplicações e adições por amostra.
*
* Nota: Para aplicações que requerem resposta de fase exata ou Q controlado,
*   usar desenho por bilinear transform com pre-warpping. Aqui priorizamos
*   estabilidade, simplicidade e baixo custo computacional.
*/
namespace cvdsp::filters
{
/**
 * @brief Base one-pole filter storage and simple processing primitives.
 *
 * Este tipo não é exposto publicamente como filtro por si só, ele provê
 * armazenamento de estado e operações atômicas seguras em tempo real.
 *
 * Template:
 * - T: tipo de ponto flutuante (float ou double)
 */
template<typename T>
struct OnePole
{
    static_assert(std::is_floating_point_v<T>, "OnePole requires floating point
type");

    using value_type = T;
    using size_type = std::size_t;

    // Coeficientes:
    // lp:  $y[n] = b_0 * x[n] + a_1 * y[n-1]$ 
    value_type b0{}; ///< feedforward coefficient (1 - alpha)
    value_type a1{}; ///< feedback coefficient (alpha)

    // Estado:
    value_type z1{}; ///< estado interno  $y[n-1]$ 

    constexpr OnePole() noexcept : b0(static_cast<value_type>(1)),
a1(static_cast<value_type>(0)), z1(static_cast<value_type>(0)) {}

    constexpr void reset() noexcept { z1 = static_cast<value_type>(0); }

    // Processa uma amostra com os coeficientes atuais (lowpass primitive)
    inline value_type processLP(const value_type x) noexcept
    {
        //  $y = b_0 * x + a_1 * z_1$ 
        const value_type y = b0 * x + a1 * z1;
        z1 = y; // atualizar estado
        return y;
    }
};
/**

```

```

* @class LowPassOnePole
* @brief Filtro passa-baixas de um pólo (first-order low-pass)
*
* Características:
* -  $y[n] = (1 - \alpha) * x[n] + \alpha * y[n-1]$ 
* -  $\alpha = \exp(-2\pi * f_c / f_s)$ 
*
* Métodos principais:
* - prepare(sampleRate)
* - reset()
* - setCutoff(freqHz)
* - process(sample)
*/
template<typename T>
class LowPassOnePole
{
public:
    static_assert(std::is_floating_point_v<T>, "LowPassOnePole requires floating
point type");
    using value_type = T;
    using size_type = std::size_t;

    constexpr LowPassOnePole() noexcept = default;
    ~LowPassOnePole() noexcept = default;

    LowPassOnePole(const LowPassOnePole&) = delete;
    LowPassOnePole& operator=(const LowPassOnePole&) = delete;

    /**
     * @brief Prepara o filtro definindo a taxa de amostragem.
     *
     * Deve ser chamado fora do audio thread. Zera estado interno.
     */
    inline void prepare(size_type sampleRate) noexcept
    {
        assert(sampleRate > 0);
        m_sampleRate = sampleRate;
        m_initialized = true;
        m_onePole.reset();
        // define default cutoff (nyquist/2) conservador
        setCutoffHz(static_cast<value_type>(m_sampleRate) /
static_cast<value_type>(2));
    }

    /**
     * @brief Reseta o estado interno do filtro (zera histórico).
     */
    inline void reset() noexcept
    {
        m_onePole.reset();
    }

    /**
     * @brief Define a frequência de corte em Hz.
     *
     * Real-time safe; O(1). Não faz alocações.
     */
    inline void setCutoffHz(value_type freqHz) noexcept
    {
        // segura limites: [minFreq, nyquist - tiny]
        const value_type minFreq = static_cast<value_type>(1e-6);
        const value_type nyquist = static_cast<value_type>(m_sampleRate) *
static_cast<value_type>(0.5);
        const value_type f = std::clamp(freqHz, minFreq, nyquist -

```

```

static_cast<value_type>(1e-12));

    // alpha = exp(-2*pi*fc / fs)
    // para estabilidade num rica, quando fc >> fs/2, f clamped
    const value_type omega = static_cast<value_type>(2.0) *
static_cast<value_type>(M_PI) * f / static_cast<value_type>(m_sampleRate);
    const value_type alpha = std::exp(-omega);

    m_cutoffHz = f;
    m_onePole.a1 = alpha;
    m_onePole.b0 = static_cast<value_type>(1) - alpha;
}

/**
 * @brief Retorna a frequ ncia de corte atual em Hz.
 */
inline value_type getCutoffHz() const noexcept { return m_cutoffHz; }

/**
 * @brief Processa uma amostra e retorna a sa da do filtro passa-baixas.
 *
 * Real-time safe: O(1), sem aloca  es, sem exce  es.
 */
inline value_type process(value_type x) noexcept
{
    return m_onePole.processLP(x);
}

private:
    OnePole<value_type> m_onePole{};
    size_type m_sampleRate{48000};
    value_type m_cutoffHz{static_cast<value_type>(24000)}; // default
    bool m_initialized{false};
};

/**
 * @class HighPassOnePole
 * @brief Filtro passa-altas de um p lo implementado como x - lowpass(x)
 *
 * Implementa  o:
 * - y_hp[n] = x[n] - y_lp[n]
 * - onde y_lp[n]   sa da do LowPassOnePole com mesma alpha
 *
 * Esta formula  o garante estabilidade e   eficiente: usa os mesmos
coeficientes
 * alpha do lowpass e um estado adicional para x[n-1] n o   necess rio aqui.
 */
template<typename T>
class HighPassOnePole
{
public:
    static_assert(std::is_floating_point_v<T>, "HighPassOnePole requires
floating point type");
    using value_type = T;
    using size_type = std::size_t;

    constexpr HighPassOnePole() noexcept = default;
    ~HighPassOnePole() noexcept = default;

    HighPassOnePole(const HighPassOnePole&) = delete;
    HighPassOnePole& operator=(const HighPassOnePole&) = delete;

    /**
     * @brief Prepara o filtro com a taxa de amostragem.

```

```

    */
inline void prepare(size_type sampleRate) noexcept
{
    assert(sampleRate > 0);
    m_sampleRate = sampleRate;
    m_initialized = true;
    reset();
}

/**
 * @brief Reseta o estado interno do filtro.
 */
inline void reset() noexcept
{
    m_lpState = static_cast<value_type>(0);
}

/**
 * @brief Define a frequência de corte em Hz para o passa-altas.
 * Internamente configura o lowpass complementar com o mesmo alpha.
 */
inline void setCutoffHz(value_type freqHz) noexcept
{
    m_cutoffHz = freqHz;
    // Calcula coeficiente do lowpass interno
    // Reusa a fórmula do LowPassOnePole
    const value_type minFreq = static_cast<value_type>(1e-6);
    const value_type nyquist = static_cast<value_type>(m_sampleRate) *
static_cast<value_type>(0.5);
    const value_type f = std::clamp(freqHz, minFreq, nyquist -
static_cast<value_type>(1e-12));
    const value_type omega = static_cast<value_type>(2.0) *
static_cast<value_type>(M_PI) * f / static_cast<value_type>(m_sampleRate);
    const value_type alpha = std::exp(-omega);

    // lowpass coefficients: b0 = 1 - alpha, a1 = alpha
    m_lpCoeffa1 = alpha;
    m_lpCoeffb0 = static_cast<value_type>(1) - alpha;
}

inline value_type getCutoffHz() const noexcept { return m_cutoffHz; }

/**
 * @brief Processa uma amostra e retorna a saída do passa-altas.
 * Implementação:
 * - primeiro calcula lowpass usando os coeficientes armazenados
 * - hp = x - lp
 * - atualiza estado do lowpass
 */
inline value_type process(value_type x) noexcept
{
    // lowpass: lp = b0 * x + a1 * z1
    const value_type lp = m_lpCoeffb0 * x + m_lpCoeffa1 * m_lpState;
    // hp = x - lp
    const value_type hp = x - lp;
    // atualiza estado do lowpass
    m_lpState = lp;
    return hp;
}

private:
    // estado do lowpass interno

```

```

    value_type m_lpState{};
    value_type m_lpCoeffb0{static_cast<value_type>(1)};
    value_type m_lpCoeffa1{static_cast<value_type>(0)};

    size_type m_sampleRate{48000};
    value_type m_cutoffHz{static_cast<value_type>(20)};
    bool m_initialized{false};
};

} // namespace cvdsp::filters

#endif // CVDSP_FILTERS_ONEPOLEFILTER_HPP

```

```

=====
FILE: CV_DSP/Filters/StateVariableFilter.hpp
=====
#ifndef CVDSP_FILTERS_STATEVARIABLEFILTER_HPP
#define CVDSP_FILTERS_STATEVARIABLEFILTER_HPP

/**
 * @file StateVariableFilter.hpp
 * @brief Topology Preserving Transform State Variable Filter
 *
 * Features:
 * - LowPass
 * - HighPass
 * - BandPass
 * - Notch
 *
 * TPT (Topology Preserving Transform)
 * Real-Time Safe
 * Header-Only
 * C++20
 */

#include <algorithm>
#include <cmath>
#include <numbers>
#include <type_traits>

namespace cvdsp::filters
{
/**
 * @brief Filter response type.
 */
enum class SVFMode
{
    LowPass,
    HighPass,
    BandPass,
    Notch
};

/**
 * @brief Topology Preserving Transform State Variable Filter.
 *
 * Based on:
 *
 * Vadim Zavalishin
 * The Art of VA Filter Design
 */

```



```

* Advantages:
*
* - Stable at high resonance
* - Stable near Nyquist
* - Real-time modulation friendly
* - Continuous parameter updates
* - Low numerical sensitivity
*/
template<typename T>
class StateVariableFilter
{
    static_assert(
        std::is_floating_point_v<T>,
        "StateVariableFilter requires floating point type");

public:

    constexpr StateVariableFilter() noexcept = default;

    /**
     * @brief Initialize filter.
     */
    void prepare(
        T sampleRate,
        SVFMode mode = SVFMode::LowPass) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));

        mode_ = mode;

        updateCoefficients();

        reset();
    }

    /**
     * @brief Reset internal states.
     */
    void reset() noexcept
    {
        ic1eq_ =
            static_cast<T>(0);

        ic2eq_ =
            static_cast<T>(0);
    }

    /**
     * @brief Set filter mode.
     */
    void setMode(
        SVFMode mode) noexcept
    {
        mode_ = mode;
    }

    /**
     * @brief Set cutoff frequency.
     *
     * Safe for sample-by-sample modulation.
     */

```

```

void setCutoff(
    T cutoffHz) noexcept
{
    constexpr T kMinCutoff =
        static_cast<T>(5);

    const T maxCutoff =
        sampleRate_
        *
        static_cast<T>(0.495);

    cutoffHz_ =
        std::clamp(
            cutoffHz,
            kMinCutoff,
            maxCutoff);

    updateCoefficients();
}

/**
 * @brief Set resonance.
 *
 * Q range:
 * 0.1 .. 40.0
 */
void setResonance(
    T q) noexcept
{
    resonance_ =
        std::clamp(
            q,
            static_cast<T>(0.1),
            static_cast<T>(40.0));

    updateCoefficients();
}

/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    const T v3 =
        input
        -
        ic2eq_;

    const T v1 =
        a1_
        *
        ic1eq_
        +
        a2_
        *
        v3;

    const T v2 =
        ic2eq_
        +
        a2_
        *

```

```

        ic1eq_
        +
        a3_
        *
        v3;

ic1eq_ =
    static_cast<T>(2)
    *
    v1
    -
    ic1eq_;

ic2eq_ =
    static_cast<T>(2)
    *
    v2
    -
    ic2eq_;

const T lowpass =
    v2;

const T bandpass =
    v1;

const T highpass =
    input
    -
    k_
    *
    v1
    -
    v2;

const T notch =
    highpass
    +
    lowpass;

switch (mode_)
{
    case SVFMode::LowPass:
        return lowpass;

    case SVFMode::HighPass:
        return highpass;

    case SVFMode::BandPass:
        return bandpass;

    case SVFMode::Notch:
        return notch;

    default:
        return lowpass;
}
}

```

private:

```

/**
 * @brief Recalculate TPT coefficients.
 */

```

```

void updateCoefficients() noexcept
{
    const T wd =
        static_cast<T>(2)
        *
        std::numbers::pi_v<T>
        *
        cutoffHz_;

    const T Tinv =
        static_cast<T>(1)
        /
        sampleRate_;

    const T wa =
        static_cast<T>(2)
        *
        sampleRate_
        *
        std::tan(
            wd
            *
            Tinv
            *
            static_cast<T>(0.5));

    g_ =
        wa
        /
        (
            static_cast<T>(2)
            *
            sampleRate_
        );

    k_ =
        static_cast<T>(1)
        /
        resonance_;

    a1_ =
        static_cast<T>(1)
        /
        (
            static_cast<T>(1)
            +
            g_
            *
            (
                g_
                +
                k_
            )
        );

    a2_ =
        g_
        *
        a1_;

    a3_ =
        g_
        *
        a2_;
}

```

```

    }

private:
    T sampleRate_ =
        static_cast<T>(44100);

    T cutoffHz_ =
        static_cast<T>(1000);

    T resonance_ =
        static_cast<T>(0.707);

    T g_ =
        static_cast<T>(0);

    T k_ =
        static_cast<T>(0);

    T a1_ =
        static_cast<T>(0);

    T a2_ =
        static_cast<T>(0);

    T a3_ =
        static_cast<T>(0);

    T ic1eq_ =
        static_cast<T>(0);

    T ic2eq_ =
        static_cast<T>(0);

    SVFMode mode_ =
        SVFMode::LowPass;
};

} // namespace cvdsp::filters

namespace cvdsp
{
using SVFMode = filters::SVFMode;

template<typename T>
using StateVariableFilter = filters::StateVariableFilter<T>;
} // namespace cvdsp

#endif

=====
FILE: CV_DSP/Guitar/AmpSimulator.hpp
=====
#ifndef CVDSP_GUITAR_AMPSIMULATOR_HPP
#define CVDSP_GUITAR_AMPSIMULATOR_HPP

/**
 * @file AmpSimulator.hpp
 * @brief Complete real-time safe guitar amplifier simulator.
 *
 * Header-Only.
 * C++20.

```

```

* No external dependencies.
* No allocations inside process().
* No exceptions thrown by audio processing.
*
* Mandatory dependencies:
* - TubePreamp.hpp
* - ToneStack.hpp
* - PowerAmp.hpp
* - CabinetSimulator.hpp
*
* Optional dependency:
* - Dynamics/NoiseGate.hpp
*
* @section signalflow Signal flow
*
* @verbatim
*     Input
*     -> NoiseGate
*     -> TubePreamp
*     -> ToneStack
*     -> PowerAmp
*     -> CabinetSimulator
*     -> Output
* @endverbatim
*
* The simulator is intentionally modular: every stage is an independent CV_DSP
* processor with its own prepare(), reset(), and process() method. The class
* only coordinates gain staging, ordering, and bypass decisions. No stage is
* reimplemented here.
*
* @section dspmath DSP model
*
* The input stage applies a linear gain @f$ g_i @f$ before nonlinear tube
* processing:
*
* @f[
*     x_0[n] = g_i x[n]
* @f]
*
* The optional gate estimates input level and multiplies the signal by a
* smoothed gain @f$ g_g[n] @f$ so pickup noise is reduced before distortion:
*
* @f[
*     x_1[n] = g_g[n] x_0[n]
* @f]
*
* The preamp then performs cascaded triode saturation. Placing it before the
* tone stack follows common guitar-amplifier topology: nonlinear harmonic
* generation happens first, and the tone network shapes the generated spectrum.
*
* The tone stack is a concrete ToneStack-compatible processor supplied by the
* template parameter. By default it is MarshallToneStack, but Fender, Vox,
Mesa,
* or user-defined compatible tone stacks can be selected without changing this
* file.
*
* The power amplifier stage models push-pull pentode coloration, supply sag,
* and level-dependent compression. Finally, the cabinet simulator applies a
* cabinet impulse response if one has been loaded; otherwise it is transparent.
*
* The output stage applies a final linear gain @f$ g_o @f$:
*
* @f[
*     y[n] = g_o x_5[n]

```

```

* @f]
*
* @tparam T Floating-point sample type. Supported types are float and double.
* @tparam ToneStackProcessor Concrete tone-stack processor type. It must expose
*     prepare(T), reset(), process(T), setBass(T), setMid(T), setTreble(T),
*     and setPresence(T).
* @tparam CabinetFFTSize FFT size used by CabinetSimulator.
* @tparam CabinetMaxIRSamples Maximum cabinet impulse-response length.
*/

#include <cstdint>
#include <string>
#include <type_traits>

#include "TubePreamp.hpp"
#include "ToneStack.hpp"
#include "MarshallToneStack.hpp"
#include "PowerAmp.hpp"
#include "CabinetSimulator.hpp"
#include "../Dynamics/NoiseGate.hpp"

namespace cvdsp
{
template<
    typename T,
    typename ToneStackProcessor = MarshallToneStack<T>,
    std::size_t CabinetFFTSize = 2048,
    std::size_t CabinetMaxIRSamples = 65536>
class AmpSimulator final
{
    static_assert(
        std::is_floating_point_v<T>,
        "AmpSimulator requires a floating point type");

public:

    using value_type = T;
    using ToneStackType = ToneStackProcessor;
    using CabinetType =
        CabinetSimulator<
            T,
            CabinetFFTSize,
            CabinetMaxIRSamples>;
    using NoiseGateType = dynamics::NoiseGate<T>;
    using PowerAmpType = PowerAmp<T>;
    using TubePreampType = TubePreamp<T>;

public:

    /**
     * @brief Constructs an amplifier simulator with deterministic defaults.
     */
    constexpr AmpSimulator() noexcept = default;

    /**
     * @brief Prepares every internal DSP stage.
     *
     * Call this outside the audio callback before process(). The method may
     * initialize cabinet-convolution memory through
    CabinetSimulator::prepare(),
     * therefore it is intentionally not part of the audio-rate path.
     *
     * @param sampleRate Host sample rate in Hz. Non-positive values fall back

```

```

*         to 44100 Hz for numerical safety.
* @return true when the cabinet convolution engine is prepared.
*/
[[nodiscard]] bool prepare(
    T sampleRate)
    noexcept
{
    sampleRate_ =
        (sampleRate > static_cast<T>(0))
        ? sampleRate
        : static_cast<T>(44100);

    noiseGate_.prepare(
        sampleRate_);

    preamp_.prepare(
        sampleRate_);

    toneStack_.prepare(
        sampleRate_);

    powerAmp_.prepare(
        sampleRate_);

    const bool cabinetPrepared =
        cabinet_.prepare(
            sampleRate_);

    reset();

    return cabinetPrepared;
}

/**
 * @brief Resets all stateful stages without changing parameters.
 *
 * Real-time safe if called by the host during a stopped or suspended audio
 * stream. It performs no dynamic allocation and throws no exceptions.
 */
void reset()
    noexcept
{
    noiseGate_.reset();
    preamp_.reset();
    toneStack_.reset();
    powerAmp_.reset();
    cabinet_.reset();
}

/**
 * @brief Processes one mono guitar sample through the full amplifier chain.
 *
 * Processing order:
 * 1. Input gain.
 * 2. Optional noise gate.
 * 3. Tube preamp.
 * 4. Tone stack.
 * 5. Power amplifier.
 * 6. Optional cabinet simulator.
 * 7. Output gain.
 *
 * @param input Input sample.
 * @return Amplified output sample.
 */

```



```

* Real-time safe: O(1) plus the selected cabinet engine work, noexcept,
* no allocation, no file I/O, and no parameter lookup by name.
*/
[[nodiscard]] T process(
    T input)
    noexcept
{
    T x =
        input
        *
        inputGain_;

    if (noiseGateEnabled_)
    {
        x =
            noiseGate_.process(
                x);
    }

    x =
        preamp_.process(
            x);

    x =
        toneStack_.process(
            x);

    x =
        powerAmp_.process(
            x);

    if (cabinetEnabled_)
    {
        x =
            cabinet_.process(
                x);
    }

    return
        x
        *
        outputGain_;
}

/**
 * @brief Enables or disables the input noise gate stage.
 */
void setNoiseGateEnabled(
    bool enabled)
    noexcept
{
    noiseGateEnabled_ = enabled;
}

/**
 * @brief Enables or disables cabinet processing.
 */
void setCabinetEnabled(
    bool enabled)
    noexcept
{
    cabinetEnabled_ = enabled;
}

```

```

/**
 * @brief Sets linear gain before the first processing stage.
 */
void setInputGain(
    T gain)
    noexcept
{
    inputGain_ = gain;
}

/**
 * @brief Sets linear gain after the final processing stage.
 */
void setOutputGain(
    T gain)
    noexcept
{
    outputGain_ = gain;
}

/**
 * @brief Convenience setter for preamp drive.
 */
void setPreampDrive(
    T drive)
    noexcept
{
    preamp_.setDrive(
        drive);
}

/**
 * @brief Convenience setter for preamp stage count.
 */
void setPreampStages(
    std::size_t stages)
    noexcept
{
    preamp_.setNumStages(
        stages);
}

/**
 * @brief Convenience setter for tone-stack bass control, normalized 0..1.
 */
void setBass(
    T value)
    noexcept
{
    toneStack_.setBass(
        clamp01(value));
}

/**
 * @brief Convenience setter for tone-stack mid control, normalized 0..1.
 */
void setMid(
    T value)
    noexcept
{
    toneStack_.setMid(
        clamp01(value));
}

```

```

/**
 * @brief Convenience setter for tone-stack treble control, normalized 0..1.
 */
void setTreble(
    T value)
    noexcept
{
    toneStack_.setTreble(
        clamp01(value));
}

/**
 * @brief Convenience setter for tone-stack presence control, normalized
0..1.
 */
void setPresence(
    T value)
    noexcept
{
    toneStack_.setPresence(
        clamp01(value));
}

/**
 * @brief Selects the power-amp tube model.
 */
void setPowerAmpModel(
    typename PowerAmp<T>::Model model)
    noexcept
{
    powerAmp_.setModel(
        model);
}

/**
 * @brief Loads a cabinet impulse response outside the audio callback.
 *
 * This method may perform file I/O and should never be called from
 * process(). The real-time process path only uses the prepared convolution
 * state.
 */
[[nodiscard]] bool loadCabinet(
    const std::string& filePath,
    bool normalize = true,
    bool trimSilence = true)
    noexcept
{
    return cabinet_.loadCabinet(
        filePath,
        normalize,
        trimSilence);
}

/**
 * @brief Unloads the current cabinet outside the audio callback.
 */
void unloadCabinet()
    noexcept
{
    cabinet_.unloadCabinet();
}

[[nodiscard]] constexpr T getSampleRate() const noexcept
{

```

```

        return sampleRate_;
    }

    [[nodiscard]] constexpr T getInputGain() const noexcept
    {
        return inputGain_;
    }

    [[nodiscard]] constexpr T getOutputGain() const noexcept
    {
        return outputGain_;
    }

    [[nodiscard]] constexpr bool isNoiseGateEnabled() const noexcept
    {
        return noiseGateEnabled_;
    }

    [[nodiscard]] constexpr bool isCabinetEnabled() const noexcept
    {
        return cabinetEnabled_;
    }

    [[nodiscard]] bool isCabinetLoaded() const noexcept
    {
        return cabinet_.isLoaded();
    }

    [[nodiscard]] NoiseGateType& getNoiseGate() noexcept
    {
        return noiseGate_;
    }

    [[nodiscard]] const NoiseGateType& getNoiseGate() const noexcept
    {
        return noiseGate_;
    }

    [[nodiscard]] TubePreampType& getPreamp() noexcept
    {
        return preamp_;
    }

    [[nodiscard]] const TubePreampType& getPreamp() const noexcept
    {
        return preamp_;
    }

    [[nodiscard]] ToneStackType& getToneStack() noexcept
    {
        return toneStack_;
    }

    [[nodiscard]] const ToneStackType& getToneStack() const noexcept
    {
        return toneStack_;
    }

    [[nodiscard]] PowerAmpType& getPowerAmp() noexcept
    {
        return powerAmp_;
    }

    [[nodiscard]] const PowerAmpType& getPowerAmp() const noexcept

```

```

{
    return powerAmp_;
}

[[nodiscard]] CabinetType& getCabinet() noexcept
{
    return cabinet_;
}

[[nodiscard]] const CabinetType& getCabinet() const noexcept
{
    return cabinet_;
}

```

private:

```

[[nodiscard]] static constexpr T clamp01(
    T value)
    noexcept
{
    return
        (value < static_cast<T>(0))
        ? static_cast<T>(0)
        :
        (
            (value > static_cast<T>(1))
            ? static_cast<T>(1)
            : value
        );
}

```

private:

```

T sampleRate_ =
    static_cast<T>(44100);

T inputGain_ =
    static_cast<T>(1);

T outputGain_ =
    static_cast<T>(1);

bool noiseGateEnabled_ =
    true;

bool cabinetEnabled_ =
    true;

NoiseGateType
    noiseGate_;

TubePreampType
    preamp_;

ToneStackType
    toneStack_;

PowerAmpType
    powerAmp_;

CabinetType
    cabinet_;
};

```

```

using AmpSimulatorF =
    AmpSimulator<float>;

using AmpSimulatorD =
    AmpSimulator<double>;

} // namespace cvdsp

#endif // CVDSP_GUITAR_AMPSIMULATOR_HPP

=====
FILE: CV_DSP/Guitar/CabinetSimulator.hpp
=====
#ifndef CVDSP_GUITAR_CABINETSIMULATOR_HPP
#define CVDSP_GUITAR_CABINETSIMULATOR_HPP

/**
 * @file CabinetSimulator.hpp
 * @brief Guitar Cabinet Simulator
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 * - ../Convolution/IRLoader.hpp
 * - ../Convolution/ConvolutionEngine.hpp
 *
 * Features:
 * - Cabinet IR Loading
 * - Mono Cabinet Processing
 * - FFT Convolution
 * - Speaker Simulation
 * - Microphone Simulation
 *
 * No dynamic allocation inside process().
 */

#include <cmath>
#include <cstdint>
#include <string>
#include <type_traits>

#include "../Convolution/IRLoader.hpp"
#include "../Convolution/ConvolutionEngine.hpp"

namespace cvdsp
{
    template<
        typename T,
        std::size_t FFTSize = 2048,
        std::size_t MaxIRSamples = 262144>
    class CabinetSimulator
    {
    public:
        static_assert(
            std::is_floating_point_v<T>,
            "CabinetSimulator requires floating point type");
    };
}

```

```

CabinetSimulator() = default;

/**
 * @brief Prepare convolution engine.
 */
bool prepare() noexcept
{
    hostSampleRate_ =
        static_cast<T>(44100);

    sampleRateCheckEnabled_ = false;
    cabinetLoaded_ = false;
    sampleRateMatched_ = true;

    return convolution_.prepare();
}

/**
 * @brief Prepare convolution engine with host sample rate.
 */
bool prepare(
    T sampleRate) noexcept
{
    hostSampleRate_ =
        (std::isfinite(sampleRate) && sampleRate > T(0))
        ? sampleRate
        : static_cast<T>(44100);

    sampleRateCheckEnabled_ = true;
    cabinetLoaded_ = false;
    sampleRateMatched_ = true;

    return convolution_.prepare();
}

/**
 * @brief Reset processing state.
 */
void reset() noexcept
{
    convolution_.reset();
}

/**
 * @brief Load cabinet IR from WAV.
 *
 * Mono IR:
 * channel 0
 *
 * Stereo IR:
 * left channel used
 */
bool loadCabinet(
    const std::string& filePath,
    bool normalize = true,
    bool trimSilence = true) noexcept
{
    cabinetLoaded_ = false;

    if (!loader_.load(
        filePath,
        normalize,
        trimSilence))
    {

```

```

        return false;
    }

    const std::uint32_t irSampleRate =
        loader_.getSampleRate();

    sampleRateMatched_ =
        !sampleRateCheckEnabled_
        ||
        irSampleRate == 0
        ||
        std::abs(
            static_cast<T>(irSampleRate)
            -
            hostSampleRate_)
        <
        static_cast<T>(1);

    if (!sampleRateMatched_)
    {
        loader_.unload();

        return false;
    }

    const T* ir =
        loader_.getChannel(0);

    const std::size_t length =
        loader_.getLength();

    if (!convolution_.loadImpulseResponse(
        ir,
        length))
    {
        return false;
    }

    cabinetLoaded_ = true;

    return true;
}

/**
 * @brief Unload current cabinet.
 */
void unloadCabinet() noexcept
{
    loader_.unload();

    convolution_.reset();

    cabinetLoaded_ = false;
    sampleRateMatched_ = true;
}

/**
 * @brief Process one sample.
 */
T process(
    T input) noexcept
{
    if (!cabinetLoaded_)
    {

```



```

        return input;
    }

    return convolution_.process(
        input);
}

/**
 * @brief Cabinet loaded?
 */
[[nodiscard]]
bool isLoaded() const noexcept
{
    return cabinetLoaded_;
}

/**
 * @brief Cabinet IR length.
 */
[[nodiscard]]
std::size_t getIRLength()
    const noexcept
{
    return loader_.getLength();
}

/**
 * @brief Cabinet IR sample rate.
 */
[[nodiscard]]
std::uint32_t getIRSampleRate()
    const noexcept
{
    return loader_.getSampleRate();
}

/**
 * @brief Host sample rate used for IR compatibility checks.
 */
[[nodiscard]]
T getHostSampleRate()
    const noexcept
{
    return hostSampleRate_;
}

/**
 * @brief True when the loaded IR sample rate matches the host rate.
 */
[[nodiscard]]
bool isSampleRateMatched()
    const noexcept
{
    return sampleRateMatched_;
}

/**
 * @brief Access IR loader.
 */
[[nodiscard]]
const IRLoader<
    T,
    MaxIRSamples>&
getIRLoader() const noexcept

```

```

    {
        return loader_;
    }

private:
    IRLoader<
        T,
        MaxIRSamples>
        loader_;

    ConvolutionEngine<
        T,
        FFTSize,
        MaxIRSamples>
        convolution_;

    T hostSampleRate_ =
        static_cast<T>(44100);

    bool cabinetLoaded_ = false;

    bool sampleRateCheckEnabled_ = false;

    bool sampleRateMatched_ = true;
};

/**
 * Common aliases.
 */

using CabinetSimulator2048F =
    CabinetSimulator<
        float,
        2048>;

using CabinetSimulator4096F =
    CabinetSimulator<
        float,
        4096>;

using CabinetSimulator8192F =
    CabinetSimulator<
        float,
        8192>;

using CabinetSimulator2048D =
    CabinetSimulator<
        double,
        2048>;

using CabinetSimulator4096D =
    CabinetSimulator<
        double,
        4096>;

using CabinetSimulator8192D =
    CabinetSimulator<
        double,
        8192>;

} // namespace cvdsp

#endif

```

```
=====
FILE: CV_DSP/Guitar/FenderToneStack.hpp
=====
```

```
#ifndef CVDSP_GUITAR_FENDERTONESTACK_HPP
#define CVDSP_GUITAR_FENDERTONESTACK_HPP
```

```
/**
 * @file FenderToneStack.hpp
 * @brief Fender Blackface Tone Stack
 *
 * Inspired by:
 *
 * - Twin Reverb
 * - Deluxe Reverb
 * - Super Reverb
 * - Bassman Blackface Era
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - ToneStack.hpp
 *
 * No allocations.
 * No exceptions.
 */
```

```
#include <cmath>
#include <algorithm>
```

```
#include "ToneStack.hpp"
```

```
namespace cvdsp
{
```

```
template<typename T>
class FenderToneStack final
    : public ToneStack<T>
{
public:
```

```
    FenderToneStack() = default;
```

```
    void prepare(
        T sampleRate)
        noexcept override
    {
        ToneStack<T>::prepare(
            sampleRate);

        updateFilters();
    }
```

```
    void reset()
        noexcept override
    {
        ToneStack<T>::reset();
    }
```

```

void setBass(
    T value)
    noexcept override
{
    ToneStack<T>::bass_ =
        ToneStack<T>::clampControl(
            value);

    updateFilters();
}

void setMid(
    T value)
    noexcept override
{
    ToneStack<T>::mid_ =
        ToneStack<T>::clampControl(
            value);

    updateFilters();
}

void setTreble(
    T value)
    noexcept override
{
    ToneStack<T>::treble_ =
        ToneStack<T>::clampControl(
            value);

    updateFilters();
}

T process(
    T input)
    noexcept override
{
    T x = input;

    /**
     * Fender topology:
     *
     * Bass Shelf
     * ->
     * Mid Cut
     * ->
     * Treble Shelf
     */

    x =
        this->bassFilter_
            .process(x);

    x =
        this->midFilter_
            .process(x);

    x =
        this->trebleFilter_
            .process(x);

    return x;
}

```

private:

```
void updateFilters()
    noexcept
{
    const T bassInteraction =
        this->bass_
        -
        static_cast<T>(0.5);

    const T trebleInteraction =
        this->treble_
        -
        static_cast<T>(0.5);

    const T bassGainDb =
        static_cast<T>(-14)
        +
        (
            this->bass_
            *
            static_cast<T>(22)
        )
        -
        (
            trebleInteraction
            *
            static_cast<T>(2)
        );

    const T midGainDb =
        static_cast<T>(-18)
        +
        (
            this->mid_
            *
            static_cast<T>(20)
        )
        +
        (
            bassInteraction
            *
            static_cast<T>(2)
        )
        -
        (
            trebleInteraction
            *
            static_cast<T>(3)
        );

    const T trebleGainDb =
        static_cast<T>(-13)
        +
        (
            this->treble_
            *
            static_cast<T>(24)
        )
        -
        (
            bassInteraction
            *
            static_cast<T>(2)
        )
    }
```

```

        );

    /**
     * Bass
     *
     * Fender low shelf.
     */

    ToneStack<T>::configureLowShelf(
        this->bassFilter_,
        static_cast<T>(80.0),
        static_cast<T>(0.707),
        bassGainDb);

    /**
     * Mid Scoop Region.
     */

    ToneStack<T>::configurePeak(
        this->midFilter_,
        static_cast<T>(450.0),
        static_cast<T>(0.7),
        midGainDb);

    /**
     * Treble Shelf.
     */

    ToneStack<T>::configureHighShelf(
        this->trebleFilter_,
        static_cast<T>(4000.0),
        static_cast<T>(0.707),
        trebleGainDb);
    }
};

using FenderToneStackF =
    FenderToneStack<float>;

using FenderToneStackD =
    FenderToneStack<double>;

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Guitar/MarshallToneStack.hpp
=====
#ifndef CVDSP_GUITAR_MARSHALLTONESTACK_HPP
#define CVDSP_GUITAR_MARSHALLTONESTACK_HPP

/**
 * @file MarshallToneStack.hpp
 * @brief Marshall Tone Stack
 *
 * Inspired by:
 *
 * - Plexi 1959
 * - JMP
 * - JCM800
 * - JCM900

```

```

*
* Header-Only
* C++20
* Real-Time Safe
*
* Dependencies:
*
* - ToneStack.hpp
*
* No allocations.
* No exceptions.
*/

#include <algorithm>
#include <cmath>

#include "ToneStack.hpp"

namespace cvdsp
{
    template<typename T>
    class MarshallToneStack final
        : public ToneStack<T>
    {
    public:
        MarshallToneStack() = default;

        void prepare(
            T sampleRate)
            noexcept override
        {
            ToneStack<T>::prepare(
                sampleRate);

            updateFilters();
        }

        void reset()
            noexcept override
        {
            ToneStack<T>::reset();
        }

        void setBass(
            T value)
            noexcept override
        {
            this->bass_ =
                ToneStack<T>::clampControl(
                    value);

            updateFilters();
        }

        void setMid(
            T value)
            noexcept override
        {
            this->mid_ =
                ToneStack<T>::clampControl(
                    value);
        }
    };
}

```

```

        updateFilters();
    }

    void setTreble(
        T value)
        noexcept override
    {
        this->treble_ =
            ToneStack<T>::clampControl(
                value);

        updateFilters();
    }

    void setPresence(
        T value)
        noexcept override
    {
        this->presence_ =
            ToneStack<T>::clampControl(
                value);

        updateFilters();
    }

    T process(
        T input)
        noexcept override
    {
        T x = input;

        /**
         * Marshall topology.
         *
         * Bass
         *  $\hat{a}^\dagger$ 
         * Mid
         *  $\hat{a}^\dagger$ 
         * Treble
         *  $\hat{a}^\dagger$ 
         * Presence
         */

        x =
            this->bassFilter_
                .process(x);

        x =
            this->midFilter_
                .process(x);

        x =
            this->trebleFilter_
                .process(x);

        x =
            this->presenceFilter_
                .process(x);

        return x;
    }

```

private:


```

void updateFilters()
    noexcept
{
    /**
     * Marshall stacks generally
     * exhibit:
     *
     * - tighter bass
     * - stronger mids
     * - aggressive upper mids
     * - bright presence region
     */

    const T bassInteraction =
        this->bass_
        -
        static_cast<T>(0.5);

    const T trebleInteraction =
        this->treble_
        -
        static_cast<T>(0.5);

    const T bassGainDb =
        static_cast<T>(-12)
        +
        (
            this->bass_
            *
            static_cast<T>(20)
        )
        -
        (
            trebleInteraction
            *
            static_cast<T>(2)
        );

    const T midGainDb =
        static_cast<T>(-12)
        +
        (
            this->mid_
            *
            static_cast<T>(28)
        )
        +
        (
            trebleInteraction
            *
            static_cast<T>(2)
        )
        -
        (
            bassInteraction
            *
            static_cast<T>(1.5)
        );

    const T trebleGainDb =
        static_cast<T>(-11)
        +
        (
            this->treble_

```

```

        *
        static_cast<T>(22)
    );

const T presenceGainDb =
    static_cast<T>(-1)
    +
    (
        this->presence_
        *
        static_cast<T>(13)
    );

/**
 * Bass
 *
 * Marshall low-end is tighter
 * than Fender.
 */

ToneStack<T>::configureLowShelf(
    this->bassFilter_,
    static_cast<T>(120.0),
    static_cast<T>(0.707),
    bassGainDb);

/**
 * Midrange emphasis.
 *
 * Classic Marshall growl.
 */

ToneStack<T>::configurePeak(
    this->midFilter_,
    static_cast<T>(700.0),
    static_cast<T>(0.8),
    midGainDb);

/**
 * Upper-mid / treble attack.
 */

ToneStack<T>::configureHighShelf(
    this->trebleFilter_,
    static_cast<T>(2500.0),
    static_cast<T>(0.707),
    trebleGainDb);

/**
 * Presence region.
 *
 * Power amp feedback style
 * brightness control.
 */

ToneStack<T>::configureHighShelf(
    this->presenceFilter_,
    static_cast<T>(4500.0),
    static_cast<T>(0.707),
    presenceGainDb);
}
};

using MarshallToneStackF =

```

```

    MarshallToneStack<float>;

using MarshallToneStackD =
    MarshallToneStack<double>;

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Guitar/MesaToneStack.hpp
=====

```

```

#ifndef CVDSP_GUITAR_MESATONESTACK_HPP
#define CVDSP_GUITAR_MESATONESTACK_HPP

/**
 * @file MesaToneStack.hpp
 * @brief Mesa Boogie Mark Series Tone Stack
 *
 * Inspired by:
 *
 * - Mesa Mark I
 * - Mesa Mark II
 * - Mesa Mark III
 * - Mesa Mark IV
 * - Mesa Mark V
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - ToneStack.hpp
 *
 * No allocations.
 * No exceptions.
 */

```

```

#include <algorithm>
#include <cmath>

#include "ToneStack.hpp"

```

```

namespace cvdsp
{
    template<typename T>
    class MesaToneStack final
        : public ToneStack<T>
    {
    public:

        MesaToneStack() = default;

        void prepare(
            T sampleRate)
            noexcept override
        {
            ToneStack<T>::prepare(
                sampleRate);

```

```

        updateFilters();
    }

    void reset()
        noexcept override
    {
        ToneStack<T>::reset();
    }

    void setBass(
        T value)
        noexcept override
    {
        this->bass_ =
            ToneStack<T>::clampControl(
                value);

        updateFilters();
    }

    void setMid(
        T value)
        noexcept override
    {
        this->mid_ =
            ToneStack<T>::clampControl(
                value);

        updateFilters();
    }

    void setTreble(
        T value)
        noexcept override
    {
        this->treble_ =
            ToneStack<T>::clampControl(
                value);

        updateFilters();
    }

    void setPresence(
        T value)
        noexcept override
    {
        this->presence_ =
            ToneStack<T>::clampControl(
                value);

        updateFilters();
    }

    T process(
        T input)
        noexcept override
    {
        T x = input;

        /**
         * Mesa Mark topology.
         *
         * Bass
         * ->

```

```

    * Mid
    * ->
    * Treble
    * ->
    * Presence
    */

    x =
        this->bassFilter_
            .process(x);

    x =
        this->midFilter_
            .process(x);

    x =
        this->trebleFilter_
            .process(x);

    x =
        this->presenceFilter_
            .process(x);

    return x;
}

```

private:

```

void updateFilters()
noexcept
{
    /**
     * Mesa Mark Series characteristics:
     *
     * Tight low end
     * Aggressive upper mids
     * Strong treble interaction
     * Wide presence range
     *
     * Tone stack positioned
     * before significant gain stages
     * in traditional Mark circuits.
     */

    const T midFocus =
        static_cast<T>(500)
        +
        (
            this->mid_
            *
            static_cast<T>(450)
        );

    const T midQ =
        static_cast<T>(0.75)
        +
        (
            this->presence_
            *
            static_cast<T>(0.35)
        );

    const T bassGainDb =
        static_cast<T>(-14)

```

```

+
(
    this->bass_
    *
    static_cast<T>(20)
);

const T midGainDb =
    static_cast<T>(-20)
+
(
    this->mid_
    *
    static_cast<T>(28)
);

const T trebleGainDb =
    static_cast<T>(-11)
+
(
    this->treble_
    *
    static_cast<T>(27)
)
+
(
    this->presence_
    *
    static_cast<T>(2)
);

const T presenceGainDb =
    static_cast<T>(-1)
+
(
    this->presence_
    *
    static_cast<T>(15)
);

/**
 * Bass.
 *
 * Mesa low-end remains tight
 * and controlled.
 */

ToneStack<T>::configureLowShelf(
    this->bassFilter_,
    static_cast<T>(90.0),
    static_cast<T>(0.707),
    bassGainDb);

/**
 * Characteristic mid control.
 *
 * Mark amplifiers often produce
 * the famous V-shaped response
 * when mids are reduced.
 */

ToneStack<T>::configurePeak(
    this->midFilter_,
    midFocus,

```

```

        midQ,
        midGainDb);

/**
 * Treble control is extremely
 * influential in Mark amplifiers.
 *
 * It affects both brightness
 * and perceived gain structure.
 */

ToneStack<T>::configureHighShelf(
    this->trebleFilter_,
    static_cast<T>(2200.0),
    static_cast<T>(0.707),
    trebleGainDb);

/**
 * Power amp presence region.
 */

ToneStack<T>::configureHighShelf(
    this->presenceFilter_,
    static_cast<T>(5000.0),
    static_cast<T>(0.707),
    presenceGainDb);
}
};

using MesaToneStackF =
    MesaToneStack<float>;

using MesaToneStackD =
    MesaToneStack<double>;

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Guitar/PentodeStage.hpp
=====
#ifndef CVDSP_GUITAR_PENTODESTAGE_HPP
#define CVDSP_GUITAR_PENTODESTAGE_HPP

/**
 * @file PentodeStage.hpp
 * @brief Pentode Power Tube Stage
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - TriodeStage.hpp
 * - Waveshaper.hpp
 *
 * Optional:
 *
 * - Oversampling.hpp
 *

```

```

* Features:
*
* - Pentode Inspired Saturation
* - Power Tube Compression
* - Screen Voltage Control
* - Even/Odd Harmonic Generation
* - Push Toward Power Amp Behaviour
*
* No allocations in process().
*/

#include <cmath>
#include <type_traits>

#include "TriodeStage.hpp"
#include "../Core/DSPUtils.hpp"
#include "../Dynamics/EnvelopeFollower.hpp"
#include "../Filters/DCBlocker.hpp"
#include "../Saturation/Waveshaper.hpp"

namespace cvdsp
{
template<typename T>
class PentodeStage
{
    static_assert(
        std::is_floating_point_v<T>,
        "PentodeStage requires floating point type");

public:
    PentodeStage() = default;

    void prepare(
        T sampleRate)
        noexcept
    {
        sampleRate_ =
            DSPUtils::isFinite(sampleRate) && sampleRate > T(0)
            ? sampleRate
            : static_cast<T>(44100);

        triode_.prepare(
            sampleRate_);

        screenEnvelope_.prepare(
            sampleRate_);

        screenEnvelope_.setMode(
            dynamics::EnvelopeMode::Peak);

        screenEnvelope_.setAttackMs(
            static_cast<T>(4));

        screenEnvelope_.setReleaseMs(
            static_cast<T>(90));

        dcBlocker_.prepare(
            sampleRate_);

        dcBlocker_.setCutoffHz(
            static_cast<T>(18));
    }
};

```



```

        updateTriodeSettings();
        reset();
    }

    void reset()
        noexcept
    {
        triode_.reset();
        screenEnvelope_.reset();
        dcBlocker_.reset();
    }

    void setDrive(
        T drive)
        noexcept
    {
        if (!DSPUtils::isFinite(drive))
        {
            drive = T(1);
        }

        drive_ =
            DSPUtils::clamp(
                drive,
                T(0),
                kMaxDrive);

        updateTriodeSettings();
    }

    void setBias(
        T bias)
        noexcept
    {
        if (!DSPUtils::isFinite(bias))
        {
            bias = T(0);
        }

        bias_ =
            DSPUtils::clamp(
                bias,
                -kMaxBias,
                kMaxBias);

        updateTriodeSettings();
    }

    void setScreenVoltage(
        T voltage)
        noexcept
    {
        if (!DSPUtils::isFinite(voltage))
        {
            voltage = kDefaultScreenVoltage;
        }

        screenVoltage_ =
            DSPUtils::clamp(
                voltage,
                kMinScreenVoltage,
                kMaxScreenVoltage);
    }

```

```

        updateTriodeSettings();
    }

void setOutputGain(
    T gain)
    noexcept
{
    if (!DSPUtils::isFinite(gain))
    {
        gain = T(1);
    }

    outputGain_ =
        DSPUtils::clamp(
            gain,
            T(0),
            kMaxOutputGain);
}

[[nodiscard]]
T getDrive() const noexcept
{
    return drive_;
}

[[nodiscard]]
T getBias() const noexcept
{
    return bias_;
}

[[nodiscard]]
T getScreenVoltage() const noexcept
{
    return screenVoltage_;
}

[[nodiscard]]
T getOutputGain() const noexcept
{
    return outputGain_;
}

T process(
    T input)
    noexcept
{
    if (!DSPUtils::isFinite(input))
    {
        input = T(0);
    }

    input =
        DSPUtils::killTiny(
            input);

    /**
     * Pentode front-end.
     */

    T x =
        input
        *
        drive_;

```

```

x += bias_;

/**
 * Screen current memory. This is intentionally based on the driven
 * control signal rather than on the already-clipped sample so that the
 * compression has attack/release behaviour instead of static per-sample
 * gain reduction.
 */

const T screenEnvelope =
    screenEnvelope_.process(
        x);

const T screenSupply =
    DSPUtils::clamp(
        T(1)
        -
        (
            kScreenSagDepth
            *
            screenEnvelope
        ),
        kMinScreenSupply,
        T(1));

/**
 * Screen voltage affects headroom. Lower dynamic screen supply reduces
 * headroom and increases power-tube compression.
 */

const T screenFactor =
    std::max(
        (
            screenVoltage_
            /
            kDefaultScreenVoltage
        )
        *
        screenSupply,
        kMinScreenFactor);

/**
 * Pentode transfer. The center is kept more linear than the companion
 * triode model, while the cubic term emphasizes odd harmonics typical
 * of pushed power tubes.
 */

T shaped =
    std::tanh(
        x
        /
        screenFactor);

const T oddAmount =
    static_cast<T>(0.16)
    +
    (
        (T(1) - screenSupply)
        *
        static_cast<T>(0.20)
    );

const T odd =

```

```

        oddAmount
        *
        shaped
        *
        shaped
        *
        shaped;

shaped += odd;

/**
 * Pentode clipping characteristic.
 */

shaped =
    static_cast<T>(0.82)
    *
    shaped
    +
    static_cast<T>(0.18)
    *
    Waveshaper<T>::process(
        shaped,
        WaveshaperMode::Arctan);

/**
 * Natural power-tube compression from the smoothed screen-current
 * estimate. This replaces the previous instantaneous sag term.
 */

const T compression =
    T(1)
    /
    (
        T(1)
        +
        (
            kCompressionDepth
            *
            screenEnvelope
        )
    );

shaped *= compression;

/**
 * Blend a smaller amount of triode behaviour for asymmetry and familiar
 * tube curvature. Parameters are synchronized in the setters rather
 * than being rewritten every sample.
 */

const T triodePart =
    triode_.process(
        input);

shaped =
    static_cast<T>(0.78)
    *
    shaped
    +
    static_cast<T>(0.22)
    *
    triodePart;

```

```

        return
            DSPUtils::killTiny(
                dcBlocker_.process(
                    shaped
                    *
                    outputGain_));
    }

```

private:

```

void updateTriodeSettings()
    noexcept
{
    triode_.setDrive(
        drive_
        *
        static_cast<T>(0.35));

    triode_.setBias(
        bias_
        *
        static_cast<T>(0.45));

    triode_.setPlateVoltage(
        screenVoltage_
        *
        static_cast<T>(0.625));

    triode_.setOutputGain(
        static_cast<T>(1));
}

```

private:

```

static constexpr T kMaxDrive =
    static_cast<T>(32);

static constexpr T kMaxBias =
    static_cast<T>(2);

static constexpr T kMinScreenVoltage =
    static_cast<T>(100);

static constexpr T kDefaultScreenVoltage =
    static_cast<T>(400);

static constexpr T kMaxScreenVoltage =
    static_cast<T>(800);

static constexpr T kMaxOutputGain =
    static_cast<T>(10);

static constexpr T kScreenSagDepth =
    static_cast<T>(0.08);

static constexpr T kCompressionDepth =
    static_cast<T>(0.10);

static constexpr T kMinScreenSupply =
    static_cast<T>(0.55);

static constexpr T kMinScreenFactor =
    static_cast<T>(0.25);

```

```

    TriodeStage<T>
        triode_;

    dynamics::EnvelopeFollower<T>
        screenEnvelope_;

    filters::DCBlocker<T>
        dcBlocker_;

    T sampleRate_ =
        static_cast<T>(44100);

    T drive_ =
        static_cast<T>(1);

    T bias_ =
        static_cast<T>(0);

    T screenVoltage_ =
        kDefaultScreenVoltage;

    T outputGain_ =
        static_cast<T>(1);
};

using PentodeStageF =
    PentodeStage<float>;

using PentodeStageD =
    PentodeStage<double>;

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Guitar/PowerAmp.hpp
=====
#ifndef CVDSP_GUITAR_POWERAMP_HPP
#define CVDSP_GUITAR_POWERAMP_HPP

/**
 * @file PowerAmp.hpp
 * @brief Guitar Power Amplifier Simulation
 *
 * Models:
 *
 * - EL34
 * - 6L6
 * - KT88
 * - EL84
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - PentodeStage.hpp
 *
 * Optional:
 *

```

```

* - Presence Filter
* - Resonance Filter
*
* No allocations.
* No exceptions.
*/

#include <cmath>
#include <type_traits>

#include "PentodeStage.hpp"
#include "../Core/DSPUtils.hpp"
#include "../Dynamics/EnvelopeFollower.hpp"

namespace cvdsp
{
template<typename T>
class PowerAmp
{
public:
    static_assert(
        std::is_floating_point_v<T>,
        "PowerAmp requires floating point type");

    enum class Model
    {
        EL34,
        SixL6,
        L6L6 = SixL6,
        KT88,
        EL84
    };

public:
    PowerAmp() = default;

    void prepare(
        T sampleRate)
        noexcept
    {
        sampleRate_ =
            DSPUtils::isFinite(sampleRate) && sampleRate > T(0)
            ? sampleRate
            : static_cast<T>(44100);

        stageA_.prepare(
            sampleRate_);

        stageB_.prepare(
            sampleRate_);

        sagEnvelope_.prepare(
            sampleRate_);

        sagEnvelope_.setMode(
            dynamics::EnvelopeMode::Peak);

        sagEnvelope_.setAttackMs(
            static_cast<T>(8));

        sagEnvelope_.setReleaseMs(

```

```

        static_cast<T>(140));

compressionEnvelope_.prepare(
    sampleRate_);

compressionEnvelope_.setMode(
    dynamics::EnvelopeMode::Peak);

compressionEnvelope_.setAttackMs(
    static_cast<T>(3));

compressionEnvelope_.setReleaseMs(
    static_cast<T>(70));

configureModel();

reset();
}

void reset()
    noexcept
{
    stageA_.reset();
    stageB_.reset();

    sagEnvelope_.reset();
    compressionEnvelope_.reset();
}

void setModel(
    Model model)
    noexcept
{
    model_ =
        model;

    configureModel();
}

Model getModel() const noexcept
{
    return model_;
}

T process(
    T input)
    noexcept
{
    if (!DSPUtils::isFinite(input))
    {
        input = T(0);
    }

    input =
        DSPUtils::killTiny(
            input);

    /**
     * Dynamic supply sag. The detector has attack/release memory and is
     * sample-rate invariant through EnvelopeFollower coefficients.
     */

    const T sagEnvelope =
        sagEnvelope_.process(

```



```

        input);

const T sagGain =
    DSPUtils::clamp(
        T(1)
        -
        (
            sagAmount_
            *
            sagEnvelope
        ),
        kMinSagGain,
        T(1));

const T driven =
    input
    *
    sagGain;

and
/**
 * Push-pull approximation: split phase, process complementary pentode
 * branches, then recombine. For a linear pair this collapses to unity;
 * under bias/saturation it yields partial even-harmonic cancellation
 * stronger power-stage odd harmonics.
 */

const T positive =
    stageA_.process(
        driven);

const T negative =
    -stageB_.process(
        -driven);

T x =
    static_cast<T>(0.5)
    *
    (
        positive
        +
        negative
    );

second
/**
 * Dynamic compression after the virtual output pair. This uses a
 * envelope so fast transients are retained while sustained level bends
 * the supply/transfer response naturally.
 */

const T compressionEnvelope =
    compressionEnvelope_.process(
        x);

const T compression =
    T(1)
    /
    (
        T(1)
        +
        compressionAmount_
        *
        compressionEnvelope
    )

```

```

        );

    x *= compression;

    return
        DSPUtils::killTiny(
            x);
}

private:

void configureModel()
    noexcept
{
    switch (model_)
    {
        case Model::EL34:
        {
            stageA_.setDrive(
                static_cast<T>(3.0));

            stageB_.setDrive(
                static_cast<T>(3.0));

            stageA_.setBias(
                static_cast<T>(0.045));

            stageB_.setBias(
                static_cast<T>(-0.045));

            stageA_.setScreenVoltage(
                static_cast<T>(420));

            stageB_.setScreenVoltage(
                static_cast<T>(420));

            sagAmount_ =
                static_cast<T>(0.12);

            compressionAmount_ =
                static_cast<T>(0.28);

            break;
        }

        case Model::SixL6:
        {
            stageA_.setDrive(
                static_cast<T>(2.2));

            stageB_.setDrive(
                static_cast<T>(2.2));

            stageA_.setBias(
                static_cast<T>(0.025));

            stageB_.setBias(
                static_cast<T>(-0.025));

            stageA_.setScreenVoltage(
                static_cast<T>(460));

            stageB_.setScreenVoltage(
                static_cast<T>(460));
        }
    }
}

```

```

        sagAmount_ =
            static_cast<T>(0.08);

        compressionAmount_ =
            static_cast<T>(0.18);

        break;
    }

    case Model::KT88:
    {
        stageA_.setDrive(
            static_cast<T>(1.8));

        stageB_.setDrive(
            static_cast<T>(1.8));

        stageA_.setBias(
            static_cast<T>(0.015));

        stageB_.setBias(
            static_cast<T>(-0.015));

        stageA_.setScreenVoltage(
            static_cast<T>(520));

        stageB_.setScreenVoltage(
            static_cast<T>(520));

        sagAmount_ =
            static_cast<T>(0.04);

        compressionAmount_ =
            static_cast<T>(0.10);

        break;
    }

    case Model::EL84:
    {
        stageA_.setDrive(
            static_cast<T>(4.0));

        stageB_.setDrive(
            static_cast<T>(4.0));

        stageA_.setBias(
            static_cast<T>(0.060));

        stageB_.setBias(
            static_cast<T>(-0.060));

        stageA_.setScreenVoltage(
            static_cast<T>(310));

        stageB_.setScreenVoltage(
            static_cast<T>(310));

        sagAmount_ =
            static_cast<T>(0.18);

        compressionAmount_ =
            static_cast<T>(0.38);
    }

```

```

        break;
    }
}

stageA_.setOutputGain(
    static_cast<T>(1));

stageB_.setOutputGain(
    static_cast<T>(1));
}

```

private:

```

    static constexpr T kMinSagGain =
        static_cast<T>(0.35);

    T sampleRate_ =
        static_cast<T>(44100);

    Model model_ =
        Model::EL34;

    PentodeStage<T>
        stageA_;

    PentodeStage<T>
        stageB_;

    dynamics::EnvelopeFollower<T>
        sagEnvelope_;

    dynamics::EnvelopeFollower<T>
        compressionEnvelope_;

    T sagAmount_ =
        static_cast<T>(0.10);

    T compressionAmount_ =
        static_cast<T>(0.20);
};

```

```

using PowerAmpF =
    PowerAmp<float>;

```

```

using PowerAmpD =
    PowerAmp<double>;

```

```

} // namespace cvdsp

```

```

#endif

```

```

=====
FILE: CV_DSP/Guitar/ToneStack.hpp
=====

```

```

#ifndef CVDSP_GUITAR_TONESTACK_FORWARD_HPP
#define CVDSP_GUITAR_TONESTACK_FORWARD_HPP

```

```

/**
 * @file ToneStack.hpp
 * @brief Forwarding header for the guitar amplifier tone stack interface.
 *

```

```

* Header-Only.
* C++20.
* Real-Time Safe.
*
* This file intentionally does not reimplement the ToneStack module. It keeps
* the guitar include layout coherent by forwarding to the existing reusable
* ToneStack definition used by the concrete guitar tone-stack models.
*/

```

```
#include "../Saturation/ToneStack.hpp"
```

```
#endif // CVDSP_GUITAR_TONESTACK_FORWARD_HPP
```

```

=====
FILE: CV_DSP/Guitar/TriodeStage.hpp
=====

```

```

#ifndef CVDSP_GUITAR_TRIODESTAGE_HPP
#define CVDSP_GUITAR_TRIODESTAGE_HPP

```

```

/**
 * @file TriodeStage.hpp
 * @brief Triode Tube Gain Stage
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - Saturation/Waveshaper.hpp
 * - Core/DSPUtils.hpp
 * - Filters/DCBlocker.hpp
 * - Math/FastMath.hpp
 * - Math/Oversampling.hpp
 *
 * Optional:
 *
 * - Core/ParameterSmoother.hpp
 *
 * Features:
 *
 * - Triode Inspired Transfer Function
 * - Even Harmonic Generation
 * - Asymmetrical Saturation
 * - Plate Voltage Control
 * - DC Blocking
 * - Oversampling Processing
 * - Guitar Preamp Style Nonlinearity
 *
 * No allocation in process().
 */

```

```
#include <cmath>
```

```
#include <type_traits>
```

```
#include "../Saturation/Waveshaper.hpp"
```

```
#include "../Core/DSPUtils.hpp"
```

```
#include "../Filters/DCBlocker.hpp"
```

```
#include "../Math/FastMath.hpp"
```

```
#include "../Math/Oversampling.hpp"
```

```
namespace cvdsp
```

```

{
template<typename T>
class TriodeStage
{
    static_assert(
        std::is_floating_point_v<T>,
        "TriodeStage requires floating point type");

public:
    TriodeStage() = default;

    void prepare(
        T sampleRate)
        noexcept
    {
        sampleRate_ =
            DSPUtils::isFinite(sampleRate) && sampleRate > T(0)
            ? sampleRate
            : static_cast<T>(44100);

        oversampler_.prepare(
            sampleRate_);

        dcBlocker_.prepare(
            sampleRate_);

        dcBlocker_.setCutoffHz(
            static_cast<T>(20));

        reset();
    }

    void reset()
        noexcept
    {
        oversampler_.reset();

        dcBlocker_.reset();
    }

    void setDrive(
        T drive)
        noexcept
    {
        if (!DSPUtils::isFinite(drive))
        {
            drive = T(0);
        }

        drive_ =
            DSPUtils::clamp(
                drive,
                T(0),
                kMaxDrive);
    }

    void setBias(
        T bias)
        noexcept
    {
        if (!DSPUtils::isFinite(bias))
        {

```

```

        bias = T(0);
    }

    bias_ =
        DSPUtils::clamp(
            bias,
            -kMaxBias,
            kMaxBias);
}

void setPlateVoltage(
    T voltage)
    noexcept
{
    if (!DSPUtils::isFinite(voltage))
    {
        voltage = kDefaultPlateVoltage;
    }

    plateVoltage_ =
        DSPUtils::clamp(
            voltage,
            kMinPlateVoltage,
            kMaxPlateVoltage);
}

void setOutputGain(
    T gain)
    noexcept
{
    if (!DSPUtils::isFinite(gain))
    {
        gain = T(1);
    }

    outputGain_ =
        DSPUtils::clamp(
            gain,
            T(0),
            kMaxOutputGain);
}

[[nodiscard]]
T getDrive() const noexcept
{
    return drive_;
}

[[nodiscard]]
T getBias() const noexcept
{
    return bias_;
}

[[nodiscard]]
T getPlateVoltage() const noexcept
{
    return plateVoltage_;
}

[[nodiscard]]
T getOutputGain() const noexcept
{
    return outputGain_;
}

```

```

}

T process(
    T input)
    noexcept
{
    if (!DSPUtils::isFinite(input))
    {
        input = T(0);
    }

    input =
        DSPUtils::killTiny(
            input);

    const auto upsampled =
        oversampler_.processUp(
            input);

    typename Oversampling<T, 4>::OversampledBlock
        processed{};

    constexpr std::size_t factor =
        4;

    for (std::size_t i = 0;
        i < factor;
        ++i)
    {
        processed[i] =
            processTriode(
                upsampled[i]);
    }

    return
        DSPUtils::killTiny(
            dcBlocker_.process(
                oversampler_
                    .processDown(
                        processed))));
}

```

private:

```

T processTriode(
    T x)
    noexcept
{
    /**
     * Input gain stage.
     */

    x *= drive_;

    /**
     * Bias point.
     */

    x += bias_;

    /**
     * Plate voltage scaling.
     *
     * Lower voltage:
     */
}

```



```

    * more compression.
    *
    * Higher voltage:
    * more headroom.
    */

const T plateFactor =
    plateVoltage_
    /
    static_cast<T>(250);

/**
 * Koren-inspired curvature.
 */

const T positive =
    static_cast<T>(1.15);

const T negative =
    static_cast<T>(0.75);

T shaped;

if (x >= T(0))
{
    shaped =
        positive
        *
        fastTanh(
            x
            /
            plateFactor);
}
else
{
    shaped =
        negative
        *
        fastTanh(
            x
            /
            plateFactor);
}

/**
 * Even harmonic enhancement.
 */

const T even =
    static_cast<T>(0.15)
    *
    shaped
    *
    shaped;

shaped += even;

/**
 * Additional tube saturation.
 */

shaped =
    static_cast<T>(0.85)
    *

```

```

        shaped
        +
        static_cast<T>(0.15)
        *
        Waveshaper<T>::process(
            shaped,
            WaveshaperMode::Tanh);

    return
        DSPUtils::killTiny(
            shaped
            *
            outputGain_);
}

private:

    static constexpr T kMaxDrive =
        static_cast<T>(64);

    static constexpr T kMaxBias =
        static_cast<T>(4);

    static constexpr T kMinPlateVoltage =
        static_cast<T>(50);

    static constexpr T kDefaultPlateVoltage =
        static_cast<T>(250);

    static constexpr T kMaxPlateVoltage =
        static_cast<T>(500);

    static constexpr T kMaxOutputGain =
        static_cast<T>(10);

    T sampleRate_ =
        static_cast<T>(44100);

    T drive_ =
        static_cast<T>(1);

    T bias_ =
        static_cast<T>(0);

    T plateVoltage_ =
        kDefaultPlateVoltage;

    T outputGain_ =
        static_cast<T>(1);

    Oversampling<
        T,
        4>
        oversampler_;

    filters::DCBlocker<T> dcBlocker_;
};

using TriodeStageF =
    TriodeStage<float>;

using TriodeStageD =
    TriodeStage<double>;

```

```
} // namespace cvdsp
```

```
#endif
```

```
=====
FILE: CV_DSP/Guitar/TubePreamp.hpp
=====
```

```
#ifndef CVDSP_GUITAR_TUBEPREAMP_HPP
#define CVDSP_GUITAR_TUBEPREAMP_HPP
```

```
/**
 * @file TubePreamp.hpp
 * @brief Cascaded Triode Tube Preamp
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - Guitar/TriodeStage.hpp
 * - Core/DSPUtils.hpp
 * - Core/ParameterSmoother.hpp
 *
 * Optional:
 *
 * - Math/Oversampling.hpp
 *
 * Features:
 *
 * - 1 Stage
 * - 2 Stages
 * - 3 Stages
 * - 4 Stages
 *
 * - Cascaded Gain Structure
 * - High Gain Preamp Topology
 * - Smooth Parameter Control
 *
 * No allocations inside process().
 */
```

```
#include <cstdint>
```

```
#include <array>
```

```
#include <type_traits>
```

```
#include "TriodeStage.hpp"
```

```
#include "../Core/DSPUtils.hpp"
```

```
#include "../Core/ParameterSmoother.hpp"
```

```
namespace cvdsp
```

```
{
```

```
template<typename T>
```

```
class TubePreamp
```

```
{
```

```
    static_assert(
        std::is_floating_point_v<T>,
        "TubePreamp requires floating point type");
```

```
public:
```

```

static constexpr std::size_t
    MaxStages = 4;

public:

    TubePreamp() = default;

    void prepare(
        T sampleRate)
        noexcept
    {
        sampleRate_ =
            DSPUtils::isFinite(sampleRate) && sampleRate > T(0)
            ? sampleRate
            : static_cast<T>(44100);

        for (auto& stage : stages_)
        {
            stage.prepare(
                sampleRate_);
        }

        driveSmoother_.prepare(
            sampleRate_,
            static_cast<T>(0.02));

        biasSmoother_.prepare(
            sampleRate_,
            static_cast<T>(0.02));

        plateVoltageSmoother_.prepare(
            sampleRate_,
            static_cast<T>(0.02));

        outputSmoother_.prepare(
            sampleRate_,
            static_cast<T>(0.02));

        reset();
    }

    void reset()
        noexcept
    {
        for (auto& stage : stages_)
        {
            stage.reset();
        }

        driveSmoother_.reset(
            drive_);

        biasSmoother_.reset(
            bias_);

        plateVoltageSmoother_.reset(
            plateVoltage_);

        outputSmoother_.reset(
            outputGain_);
    }

    void setNumStages(
        std::size_t stages)

```

```

    noexcept
{
    if (stages < 1)
    {
        stages = 1;
    }

    if (stages > MaxStages)
    {
        stages = MaxStages;
    }

    numStages_ = stages;
}

void setDrive(
    T drive)
    noexcept
{
    if (!DSPUtils::isFinite(drive))
    {
        drive = T(0);
    }

    drive_ =
        DSPUtils::clamp(
            drive,
            T(0),
            kMaxDrive);

    driveSmoother_.setTarget(
        drive_);
}

void setBias(
    T bias)
    noexcept
{
    if (!DSPUtils::isFinite(bias))
    {
        bias = T(0);
    }

    bias_ =
        DSPUtils::clamp(
            bias,
            -kMaxBias,
            kMaxBias);

    biasSmoother_.setTarget(
        bias_);
}

void setPlateVoltage(
    T voltage)
    noexcept
{
    if (!DSPUtils::isFinite(voltage))
    {
        voltage = kDefaultPlateVoltage;
    }

    plateVoltage_ =
        DSPUtils::clamp(

```

```

        voltage,
        kMinPlateVoltage,
        kMaxPlateVoltage);

    plateVoltageSmoother_.setTarget(
        plateVoltage_);
}

void setOutputGain(
    T gain)
    noexcept
{
    if (!DSPUtils::isFinite(gain))
    {
        gain = T(1);
    }

    outputGain_ =
        DSPUtils::clamp(
            gain,
            T(0),
            kMaxOutputGain);

    outputSmoother_.setTarget(
        outputGain_);
}

[[nodiscard]]
std::size_t getNumStages()
    const noexcept
{
    return numStages_;
}

[[nodiscard]]
T getDrive()
    const noexcept
{
    return drive_;
}

[[nodiscard]]
T getBias()
    const noexcept
{
    return bias_;
}

[[nodiscard]]
T getPlateVoltage()
    const noexcept
{
    return plateVoltage_;
}

[[nodiscard]]
T getOutputGain()
    const noexcept
{
    return outputGain_;
}

T process(
    T input)

```

```

noexcept
{
    const T drive =
        driveSmoother_.process();

    const T bias =
        biasSmoother_.process();

    const T plateVoltage =
        plateVoltageSmoother_.process();

    const T output =
        outputSmoother_.process();

    if (!DSPUtils::isFinite(input))
    {
        input = T(0);
    }

    T x =
        DSPUtils::killTiny(
            input);

    for (std::size_t i = 0;
        i < numStages_;
        ++i)
    {
        const T stageDrive =
            computeStageDrive(
                drive,
                i);

        stages_[i].setDrive(
            stageDrive);

        stages_[i].setBias(
            bias);

        stages_[i].setPlateVoltage(
            plateVoltage);

        stages_[i].setOutputGain(
            computeStageOutputGain(
                i));

        x =
            stages_[i].process(
                x);
    }

    x *= output;

    if (!DSPUtils::isFinite(x))
    {
        return T(0);
    }

    return
        DSPUtils::killTiny(
            x);
}

```

private:

```

T computeStageDrive(
    T globalDrive,
    std::size_t stage)
    const noexcept
{
    switch (stage)
    {
        case 0:
            return
                globalDrive;

        case 1:
            return
                globalDrive
                *
                static_cast<T>(0.90);

        case 2:
            return
                globalDrive
                *
                static_cast<T>(0.80);

        case 3:
            return
                globalDrive
                *
                static_cast<T>(0.70);

        default:
            return
                globalDrive;
    }
}

```

```

T computeStageOutputGain(
    std::size_t stage)
    const noexcept
{
    if (stage + 1 >= numStages_)
    {
        return T(1);
    }

    switch (stage)
    {
        case 0:
            return static_cast<T>(0.85);

        case 1:
            return static_cast<T>(0.80);

        case 2:
            return static_cast<T>(0.75);

        default:
            return T(1);
    }
}

```

private:

```

static constexpr T kMaxDrive =
    static_cast<T>(64);

```



```

static constexpr T kMaxBias =
    static_cast<T>(4);

static constexpr T kMinPlateVoltage =
    static_cast<T>(50);

static constexpr T kDefaultPlateVoltage =
    static_cast<T>(250);

static constexpr T kMaxPlateVoltage =
    static_cast<T>(500);

static constexpr T kMaxOutputGain =
    static_cast<T>(10);

T sampleRate_ =
    static_cast<T>(44100);

std::size_t numStages_ = 1;

T drive_ =
    static_cast<T>(1);

T bias_ =
    static_cast<T>(0);

T plateVoltage_ =
    kDefaultPlateVoltage;

T outputGain_ =
    static_cast<T>(1);

std::array<
    TriodeStage<T>,
    MaxStages>
    stages_;

OnePoleSmoother<T>
    driveSmoother_;

OnePoleSmoother<T>
    biasSmoother_;

OnePoleSmoother<T>
    plateVoltageSmoother_;

OnePoleSmoother<T>
    outputSmoother_;
};

using TubePreampF =
    TubePreamp<float>;

using TubePreampD =
    TubePreamp<double>;

} // namespace cvdsp

#endif

```

```
=====
```

FILE: CV_DSP/Guitar/VoxToneStack.hpp

```
=====
#ifndef CVDSP_GUITAR_VOXTONESTACK_HPP
#define CVDSP_GUITAR_VOXTONESTACK_HPP

/**
 * @file VoxToneStack.hpp
 * @brief Vox AC15 / AC30 Top Boost Tone Stack
 *
 * Inspired by:
 *
 * - Vox AC15
 * - Vox AC30
 * - Top Boost Channel
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - ToneStack.hpp
 *
 * No allocations.
 * No exceptions.
 */

#include <algorithm>
#include <cmath>

#include "ToneStack.hpp"

namespace cvdsp
{
    template<typename T>
    class VoxToneStack final
        : public ToneStack<T>
    {
    public:
        VoxToneStack() = default;

        void prepare(
            T sampleRate)
            noexcept override
        {
            ToneStack<T>::prepare(
                sampleRate);

            updateFilters();
        }

        void reset()
            noexcept override
        {
            ToneStack<T>::reset();
        }

        void setBass(
            T value)
            noexcept override
        {
            this->bass_ =

```

```

        ToneStack<T>::clampControl(
            value);
    updateFilters();
}

void setMid(
    T value)
    noexcept override
{
    this->mid_ =
        ToneStack<T>::clampControl(
            value);

    updateFilters();
}

void setTreble(
    T value)
    noexcept override
{
    this->treble_ =
        ToneStack<T>::clampControl(
            value);

    updateFilters();
}

void setPresence(
    T value)
    noexcept override
{
    this->presence_ =
        ToneStack<T>::clampControl(
            value);

    updateFilters();
}

T process(
    T input)
    noexcept override
{
    T x = input;

    /**
     * Vox Top Boost topology.
     *
     * Bass
     * ->
     * Mid shaping
     * ->
     * Treble
     * ->
     * Presence
     */

    x =
        this->bassFilter_
            .process(x);

    x =
        this->midFilter_
            .process(x);

```

```

    x =
        this->trebleFilter_
            .process(x);

    x =
        this->presenceFilter_
            .process(x);

    return x;
}

```

private:

```

void updateFilters()
    noexcept
{
    /**
     * AC15 / AC30 Top Boost
     *
     * Character:
     *
     * - Chime
     * - Sparkle
     * - Upper-mid emphasis
     * - Bright top end
     * - Less low-end than Fender
     */

    const T cutInteraction =
        this->treble_
        -
        this->bass_;

    const T bassGainDb =
        static_cast<T>(-13)
        +
        (
            this->bass_
            *
            static_cast<T>(18)
        )
        -
        (
            this->treble_
            *
            static_cast<T>(3)
        );

    const T midGainDb =
        static_cast<T>(-8)
        +
        (
            this->mid_
            *
            static_cast<T>(14)
        )
        +
        (
            cutInteraction
            *
            static_cast<T>(2)
        );
}

```

```

const T trebleGainDb =
    static_cast<T>(-13)
    +
    (
        this->treble_
        *
        static_cast<T>(25)
    )
    -
    (
        this->bass_
        *
        static_cast<T>(2)
    );

const T presenceGainDb =
    static_cast<T>(-1)
    +
    (
        this->presence_
        *
        static_cast<T>(11)
    );

/**
 * Tight bass response.
 */

ToneStack<T>::configureLowShelf(
    this->bassFilter_,
    static_cast<T>(100.0),
    static_cast<T>(0.707),
    bassGainDb);

/**
 * Upper-mid chime region.
 */

ToneStack<T>::configurePeak(
    this->midFilter_,
    static_cast<T>(1200.0),
    static_cast<T>(0.8),
    midGainDb);

/**
 * Top Boost treble section.
 */

ToneStack<T>::configureHighShelf(
    this->trebleFilter_,
    static_cast<T>(3500.0),
    static_cast<T>(0.707),
    trebleGainDb);

/**
 * Air and brilliance.
 */

ToneStack<T>::configureHighShelf(
    this->presenceFilter_,
    static_cast<T>(7000.0),
    static_cast<T>(0.707),
    presenceGainDb);
}

```

```

};

using VoxToneStackF =
    VoxToneStack<float>;

using VoxToneStackD =
    VoxToneStack<double>;

} // namespace cvdsp

#endif

=====
FILE: CV_DSP/Math/FastMath.hpp
=====
#ifndef CVDSP_MATH_FASTMATH_HPP
#define CVDSP_MATH_FASTMATH_HPP

/**
 * @file FastMath.hpp
 * @brief Fast constexpr-friendly mathematical approximations for real-time DSP.
 *
 * This module intentionally favors deterministic, allocation-free arithmetic
 * over libm precision. The functions are small, force-inlined, `noexcept`, and
 * designed for hot audio paths where a bounded approximation is often more
 * useful than a fully rounded standard-library result.
 *
 * Theoretical scalar cost compared with common `std::` alternatives:
 *
 * | Function group | CV_DSP theoretical cost | Typical `std::` equivalent |
 * | --- | --- | --- |
 * | abs/min/max/clamp/sign | 1-3 comparisons, no calls | Similar for simple
overloads; may not be force-inlined at every call site |
 * | lerp | 2 additions + 2 multiplications | `std::lerp` may add edge-case
handling for exact IEEE behavior |
 * | wrap | 1 division + 1 floor-like cast + arithmetic |
`std::fmod`/`std::remainder` usually call libm and handle many IEEE edge cases |
 * | pow2/pow3 | 1 or 2 multiplications | `std::pow(x, 2/3)` usually performs a
generic exponent path |
 * | dB to gain | range reduction + cubic exp2 polynomial | `std::pow(10, db /
20)` usually calls libm |
 * | gain to dB | normalization + fifth-order atanh log polynomial |
`std::log10` usually calls libm |
 * | tanh/atan/soft clip | low-order rational/polynomial approximation |
`std::tanh`/`std::atan` usually call libm |
 *
 * @note These approximations are intended for audio-rate modulation,
waveshaping,
 * metering, and control smoothing. Use the standard library when exact
IEEE
 * special-value behavior or maximum numerical accuracy is required.
 * @note No external libraries are used. The header performs no allocation and
does not throw exceptions.
 */

#include "../Core/Constants.hpp"
#include "../Core/Namespace.hpp"
#include "../Core/Types.hpp"

#include <type_traits>

namespace cvdsp

```

```

{
namespace detail
{
template <typename T>
CVDSP_FORCE_INLINE constexpr void requireArithmetic() noexcept
{
    static_assert(std::is_arithmetic_v<T>, "CV_DSP fast math functions require
arithmetic scalar types.");
}

template <typename T>
CVDSP_FORCE_INLINE constexpr T fastFloor(T value) noexcept
{
    requireArithmetic<T>();

    if constexpr (std::is_integral_v<T>)
    {
        return value;
    }
    else
    {
        const auto truncated = static_cast<i64>(value);
        const auto asValue = static_cast<T>(truncated);
        return value < asValue ? static_cast<T>(truncated - 1) : asValue;
    }
}

template <typename T>
CVDSP_FORCE_INLINE constexpr T exp2Polynomial01(T fraction) noexcept
{
    // 2^f on [0, 1): cubic minimax-style Taylor approximation around zero.
    // Cost: 3 multiplications + 3 fused-add friendly additions.
    constexpr T ln2 = static_cast<T>(0.693147180559945309417232121458176568L);
    constexpr T c2 = static_cast<T>(0.240226506959100712333551263163332485L); //
ln(2)^2 / 2
    constexpr T c3 = static_cast<T>(0.055504108664821579953005696965521856L); //
ln(2)^3 / 6
    return static_cast<T>(1) + fraction * (ln2 + fraction * (c2 + fraction *
c3));
}

template <typename T>
CVDSP_FORCE_INLINE constexpr T scaleByPowerOfTwo(T value, i64 exponent) noexcept
{
    // Cost: bounded repeated squaring over the exponent bits; no libm call.
    // The loop count is proportional to the number of bits in |exponent|, not
    // to the magnitude itself, and is therefore small for DSP control ranges.
    if (value == static_cast<T>(0))
    {
        return static_cast<T>(0);
    }

    T factor = static_cast<T>(2);
    T result = static_cast<T>(1);
    auto power = exponent < 0 ? static_cast<u64>(-exponent) :
static_cast<u64>(exponent);

    while (power != 0U)
    {
        if ((power & 1U) != 0U)
        {
            result *= factor;
        }
        factor *= factor;
    }
}

```

```

        power >>= 1U;
    }

    return exponent < 0 ? value / result : value * result;
}

template <typename T>
CVDSP_FORCE_INLINE constexpr T fastExp2(T value) noexcept
{
    requireArithmetic<T>();

    const auto integerPart = static_cast<i64>(fastFloor(value));
    const auto fraction = value - static_cast<T>(integerPart);
    return scaleByPowerOfTwo(exp2Polynomial01(fraction), integerPart);
}

template <typename T>
CVDSP_FORCE_INLINE constexpr T fastLog2Positive(T value) noexcept
{
    requireArithmetic<T>();

    if (value <= static_cast<T>(0))
    {
        return static_cast<T>(-120); // Practical floor for invalid/silent
gains.
    }

    i64 exponent = 0;
    while (value >= static_cast<T>(2))
    {
        value *= static_cast<T>(0.5);
        ++exponent;
    }
    while (value < static_cast<T>(1))
    {
        value *= static_cast<T>(2);
        --exponent;
    }

    // log(m) = 2 * atanh((m - 1) / (m + 1)), m in [1, 2).
    // Cost: 1 division + 5 multiplications + additions; avoids std::log/log10.
    constexpr T invLn2 =
static_cast<T>(1.442695040888963407359924681001892137L);
    const T y = (value - static_cast<T>(1)) / (value + static_cast<T>(1));
    const T y2 = y * y;
    const T ln = static_cast<T>(2) * y * (static_cast<T>(1) + y2 *
(static_cast<T>(1.0 / 3.0) + y2 * static_cast<T>(1.0 / 5.0)));
    return static_cast<T>(exponent) + ln * invLn2;
}
} // namespace detail

/**
 * @brief Fast absolute value.
 * @details Cost: 1 comparison + optional negation; avoids function-call
overhead
 *         and is comparable to `std::abs` for simple scalar types.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastAbs(T value) noexcept
{
    detail::requireArithmetic<T>();

    if constexpr (std::is_unsigned_v<T>)
    {

```



```

        return value;
    }
    else
    {
        return value < static_cast<T>(0) ? static_cast<T>(-value) : value;
    }
}

/**
 * @brief Fast minimum of two scalar values.
 * @details Cost: 1 comparison; comparable to `std::min` but force-inlined.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastMin(T a, T b) noexcept
{
    detail::requireArithmetic<T>();
    return b < a ? b : a;
}

/**
 * @brief Fast maximum of two scalar values.
 * @details Cost: 1 comparison; comparable to `std::max` but force-inlined.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastMax(T a, T b) noexcept
{
    detail::requireArithmetic<T>();
    return a < b ? b : a;
}

/**
 * @brief Clamp a value to the inclusive range [low, high].
 * @details Cost: 2 comparisons; comparable to `std::clamp` but force-inlined.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastClamp(T value, T low, T high)
noexcept
{
    detail::requireArithmetic<T>();
    return fastMin(fastMax(value, low), high);
}

/**
 * @brief Linear interpolation between a and b.
 * @details Cost: 2 multiplications + 2 additions; simpler than `std::lerp`
 *         because it does not add special handling for all IEEE edge cases.
 */
template <typename T, typename U>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr auto fastLerp(T a, T b, U t)
noexcept -> decltype(a + (b - a) * t)
{
    detail::requireArithmetic<T>();
    detail::requireArithmetic<U>();
    return a + (b - a) * t;
}

/**
 * @brief Wrap a value into the half-open range [low, high).
 * @details Floating-point cost: 1 division + floor-like cast + arithmetic;
 *         usually cheaper than `std::fmod`/`std::remainder` in scalar DSP
 *         paths. Integral cost: modulo + a correction branch.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastWrap(T value, T low, T high)

```

```

noexcept
{
    detail::requireArithmetic<T>();

    const T range = high - low;
    if (range == static_cast<T>(0))
    {
        return low;
    }

    if constexpr (std::is_integral_v<T>)
    {
        T wrapped = static_cast<T>((value - low) % range);
        if (wrapped < static_cast<T>(0))
        {
            wrapped = static_cast<T>(wrapped + range);
        }
        return static_cast<T>(wrapped + low);
    }
    else
    {
        return value - range * detail::fastFloor((value - low) / range);
    }
}

/**
 * @brief Return -1 for negative values, 0 for zero, and +1 for positive values.
 * @details Cost: 2 comparisons; avoids branching-heavy sign helper code.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastSign(T value) noexcept
{
    detail::requireArithmetic<T>();
    return static_cast<T>((static_cast<T>(0) < value) - (value <
static_cast<T>(0)));
}

/**
 * @brief Fast square (x^2).
 * @details Cost: 1 multiplication; much cheaper than generic `std::pow(x, 2)`.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastPow2(T value) noexcept
{
    detail::requireArithmetic<T>();
    return value * value;
}

/**
 * @brief Fast cube (x^3).
 * @details Cost: 2 multiplications; much cheaper than generic `std::pow(x, 3)`.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastPow3(T value) noexcept
{
    detail::requireArithmetic<T>();
    return value * value * value;
}

/**
 * @brief Approximate decibels to linear gain.
 * @details Cost: 1 multiplication + fast base-2 exponential approximation;
 * avoids `std::pow(10, dB / 20)` and is suitable for smoothed gain
 * changes and meters.

```

```

*/
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastDbToGain(T decibels) noexcept
{
    detail::requireArithmetic<T>();
    constexpr T dbToLog2Gain =
static_cast<T>(0.166096404744368117393515971474267747L); // 1 / (20 * log10(2))
    return detail::fastExp2(decibels * dbToLog2Gain);
}

/**
 * @brief Approximate linear gain to decibels.
 * @details Cost: fast base-2 logarithm approximation + 1 multiplication;
 *             avoids `std::log10` and clamps non-positive gains to a practical
 *             silence floor before conversion.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastGainToDb(T gain) noexcept
{
    detail::requireArithmetic<T>();
    constexpr T log2ToDb =
static_cast<T>(6.02059991327962390427477789448984365L); // 20 * log10(2)
    return detail::fastLog2Positive(gain) * log2ToDb;
}

/**
 * @brief Fast hyperbolic tangent approximation.
 * @details Cost: 1 division + 4 multiplications + additions. The rational
 *             approximation  $x * (27 + x^2) / (27 + 9x^2)$  is far cheaper than
 *             `std::tanh` and saturates explicitly outside a useful audio range.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastTanh(T value) noexcept
{
    detail::requireArithmetic<T>();

    if (value <= static_cast<T>(-3))
    {
        return static_cast<T>(-1);
    }
    if (value >= static_cast<T>(3))
    {
        return static_cast<T>(1);
    }

    const T x2 = value * value;
    return value * (static_cast<T>(27) + x2) / (static_cast<T>(27) +
static_cast<T>(9) * x2);
}

/**
 * @brief Fast arctangent approximation in radians.
 * @details Cost for  $|x| \leq 1$ : 1 division + 2 multiplications + additions;
 *             large magnitudes use the reciprocal identity and remain much cheaper
 *             than `std::atan` for real-time waveshaping/filter code.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastAtan(T value) noexcept
{
    detail::requireArithmetic<T>();

    const T absValue = fastAbs(value);
    const T sign = fastSign(value);

```

```

        if (absValue <= static_cast<T>(1))
        {
            return value / (static_cast<T>(1) + static_cast<T>(0.28) * value *
value);
        }

        const T reciprocal = static_cast<T>(1) / absValue;
        const T small = reciprocal / (static_cast<T>(1) + static_cast<T>(0.28) *
reciprocal * reciprocal);
        return sign * (halfPi<T> - small);
    }

/**
 * @brief Cubic soft-clip waveshaper.
 * @details Cost in the active region: 2 multiplications + subtraction. This is
 *         far cheaper than tanh-based clipping with `std::tanh` and has a
 *         continuous first derivative at the saturation boundary.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE constexpr T fastSoftClip(T value) noexcept
{
    detail::requireArithmetic<T>();

    if (value <= static_cast<T>(-1))
    {
        return static_cast<T>(-2.0 / 3.0);
    }
    if (value >= static_cast<T>(1))
    {
        return static_cast<T>(2.0 / 3.0);
    }

    return value - fastPow3(value) * static_cast<T>(1.0 / 3.0);
}
} // namespace cvdsp

#endif // CVDSP_MATH_FASTMATH_HPP

```

```

=====
FILE: CV_DSP/Math/Interpolation.hpp
=====

```

```

#ifndef CVDSP_MATH_INTERPOLATION_HPP
#define CVDSP_MATH_INTERPOLATION_HPP

```

```

/**
 * @file Interpolation.hpp
 * @brief Header-only interpolation algorithms for real-time audio DSP.
 *
 * Interpolation estimates a continuous value between discrete samples. In audio
 * DSP this is required whenever a read position is fractional rather than an
 * integer sample index. Common examples are variable delay lines, pitch
 * shifters, time-stretch engines, chorus, flanger, wavetable playback, and
 * sample-rate conversion. This header provides small stateless interpolators
 * that are suitable for the audio thread: no allocation, no exceptions, no
RTTI,
 * and no external dependencies.
 *
 * All algorithms use the normalized fractional coordinate `fraction` in the
 * range [0, 1], where 0 means the current sample and 1 means the next sample.
 * The functions do not clamp `fraction`; callers may intentionally evaluate
 * outside [0, 1] for extrapolation, but real-time audio delay-line readers
 * should normally pass the fractional part returned by their address logic.

```

```

*
* Required DSP applications:
* - Delay Lines: fractional read heads use interpolation to realize sub-sample
*   delay times without moving memory.
* - Pitch Shifters: modulated fractional readers resample audio at a variable
*   ratio; interpolation controls aliasing, brightness, and modulation noise.
* - Time Stretch: overlap-add and granular readers often need fractional grain
*   positions for smooth playback-rate changes.
* - Chorus: low-frequency delay modulation continuously sweeps fractional read
*   positions; cubic-family methods reduce stepping noise compared with nearest
*   or linear reads.
* - Flanger: very short modulated delays are sensitive to phase error; higher
*   order interpolation improves comb-filter stability.
* - Sample Rate Conversion: arbitrary input/output ratios require evaluating a
*   discrete stream at non-integer positions.
*
* Example: linear fractional delay read from a circular buffer. The storage is
* owned by the object and is prepared before process(), so the audio callback
* performs only bounded arithmetic and fixed-size array access.
* @code{.cpp}
* #include "CV_DSP/Math/Interpolation.hpp"
* #include <array>
* #include <cstdint>
*
* class FractionalDelayExample
* {
* public:
*     void prepare(float delaySamples) noexcept
*     {
*         delaySamples_ = delaySamples;
*         writeIndex_ = 0;
*         buffer_.fill(0.0F);
*         cvdsp::LinearInterpolation::prepare();
*     }
*
*     void reset() noexcept
*     {
*         writeIndex_ = 0;
*         buffer_.fill(0.0F);
*         cvdsp::LinearInterpolation::reset();
*     }
*
*     void process(float* output, const float* input, std::size_t frames)
noexcept
*     {
*         for (std::size_t n = 0; n < frames; ++n)
*         {
*             buffer_[writeIndex_] = input[n];
*
*             const float readPosition = static_cast<float>(writeIndex_) -
delaySamples_ + static_cast<float>(size);
*             const std::size_t integerPart =
static_cast<std::size_t>(readPosition);
*             const std::size_t base = integerPart & mask;
*             const float fraction = readPosition -
static_cast<float>(integerPart);
*             const float y0 = buffer_[base];
*             const float y1 = buffer_[(base + 1U) & mask];
*
*             output[n] = cvdsp::LinearInterpolation::interpolate(y0, y1,
fraction);
*             writeIndex_ = (writeIndex_ + 1U) & mask;
*         }
*     }
}

```

```

*
* private:
*     static constexpr std::size_t size = 2048U;
*     static constexpr std::size_t mask = size - 1U;
*     std::array<float, size> buffer_{};
*     std::size_t writeIndex_ = 0;
*     float delaySamples_ = 1.0F;
* };
* @endcode
*
* Example: Catmull-Rom interpolation for chorus/flanger/pitch readers.
* @code{.cpp}
* #include "CV_DSP/Math/Interpolation.hpp"
* #include <array>
* #include <cstdint>
*
* float readCatmullRom(const std::array<float, 4096>& buffer,
*                     std::size_t integerIndex,
*                     float fraction) noexcept
* {
*     constexpr std::size_t mask = 4096U - 1U;
*     const float ym1 = buffer[(integerIndex - 1U) & mask];
*     const float y0  = buffer[integerIndex & mask];
*     const float y1  = buffer[(integerIndex + 1U) & mask];
*     const float y2  = buffer[(integerIndex + 2U) & mask];
*
*     return cvdsp::CatmullRomInterpolation::interpolate(ym1, y0, y1, y2,
fraction);
* }
* @endcode
*
* Example: Hermite interpolation with explicit slopes for sample-rate
conversion.
* @code{.cpp}
* #include "CV_DSP/Math/Interpolation.hpp"
*
* double resampleSegment(double y0, double y1, double previous, double next,
double mu) noexcept
* {
*     const double m0 = (y1 - previous) * 0.5;
*     const double m1 = (next - y0) * 0.5;
*     return cvdsp::HermiteInterpolation::interpolate(y0, y1, m0, m1, mu);
* }
* @endcode
*/

#include "../Core/Namespace.hpp"

#include <type_traits>

namespace cvdsp
{
namespace detail
{
template <typename T>
CVDSP_FORCE_INLINE constexpr void requireInterpolationScalar() noexcept
{
    static_assert(std::is_same_v<T, float> || std::is_same_v<T, double>,
"CV_DSP interpolation requires float or double scalar
types.");
}
} // namespace detail
}

/**

```

```

* @brief First-order interpolation between two adjacent samples.
*
* Mathematical theory:
* Linear interpolation fits a first-degree polynomial through the points
* `(0, y0)` and `(1, y1)`. The basis functions are `(1 - t)` and `t`, so the
* estimated value is `y(t) = y0 * (1 - t) + y1 * t`, equivalently
* `y0 + t * (y1 - y0)`. It is continuous in amplitude but its derivative is
* discontinuous at sample boundaries.
*
* DSP applications:
* It is commonly used in delay lines, pitch shifters, time stretchers, chorus,
* flangers, and low/medium quality sample-rate conversion when minimum latency
* and CPU cost are more important than high-frequency accuracy. In a modulated
* delay, it prevents hard sample jumps but can introduce mild high-frequency
* attenuation and modulation sidebands.
*
* Computational cost:
* 1 subtraction, 1 multiplication, and 1 addition. This is the cheapest method
* in this file and is highly compiler-friendly for scalar and auto-vectorized
* processing.
*/
struct LinearInterpolation
{
    /** @brief Stateless setup hook supplied for a uniform CV_DSP module API. */
    CVDSP_FORCE_INLINE static constexpr void prepare() noexcept {}

    /** @brief Stateless reset hook supplied for a uniform CV_DSP module API. */
    CVDSP_FORCE_INLINE static constexpr void reset() noexcept {}

    /**
     * @brief Interpolate between `y0` at t=0 and `y1` at t=1.
     * @tparam T Floating-point scalar type: float or double.
     * @param y0 Sample at the lower integer position.
     * @param y1 Sample at the next integer position.
     * @param fraction Normalized fractional position.
     * @return Interpolated value.
     */
    template <typename T>
    CVDSP_NODISCARD CVDSP_FORCE_INLINE static constexpr T interpolate(T y0, T
y1, T fraction) noexcept
    {
        detail::requireInterpolationScalar<T>();
        return y0 + fraction * (y1 - y0);
    }
};

/**
* @brief Four-point cubic interpolation optimized for fractional audio reads.
*
* Mathematical theory:
* This interpolator constructs a third-degree polynomial over the segment from
* `y1` at t=0 to `y2` at t=1 using the neighboring samples `y0` and `y3` to
* shape curvature. In Horner form the polynomial is
* `(((a0 * t) + a1) * t + a2) * t + a3`, with
* `a0 = y3 - y2 - y0 + y1`, `a1 = y0 - y1 - a0`, `a2 = y2 - y0`, and
* `a3 = y1`. The polynomial exactly reaches `y1` and `y2`, while the outer
* samples influence the slope and curvature. This classic audio cubic is not
* identical to Catmull-Rom or Lagrange; it trades a little mathematical
* exactness for a compact coefficient set.
*
* DSP applications:
* Useful for fractional delay lines, pitch shifters, time stretch engines,
* chorus, flanger, and sample-rate conversion where linear interpolation is too
* grainy but a slightly more expensive four-sample read is acceptable.

```

```

*
* Computational cost:
* Coefficient construction uses several additions/subtractions. Evaluation uses
* 3 multiplications and 3 additions in Horner form. No divisions are required.
*/
struct CubicInterpolation
{
    /** @brief Stateless setup hook supplied for a uniform CV_DSP module API. */
    CVDSP_FORCE_INLINE static constexpr void prepare() noexcept {}

    /** @brief Stateless reset hook supplied for a uniform CV_DSP module API. */
    CVDSP_FORCE_INLINE static constexpr void reset() noexcept {}

    /**
     * @brief Interpolate between center samples `y1` and `y2` using four
points.
     * @tparam T Floating-point scalar type: float or double.
     * @param y0 Sample before the interpolation segment.
     * @param y1 Segment start sample at t=0.
     * @param y2 Segment end sample at t=1.
     * @param y3 Sample after the interpolation segment.
     * @param fraction Normalized fractional position.
     * @return Interpolated value.
     */
    template <typename T>
    CVDSP_NODISCARD CVDSP_FORCE_INLINE static constexpr T interpolate(T y0, T
y1, T y2, T y3, T fraction) noexcept
    {
        detail::requireInterpolationScalar<T>();
        const T a0 = y3 - y2 - y0 + y1;
        const T a1 = y0 - y1 - a0;
        const T a2 = y2 - y0;
        const T a3 = y1;
        return ((a0 * fraction + a1) * fraction + a2) * fraction + a3;
    }
};

/**
 * @brief Cubic Hermite interpolation with explicit endpoint tangents.
 *
 * Mathematical theory:
 * Hermite interpolation fits a cubic polynomial to two endpoint values and two
 * endpoint derivatives. For t in [0, 1], the basis functions are
 * `h00 = 2t^3 - 3t^2 + 1`, `h10 = t^3 - 2t^2 + t`,
 * `h01 = -2t^3 + 3t^2`, and `h11 = t^3 - t^2`. The result is
 * `h00*y0 + h10*m0 + h01*y1 + h11*m1`, where `m0` and `m1` are slopes measured
 * in sample-value units per normalized segment. Explicit tangents let the
caller
 * control smoothness, monotonicity, and transient behavior.
 *
 * DSP applications:
 * Hermite interpolation is useful when a resampler, time-stretch reader, pitch
 * shifter, delay line, chorus, or flanger already has reliable derivative
 * estimates. It can preserve smoother motion than linear interpolation and can
 * be tuned to avoid excessive overshoot in sensitive feedback structures.
 *
 * Computational cost:
 * Builds t^2 and t^3, evaluates four basis functions, then combines four terms.
 * It is more expensive than linear interpolation but remains small and branch
 * free. When Catmull-Rom style tangents are desired, use
 * CatmullRomInterpolation to avoid repeated caller-side tangent code.
 */
struct HermiteInterpolation
{

```



```

/** @brief Stateless setup hook supplied for a uniform CV_DSP module API. */
CVDSP_FORCE_INLINE static constexpr void prepare() noexcept {}

/** @brief Stateless reset hook supplied for a uniform CV_DSP module API. */
CVDSP_FORCE_INLINE static constexpr void reset() noexcept {}

/**
 * @brief Interpolate between two samples using explicit tangents.
 * @tparam T Floating-point scalar type: float or double.
 * @param y0 Segment start sample at t=0.
 * @param y1 Segment end sample at t=1.
 * @param m0 Tangent at y0 in value units per normalized segment.
 * @param m1 Tangent at y1 in value units per normalized segment.
 * @param fraction Normalized fractional position.
 * @return Interpolated value.
 */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE static constexpr T interpolate(T y0, T
y1, T m0, T m1, T fraction) noexcept
{
    detail::requireInterpolationScalar<T>();
    const T t2 = fraction * fraction;
    const T t3 = t2 * fraction;
    const T h00 = static_cast<T>(2) * t3 - static_cast<T>(3) * t2 +
static_cast<T>(1);
    const T h10 = t3 - static_cast<T>(2) * t2 + fraction;
    const T h01 = static_cast<T>(-2) * t3 + static_cast<T>(3) * t2;
    const T h11 = t3 - t2;
    return h00 * y0 + h10 * m0 + h01 * y1 + h11 * m1;
}
};

/**
 * @brief Catmull-Rom spline interpolation through the two center samples.
 *
 * Mathematical theory:
 * Catmull-Rom is a cubic Hermite spline whose tangents are estimated from
 * neighboring samples:  $m_0 = 0.5 * (y_1 - y_{m1})$  and  $m_1 = 0.5 * (y_2 - y_0)$ .
 * With samples  $y_{m1}, y_0, y_1, y_2$ , it interpolates the segment from  $y_0$  at t=0
 * to  $y_1$  at t=1 and passes exactly through the center points. The 0.5 factor
 * gives the standard uniform Catmull-Rom spline, a good default for evenly
 * spaced audio samples.
 *
 * DSP applications:
 * This is a strong general-purpose choice for modulated delay lines, pitch
 * shifters, time stretch readers, chorus, flanger, and sample-rate conversion.
 * It usually sounds brighter and smoother than linear interpolation while
 * requiring only four neighboring samples and no state.
 *
 * Computational cost:
 * Tangent estimation uses two subtractions and two multiplications by 0.5, then
 * a cubic Hermite evaluation. The optimized Horner form below reduces the final
 * polynomial to 3 multiplications after coefficient construction.
 */
struct CatmullRomInterpolation
{
    /** @brief Stateless setup hook supplied for a uniform CV_DSP module API. */
    CVDSP_FORCE_INLINE static constexpr void prepare() noexcept {}

    /** @brief Stateless reset hook supplied for a uniform CV_DSP module API. */
    CVDSP_FORCE_INLINE static constexpr void reset() noexcept {}

    /**
     * @brief Interpolate between  $y_0$  and  $y_1$  using their immediate neighbors.

```

```

    * @tparam T Floating-point scalar type: float or double.
    * @param ym1 Sample before y0.
    * @param y0 Segment start sample at t=0.
    * @param y1 Segment end sample at t=1.
    * @param y2 Sample after y1.
    * @param fraction Normalized fractional position.
    * @return Interpolated value.
    */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE static constexpr T interpolate(T ym1, T
y0, T y1, T y2, T fraction) noexcept
{
    detail::requireInterpolationScalar<T>();
    const T half = static_cast<T>(0.5);
    const T a0 = half * (static_cast<T>(2) * y0);
    const T a1 = half * (-ym1 + y1);
    const T a2 = half * (static_cast<T>(2) * ym1 - static_cast<T>(5) * y0 +
static_cast<T>(4) * y1 - y2);
    const T a3 = half * (-ym1 + static_cast<T>(3) * y0 - static_cast<T>(3) *
y1 + y2);
    return ((a3 * fraction + a2) * fraction + a1) * fraction + a0;
}
};

/**
 * @brief Four-point third-order Lagrange interpolation.
 *
 * Mathematical theory:
 * Lagrange interpolation constructs the unique cubic polynomial that passes
 * exactly through four uniformly spaced samples. This implementation uses the
 * support points  $x=-1, 0, 1$ , and  $2$  with values `ym1, y0, y1, y2`, then
evaluates
 * the polynomial for  $t$  in  $[0, 1]$ . The basis functions are products of distance
 * ratios from all other sample positions, so each input sample contributes a
 * weight that becomes 1 at its own grid position and 0 at the others.
 *
 * DSP applications:
 * Lagrange interpolation is widely used for fractional delay lines, pitch
 * shifters, time stretchers, chorus, flangers, and sample-rate conversion when
 * exact passage through the four local samples and good passband behavior are
 * desired. It can overshoot on sharp transients, so feedback delay networks and
 * nonlinear processors should be gain-staged carefully.
 *
 * Computational cost:
 * The direct basis form uses several multiplications and constant divisions.
 * Constant denominators are represented as compile-time multipliers, avoiding
 * runtime division. It is typically more expensive than the compact cubic form
 * but provides a well-defined polynomial interpolant through all four points.
 */
struct LagrangeInterpolation
{
    /** @brief Stateless setup hook supplied for a uniform CV_DSP module API. */
    CVDSP_FORCE_INLINE static constexpr void prepare() noexcept {}

    /** @brief Stateless reset hook supplied for a uniform CV_DSP module API. */
    CVDSP_FORCE_INLINE static constexpr void reset() noexcept {}

    /**
     * @brief Interpolate between `y0` and `y1` with a four-point Lagrange
polynomial.
     * @tparam T Floating-point scalar type: float or double.
     * @param ym1 Sample at  $x=-1$ .
     * @param y0 Segment start sample at  $x=0$ .
     * @param y1 Segment end sample at  $x=1$ .

```

```

    * @param y2 Sample at x=2.
    * @param fraction Normalized fractional position.
    * @return Interpolated value.
    */
template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE static constexpr T interpolate(T ym1, T
y0, T y1, T y2, T fraction) noexcept
{
    detail::requireInterpolationScalar<T>();
    const T t = fraction;
    const T l0 = -((t) * (t - static_cast<T>(1)) * (t - static_cast<T>(2)))
* static_cast<T>(1.0 / 6.0);
    const T l1 = (t + static_cast<T>(1)) * (t - static_cast<T>(1)) * (t -
static_cast<T>(2)) * static_cast<T>(0.5);
    const T l2 = -((t + static_cast<T>(1)) * t * (t - static_cast<T>(2))) *
static_cast<T>(0.5);
    const T l3 = (t + static_cast<T>(1)) * t * (t - static_cast<T>(1)) *
static_cast<T>(1.0 / 6.0);
    return ym1 * l0 + y0 * l1 + y1 * l2 + y2 * l3;
}
};
} // namespace cvdsp

#endif // CVDSP_MATH_INTERPOLATION_HPP

```

```

=====
FILE: CV_DSP/Math/LookupTable.hpp
=====

```

```

#ifndef CVDSP_MATH_LOOKUPTABLE_HPP
#define CVDSP_MATH_LOOKUPTABLE_HPP

```

```

/**
 * @file LookupTable.hpp
 * @brief Generic linearly interpolated lookup tables for real-time DSP.
 *
 * The module provides allocation-free lookup tables for sine, cosine, and tanh
 * transfer functions. Tables are templated by scalar type, function kind, and
 * capacity so the compiler can optimize hot `lookup()` calls without virtual
 * dispatch. `prepare()` performs the expensive standard-library function calls
 * and should be executed during plug-in initialization or parameter changes,
 * never from a real-time audio callback.
 *
 * Supported capacities are 4096, 8192, 16384, and 32768 points. Lookup always
 * uses linear interpolation between adjacent samples. Sine and cosine inputs
are
 * interpreted as radians and wrapped to  $[0, 2\pi)$ . Tanh inputs are interpreted
as
 * waveshaper/control values over a configurable finite domain, defaulting to
 *  $[-5, 5]$ , and are clamped to the edge values outside that domain.
 *
 * Mathematical error analysis for linear interpolation:
 *
 * For a twice-differentiable function  $f$  sampled with spacing  $h$ , the absolute
 * interpolation error is bounded by
 *
 * 
$$\max(|f''(x)|) * h^2 / 8.$$

 *
 * Therefore, sine and cosine use  $\max(|f''|) = 1$  and  $h = 2\pi / \text{Points}$ . Tanh uses
 *  $f''(x) = -2 * \text{sech}^2(x) * \tanh(x)$ , whose global maximum absolute value is
 *  $4 / (3 * \sqrt{3})$ ; with  $h = (\text{max} - \text{min}) / \text{Points}$ . Outside the finite tanh
 * domain the table clamps to  $\tanh(\text{min}/\text{max})$ , so the additional saturation error
 * is bounded by  $\max(1 - |\tanh(\text{max})|, 1 - |\tanh(\text{min})|)$  for the default monotonic

```

```

* symmetric use case.
*
* Approximate interpolation-only error bounds:
*
* | Points | sine/cosine bound | tanh [-5, 5] bound |
* | --- | --- | --- |
* | 4096 | 2.94e-7 | 5.74e-7 |
* | 8192 | 7.35e-8 | 1.43e-7 |
* | 16384 | 1.84e-8 | 3.59e-8 |
* | 32768 | 4.59e-9 | 8.96e-9 |
*
* The actual total error also includes floating-point rounding from table
* generation, index arithmetic, and interpolation. For f32, rounding normally
* dominates at the largest table sizes; for f64, the interpolation term remains
* meaningful across the supported sizes.
*/

#include "../Core/Constants.hpp"
#include "../Core/Namespace.hpp"

#include <array>
#include <cmath>
#include <cstdint>
#include <type_traits>

namespace cvdsp
{
/**
 * @brief Function represented by a lookup table.
 */
enum class LookupTableFunction
{
    Sine,
    Cosine,
    Tanh
};

/**
 * @brief Named capacities supported by LookupTable.
 */
enum class LookupTableCapacity : std::size_t
{
    Points4096 = 4096,
    Points8192 = 8192,
    Points16384 = 16384,
    Points32768 = 32768
};

namespace detail
{
template <typename T>
CVDSP_FORCE_INLINE constexpr void requireLookupScalar() noexcept
{
    static_assert(std::is_same_v<T, float> || std::is_same_v<T, double>,
                  "CV_DSP lookup tables require float or double scalar types.");
}

template <std::size_t Points>
CVDSP_FORCE_INLINE constexpr void requireLookupCapacity() noexcept
{
    static_assert(Points == 4096U || Points == 8192U || Points == 16384U ||
                  Points == 32768U,
                  "CV_DSP lookup table capacity must be 4096, 8192, 16384, or
                  32768 points.");
}
}
}

```

```

}

template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE T sinValue(T radians) noexcept
{
    return static_cast<T>(std::sin(radians));
}

template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE T cosValue(T radians) noexcept
{
    return static_cast<T>(std::cos(radians));
}

template <typename T>
CVDSP_NODISCARD CVDSP_FORCE_INLINE T tanhValue(T value) noexcept
{
    return static_cast<T>(std::tanh(value));
}
} // namespace detail

/**
 * @brief Allocation-free, linearly interpolated DSP lookup table.
 *
 * @tparam T Floating-point scalar type: float or double.
 * @tparam Function Function represented by the table.
 * @tparam Points Table capacity: 4096, 8192, 16384, or 32768.
 */
template <typename T, LookupTableFunction Function, std::size_t Points>
class LookupTable
{
    static_assert(std::is_same_v<T, float> || std::is_same_v<T, double>,
                  "CV_DSP lookup tables require float or double scalar types.");

    static_assert(Points == 4096U || Points == 8192U || Points == 16384U ||
                  Points == 32768U,
                  "CV_DSP lookup table capacity must be 4096, 8192, 16384, or 32768 points.");

public:
    using Scalar = T;

    static constexpr LookupTableFunction function = Function;
    static constexpr std::size_t capacity = Points;
    static constexpr std::size_t storageSize = Points + 1U;

    /**
     * @brief Creates a table with the default tanh domain [-5, 5].
     * @note Call prepare() before lookup() to populate the table.
     */
    constexpr LookupTable() noexcept = default;

    /**
     * @brief Creates a table with a custom tanh domain.
     * @details The domain only affects Tanh tables. Sine and cosine tables keep
     *          their fixed [0, 2 $\pi$ ) radian domain.
     */
    constexpr LookupTable(T minimumInput, T maximumInput) noexcept
        : minInput_(minimumInput)
        , maxInput_(maximumInput)
    {
    }
}

/**

```

```

* @brief Fill the table using std::sin, std::cos, or std::tanh.
* @details This method performs all non-real-time setup work. The final
*          guard sample duplicates the periodic endpoint for sine/cosine or
*          stores the tanh maximum-domain value, allowing lookup() to use a
*          branch-light interpolation path.
*/
void prepare() noexcept
{
    detail::requireLookupScalar<T>();
    detail::requireLookupCapacity<Points>();

    if constexpr (Function == LookupTableFunction::Sine)
    {
        preparePeriodicSine();
    }
    else if constexpr (Function == LookupTableFunction::Cosine)
    {
        preparePeriodicCosine();
    }
    else
    {
        normalizeTanhDomain();
        prepareTanh();
    }
}

/**
* @brief Read the table with mandatory linear interpolation.
*
* Sine and cosine wrap `inputRadians` into  $[0, 2\pi)$ . Tanh maps the input
from
* the configured [minInput(), maxInput()] domain and clamps outside it.
*/
CVDSP_NODISCARD CVDSP_FORCE_INLINE T lookup(T input) const noexcept
{
    detail::requireLookupScalar<T>();
    detail::requireLookupCapacity<Points>();

    if constexpr (Function == LookupTableFunction::Sine || Function ==
LookupTableFunction::Cosine)
    {
        return lookupPeriodic(input);
    }
    else
    {
        return lookupClamped(input);
    }
}

/**
* @brief Current minimum input for tanh tables.
*/
CVDSP_NODISCARD constexpr T minInput() const noexcept
{
    return minInput_;
}

/**
* @brief Current maximum input for tanh tables.
*/
CVDSP_NODISCARD constexpr T maxInput() const noexcept
{
    return maxInput_;
}

```

```

/**
 * @brief Set the finite tanh input domain used by prepare() and lookup().
 * @note This does not rebuild the table automatically; call prepare() after
 *       changing the domain.
 */
void setInputRange(T minimumInput, T maximumInput) noexcept
{
    minInput_ = minimumInput;
    maxInput_ = maximumInput;
}

/**
 * @brief Direct read-only access to table storage for tests or inspection.
 */
CVDSP_NODISCARD constexpr const std::array<T, storageSize>& data() const
noexcept
{
    return table_;
}

/**
 * @brief Interpolation error bound from  $\max(|f'|) * h^2 / 8$ .
 */
CVDSP_NODISCARD constexpr T interpolationErrorBound() const noexcept
{
    if constexpr (Function == LookupTableFunction::Sine || Function ==
LookupTableFunction::Cosine)
    {
        constexpr T h = twoPi<T> / static_cast<T>(Points);
        return h * h * static_cast<T>(0.125);
    }
    else
    {
        // 4 / (3 * sqrt(3)) is the global maximum of |tanh''(x)|.
        constexpr T maxAbsSecondDerivative = static_cast<T>(4.0L /
5.1961524227066318805823390245176171008L);
        const T h = (maxInput_ - minInput_) / static_cast<T>(Points);
        return maxAbsSecondDerivative * h * h * static_cast<T>(0.125);
    }
}

/**
 * @brief Additional tanh-domain clamp error bound; zero for sine/cosine.
 */
CVDSP_NODISCARD T saturationErrorBound() const noexcept
{
    if constexpr (Function == LookupTableFunction::Tanh)
    {
        const T lowError = static_cast<T>(1) + detail::tanhValue(minInput_);
        const T highError = static_cast<T>(1) -
detail::tanhValue(maxInput_);
        return lowError > highError ? lowError : highError;
    }
    else
    {
        return static_cast<T>(0);
    }
}

private:
    std::array<T, storageSize> table_{};
    T minInput_ = static_cast<T>(-5);
    T maxInput_ = static_cast<T>(5);

```

```

void preparePeriodicSine() noexcept
{
    constexpr T step = twoPi<T> / static_cast<T>(Points);
    for (std::size_t i = 0; i < Points; ++i)
    {
        table_[i] = detail::sinValue(static_cast<T>(i) * step);
    }
    table_[Points] = table_[0];
}

void preparePeriodicCosine() noexcept
{
    constexpr T step = twoPi<T> / static_cast<T>(Points);
    for (std::size_t i = 0; i < Points; ++i)
    {
        table_[i] = detail::cosValue(static_cast<T>(i) * step);
    }
    table_[Points] = table_[0];
}

void normalizeTanhDomain() noexcept
{
    if (!(maxInput_ > minInput_))
    {
        minInput_ = static_cast<T>(-5);
        maxInput_ = static_cast<T>(5);
    }
}

void prepareTanh() noexcept
{
    const T step = (maxInput_ - minInput_) / static_cast<T>(Points);
    for (std::size_t i = 0; i <= Points; ++i)
    {
        table_[i] = detail::tanhValue(minInput_ + static_cast<T>(i) * step);
    }
}

CVDSP_NODISCARD CVDSP_FORCE_INLINE T lookupPeriodic(T radians) const
noexcept
{
    T normalized = radians / twoPi<T>;
    normalized -= std::floor(normalized);

    const T position = normalized * static_cast<T>(Points);
    const auto index = static_cast<std::size_t>(position);
    if (index >= Points)
    {
        return table_[0];
    }

    const T fraction = position - static_cast<T>(index);
    return table_[index] + (table_[index + 1U] - table_[index]) * fraction;
}

CVDSP_NODISCARD CVDSP_FORCE_INLINE T lookupClamped(T input) const noexcept
{
    if (input <= minInput_)
    {
        return table_[0];
    }
    if (input >= maxInput_)
    {

```



```

        return table_[Points];
    }

    const T position = (input - minInput_) * (static_cast<T>(Points) /
(maxInput_ - minInput_));
    auto index = static_cast<std::size_t>(position);
    if (index >= Points)
    {
        index = Points - 1U;
    }

    const T fraction = position - static_cast<T>(index);
    return table_[index] + (table_[index + 1U] - table_[index]) * fraction;
}
};

/**
 * @brief Convenience alias for callers that prefer the named capacity enum.
 */
template <typename T, LookupTableFunction Function, LookupTableCapacity
Capacity>
using LookupTableWithCapacity = LookupTable<T, Function,
static_cast<std::size_t>(Capacity)>;

using SineTable4096f = LookupTable<float, LookupTableFunction::Sine, 4096>;
using SineTable8192f = LookupTable<float, LookupTableFunction::Sine, 8192>;
using SineTable16384f = LookupTable<float, LookupTableFunction::Sine, 16384>;
using SineTable32768f = LookupTable<float, LookupTableFunction::Sine, 32768>;

using CosineTable4096f = LookupTable<float, LookupTableFunction::Cosine, 4096>;
using CosineTable8192f = LookupTable<float, LookupTableFunction::Cosine, 8192>;
using CosineTable16384f = LookupTable<float, LookupTableFunction::Cosine,
16384>;
using CosineTable32768f = LookupTable<float, LookupTableFunction::Cosine,
32768>;

using TanhTable4096f = LookupTable<float, LookupTableFunction::Tanh, 4096>;
using TanhTable8192f = LookupTable<float, LookupTableFunction::Tanh, 8192>;
using TanhTable16384f = LookupTable<float, LookupTableFunction::Tanh, 16384>;
using TanhTable32768f = LookupTable<float, LookupTableFunction::Tanh, 32768>;

using SineTable4096d = LookupTable<double, LookupTableFunction::Sine, 4096>;
using SineTable8192d = LookupTable<double, LookupTableFunction::Sine, 8192>;
using SineTable16384d = LookupTable<double, LookupTableFunction::Sine, 16384>;
using SineTable32768d = LookupTable<double, LookupTableFunction::Sine, 32768>;

using CosineTable4096d = LookupTable<double, LookupTableFunction::Cosine, 4096>;
using CosineTable8192d = LookupTable<double, LookupTableFunction::Cosine, 8192>;
using CosineTable16384d = LookupTable<double, LookupTableFunction::Cosine,
16384>;
using CosineTable32768d = LookupTable<double, LookupTableFunction::Cosine,
32768>;

using TanhTable4096d = LookupTable<double, LookupTableFunction::Tanh, 4096>;
using TanhTable8192d = LookupTable<double, LookupTableFunction::Tanh, 8192>;
using TanhTable16384d = LookupTable<double, LookupTableFunction::Tanh, 16384>;
using TanhTable32768d = LookupTable<double, LookupTableFunction::Tanh, 32768>;

} // namespace cvdsp

#endif // CVDSP_MATH_LOOKUPTABLE_HPP

```

```

=====
FILE: CV_DSP/Math/Oversampling.hpp
=====
#ifndef CVDSP_MATH_OVERSAMPLING_HPP
#define CVDSP_MATH_OVERSAMPLING_HPP

/**
 * @file Oversampling.hpp
 * @brief Oversampling Engine
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - 2x Oversampling
 * - 4x Oversampling
 * - 8x Oversampling
 * - Upsampling
 * - Downsampling
 * - Anti-Imaging Filter
 * - Anti-Aliasing Filter
 *
 * No dynamic allocation inside process.
 */

#include <array>
#include <cstdint>
#include <type_traits>

namespace cvdsp
{
/**
 * @brief Supported oversampling factors.
 */
enum class OversamplingFactor
{
    x2 = 2,
    x4 = 4,
    x8 = 8
};

/**
 * @brief Simple Halfband FIR.
 *
 * Linear phase.
 *
 * Used for:
 * - Anti Imaging
 * - Anti Aliasing
 *
 * Symmetric coefficients.
 */
template<typename T>
class HalfbandFIR
{
    static_assert(
        std::is_floating_point_v<T>,
        "HalfbandFIR requires floating point type");

public:
    static constexpr std::size_t NumTaps = 7;

```

```

constexpr HalfbandFIR() noexcept = default;

void reset() noexcept
{
    buffer_.fill(
        static_cast<T>(0));

    index_ = 0;
}

inline T process(
    T input) noexcept
{
    buffer_[index_] = input;

    T output =
        coeffs_[0] * get(0)
        +
        coeffs_[1] * get(1)
        +
        coeffs_[2] * get(2)
        +
        coeffs_[3] * get(3)
        +
        coeffs_[4] * get(4)
        +
        coeffs_[5] * get(5)
        +
        coeffs_[6] * get(6);

    index_++;

    if (index_ >= NumTaps)
    {
        index_ = 0;
    }

    return output;
}

private:

inline T get(
    std::size_t delay) const noexcept
{
    std::size_t pos =
        (
            index_
            +
            NumTaps
            -
            delay
        )
        %
        NumTaps;

    return buffer_[pos];
}

private:

std::array<T, NumTaps> buffer_{};

```

```

std::size_t index_ = 0;

static constexpr std::array<T, NumTaps> coeffs_
{
    static_cast<T>(-0.045),
    static_cast<T>(0.0),
    static_cast<T>(0.275),
    static_cast<T>(0.540),
    static_cast<T>(0.275),
    static_cast<T>(0.0),
    static_cast<T>(-0.045)
};
};

/**
 * @brief Oversampling engine.
 *
 * Template factor:
 *
 * 2
 * 4
 * 8
 */
template<
    typename T,
    std::size_t Factor>
class Oversampling
{
    static_assert(
        std::is_floating_point_v<T>,
        "Oversampling requires floating point type");

    static_assert(
        Factor == 2 ||
        Factor == 4 ||
        Factor == 8,
        "Factor must be 2,4 or 8");

public:

    static constexpr std::size_t OversamplingFactorValue =
        Factor;

    static constexpr std::size_t BlockSize =
        Factor;

    using OversampledBlock =
        std::array<T, Factor>;

public:

    constexpr Oversampling() noexcept = default;

    /**
     * @brief Initialize.
     */
    void prepare(
        T sampleRate) noexcept
    {
        sampleRate_ = sampleRate;

        reset();
    }
}

```

```

/**
 * @brief Reset state.
 */
void reset() noexcept
{
    upFilter_.reset();
    downFilter_.reset();
}

/**
 * @brief Upsample one sample.
 *
 * Returns Factor samples.
 */
[[nodiscard]]
inline OversampledBlock processUp(
    T input) noexcept
{
    OversampledBlock block{};

    /**
     * Zero-stuffing.
     */
    block[0] =
        upFilter_.process(
            input);

    for (std::size_t i = 1;
         i < Factor;
         ++i)
    {
        block[i] =
            upFilter_.process(
                static_cast<T>(0));
    }

    return block;
}

/**
 * @brief Downsample oversampled block.
 *
 * Anti-aliasing filtering
 * before decimation.
 */
[[nodiscard]]
inline T processDown(
    const OversampledBlock& block) noexcept
{
    T filtered =
        static_cast<T>(0);

    for (std::size_t i = 0;
         i < Factor;
         ++i)
    {
        filtered =
            downFilter_.process(
                block[i]);
    }

    return filtered;
}

```

```

private:
    T sampleRate_ =
        static_cast<T>(44100);

    HalfbandFIR<T> upFilter_;
    HalfbandFIR<T> downFilter_;
};

using Oversampling2xF = Oversampling<float, 2>;
using Oversampling4xF = Oversampling<float, 4>;
using Oversampling8xF = Oversampling<float, 8>;

using Oversampling2xD = Oversampling<double, 2>;
using Oversampling4xD = Oversampling<double, 4>;
using Oversampling8xD = Oversampling<double, 8>;

} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Modulation/ADSR.hpp
=====
#ifndef CVDSP_MODULATION_ADSR_HPP
#define CVDSP_MODULATION_ADSR_HPP

/**
 * @file ADSR.hpp
 * @brief ADSR Envelope Generator
 *
 * Header-only
 * C++20
 * Real-Time Safe
 * No dynamic allocation
 */

#include <algorithm>
#include <type_traits>

namespace cvdsp::modulation
{
/**
 * @brief ADSR envelope states.
 */
enum class ADSRState
{
    Idle,
    Attack,
    Decay,
    Sustain,
    Release
};

/**
 * @brief ADSR Envelope Generator.
 *
 * Output Range:
 *
 * 0.0 -> 1.0
 */

```

```

* Stages:
*
* Attack
* Decay
* Sustain
* Release
*
* Real-Time Safe
* Header Only
*/
template<typename T>
class ADSR
{
    static_assert(
        std::is_floating_point_v<T>,
        "ADSR requires floating point type");

public:

    constexpr ADSR() noexcept = default;

    /**
     * @brief Prepare envelope.
     *
     * @param sampleRate Processing sample rate.
     * @param attackMs Attack time.
     * @param decayMs Decay time.
     * @param sustain Sustain level.
     * @param releaseMs Release time.
     */
    void prepare(
        T sampleRate,
        T attackMs,
        T decayMs,
        T sustain,
        T releaseMs) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));

        attackMs_ =
            std::max(
                attackMs,
                static_cast<T>(0.001));

        decayMs_ =
            std::max(
                decayMs,
                static_cast<T>(0.001));

        sustainLevel_ =
            std::clamp(
                sustain,
                static_cast<T>(0),
                static_cast<T>(1));

        releaseMs_ =
            std::max(
                releaseMs,
                static_cast<T>(0.001));

        updateCoefficients();
    }
};

```

```

        reset();
    }

    /**
     * @brief Reset envelope state.
     */
    void reset() noexcept
    {
        value_ =
            static_cast<T>(0);

        releaseStartLevel_ =
            static_cast<T>(0);

        state_ =
            ADSRState::Idle;
    }

    /**
     * @brief Start envelope.
     */
    void noteOn() noexcept
    {
        state_ =
            ADSRState::Attack;
    }

    /**
     * @brief Release envelope.
     */
    void noteOff() noexcept
    {
        releaseStartLevel_ =
            value_;

        state_ =
            ADSRState::Release;
    }

    /**
     * @brief Set attack time.
     */
    void setAttack(
        T attackMs) noexcept
    {
        attackMs_ =
            std::max(
                attackMs,
                static_cast<T>(0.001));

        updateAttack();
    }

    /**
     * @brief Set decay time.
     */
    void setDecay(
        T decayMs) noexcept
    {
        decayMs_ =
            std::max(
                decayMs,
                static_cast<T>(0.001));
    }

```



```

        updateDecay();
    }

    /**
     * @brief Set sustain level.
     */
    void setSustain(
        T sustain) noexcept
    {
        sustainLevel_ =
            std::clamp(
                sustain,
                static_cast<T>(0),
                static_cast<T>(1));
    }

    /**
     * @brief Set release time.
     */
    void setRelease(
        T releaseMs) noexcept
    {
        releaseMs_ =
            std::max(
                releaseMs,
                static_cast<T>(0.001));

        updateRelease();
    }

    /**
     * @brief Process one sample.
     *
     * @return Envelope value.
     */
    inline T process() noexcept
    {
        switch (state_)
        {
            case ADSRState::Idle:
            {
                value_ =
                    static_cast<T>(0);
                break;
            }

            case ADSRState::Attack:
            {
                value_ += attackIncrement_;

                if (value_ >= static_cast<T>(1))
                {
                    value_ =
                        static_cast<T>(1);

                    state_ =
                        ADSRState::Decay;
                }

                break;
            }

            case ADSRState::Decay:

```

```

    {
        value_ -= decayIncrement_;

        if (value_ <= sustainLevel_)
        {
            value_ =
                sustainLevel_;

            state_ =
                ADSRState::Sustain;
        }

        break;
    }

    case ADSRState::Sustain:
    {
        value_ =
            sustainLevel_;
        break;
    }

    case ADSRState::Release:
    {
        value_ -= releaseIncrement_;

        if (value_ <= static_cast<T>(0))
        {
            value_ =
                static_cast<T>(0);

            state_ =
                ADSRState::Idle;
        }

        break;
    }

    default:
    {
        value_ =
            static_cast<T>(0);

        state_ =
            ADSRState::Idle;
        break;
    }
}

return value_;
}

/**
 * @brief Current state.
 */
[[nodiscard]]
ADSRState getState() const noexcept
{
    return state_;
}

/**
 * @brief Current envelope value.
 */

```

```

[[nodiscard]]
T getValue() const noexcept
{
    return value_;
}

```

private:

```

void updateCoefficients() noexcept
{
    updateAttack();
    updateDecay();
    updateRelease();
}

```

```

void updateAttack() noexcept
{
    const T attackSamples =
        attackMs_
        *
        static_cast<T>(0.001)
        *
        sampleRate_;

    attackIncrement_ =
        static_cast<T>(1)
        /
        std::max(
            attackSamples,
            static_cast<T>(1));
}

```

```

void updateDecay() noexcept
{
    const T decaySamples =
        decayMs_
        *
        static_cast<T>(0.001)
        *
        sampleRate_;

    decayIncrement_ =
        (
            static_cast<T>(1)
            -
            sustainLevel_
        )
        /
        std::max(
            decaySamples,
            static_cast<T>(1));
}

```

```

void updateRelease() noexcept
{
    const T releaseSamples =
        releaseMs_
        *
        static_cast<T>(0.001)
        *
        sampleRate_;

    releaseIncrement_ =
        static_cast<T>(1)

```

```

        /
        std::max(
            releaseSamples,
            static_cast<T>(1));
    }

private:
    T sampleRate_ =
        static_cast<T>(44100);

    T attackMs_ =
        static_cast<T>(10);

    T decayMs_ =
        static_cast<T>(100);

    T sustainLevel_ =
        static_cast<T>(0.7);

    T releaseMs_ =
        static_cast<T>(250);

    T attackIncrement_ =
        static_cast<T>(0);

    T decayIncrement_ =
        static_cast<T>(0);

    T releaseIncrement_ =
        static_cast<T>(0);

    T value_ =
        static_cast<T>(0);

    T releaseStartLevel_ =
        static_cast<T>(0);

    ADSRState state_ =
        ADSRState::Idle;
};

} // namespace cvdsp::modulation

namespace cvdsp
{
template<typename T>
using ADSR = modulation::ADSR<T>;

using ADSRState = modulation::ADSRState;
} // namespace cvdsp

#endif

=====
FILE: CV_DSP/Modulation/LFO.hpp
=====
#ifndef CVDSP_MODULATION_LFO_HPP
#define CVDSP_MODULATION_LFO_HPP

/**
 * @file LFO.hpp

```

```

* @brief Low Frequency Oscillator
*
* Header-only
* C++20
* Real-Time Safe
* No dynamic allocation
*/

#include <algorithm>
#include <cmath>
#include <type_traits>

namespace cvdsp::modulation
{
    /**
     * @brief LFO waveform types.
     */
    enum class LFOWaveform
    {
        Sine,
        Triangle,
        Saw,
        Square
    };

    /**
     * @brief Low Frequency Oscillator.
     *
     * Frequency Range:
     *
     * 0.01 Hz
     * to
     * 50 Hz
     *
     * Output Range:
     *
     * depth = 1.0
     *
     * [-1, +1]
     *
     * depth = 0.5
     *
     * [-0.5, +0.5]
     *
     * Suitable for:
     *
     * - Chorus
     * - Flanger
     * - Phaser
     * - Tremolo
     * - Auto Pan
     * - Filter Modulation
     */
    template<typename T>
    class LFO
    {
    public:
        static_assert(
            std::is_floating_point_v<T>,
            "LFO requires floating point type");

        constexpr LFO() noexcept = default;
    };
}

```

```

/**
 * @brief Prepare LFO.
 *
 * @param sampleRate Processing sample rate.
 * @param rateHz Initial rate.
 * @param depth Initial depth.
 * @param waveform Waveform.
 */
void prepare(
    T sampleRate,
    T rateHz,
    T depth,
    LFOWaveform waveform) noexcept
{
    sampleRate_ =
        std::max(
            sampleRate,
            static_cast<T>(1));

    waveform_ = waveform;

    setRate(rateHz);
    setDepth(depth);

    reset();
}

/**
 * @brief Reset phase accumulator.
 */
void reset() noexcept
{
    phase_ = static_cast<T>(0);
}

/**
 * @brief Set LFO frequency.
 *
 * Range:
 *
 * 0.01 Hz
 * to
 * 50 Hz
 */
void setRate(
    T rateHz) noexcept
{
    constexpr T kMinRate =
        static_cast<T>(0.01);

    constexpr T kMaxRate =
        static_cast<T>(50.0);

    rateHz_ =
        std::clamp(
            rateHz,
            kMinRate,
            kMaxRate);

    phaseIncrement_ =
        rateHz_ / sampleRate_;
}

```

```

/**
 * @brief Set modulation depth.
 *
 * Range:
 *
 * 0.0 ... 1.0
 */
void setDepth(
    T depth) noexcept
{
    depth_ =
        std::clamp(
            depth,
            static_cast<T>(0),
            static_cast<T>(1));
}

/**
 * @brief Select waveform.
 */
void setWaveform(
    LFOWaveform waveform) noexcept
{
    waveform_ = waveform;
}

/**
 * @brief Current rate.
 */
[[nodiscard]]
T getRate() const noexcept
{
    return rateHz_;
}

/**
 * @brief Current depth.
 */
[[nodiscard]]
T getDepth() const noexcept
{
    return depth_;
}

/**
 * @brief Generate one LFO sample.
 *
 * Output:
 *
 * [-depth,+depth]
 */
inline T process() noexcept
{
    T value {};

    switch (waveform_)
    {
        case LFOWaveform::Sine:
        {
            value = processSine();
            break;
        }

        case LFOWaveform::Triangle:

```

```

    {
        value = processTriangle();
        break;
    }

    case LFOWaveform::Saw:
    {
        value = processSaw();
        break;
    }

    case LFOWaveform::Square:
    {
        value = processSquare();
        break;
    }

    default:
    {
        value = static_cast<T>(0);
        break;
    }
}

advancePhase();

return value * depth_;
}

```

private:

```

/**
 * @brief Advance normalized phase.
 *
 * Range:
 *
 * [0,1)
 */
inline void advancePhase() noexcept
{
    phase_ += phaseIncrement_;

    if (phase_ >= static_cast<T>(1))
    {
        phase_ -= static_cast<T>(1);
    }
}

/**
 * @brief Sine waveform.
 */
inline T processSine() const noexcept
{
    constexpr T kTwoPi =
        static_cast<T>(
            6.283185307179586476925286766559);

    return std::sin(
        phase_ * kTwoPi);
}

/**
 * @brief Triangle waveform.
 */

```



```

inline T processTriangle() const noexcept
{
    return
        static_cast<T>(1)
        -
        static_cast<T>(4)
        *
        std::abs(
            phase_
            -
            static_cast<T>(0.5));
}

/**
 * @brief Saw waveform.
 */
inline T processSaw() const noexcept
{
    return
        static_cast<T>(2)
        *
        phase_
        -
        static_cast<T>(1);
}

/**
 * @brief Square waveform.
 */
inline T processSquare() const noexcept
{
    return
        (phase_ < static_cast<T>(0.5))
        ? static_cast<T>(1)
        : static_cast<T>(-1);
}

private:

    T sampleRate_ =
        static_cast<T>(44100);

    T rateHz_ =
        static_cast<T>(1);

    T depth_ =
        static_cast<T>(1);

    T phase_ =
        static_cast<T>(0);

    T phaseIncrement_ =
        static_cast<T>(0);

    LFOWaveform waveform_ =
        LFOWaveform::Sine;
};

} // namespace cvdsp::modulation

namespace cvdsp
{
template<typename T>
using LFO = modulation::LFO<T>;

```

```
using LFOWaveform = modulation::LFOWaveform;
} // namespace cvdsp
```

```
#endif
```

```
=====
FILE: CV_DSP/Modulation/Oscillator.hpp
=====
```

```
#ifndef CVDSP_MODULATION_OSCILLATOR_HPP
#define CVDSP_MODULATION_OSCILLATOR_HPP
```

```
/**
 * @file Oscillator.hpp
 * @brief Digital Oscillator
 *
 * Header-only
 * C++20
 * Real-Time Safe
 * No dynamic allocation
 */
```

```
#include <cmath>
#include <type_traits>
#include <algorithm>
```

```
namespace cvdsp::modulation
{
```

```
/**
 * @brief Oscillator waveform types.
 */
```

```
enum class OscillatorWaveform
{
    Sine,
    Triangle,
    Saw,
    Square
};
```

```
/**
 * @brief Digital oscillator using phase accumulator.
 *
 * Supports:
 * - Sine
 * - Triangle
 * - Saw
 * - Square
 *
 * Template:
 *
 * float
 * double
 */
```

```
template<typename T>
class Oscillator
{
    static_assert(
        std::is_floating_point_v<T>,
        "Oscillator requires floating point type");
```

```
public:
```

```

constexpr Oscillator() noexcept = default;

/**
 * @brief Initialize oscillator.
 *
 * @param sampleRate Processing sample rate.
 * @param frequency Initial frequency.
 * @param waveform Waveform type.
 */
void prepare(
    T sampleRate,
    T frequency,
    OscillatorWaveform waveform) noexcept
{
    sampleRate_ =
        std::max(
            sampleRate,
            static_cast<T>(1));

    waveform_ = waveform;

    setFrequency(frequency);

    reset();
}

/**
 * @brief Reset oscillator state.
 */
void reset() noexcept
{
    phase_ = static_cast<T>(0);
}

/**
 * @brief Set oscillator frequency.
 *
 * @param frequency Frequency in Hz.
 */
void setFrequency(
    T frequency) noexcept
{
    constexpr T kMinFreq =
        static_cast<T>(0);

    const T nyquist =
        sampleRate_ *
        static_cast<T>(0.5);

    frequency_ =
        std::clamp(
            frequency,
            kMinFreq,
            nyquist);

    phaseIncrement_ =
        frequency_ / sampleRate_;
}

/**
 * @brief Set waveform.
 *
 * @param waveform New waveform.

```

```

    */
void setWaveform(
    OscillatorWaveform waveform) noexcept
{
    waveform_ = waveform;
}

/**
 * @brief Get current frequency.
 */
[[nodiscard]]
T getFrequency() const noexcept
{
    return frequency_;
}

/**
 * @brief Generate one sample.
 */
inline T process() noexcept
{
    T output {};

    switch (waveform_)
    {
        case OscillatorWaveform::Sine:
        {
            output = processSine();
            break;
        }

        case OscillatorWaveform::Triangle:
        {
            output = processTriangle();
            break;
        }

        case OscillatorWaveform::Saw:
        {
            output = processSaw();
            break;
        }

        case OscillatorWaveform::Square:
        {
            output = processSquare();
            break;
        }

        default:
        {
            output = static_cast<T>(0);
            break;
        }
    }

    advancePhase();

    return output;
}

private:

/**

```

```

    * @brief Advance normalized phase.
    *
    * Range:
    *
    * [0,1)
    */
inline void advancePhase() noexcept
{
    phase_ += phaseIncrement_;

    if (phase_ >= static_cast<T>(1))
    {
        phase_ -= static_cast<T>(1);
    }
}

/**
 * @brief Sine oscillator.
 */
inline T processSine() const noexcept
{
    constexpr T kTwoPi =
        static_cast<T>(
            6.283185307179586476925286766559);

    return std::sin(
        phase_ * kTwoPi);
}

/**
 * @brief Saw oscillator.
 *
 * Range:
 *
 * -1 ... +1
 */
inline T processSaw() const noexcept
{
    return
        static_cast<T>(2) * phase_
        -
        static_cast<T>(1);
}

/**
 * @brief Square oscillator.
 *
 * 50% duty cycle.
 */
inline T processSquare() const noexcept
{
    return
        (phase_ < static_cast<T>(0.5))
        ? static_cast<T>(1)
        : static_cast<T>(-1);
}

/**
 * @brief Triangle oscillator.
 *
 * Range:
 *
 * -1 ... +1
 */

```

```

inline T processTriangle() const noexcept
{
    return
        static_cast<T>(1)
        -
        static_cast<T>(4)
        *
        std::abs(
            phase_
            -
            static_cast<T>(0.5));
}

private:

    T sampleRate_ =
        static_cast<T>(44100);

    T frequency_ =
        static_cast<T>(440);

    T phase_ =
        static_cast<T>(0);

    T phaseIncrement_ =
        static_cast<T>(0);

    OscillatorWaveform waveform_ =
        OscillatorWaveform::Sine;
};

} // namespace cvdsp::modulation

namespace cvdsp
{
template<typename T>
using Oscillator = modulation::Oscillator<T>;

using OscillatorWaveform = modulation::OscillatorWaveform;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Saturation/TapeSaturation.hpp
=====
#ifndef CVDSP_SATURATION_TAPESATURATION_HPP
#define CVDSP_SATURATION_TAPESATURATION_HPP

/**
 * @file TapeSaturation.hpp
 * @brief Tape Saturation Processor
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Drive
 * - Bias
 * - Compression
 * - Soft Limiting

```

```

* - Magnetic Saturation
*
* Inspired by analog tape machines.
*/

#include <algorithm>
#include <cmath>
#include <type_traits>

namespace cvdsp::saturation
{
/**
 * @brief Tape saturation processor.
 *
 * Simulates:
 *
 * - Magnetic tape saturation
 * - Soft compression
 * - Soft limiting
 * - Harmonic enrichment
 *
 * Designed for:
 *
 * - Mix bus
 * - Master bus
 * - Tape coloration
 * - Instrument processing
 */
template<typename T>
class TapeSaturation
{
    static_assert(
        std::is_floating_point_v<T>,
        "TapeSaturation requires floating point type");

public:
    constexpr TapeSaturation() noexcept = default;

    /**
     * @brief Initialize processor.
     *
     * Present for API consistency.
     */
    void prepare(
        T sampleRate) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));

        reset();
    }

    /**
     * @brief Reset internal state.
     */
    void reset() noexcept
    {
        envelope_ =
            static_cast<T>(0);
    }

```

```

        gainReduction_ =
            static_cast<T>(1);
    }

    /**
     * @brief Set input drive.
     *
     * Range:
     *
     * 1.0 .. 30.0
     */
    void setDrive(
        T drive) noexcept
    {
        drive_ =
            std::clamp(
                drive,
                static_cast<T>(1),
                static_cast<T>(30));
    }

    /**
     * @brief Set tape bias.
     *
     * Range:
     *
     * -1.0 .. +1.0
     */
    void setBias(
        T bias) noexcept
    {
        bias_ =
            std::clamp(
                bias,
                static_cast<T>(-1),
                static_cast<T>(1));
    }

    /**
     * @brief Set compression amount.
     *
     * Range:
     *
     * 0.0 .. 1.0
     */
    void setCompression(
        T compression) noexcept
    {
        compression_ =
            std::clamp(
                compression,
                static_cast<T>(0),
                static_cast<T>(1));
    }

    /**
     * @brief Process one sample.
     */
    inline T process(
        T input) noexcept
    {
        /**
         * Input drive.
         */

```



```

T x =
    input
    *
    drive_;

/**
 * Tape bias.
 */
x +=
    bias_;

/**
 * Envelope follower.
 *
 * Slow and smooth.
 */
const T detector =
    std::fabs(x);

envelope_ +=
    envelopeCoeff_
    *
    (
        detector
        -
        envelope_
    );

/**
 * Tape compression.
 */
const T compressionAmount =
    static_cast<T>(1)
    +
    (
        envelope_
        *
        compression_
        *
        static_cast<T>(4)
    );

gainReduction_ =
    static_cast<T>(1)
    /
    compressionAmount;

x *= gainReduction_;

/**
 * Magnetic transfer curve.
 *
 * Smooth saturation.
 */
T y =
    magneticCurve(x);

/**
 * Remove part of bias.
 */
y -=
    bias_
    *
    static_cast<T>(0.35);

```

```

    /**
     * Tape output normalization.
     */
    y *=
        outputTrim_;

    return y;
}

```

private:

```

/**
 * @brief Magnetic tape transfer curve.
 *
 * Softer than tube clipping.
 */
static inline T magneticCurve(
    T x) noexcept
{
    return
        std::tanh(
            x
            *
            static_cast<T>(0.85));
}

```

private:

```

T sampleRate_ =
    static_cast<T>(44100);

T drive_ =
    static_cast<T>(1);

T bias_ =
    static_cast<T>(0);

T compression_ =
    static_cast<T>(0.5);

/**
 * Internal compressor state.
 */
T envelope_ =
    static_cast<T>(0);

T gainReduction_ =
    static_cast<T>(1);

/**
 * Fixed tape-style smoothing.
 *
 * Produces slow gain adaptation
 * similar to magnetic saturation.
 */
T envelopeCoeff_ =
    static_cast<T>(0.001);

/**
 * Output compensation.
 */
T outputTrim_ =
    static_cast<T>(1.15);

```

```
};

} // namespace cvdsp::saturation

namespace cvdsp
{
template<typename T>
using TapeSaturation = saturation::TapeSaturation<T>;
} // namespace cvdsp

#endif
```

```
=====
FILE: CV_DSP/Saturation/ToneStack.hpp
=====
#ifndef CVDSP_GUITAR_TONESTACK_HPP
#define CVDSP_GUITAR_TONESTACK_HPP

/**
 * @file ToneStack.hpp
 * @brief Base class for guitar amplifier tone stacks.
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - Filters/Biquad.hpp
 *
 * Optional:
 *
 * - Filters/StateVariableFilter.hpp
 *
 * Intended base for:
 *
 * - FenderToneStack
 * - MarshallToneStack
 * - MesaToneStack
 * - VoxToneStack
 */

#include <algorithm>
#include <cmath>
#include <type_traits>

#include "../Filters/Biquad.hpp"

namespace cvdsp::saturation
{
template<typename T>
class ToneStack
{
public:
    static_assert(
        std::is_floating_point_v<T>,
        "ToneStack requires float or double");

    virtual ~ToneStack() = default;
```

```

virtual void prepare(
    T sampleRate)
    noexcept
{
    sampleRate_ =
        (std::isfinite(sampleRate) && sampleRate > static_cast<T>(0))
        ? sampleRate
        : static_cast<T>(44100);

    bassFilter_.prepare(sampleRate_);
    midFilter_.prepare(sampleRate_);
    trebleFilter_.prepare(sampleRate_);
    presenceFilter_.prepare(sampleRate_);

    reset();
}

virtual void reset()
    noexcept
{
    bassFilter_.reset();
    midFilter_.reset();
    trebleFilter_.reset();
    presenceFilter_.reset();
}

virtual T process(
    T input)
    noexcept = 0;

virtual void setBass(
    T value)
    noexcept
{
    bass_ = clampControl(value);
}

virtual void setMid(
    T value)
    noexcept
{
    mid_ = clampControl(value);
}

virtual void setTreble(
    T value)
    noexcept
{
    treble_ = clampControl(value);
}

virtual void setPresence(
    T value)
    noexcept
{
    presence_ = clampControl(value);
}

[[nodiscard]]
T getBass() const noexcept
{
    return bass_;
}

```

```

[[nodiscard]]
T getMid() const noexcept
{
    return mid_;
}

[[nodiscard]]
T getTreble() const noexcept
{
    return treble_;
}

[[nodiscard]]
T getPresence() const noexcept
{
    return presence_;
}

[[nodiscard]]
T getSampleRate() const noexcept
{
    return sampleRate_;
}

```

protected:

```

[[nodiscard]]
static T clampControl(
    T value) noexcept
{
    if (!std::isfinite(value))
    {
        return static_cast<T>(0.5);
    }

    return std::clamp(
        value,
        static_cast<T>(0),
        static_cast<T>(1));
}

static void configurePeak(
    cvdsp::filters::Biquad<T>& filter,
    T frequencyHz,
    T q,
    T gainDb) noexcept
{
    filter.setType(
        filters::BiquadType::PeakingEQ);

    filter.setFrequency(
        frequencyHz);

    filter.setQ(
        q);

    filter.setGainDB(
        gainDb);

    filter.updateCoefficients();
}

static void configureLowShelf(
    cvdsp::filters::Biquad<T>& filter,

```

```

        T frequencyHz,
        T q,
        T gainDb) noexcept
{
    filter.setType(
        filters::BiquadType::LowShelf);

    filter.setFrequency(
        frequencyHz);

    filter.setQ(
        q);

    filter.setGainDB(
        gainDb);

    filter.updateCoefficients();
}

static void configureHighShelf(
    cvdsp::filters::Biquad<T>& filter,
    T frequencyHz,
    T q,
    T gainDb) noexcept
{
    filter.setType(
        filters::BiquadType::HighShelf);

    filter.setFrequency(
        frequencyHz);

    filter.setQ(
        q);

    filter.setGainDB(
        gainDb);

    filter.updateCoefficients();
}

```

protected:

```

    T sampleRate_ =
        static_cast<T>(44100);

    T bass_ =
        static_cast<T>(0.5);

    T mid_ =
        static_cast<T>(0.5);

    T treble_ =
        static_cast<T>(0.5);

    T presence_ =
        static_cast<T>(0.5);

/**
 * Reutilizados pelas implementações derivadas.
 */

cvdsp::filters::Biquad<T>
    bassFilter_;

```

```

        cvdsp::filters::Biquad<T>
            midFilter_;

        cvdsp::filters::Biquad<T>
            trebleFilter_;

        cvdsp::filters::Biquad<T>
            presenceFilter_;
};

using ToneStackF =
    ToneStack<float>;

using ToneStackD =
    ToneStack<double>;

} // namespace cvdsp::saturation

namespace cvdsp
{
template<typename T>
using ToneStack = saturation::ToneStack<T>;

using ToneStackF = saturation::ToneStackF;
using ToneStackD = saturation::ToneStackD;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Saturation/TubeSaturation.hpp
=====
#ifndef CVDSP_SATURATION_TUBESATURATION_HPP
#define CVDSP_SATURATION_TUBESATURATION_HPP

/**
 * @file TubeSaturation.hpp
 * @brief Tube Style Saturation
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Drive
 * - Bias
 * - Output Gain
 * - Even Harmonic Generation
 * - Asymmetrical Saturation
 */

#include <algorithm>
#include <cmath>
#include <type_traits>

namespace cvdsp::saturation
{
/**
 * @brief Tube style saturation processor.
 *
 * Inspired by triode preamp stages.

```

```

*
* Produces:
*
* - Soft compression
* - Even harmonics
* - Asymmetrical distortion
*
* Real-Time Safe
*/
template<typename T>
class TubeSaturation
{
    static_assert(
        std::is_floating_point_v<T>,
        "TubeSaturation requires floating point type");

public:
    constexpr TubeSaturation() noexcept = default;

    /**
     * @brief Initialize processor.
     *
     * Present for API consistency.
     */
    void prepare(
        T sampleRate) noexcept
    {
        sampleRate_ =
            std::max(
                sampleRate,
                static_cast<T>(1));

        reset();
    }

    /**
     * @brief Reset internal state.
     */
    void reset() noexcept
    {
        previousOutput_ =
            static_cast<T>(0);
    }

    /**
     * @brief Set drive amount.
     *
     * Range:
     *
     * 1.0 .. 50.0
     */
    void setDrive(
        T drive) noexcept
    {
        drive_ =
            std::clamp(
                drive,
                static_cast<T>(1),
                static_cast<T>(50));
    }

    /**
     * @brief Set DC bias.

```



```

*
* Range:
*
* -1.0 .. +1.0
*/
void setBias(
    T bias) noexcept
{
    bias_ =
        std::clamp(
            bias,
            static_cast<T>(-1),
            static_cast<T>(1));
}

/**
 * @brief Set output gain.
 *
 * Linear gain.
 *
 * Range:
 *
 * 0.0 .. 10.0
 */
void setOutputGain(
    T gain) noexcept
{
    outputGain_ =
        std::clamp(
            gain,
            static_cast<T>(0),
            static_cast<T>(10));
}

/**
 * @brief Process one sample.
 */
inline T process(
    T input) noexcept
{
    T x =
        input
        *
        drive_;

    /**
     * Bias shift.
     */
    x += bias_;

    /**
     * Asymmetrical triode response.
     *
     * Positive half:
     * softer
     *
     * Negative half:
     * stronger compression
     */
    T y;

    if (x >= static_cast<T>(0))
    {
        y =

```

```

        positiveShape(x);
    }
    else
    {
        y =
            negativeShape(x);
    }

    /**
     * Remove part of the bias.
     */
    y -=
        bias_
        *
        static_cast<T>(0.5);

    y *= outputGain_;

    previousOutput_ = y;

    return y;
}

```

private:

```

/**
 * @brief Positive triode curve.
 */
static inline T positiveShape(
    T x) noexcept
{
    return
        static_cast<T>(1)
        -
        std::exp(
            -x);
}

/**
 * @brief Negative triode curve.
 */
static inline T negativeShape(
    T x) noexcept
{
    return
        -(
            static_cast<T>(1)
            -
            std::exp(
                static_cast<T>(1.5)
                *
                x));
}

```

private:

```

T sampleRate_ =
    static_cast<T>(44100);

T drive_ =
    static_cast<T>(1);

T bias_ =
    static_cast<T>(0);

```

```

        T outputGain_ =
            static_cast<T>(1);

        T previousOutput_ =
            static_cast<T>(0);
};

} // namespace cvdsp::saturation

namespace cvdsp
{
template<typename T>
using TubeSaturation = saturation::TubeSaturation<T>;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Saturation/Waveshaper.hpp
=====

```

```

#ifndef CVDSP_SATURATION_WAVESHAPER_HPP
#define CVDSP_SATURATION_WAVESHAPER_HPP

/**
 * @file Waveshaper.hpp
 * @brief Collection of Waveshaping Algorithms
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 */

#include <algorithm>
#include <cmath>
#include <numbers>
#include <type_traits>

namespace cvdsp::saturation
{
/**
 * @brief Waveshaper modes.
 */
enum class WaveshaperMode
{
    HardClip,
    SoftClip,
    Tanh,
    Arctan,
    Cubic,
    Foldback
};

/**
 * @brief Collection of static waveshaping algorithms.
 *
 * Stateless.
 * Thread-safe.
 * Real-Time Safe.

```

```

*/
template<typename T>
class Waveshaper
{
    static_assert(
        std::is_floating_point_v<T>,
        "Waveshaper requires floating point type");

public:
    /**
     * @brief Process sample.
     *
     * @param input Input sample.
     * @param mode Waveshaper mode.
     *
     * @return Processed sample.
     */
    static inline T process(
        T input,
        WaveshaperMode mode) noexcept
    {
        switch (mode)
        {
            case WaveshaperMode::HardClip:
                return hardClip(input);

            case WaveshaperMode::SoftClip:
                return softClip(input);

            case WaveshaperMode::Tanh:
                return tanhShape(input);

            case WaveshaperMode::Arctan:
                return arctanShape(input);

            case WaveshaperMode::Cubic:
                return cubicShape(input);

            case WaveshaperMode::Foldback:
                return foldbackShape(input);

            default:
                return input;
        }
    }

private:
    /**
     * @brief Hard Clipper.
     *
     * Abrupt clipping.
     */
    static inline T hardClip(
        T x) noexcept
    {
        return std::clamp(
            x,
            static_cast<T>(-1),
            static_cast<T>(1));
    }
    /**

```

```

    * @brief Soft Clipper.
    *
    * Smooth transition.
    */
static inline T softClip(
    T x) noexcept
{
    constexpr T threshold =
        static_cast<T>(1);

    if (x > threshold)
    {
        return threshold;
    }

    if (x < -threshold)
    {
        return -threshold;
    }

    return
        x
        -
        (
            x
            *
            x
            *
            x
            /
            static_cast<T>(3)
        );
}

/**
 * @brief Hyperbolic tangent.
 */
static inline T tanhShape(
    T x) noexcept
{
    return std::tanh(x);
}

/**
 * @brief Arctangent saturator.
 */
static inline T arctanShape(
    T x) noexcept
{
    constexpr T kScale =
        static_cast<T>(2)
        /
        std::numbers::pi_v<T>;

    return
        std::atan(x)
        *
        kScale;
}

/**
 * @brief Cubic saturation.
 *
 * Third-order polynomial.

```

```

    */
    static inline T cubicShape(
        T x) noexcept
    {
        constexpr T limit =
            static_cast<T>(1);

        x =
            std::clamp(
                x,
                -limit,
                limit);

        return
            x
            -
            (
                static_cast<T>(1.0 / 3.0)
                *
                x
                *
                x
                *
                x
            );
    }

    /**
     * @brief Foldback distortion.
     */
    static inline T foldbackShape(
        T x) noexcept
    {
        constexpr T threshold =
            static_cast<T>(1);

        if (x > threshold || x < -threshold)
        {
            x =
                std::fabs(
                    std::fmod(
                        x - threshold,
                        threshold * static_cast<T>(4)));

            x =
                std::fabs(
                    x
                    -
                    threshold
                    *
                    static_cast<T>(2))
                -
                threshold;
        }

        return x;
    }
};

} // namespace cvdsp::saturation

namespace cvdsp
{
    template<typename T>

```

```

using Waveshaper = saturation::Waveshaper<T>;

using WaveshaperMode = saturation::WaveshaperMode;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Spatial/MidSide.hpp
=====

```

```

#ifndef CVDSP_SPATIAL_MIDSIDE_HPP
#define CVDSP_SPATIAL_MIDSIDE_HPP

```

```

/**
 * @file MidSide.hpp
 * @brief Mid/Side Encoder and Decoder
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Features:
 * - Mid/Side Encode
 * - Mid/Side Decode
 * - Stereo Manipulation
 * - Mastering Support
 */

```

```

#include <type_traits>

```

```

namespace cvdsp::spatial
{

```

```

/**
 * @brief Mid/Side stereo utilities.
 *
 * Static-only class.
 *
 * No internal state.
 */

```

```

template<typename T>
class MidSide
{
    static_assert(
        std::is_floating_point_v<T>,
        "MidSide requires floating point type");

```

```

public:

```

```

    /**
     * @brief Mid/Side frame.
     */
    struct MSFrame
    {
        T mid;
        T side;
    };

```

```

    /**
     * @brief Stereo frame.
     */
    struct StereoFrame

```

```

{
    T left;
    T right;
};

/**
 * @brief Encode Left/Right to Mid/Side.
 *
 * Energy preserving form:
 *
 *  $Mid = (L + R) / \sqrt{2}$ 
 *  $Side = (L - R) / \sqrt{2}$ 
 */
[[nodiscard]]
static constexpr MSFrame encode(
    T left,
    T right) noexcept
{
    constexpr T kNorm =
        static_cast<T>(
            0.70710678118654752440);

    return
    {
        (left + right) * kNorm,
        (left - right) * kNorm
    };
}

/**
 * @brief Decode Mid/Side to Left/Right.
 *
 *  $L = (M + S) / \sqrt{2}$ 
 *  $R = (M - S) / \sqrt{2}$ 
 */
[[nodiscard]]
static constexpr StereoFrame decode(
    T mid,
    T side) noexcept
{
    constexpr T kNorm =
        static_cast<T>(
            0.70710678118654752440);

    return
    {
        (mid + side) * kNorm,
        (mid - side) * kNorm
    };
}

/**
 * @brief Compute Mid only.
 */
[[nodiscard]]
static constexpr T mid(
    T left,
    T right) noexcept
{
    constexpr T kNorm =
        static_cast<T>(
            0.70710678118654752440);

    return

```



```

        (left + right)
        * kNorm;
    }

    /**
     * @brief Compute Side only.
     */
    [[nodiscard]]
    static constexpr T side(
        T left,
        T right) noexcept
    {
        constexpr T kNorm =
            static_cast<T>(
                0.70710678118654752440);

        return
            (left - right)
            * kNorm;
    }
};

```

```

} // namespace cvdsp::spatial

```

```

namespace cvdsp
{
    template<typename T>
    using MidSide = spatial::MidSide<T>;
} // namespace cvdsp

```

```

#endif

```

```

=====
FILE: CV_DSP/Spatial/StereoWidth.hpp
=====

```

```

#ifndef CVDSP_SPATIAL_STEREOWIDTH_HPP
#define CVDSP_SPATIAL_STEREOWIDTH_HPP

```

```

/**
 * @file StereoWidth.hpp
 * @brief Stereo Width Processor
 *
 * Dependency:
 * - MidSide.hpp
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 */

```

```

#include <algorithm>
#include <type_traits>

```

```

#include "MidSide.hpp"

```

```

namespace cvdsp::spatial
{

```

```

/**
 * @brief Stereo Width Processor.
 *
 * Mid/Side based width control.

```

```

*
* Width:
*
* 0.0 = Mono
* 1.0 = Original Width
* 2.0 = 200% Width
*
* Processing:
*
* L/R
*  â†“
* M/S Encode
*  â†“
* Side Gain
*  â†“
* M/S Decode
*  â†“
* L/R
*/
template<typename T>
class StereoWidth
{
    static_assert(
        std::is_floating_point_v<T>,
        "StereoWidth requires floating point type");

public:
    /**
     * @brief Stereo frame.
     */
    struct StereoFrame
    {
        T left;
        T right;
    };

public:
    constexpr StereoWidth() noexcept = default;

    /**
     * @brief Process stereo frame.
     *
     * Width Range:
     *
     * 0.0 -> 2.0
     *
     * 0.0 = Mono
     * 1.0 = Original
     * 2.0 = 200%
     */
    [[nodiscard]]
    inline StereoFrame process(
        T left,
        T right,
        T width) const noexcept
    {
        width =
            std::clamp(
                width,
                static_cast<T>(0),
                static_cast<T>(2));
    }
};

```

```

        const auto ms =
            MidSide<T>::encode(
                left,
                right);

        const T mid =
            ms.mid;

        const T side =
            ms.side
            *
            width;

        const auto lr =
            MidSide<T>::decode(
                mid,
                side);

        return
        {
            lr.left,
            lr.right
        };
    }

    /**
     * @brief Process in-place.
     */
    inline void process(
        T& left,
        T& right,
        T width) const noexcept
    {
        const auto result =
            process(
                left,
                right,
                width);

        left = result.left;
        right = result.right;
    }
};

} // namespace cvdsp::spatial

namespace cvdsp
{
    template<typename T>
    using StereoWidth = spatial::StereoWidth<T>;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Spectral/FFT.hpp
=====

```

```

#ifndef CVDSP_SPECTRAL_FFT_HPP
#define CVDSP_SPECTRAL_FFT_HPP

```

```

/**
 * @file FFT.hpp

```

```

* @brief Iterative Cooley-Tukey FFT
*
* Header-Only
* C++20
* Real-Time Safe
*
* Features:
* - Iterative FFT
* - Iterative IFFT
* - Bit Reversal
* - Radix-2
* - std::complex
* - No recursion
*
* Supported Sizes:
* - 512
* - 1024
* - 2048
* - 4096
* - 8192
*
* Dependencies:
* - Core/Constants.hpp
* - Math/FastMath.hpp
*/

#include <array>
#include <complex>
#include <cstdint>
#include <type_traits>
#include <cmath>

namespace cvdsp::spectral
{
template<typename T>
class FFT
{
    static_assert(
        std::is_floating_point_v<T>,
        "FFT requires floating point type");

public:
    using Complex = std::complex<T>;

    static constexpr std::size_t MaxFFTSize = 8192;

public:
    constexpr FFT() noexcept = default;

    /**
     * @brief Prepare FFT instance.
     *
     * Allowed:
     *
     * 512
     * 1024
     * 2048
     * 4096
     * 8192
     */

```

```

bool prepare(
    std::size_t fftSize) noexcept
{
    if (
        fftSize != 512 &&
        fftSize != 1024 &&
        fftSize != 2048 &&
        fftSize != 4096 &&
        fftSize != 8192)
    {
        return false;
    }

    fftSize_ = fftSize;

    buildBitReverseTable();
    buildTwiddleTable();

    prepared_ = true;

    return true;
}

/**
 * @brief Reset state.
 */
void reset() noexcept
{
    fftSize_ = 0;

    prepared_ = false;

    bitReverse_.fill(0);
    twiddles_.fill(
        Complex(
            static_cast<T>(0),
            static_cast<T>(0)));
}

/**
 * @brief Forward FFT.
 *
 * Input and output may be
 * the same buffer.
 */
void forward(
    Complex* data) const noexcept
{
    if (!prepared_)
    {
        return;
    }

    bitReversePermutation(
        data);

    iterativeFFT(
        data,
        false);
}

/**
 * @brief Inverse FFT.
 */

```

```

void inverse(
    Complex* data) const noexcept
{
    if (!prepared_)
    {
        return;
    }

    bitReversePermutation(
        data);

    iterativeFFT(
        data,
        true);

    const T scale =
        static_cast<T>(1)
        /
        static_cast<T>(fftSize_);

    for (std::size_t i = 0;
        i < fftSize_;
        ++i)
    {
        data[i] *= scale;
    }
}

[[nodiscard]]
constexpr std::size_t getFFTSize() const noexcept
{
    return fftSize_;
}

```

private:

```

void buildBitReverseTable() noexcept
{
    const std::size_t bits =
        log2Int(fftSize_);

    for (std::size_t i = 0;
        i < fftSize_;
        ++i)
    {
        bitReverse_[i] =
            reverseBits(
                i,
                bits);
    }
}

void buildTwiddleTable() noexcept
{
    constexpr T pi =
        static_cast<T>(
            3.1415926535897932384626433832795);

    for (std::size_t k = 0;
        k < fftSize_ / 2;
        ++k)
    {
        const T angle =
            static_cast<T>(-2)

```

```

        *
        pi
        *
        static_cast<T>(k)
        /
        static_cast<T>(fftSize_);

    twiddles_[k] =
        Complex(
            std::cos(angle),
            std::sin(angle));
}
}

void bitReversePermutation(
    Complex* data) const noexcept
{
    for (std::size_t i = 0;
        i < fftSize_;
        ++i)
    {
        const std::size_t j =
            bitReverse_[i];

        if (j > i)
        {
            const Complex temp =
                data[i];

            data[i] =
                data[j];

            data[j] =
                temp;
        }
    }
}

void iterativeFFT(
    Complex* data,
    bool inverse) const noexcept
{
    for (
        std::size_t stage = 2;
        stage <= fftSize_;
        stage <<= 1)
    {
        const std::size_t half =
            stage >> 1;

        const std::size_t twiddleStep =
            fftSize_
            /
            stage;

        for (
            std::size_t start = 0;
            start < fftSize_;
            start += stage)
        {
            for (
                std::size_t k = 0;
                k < half;
                ++k)

```

```

        {
            Complex twiddle =
                twiddles_[
                    k * twiddleStep];

            if (inverse)
            {
                twiddle =
                    std::conj(
                        twiddle);
            }

            const Complex even =
                data[start + k];

            const Complex odd =
                twiddle
                *
                data[start + k + half];

            data[start + k] =
                even + odd;

            data[start + k + half] =
                even - odd;
        }
    }
}

```

```

[[nodiscard]]
static constexpr std::size_t
log2Int(
    std::size_t value) noexcept
{
    std::size_t result = 0;

    while (value > 1)
    {
        value >>= 1;
        ++result;
    }

    return result;
}

```

```

[[nodiscard]]
static constexpr std::size_t
reverseBits(
    std::size_t value,
    std::size_t bits) noexcept
{
    std::size_t result = 0;

    for (std::size_t i = 0;
        i < bits;
        ++i)
    {
        result <<= 1;

        result |=
            (value & 1);

        value >>= 1;
    }
}

```



```

        }

        return result;
    }

private:
    bool prepared_ = false;

    std::size_t fftSize_ = 0;

    std::array<
        std::size_t,
        MaxFFTSize> bitReverse_{};

    std::array<
        Complex,
        MaxFFTSize / 2> twiddles_{};
};

using FFTf = FFT<float>;
using FFTd = FFT<double>;

} // namespace cvdsp::spectral

namespace cvdsp
{
    template<typename T>
    using FFT = spectral::FFT<T>;

    using FFTf = spectral::FFTf;
    using FFTd = spectral::FFTd;
} // namespace cvdsp

#endif

=====
FILE: CV_DSP/Spectral/SpectrumAnalyzer.hpp
=====
#ifndef CVDSP_SPECTRAL_SPECTRUMANALYZER_HPP
#define CVDSP_SPECTRAL_SPECTRUMANALYZER_HPP

/**
 * @file SpectrumAnalyzer.hpp
 * @brief FFT Spectrum Analyzer
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 * - FFT.hpp
 * - WindowFunctions.hpp
 * - STFT.hpp
 */

#include <array>
#include <complex>
#include <cmath>
#include <cstdint>
#include <type_traits>

```

```

#include "FFT.hpp"
#include "WindowFunctions.hpp"

namespace cvdsp::spectral
{
    template<
        typename T,
        std::size_t FFTSize>
    class SpectrumAnalyzer
    {
    public:
        static_assert(
            std::is_floating_point_v<T>,
            "SpectrumAnalyzer requires floating point type");

        static constexpr std::size_t NumBins =
            FFTSize / 2;

        using Complex =
            std::complex<T>;

        using MagnitudeArray =
            std::array<T, NumBins>;

    public:
        SpectrumAnalyzer() = default;

        bool prepare(
            WindowType windowType =
                WindowType::BlackmanHarris,
            T peakHoldReleaseDbPerFrame =
                static_cast<T>(0.25))
            noexcept
        {
            peakDecay_ =
                peakHoldReleaseDbPerFrame;

            if (!fft_.prepare(
                FFTSize))
            {
                return false;
            }

            window_ =
                WindowFunctions<
                    T,
                    FFTSize>::generate(
                        windowType);

            reset();

            return true;
        }

        void reset() noexcept
        {
            for (auto& v : inputFrame_)
                v = T(0);

            for (auto& v : spectrum_)
                v = Complex(T(0), T(0));
        }
    };
}

```

```

        for (auto& v : magnitudes_)
            v = T(0);

        for (auto& v : rmsSpectrum_)
            v = T(0);

        for (auto& v : peakSpectrum_)
            v = T(0);
    }

    /**
     * Process one FFT frame.
     *
     * Input size must be FFTSize.
     */
    void process(
        const T* input) noexcept
    {
        copyInput(input);

        WindowFunctions<
            T,
            FFTSize>::apply(
                inputFrame_.data(),
                window_);

        buildComplexBuffer();

        fft_.forward(
            spectrum_.data());

        computeMagnitudeSpectrum();

        computeRMSSpectrum();

        updatePeakHold();
    }

    /**
     * Linear magnitudes.
     */
    [[nodiscard]]
    const MagnitudeArray&
    getMagnitudes() const noexcept
    {
        return magnitudes_;
    }

    /**
     * RMS spectrum.
     */
    [[nodiscard]]
    const MagnitudeArray&
    getRMSSpectrum() const noexcept
    {
        return rmsSpectrum_;
    }

    /**
     * Peak hold spectrum.
     */
    [[nodiscard]]
    const MagnitudeArray&

```

```

getPeakSpectrum() const noexcept
{
    return peakSpectrum_;
}

/**
 * Magnitude in dBFS.
 */
[[nodiscard]]
T getMagnitudeDB(
    std::size_t bin) const noexcept
{
    if (bin >= NumBins)
    {
        return static_cast<T>(-120);
    }

    return linearToDB(
        magnitudes_[bin]);
}

/**
 * Peak Hold in dBFS.
 */
[[nodiscard]]
T getPeakDB(
    std::size_t bin) const noexcept
{
    if (bin >= NumBins)
    {
        return static_cast<T>(-120);
    }

    return linearToDB(
        peakSpectrum_[bin]);
}

/**
 * RMS in dBFS.
 */
[[nodiscard]]
T getRMSDB(
    std::size_t bin) const noexcept
{
    if (bin >= NumBins)
    {
        return static_cast<T>(-120);
    }

    return linearToDB(
        rmsSpectrum_[bin]);
}

/**
 * Bin frequency.
 */
[[nodiscard]]
static constexpr T
binFrequency(
    std::size_t bin,
    T sampleRate) noexcept
{
    return
        static_cast<T>(bin)

```

```

        *
        sampleRate
        /
        static_cast<T>(FFTSize);
    }

private:

    void copyInput(
        const T* input) noexcept
    {
        for (std::size_t i = 0;
            i < FFTSize;
            ++i)
        {
            inputFrame_[i] =
                input[i];
        }
    }

    void buildComplexBuffer()
        noexcept
    {
        for (std::size_t i = 0;
            i < FFTSize;
            ++i)
        {
            spectrum_[i] =
                Complex(
                    inputFrame_[i],
                    T(0));
        }
    }

    void computeMagnitudeSpectrum()
        noexcept
    {
        const T scale =
            static_cast<T>(2)
            /
            static_cast<T>(FFTSize);

        for (std::size_t i = 0;
            i < NumBins;
            ++i)
        {
            magnitudes_[i] =
                std::abs(
                    spectrum_[i])
                *
                scale;
        }
    }

    void computeRMSSpectrum()
        noexcept
    {
        constexpr T alpha =
            static_cast<T>(0.15);

        for (std::size_t i = 0;
            i < NumBins;
            ++i)
        {

```

```

        const T power =
            magnitudes_[i]
            *
            magnitudes_[i];

        rmsSpectrum_[i] =
            std::sqrt(
                alpha * power
                +
                (static_cast<T>(1) - alpha)
                *
                rmsSpectrum_[i]
                *
                rmsSpectrum_[i]);
    }
}

```

```

void updatePeakHold()
    noexcept
{
    const T decay =
        dbToLinear(
            -peakDecay_);

    for (std::size_t i = 0;
        i < NumBins;
        ++i)
    {
        if (magnitudes_[i] >
            peakSpectrum_[i])
        {
            peakSpectrum_[i] =
                magnitudes_[i];
        }
        else
        {
            peakSpectrum_[i] *=
                decay;
        }
    }
}

```

```

[[nodiscard]]
static T linearToDB(
    T value) noexcept
{
    constexpr T minimum =
        static_cast<T>(1e-12);

    if (value < minimum)
    {
        value = minimum;
    }

    return
        static_cast<T>(20)
        *
        std::log10(value);
}

```

```

[[nodiscard]]
static T dbToLinear(
    T db) noexcept
{

```

```

        return
            std::pow(
                static_cast<T>(10),
                db /
                static_cast<T>(20));
    }

private:
    FFT<T> fft_;

    typename WindowFunctions<
        T,
        FFTSize>::WindowBuffer window_;

    std::array<T, FFTSize>
        inputFrame_{};

    std::array<Complex, FFTSize>
        spectrum_{};

    MagnitudeArray magnitudes_{};

    MagnitudeArray peakSpectrum_{};

    MagnitudeArray rmsSpectrum_{};

    T peakDecay_ =
        static_cast<T>(0.25);
};

using SpectrumAnalyzer512F =
    SpectrumAnalyzer<float, 512>;

using SpectrumAnalyzer1024F =
    SpectrumAnalyzer<float, 1024>;

using SpectrumAnalyzer2048F =
    SpectrumAnalyzer<float, 2048>;

using SpectrumAnalyzer4096F =
    SpectrumAnalyzer<float, 4096>;

using SpectrumAnalyzer8192F =
    SpectrumAnalyzer<float, 8192>;

using SpectrumAnalyzer512D =
    SpectrumAnalyzer<double, 512>;

using SpectrumAnalyzer1024D =
    SpectrumAnalyzer<double, 1024>;

using SpectrumAnalyzer2048D =
    SpectrumAnalyzer<double, 2048>;

using SpectrumAnalyzer4096D =
    SpectrumAnalyzer<double, 4096>;

using SpectrumAnalyzer8192D =
    SpectrumAnalyzer<double, 8192>;

} // namespace cvdsp::spectral

namespace cvdsp

```

```

{
template<typename T, std::size_t FFTSize>
using SpectrumAnalyzer = spectral::SpectrumAnalyzer<T, FFTSize>;

using SpectrumAnalyzer512F = spectral::SpectrumAnalyzer512F;
using SpectrumAnalyzer1024F = spectral::SpectrumAnalyzer1024F;
using SpectrumAnalyzer2048F = spectral::SpectrumAnalyzer2048F;
using SpectrumAnalyzer4096F = spectral::SpectrumAnalyzer4096F;
using SpectrumAnalyzer8192F = spectral::SpectrumAnalyzer8192F;
using SpectrumAnalyzer512D = spectral::SpectrumAnalyzer512D;
using SpectrumAnalyzer1024D = spectral::SpectrumAnalyzer1024D;
using SpectrumAnalyzer2048D = spectral::SpectrumAnalyzer2048D;
using SpectrumAnalyzer4096D = spectral::SpectrumAnalyzer4096D;
using SpectrumAnalyzer8192D = spectral::SpectrumAnalyzer8192D;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Spectral/STFT.hpp
=====

```

```

#ifndef CVDSP_SPECTRAL_STFT_HPP
#define CVDSP_SPECTRAL_STFT_HPP

/**
 * @file STFT.hpp
 * @brief Short Time Fourier Transform
 *
 * Header-Only
 * C++20
 * Real-Time Safe
 *
 * Dependencies:
 *
 * - FFT.hpp
 * - WindowFunctions.hpp
 * - ../Core/CircularBuffer.hpp
 *
 * Features:
 *
 * - Forward STFT
 * - Inverse STFT
 * - Overlap Add (OLA)
 * - Overlap Save (OLS)
 *
 * Supported Overlap:
 *
 * - 25%
 * - 50%
 * - 75%
 *
 * No dynamic allocation inside process().
 */

```

```

#include <array>
#include <complex>
#include <cstdint>
#include <type_traits>

```

```

#include "FFT.hpp"
#include "WindowFunctions.hpp"
#include "../Core/CircularBuffer.hpp"

```



```

namespace cvdsp::spectral
{
enum class STFTMode
{
    OverlapAdd,
    OverlapSave
};

template<
    typename T,
    std::size_t FFTSize,
    std::size_t OverlapPercent = 50>
class STFT
{
    static_assert(
        std::is_floating_point_v<T>,
        "STFT requires floating point type");

    static_assert(
        FFTSize == 512 ||
        FFTSize == 1024 ||
        FFTSize == 2048 ||
        FFTSize == 4096 ||
        FFTSize == 8192,
        "Unsupported FFT size");

    static_assert(
        OverlapPercent == 25 ||
        OverlapPercent == 50 ||
        OverlapPercent == 75,
        "Overlap must be 25, 50 or 75");

public:
    using Complex =
        std::complex<T>;

    static constexpr std::size_t WindowSize =
        FFTSize;

    static constexpr std::size_t HopSize =
        FFTSize
        *
        (100 - OverlapPercent)
        / 100;

public:
    STFT() = default;

    /**
     * Prepare STFT.
     */
    bool prepare(
        WindowType windowType =
            WindowType::Hann,
        STFTMode mode =
            STFTMode::OverlapAdd)
        noexcept
    {
        mode_ = mode;
    }

```

```

        if (!fft_.prepare(
            FFTSize))
        {
            return false;
        }

        window_ =
            WindowFunctions<
                T,
                FFTSize>::generate(
                    windowType);

        reset();

        return true;
    }

/**
 * Reset state.
 */
void reset() noexcept
{
    inputBuffer_.reset();
    outputBuffer_.reset();

    frameCounter_ = 0;

    for (auto& v : timeDomain_)
        v = T(0);

    for (auto& v : overlapBuffer_)
        v = T(0);

    for (auto& v : spectrum_)
        v = Complex(T(0), T(0));
}

/**
 * Process one sample.
 *
 * Returns reconstructed output.
 */
T process(
    T inputSample)
    noexcept
{
    inputBuffer_.push(
        inputSample);

    T output =
        outputBuffer_.read(0);

    outputBuffer_.push(
        T(0));

    ++frameCounter_;

    if (frameCounter_ >= HopSize)
    {
        frameCounter_ = 0;

        if (mode_ ==
            STFTMode::OverlapAdd)
        {

```

```

        processOLA();
    }
    else
    {
        processOLS();
    }
}

return output;
}

/**
 * Direct spectrum access.
 */
[[nodiscard]]
Complex* getSpectrum()
    noexcept
{
    return spectrum_.data();
}

/**
 * Const spectrum access.
 */
[[nodiscard]]
const Complex* getSpectrum() const
    noexcept
{
    return spectrum_.data();
}

/**
 * FFT size.
 */
[[nodiscard]]
static constexpr std::size_t
getFFTSize() noexcept
{
    return FFTSize;
}

/**
 * Window size.
 */
[[nodiscard]]
static constexpr std::size_t
getWindowSize() noexcept
{
    return WindowSize;
}

/**
 * Hop size.
 */
[[nodiscard]]
static constexpr std::size_t
getHopSize() noexcept
{
    return HopSize;
}

```

private:

```

/**

```

```

    * Overlap Add.
    */
void processOLA()
    noexcept
{
    acquireFrame();

    applyWindow();

    performForwardFFT();

    /**
     * Spectral processing hook.
     */

    performInverseFFT();

    applyWindow();

    overlapAdd();
}

/**
 * Overlap Save.
 */
void processOLS()
    noexcept
{
    acquireFrame();

    applyWindow();

    performForwardFFT();

    /**
     * Spectral processing hook.
     */

    performInverseFFT();

    saveOverlap();
}

void acquireFrame()
    noexcept
{
    for (std::size_t i = 0;
         i < FFTSize;
         ++i)
    {
        timeDomain_[i] =
            inputBuffer_.read(
                FFTSize - 1 - i);
    }
}

void applyWindow()
    noexcept
{
    WindowFunctions<
        T,
        FFTSize>::apply(
        timeDomain_.data(),
        window_);
}

```

```

}

void performForwardFFT()
    noexcept
{
    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        spectrum_[i] =
            Complex(
                timeDomain_[i],
                T(0));
    }

    fft_.forward(
        spectrum_.data());
}

void performInverseFFT()
    noexcept
{
    fft_.inverse(
        spectrum_.data());

    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        timeDomain_[i] =
            spectrum_[i].real();
    }
}

void overlapAdd()
    noexcept
{
    for (std::size_t i = 0;
        i < FFTSize;
        ++i)
    {
        overlapBuffer_[i] +=
            timeDomain_[i];
    }

    for (std::size_t i = 0;
        i < HopSize;
        ++i)
    {
        outputBuffer_.push(
            overlapBuffer_[i]);
    }

    for (std::size_t i = 0;
        i < FFTSize - HopSize;
        ++i)
    {
        overlapBuffer_[i] =
            overlapBuffer_[
                i + HopSize];
    }

    for (std::size_t i =
        FFTSize - HopSize;

```

```

        i < FFTSize;
        ++i)
    {
        overlapBuffer_[i] =
            T(0);
    }
}

void saveOverlap()
    noexcept
{
    constexpr std::size_t overlap =
        FFTSize - HopSize;

    for (std::size_t i = overlap;
        i < FFTSize;
        ++i)
    {
        outputBuffer_.push(
            timeDomain_[i]);
    }
}

private:

    FFT<T> fft_;

    STFTMode mode_ =
        STFTMode::OverlapAdd;

    typename WindowFunctions<
        T,
        FFTSize>::WindowBuffer window_;

    std::array<T, FFTSize>
        timeDomain_{};

    std::array<T, FFTSize>
        overlapBuffer_{};

    std::array<Complex, FFTSize>
        spectrum_{};

    CircularBuffer<
        T,
        FFTSize * 2>
        inputBuffer_;

    CircularBuffer<
        T,
        FFTSize * 2>
        outputBuffer_;

    std::size_t frameCounter_ = 0;
};

/**
 * Common aliases.
 */

using STFT512F =
    STFT<float, 512>;

using STFT1024F =

```

```

        STFT<float, 1024>;

using STFT2048F =
    STFT<float, 2048>;

using STFT4096F =
    STFT<float, 4096>;

using STFT8192F =
    STFT<float, 8192>;

using STFT512D =
    STFT<double, 512>;

using STFT1024D =
    STFT<double, 1024>;

using STFT2048D =
    STFT<double, 2048>;

using STFT4096D =
    STFT<double, 4096>;

using STFT8192D =
    STFT<double, 8192>;

} // namespace cvdsp::spectral

namespace cvdsp
{
using STFTMode = spectral::STFTMode;

template<typename T, std::size_t FFTSize, std::size_t OverlapPercent = 50>
using STFT = spectral::STFT<T, FFTSize, OverlapPercent>;

using STFT512F = spectral::STFT512F;
using STFT1024F = spectral::STFT1024F;
using STFT2048F = spectral::STFT2048F;
using STFT4096F = spectral::STFT4096F;
using STFT8192F = spectral::STFT8192F;
using STFT512D = spectral::STFT512D;
using STFT1024D = spectral::STFT1024D;
using STFT2048D = spectral::STFT2048D;
using STFT4096D = spectral::STFT4096D;
using STFT8192D = spectral::STFT8192D;
} // namespace cvdsp

#endif

```

```

=====
FILE: CV_DSP/Spectral/WindowFunctions.hpp
=====
#ifndef CVDSP_SPECTRAL_WINDOWFUNCTIONS_HPP
#define CVDSP_SPECTRAL_WINDOWFUNCTIONS_HPP

/**
 * @file WindowFunctions.hpp
 * @brief Spectral Window Functions
 *
 * Header-Only
 * C++20
 * Real-Time Safe

```

```

*
* Dependencies:
* - FFT.hpp
*
* Supported Windows:
* - Rectangular
* - Hann
* - Hamming
* - Blackman
* - Blackman-Harris
* - Kaiser
*/

#include <array>
#include <cstdint>
#include <cmath>
#include <type_traits>

namespace cvdsp::spectral
{
/**
 * @brief Supported window types.
 */
enum class WindowType
{
    Rectangular,
    Hann,
    Hamming,
    Blackman,
    BlackmanHarris,
    Kaiser
};

/**
 * @brief Window generator.
 *
 * Template parameter N defines the
 * window size at compile time.
 */
template<typename T, std::size_t N>
class WindowFunctions
{
    static_assert(
        std::is_floating_point_v<T>,
        "WindowFunctions requires floating point type");

public:
    using WindowBuffer =
        std::array<T, N>;

public:
    /**
     * @brief Generate window coefficients.
     *
     * Kaiser beta is ignored by all
     * other window types.
     */
    [[nodiscard]]
    static WindowBuffer generate(
        WindowType type,

```



```

    T beta =
        static_cast<T>(8.6)) noexcept
{
    WindowBuffer window{};

    switch(type)
    {
        case WindowType::Rectangular:
            generateRectangular(window);
            break;

        case WindowType::Hann:
            generateHann(window);
            break;

        case WindowType::Hamming:
            generateHamming(window);
            break;

        case WindowType::Blackman:
            generateBlackman(window);
            break;

        case WindowType::BlackmanHarris:
            generateBlackmanHarris(window);
            break;

        case WindowType::Kaiser:
            generateKaiser(
                window,
                beta);
            break;
    }

    return window;
}

/**
 * @brief Apply window in-place.
 */
static void apply(
    T* samples,
    const WindowBuffer& window) noexcept
{
    for (std::size_t i = 0;
        i < N;
        ++i)
    {
        samples[i] *=
            window[i];
    }
}

/**
 * @brief Apply window from source
 * to destination.
 */
static void apply(
    const T* input,
    T* output,
    const WindowBuffer& window) noexcept
{
    for (std::size_t i = 0;
        i < N;

```

```

        ++i)
    {
        output[i] =
            input[i]
            *
            window[i];
    }
}

```

private:

```

static constexpr T pi() noexcept
{
    return static_cast<T>(
        3.1415926535897932384626433832795);
}

```

```

static void generateRectangular(
    WindowBuffer& w) noexcept
{
    for (auto& v : w)
    {
        v =
            static_cast<T>(1);
    }
}

```

```

static void generateHann(
    WindowBuffer& w) noexcept
{
    constexpr T twoPi =
        static_cast<T>(2)
        *
        pi();

    for (std::size_t n = 0;
        n < N;
        ++n)
    {
        w[n] =
            static_cast<T>(0.5)
            -
            static_cast<T>(0.5)
            *
            std::cos(
                twoPi
                *
                static_cast<T>(n)
                /
                static_cast<T>(N - 1));
    }
}

```

```

static void generateHamming(
    WindowBuffer& w) noexcept
{
    constexpr T twoPi =
        static_cast<T>(2)
        *
        pi();

    for (std::size_t n = 0;
        n < N;
        ++n)
    {

```

```

{
    w[n] =
        static_cast<T>(0.54)
        -
        static_cast<T>(0.46)
        *
        std::cos(
            twoPi
            *
            static_cast<T>(n)
            /
            static_cast<T>(N - 1));
}
}

```

```

static void generateBlackman(
    WindowBuffer& w) noexcept
{
    constexpr T a0 =
        static_cast<T>(0.42);

    constexpr T a1 =
        static_cast<T>(0.50);

    constexpr T a2 =
        static_cast<T>(0.08);

    constexpr T twoPi =
        static_cast<T>(2)
        *
        pi();

    constexpr T fourPi =
        static_cast<T>(4)
        *
        pi();

    for (std::size_t n = 0;
        n < N;
        ++n)
    {
        const T phase =
            static_cast<T>(n)
            /
            static_cast<T>(N - 1);

        w[n] =
            a0
            -
            a1
            *
            std::cos(
                twoPi * phase)
            +
            a2
            *
            std::cos(
                fourPi * phase);
    }
}

```

```

static void generateBlackmanHarris(
    WindowBuffer& w) noexcept
{

```

```

constexpr T a0 =
    static_cast<T>(0.35875);

constexpr T a1 =
    static_cast<T>(0.48829);

constexpr T a2 =
    static_cast<T>(0.14128);

constexpr T a3 =
    static_cast<T>(0.01168);

constexpr T twoPi =
    static_cast<T>(2)
    *
    pi();

constexpr T fourPi =
    static_cast<T>(4)
    *
    pi();

constexpr T sixPi =
    static_cast<T>(6)
    *
    pi();

for (std::size_t n = 0;
     n < N;
     ++n)
{
    const T phase =
        static_cast<T>(n)
        /
        static_cast<T>(N - 1);

    w[n] =
        a0
        -
        a1
        *
        std::cos(
            twoPi * phase)
        +
        a2
        *
        std::cos(
            fourPi * phase)
        -
        a3
        *
        std::cos(
            sixPi * phase);
}

static void generateKaiser(
    WindowBuffer& w,
    T beta) noexcept
{
    const T denom =
        besseli0(beta);

    for (std::size_t n = 0;

```

```

        n < N;
        ++n)
    {
        const T ratio =
            (
                static_cast<T>(2)
                *
                static_cast<T>(n)
            )
            /
            static_cast<T>(N - 1)
            -
            static_cast<T>(1);

        const T value =
            beta
            *
            std::sqrt(
                static_cast<T>(1)
                -
                ratio * ratio);

        w[n] =
            besseli0(value)
            /
            denom;
    }
}

/**
 * @brief Modified Bessel I0.
 *
 * Polynomial approximation.
 */
[[nodiscard]]
static T besseli0(
    T x) noexcept
{
    T sum =
        static_cast<T>(1);

    T term =
        static_cast<T>(1);

    const T xx =
        x * x
        *
        static_cast<T>(0.25);

    for (std::size_t k = 1;
        k < 30;
        ++k)
    {
        term *=
            xx
            /
            (
                static_cast<T>(k)
                *
                static_cast<T>(k));

        sum += term;
    }
}

```

```
        return sum;
    }
};

} // namespace cvdsp::spectral

namespace cvdsp
{
using WindowType = spectral::WindowType;

template<typename T, std::size_t N>
using WindowFunctions = spectral::WindowFunctions<T, N>;
} // namespace cvdsp

#endif
```